Machine Learning

Kon Yi

November 6, 2026

Preface

These are my notes for *Machine Learning* (機器學習), taught by Hsuan-Tien Lin (林軒田) at National Taiwan University in the fall of 2026. This preface reproduces the course-introduction handout of 09/09/2026, so that the ground rules sit next to the material they govern; everything after it is the course itself.

Five Reasons for NOT Taking the Course

The course opens, by tradition, with an argument against taking it.

Complicated contents. From a Taiwanese student taking the MIT ML class (translated):

> *The professor started writing math equations as if he was using some writing accelerator. After class I always felt feeble. The worst part is: I needed to understand the contents as soon as I can. Otherwise I cannot finish the homework and cannot follow up in the next class.*

The NTU ML class is designed to be as good as the best classes in the world, so similar things will happen to you. *If you are not willing to be so miserable, ...*

Strict instructor. Will you give me a second chance if I copy homework from other people? No. Could you let me pass because I will be kicked out by the 1/2 rule? No. Will you change my score from F to C? No. How many will pass? Any, if necessary. *If you do not like a strict instructor, ...*

Huge loads. From a student taking ML 2010, posted on BBS (translated):

> *Only 1 problem requires running 100 test experiments? (*100×10 *min* $= 16$ *hr); I double-checked multiple times — the one who designed this homework is heartless.*

This class is two to six times harder than a normal one at NTU: six homework sets are planned, plus the final project, each due within two weeks in general. Homework 0 arrives in the first week — already hard, worth very few points, and there to remind you of the prerequisites. *If you do not want to spend so much time on homework, ...*

Online-learnable. Two of the course's incarnations were invited by NTU as Massive Online Open Courses on NTU-Coursera, *Machine Learning Foundations* and *Machine Learning Techniques*, which cover 70% of what we teach; it is much easier to simply take the recorded version at home. Indeed we will do pseudo-flipped teaching. *If you want to learn at your own pace, ...*

Shallow and old-fashioned. This is not a deep learning class — it surrounds shallow (old) machine learning. Yes, many of the materials are ten or more years old; and yes, AI has knowledge of those materials, just as AI can write code. *If you want to chase newer buzzwords, ...*

From a student in ML 2013, in the final feedback (translated): *"this is a class that makes you experience all kinds of devastation and desperation."*

May the Brave Ones Stay.

Basic Information

- **Mode**: physical, with a live YouTube screencast and Slido questions (`#HTML2026FALL`), if everything goes smoothly.
- **Instructor**: Hsuan-Tien Lin, htlin@csie.ntu.edu.tw. Office hour after class, or by appointment.
- **Public course infopage**: https://www.csie.ntu.edu.tw/~htlin/course/ml26fall/.
- **Semi-private course platform**: NTU COOL, for announcements, recorded videos and the like — be sure to check emails from NTU COOL frequently.
- **Class chats**: Discord (invitation URL on NTU COOL).
- **Homework grading** (registered students only): Gradescope, linked from NTU COOL.
- **Languages**: teaching in Mandarin; Slido questions in English, traditional or simplified Chinese; homework written in English or Chinese; TA and instructor hours in Mandarin, with English welcome. Don't be afraid of English.

The Course and Its History

- 2008–2012: 3 credits, taught in English.
- 2013–2014: 3 credits, Mandarin, with MOOC.
- 2015: 4 credits, Mandarin, with MOOC.
- 2016–2020: mostly $2 + 2$ credits (MLF + MLT), Mandarin, with MOOC.
- 2021 Fall and 2023 Spring: 3 credits, English.
- 2023 Fall: 3 credits, Mandarin, with MOOC offline and physical discussions stimulated by ChatGPT.
- 2024: 3 credits, English, TAICA.
- 2026: 3 credits, Mandarin, TAICA, with MOOC offline and discussions stimulated by ChatGPT.

The reason for moving the lectures offline is that many students prefer watching a recording on their own time, which frees the precious class time to encourage speculation rather than lecturing. Three things are new this semester.

Cross-university teaching (2nd time). Taiwan needs more AI talent, so the course accepted an invitation from TAICA (Taiwan AI College Alliance) to be offered to 50+ universities — with thanks to the MOE for more TAs, a YouTube screencast to break the time and space boundary, and some online TA hours to be planned. An attempt to further increase the impact in this new AI age.

Final exam. AI can solve all of your homework with a high success rate. That is not a totally bad thing for teaching, but it makes fair grading exponentially difficult, so we will experiment with a closed-book final exam bearing some similarity to the homework — an old-fashioned yet fair grading protocol, until something better is designed.

AI-assisted TAship. AI can answer many questions with 90%+ correctness in a friendly manner; the question is how to best help learning. We are building an AI agent to answer students' questions, guarded by the TAs' regular human checking — human-computer collaboration towards better teaching and learning.

On enrollment: per school policy, students who have taken another version of the course (ML Foundations, ML Techniques, or the 4-credit ML) are generally *not* allowed to take it again. We will not check, but the school may, and multiple takings may not look nice on a transcript anyway. NTU students sign up through the link on the course infopage, processed progressively; a limited quota is reserved for TAICA students from other universities. Auditing is welcome in general, as long as it does not interfere with official members. *Leave as soon as possible — give your classmates a chance to be miserable.*

The course is supported by sixteen teaching assistants, reachable at html_ta@csie.ntu.edu.tw.

- Cheng-Han Hsieh (∗), CSIE
- Guan-Chen Chen, CSIE
- Chun-Yang Cheng, CSIE
- Sheng-Yuan Chang, CSIE
- Tsung-Lun Chiang, EE
- Ting-Chieh Chiu, CSIE
- Ting-Wei Chuang, CSIE
- Kang-Yu Fan, CSIE
- Yen-Hung Hsueh, EE
- Li-Cheng Hsu, EE
- Jung-Sheng Hung, GINM
- Yun-Shi Lee, IM
- Yi-Ting Lo, CSIE
- Han-Hwa Shih, GICE
- Yen-Chu Tsai, CSIE
- Yi-Ming Yao, Math

THE Book

Learning from Data: A Short Course
Y. Abu-Mostafa (Caltech), M. Magdon-Ismail (RPI), H.-T. Lin (NTU)

The idea was initiated in 2008. Its five chapters are closely needed for the first part of the class, and further e-Chapters are used in the second part; the teaching follows the book, with suggested reading inside it. Copies can be found at the NTU Library (a reserved copy in the shared course material area) and in R536 (shared copies to be read in the room, if open); the Chuan-Hwa Book Company has imported some copies, and Amazon/Kindle is the main selling channel in the US. If the book is not affordable to you but you really want to read it, email the instructor and he will see how he can help.

THE Principle

Taking any unfair advantage over other class members is not allowed. It is everyone's responsibility to maximize the level of fairness.

NO CHEATING
NO LYING
NO PLAGIARISM
NO PIRATING of THE BOOK

There are very serious consequences. In practice this means: homework discussions are encouraged, but write the final solutions independently and understand them fully; references — books, notes, the Internet, AI tools — can be consulted but not copied from, and you are strongly encouraged to save your AI prompts and interaction logs. There is no need to lend, borrow, sell, buy or trade a solution, a piece of code or a dialogue, and doing so is not allowed.

Grading

There is no midterm. The main reference is

homework sets	30%
final exam	30%
final project	40%

after which the raw score goes through an order-preserving normalization: a raw score of 80 with term rank A is possible, and so is a raw score of 60 with term rank F; raw scores 80 and 60 both mapping to B is possible but unlikely; raw scores 80 and 60 mapping to F and B respectively is impossible. For non-NTU students the rank will tentatively be converted by the NTU metric back to a score in $[0, 100]$, which the student's own university then converts back to a rank.

Homework uses multiple-choice problems for auto-grading, with photographed or scanned details and code uploaded for selective grading by the TAs — there is a penalty if your details are not logical — and for plagiarism detection. Late parts are worth 90% if twelve hours late, 80% if one day late, and so on; each person is granted four penalty-free late half-days, the gold medals. About a quarter to a third of the problems are programming assignments, in any language, on any platform, with any packages; AI-generated code is fine if you co-wrote it with the AI and your logs justify that.

The *final project* is a teamed in-class competition, with team size to be decided, co-run with Prof. Shou-de Lin's *Machine Learning: Theory and Practice* class — so the two classes cannot both be taken this semester. You will need to sign and abide by a data policy. It can make you rich, if you know what I mean.

Contents

1. The Learning Problem **8**
- 1.1. Course Introduction . . . 8
- 1.2. What is Machine Learning . . . 9
- 1.3. Applications of Machine Learning . . . 11
- 1.4. Components of Machine Learning . . . 14
- 1.5. Machine Learning and Other Fields . . . 16

2. Learning to Answer Yes/No **19**
- 2.1. Perceptron Hypothesis Set . . . 19
- 2.2. Perceptron Learning Algorithm (PLA) . . . 22
- 2.3. Guarantee of PLA . . . 25
- 2.4. Non-Separable Data . . . 29

3. Types of Learning **32**
- 3.1. Learning with Different Output Space $\mathcal{Y}$ . . . 32
- 3.2. Learning with Different Data Label y_n . . . 35
- 3.3. Learning with Different Protocol $f \Rightarrow (\boldsymbol{x}_n, y_n)$ . . . 38
- 3.4. Learning with Different Input Space $\mathcal{X}$ . . . 40

4. Feasibility of Learning **44**
- 4.1. Learning is Impossible? . . . 44
- 4.2. Probability to the Rescue . . . 47
- 4.3. Connection to Learning . . . 49
- 4.4. Connection to Real Learning . . . 52

5. Training versus Testing **57**
- 5.1. Recap and Preview . . . 57
- 5.2. Effective Number of Lines . . . 59
- 5.3. Effective Number of Hypotheses . . . 63
- 5.4. Break Point . . . 67

6. Theory of Generalization **70**
- 6.1. Restriction of Break Point . . . 70
- 6.2. Bounding Function: Basic Cases . . . 72
- 6.3. Bounding Function: Inductive Cases . . . 74
- 6.4. A Pictorial Proof . . . 78

7. The VC Dimension **83**
- 7.1. Definition of VC Dimension . . . 83
- 7.2. VC Dimension of Perceptrons . . . 86
- 7.3. Physical Intuition of VC Dimension . . . 90
- 7.4. Interpreting VC Dimension . . . 92

8. Noise and Error **96**
- 8.1. Noise and Probabilistic Target . . . 96
- 8.2. Error Measure . . . 99
- 8.3. Algorithmic Error Measure . . . 102
- 8.4. Weighted Classification . . . 105

9. Linear Regression **109**
9.1. Linear Regression Problem . . . 109
9.2. Linear Regression Algorithm . . . 111
9.3. Generalization Issue . . . 115
9.4. Linear Regression for Binary Classification . . . 118

10. Logistic Regression **121**
10.1. Logistic Regression Problem . . . 121
10.2. Logistic Regression Error . . . 124
10.3. Gradient of Logistic Regression Error . . . 127
10.4. Gradient Descent . . . 129

11. Linear Models for Classification **135**
11.1. Linear Models for Binary Classification . . . 135
11.2. Stochastic Gradient Descent . . . 139
11.3. Multiclass via Logistic Regression . . . 143
11.4. Multiclass via Binary Classification . . . 147

12. Nonlinear Transformation **150**
12.1. Quadratic Hypotheses . . . 150
12.2. Nonlinear Transform . . . 153
12.3. Price of Nonlinear Transform . . . 157
12.4. Structured Hypothesis Sets . . . 160

13. Hazard of Overfitting **163**
13.1. What is Overfitting? . . . 163
13.2. The Role of Noise and Data Size . . . 165
13.3. Deterministic Noise . . . 168
13.4. Dealing with Overfitting . . . 172

14. Regularization **175**
14.1. Regularized Hypothesis Set . . . 175
14.2. Weight Decay Regularization . . . 177
14.3. Regularization and VC Theory . . . 181
14.4. General Regularizers . . . 182

1. The Learning Problem

Lecture 1

§1.1. Course Introduction

9 Sep.

slides p.1–5

Machine learning is a mixture of theoretical and practical tools, so any course about it has to decide how to mix them. There are two natural extremes, and both of them fail in their own way.

- *Theory oriented*: derive everything deeply, so that every statement rests on a proof and the understanding is solid. The cost is that it is less interesting to a general audience — one may finish such a course able to prove a generalization bound, yet unable to fit a model to a real data set.
- *Techniques oriented*: flash over the sexiest techniques broadly, so that the coverage looks shiny. The cost is that there are simply too many techniques: they become hard to choose between, and hard to use properly, because nothing tells you *why* one of them should work here and not there.

Our approach: foundation oriented.

By *foundation* we mean the part that does not change when the fashionable technique changes. Concretely, the course is a mixture of philosophical illustrations, key theory, core techniques, usage in practice, and hopefully jokes; the selecting principle is always *what every machine learning user should know*. The payoff of insisting on foundations is that they transfer: a student who owns them can pick up a future or untaught technique, or go read deeper theory, without being taught each one explicitly.

The material is organized **story-like**, as a sequence of questions, each one asked only after the previous one has been answered:

1. **When** can machines learn? (illustrative + technical)
2. **Why** can machines learn? (theoretical + illustrative)
3. **How** can machines learn? (technical + practical)
4. **How** can machines learn **better**? (practical + theoretical)

Note 1.1.1. The four questions above are the backbone of the *foundations* half of the story. The NTU version continues it with three more chapters, each one a different answer to "where do good features come from":

5. How can machines learn by *embedding numerous* features? (support vector machines)
6. How can machines learn by *combining predictive* features? (aggregation, boosting, trees and forests)
7. How can machines learn by *distilling hidden* features? (neural networks and deep learning)

This first chapter sits entirely inside question 1, and it is the most basic piece of it: before asking when machines *can* learn, we must say what the learning problem even *is*. That is the plan of the chapter: we first ask what machine learning is (Section 1.2), then look at where it already lives (Section 1.3), then cut it into its formal components (Section 1.4), and finally place it next to the neighbouring fields it is often confused with (Section 1.5).

Exercise 1.1.2 (Fun Time)**.** Which of the following descriptions of this course is true?

(1) the course will be taught in Taiwanese;

(2) the course will tell me the techniques that create the android Lieutenant Commander Data in *Star Trek*;

(3) the course will be 15 weeks long;

(4) the course will be story-like.

Reference Answer. (4). As for the others: (1) no, the instructor's Taiwanese is unfortunately not good enough for teaching (yet); (2) no, although what we teach may serve as foundations of those (future) techniques; (3) no, unless you choose to join the next course. So: let us begin the story. ■

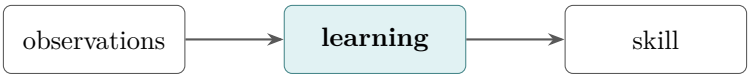
§1.2. What is Machine Learning

slides p.6–11

§1.2.i. From Learning to Machine Learning

The safest way into a definition is by analogy with the thing we already do ourselves.

Definition 1.2.1 (Learning)**.** **Learning** is acquiring *skill* with experience accumulated from *observations*.



A machine has no eyes and no childhood, so the only observations it can accumulate are the ones we hand it in recorded form. Replacing *observations* by *data* and *learning* by *computation* gives the machine version.

Definition 1.2.2 (Machine learning, first attempt)**.** **Machine learning** is acquiring skill with experience accumulated/computed from *data*.



The word *data* is concrete enough to be useful. The word *skill* is not: it is exactly the kind of word that cannot be handed to a computer, so the definition is not finished yet.

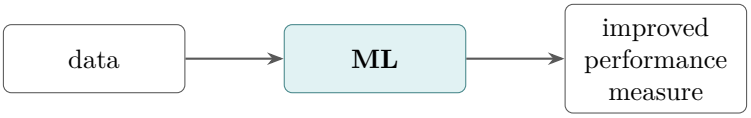
Question 1.2.3. What is skill?

§1.2.ii. A More Concrete Definition

Intuition . skill ⇔ improve some *performance measure* (e.g. prediction accuracy).

This is the move that makes the whole subject possible: a vague goal ("be good at recognizing trees") is replaced by a number that can be computed and watched ("fraction of tree images classified correctly"). Every later chapter is, in one way or another, an argument about which number to watch and what watching it really guarantees.

Definition 1.2.4 (Machine learning)**.** **Machine learning** is improving some *performance measure* with experience computed from *data*.



Example 1.2.5 (An application in computational finance)**.** Take the data to be historical stock data, and the performance measure to be investment gain. Then machine learning here means: use the stock data to compute a trading rule whose gain is higher than what we had before.



§1.2.iii. Why Use Machine Learning?

We now know what machine learning *is*. A separate matter is why one would ever want it, given that a computer can already be programmed directly.

Question 1.2.6. Why use machine learning?

Example 1.2.7 (Tree recognition)**.** Suppose we want a system that looks at a photograph and answers whether there is a tree in it.

- To *'define' trees and hand-program* the answer is difficult: try writing down, in code, a condition that accepts every pine, palm and bonsai and rejects every bush and lamppost.
- To *learn from data (observations) and recognize* is easy enough that a 3-year-old can do it, having never received a definition of "tree" in their life — only examples.

So an 'ML-based tree recognition system' can be easier to *build* than a hand-programmed one, even though it is harder to *explain*.

ML is an alternative route to build complicated systems.

The useful way to read that moral is as a list of situations where the direct route is blocked:

- when humans cannot program the system manually — navigating on Mars;
- when humans cannot 'define the solution' easily — speech/visual recognition;
- when rapid decisions are needed that humans cannot make — high-frequency trading;
- when the system must be user-oriented at a massive scale — consumer-targeted marketing.

Remark 1.2.8. Give a computer a fish, you feed it for a day; teach it how to fish, you feed it for a lifetime. :-)

§1.2.iv. Key Essence of Machine Learning

Machine learning is not always the right tool, and the definition above already tells us when it is not. Reading Definition 1.2.4 backwards, three things must hold before the phrase even makes sense.

1. There exists some **underlying pattern** to be learned — so that a 'performance measure' *can* be improved. Without a pattern there is nothing to be better at.
2. But there is no programmable (easy) *definition* of that pattern — so that 'ML' is *needed*. If you can just write the rule down, write it down.
3. Somehow there is **data** about the pattern — so that ML has some 'inputs' to learn from. A pattern nobody has recorded cannot be computed from.

Key essence: it helps decide whether to use ML at all.

Exercise 1.2.9 (Fun Time)**.** Which of the following is best suited for machine learning?

(1) predicting whether the next cry of a baby girl happens at an even-numbered minute or not;

(2) determining whether a given graph contains a cycle;

(3) deciding whether to approve a credit card to some customer;

(4) guessing whether the earth will be destroyed by the misuse of nuclear power in the next ten years.

Reference Answer. (3), and the three essences say exactly why each of the others fails.

(1) Fails essence 1: there is no pattern.

(2) Fails essence 2: the definition is programmable (run a cycle detection algorithm).

(3) Passes all three — pattern: customer behaviour; definition: not easily programmable; data: the history of bank operations.

(4) Fails essence 3: there is arguably no (or not enough) data yet.

■

§1.3. Applications of Machine Learning

slides p.12–16

Every application below is the same picture,



with the two ends filled in differently. So the exercise in reading an application is always: *what is the data, and what is the skill?*

§1.3.i. Daily Needs: Food, Clothing, Housing, Transportation

1. **Food** (Sadilek et al., 2013). *Data*: Twitter data (words + location). *Skill*: tell the food-poisoning likeliness of a restaurant properly.
2. **Clothing** (Abu-Mostafa, 2012). *Data*: sales figures + client surveys. *Skill*: give good fashion recommendations to clients.
3. **Housing** (Tsanas and Xifara, 2012). *Data*: characteristics of buildings and their energy load. *Skill*: predict the energy load of other buildings closely.
4. **Transportation** (Stallkamp et al., 2012). *Data*: some traffic sign images and their meanings. *Skill*: recognize traffic signs accurately.

ML is everywhere!

§1.3.ii. Education

Data: students' records on quizzes in a mathematics tutoring system. *Skill*: predict whether a student can give a correct answer to *another* quiz question.

What makes this example instructive is that the pattern one guesses is not a formula in the recorded variables at all; it is a formula in quantities nobody ever measured:

$$\text{answer correctly} \approx [\![\,\text{recent strength of student} > \text{difficulty of question}\,]\!].$$

Notation . $[\![\cdot]\!]$ is the *Iverson bracket*: it equals 1 when the statement inside is true and 0 when it is false.

Neither "strength" nor "difficulty" appears in the data — only who answered what, and whether it was right. Feeding roughly 9 million records from 3000 students to a learning algorithm lets it *reverse-engineer* both numbers automatically, purely from the requirement that the formula above should reproduce the observed answers.

This was the key part of the world-champion system from National Taiwan University in KDDCup 2010.

§1.3.iii. Entertainment: Recommender Systems

Data: how many users have rated some movies. *Skill*: predict how a user would rate an unrated movie.

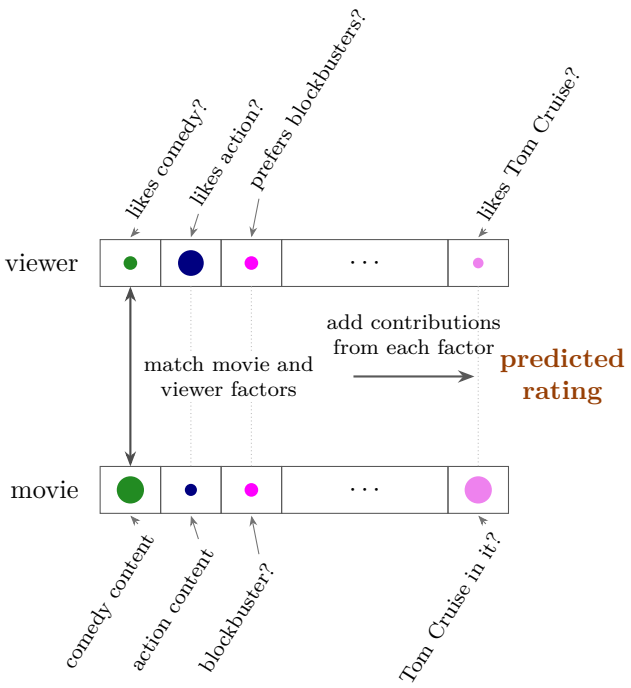
Example 1.3.1 (A hot problem).

- A competition held by Netflix in 2006: 100,480,507 ratings that 480,189 users gave to 17,770 movies, with a 10% improvement worth a one-million-dollar prize.
- A similar competition (movies → songs) held by Yahoo! in KDDCup 2011: 252,800,275 ratings that 1,000,990 users gave to 624,961 songs.

Question 1.3.2. How can machines learn our preferences?

The same trick as in the education example applies, and it is worth seeing that it is literally the same trick. Postulate that a viewer and a movie are each described by the *same* list of hidden **factors** — how much comedy, how much action, how much of a blockbuster — read on

the viewer's side as *how much this viewer wants it* and on the movie's side as *how much this movie has it*.



Both rows carry the same factors in the same order, so a viewer and a movie live in one common space; the size of a dot is the size of a number. The vertical arrow pairs slot k with slot k, the horizontal one sums the pairs, and together they are a single formula: writing v_k for how much the viewer wants factor k and m_k for how much the movie has of it,

$$\text{predicted rating} \approx \sum_{k=1}^{K} v_k m_k,$$

the inner product of the two factor vectors. The product is the point — a factor contributes only where the two sides *agree*, so our action-loving viewer would rate this comedy rather low.

The dashes say the list is long, and the last slot says how arbitrary it may be. Nobody drew that list up: as with the students' strength and difficulty, the factors are never observed, and the learning is what invents them.

pattern: rating ⟵ viewer/movie factors;
learning: known ratings ⟶ learned factors ⟶ unknown rating prediction.

Again the key part of the world-champion system from National Taiwan University, this time in KDDCup 2011.

Exercise 1.3.3 (Fun Time)**.** Which of the following fields *cannot* use machine learning?

(1) Finance;
(2) Medicine;
(3) Law;

(4) none of the above.

Reference Answer. (4). Finance can predict stock prices from data, medicine can predict medicine effects from data, and law can summarize legal documents from data. Welcome to study this hot topic. ■

§1.4. Components of Machine Learning

slides p.17–22

So far "machine learning" has been a box in a picture. This section opens the box and names its parts. The running metaphor is *credit approval*: a bank receives an application and must decide whether issuing a card to this person is good for the bank.

Applicant Information	
age	23 years
gender	female
annual salary	NTD 1,000,000
year in residence	1 year
year in job	0.5 year
current debt	200,000

The unknown pattern to be learned is: *is approving this credit card good for the bank?*

§1.4.i. Formalizing the Learning Problem

Notation .

- **Input**: $x \in \mathcal{X}$ (a customer application).
- **Output**: $y \in \mathcal{Y}$ (good/bad after approving the credit card).
- The unknown pattern to be learned $\Leftrightarrow$ the **target function**

$$f : \mathcal{X} \to \mathcal{Y}$$

(the ideal credit approval formula).

- The data $\Leftrightarrow$ the **training examples**

$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$$

(historical records in the bank).

- The skill, hopefully with good performance, $\Leftrightarrow$ the **hypothesis**

$$g : \mathcal{X} \to \mathcal{Y}$$

(the 'learned' formula to be used).

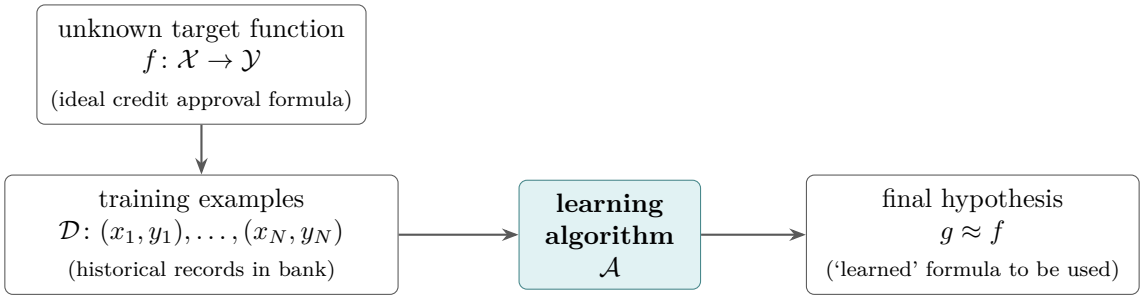


Remark 1.4.1. The asymmetry between f and g is the whole difficulty of the subject, so it is worth stating plainly.

- f is *unknown* — indeed, by essence 2 of the previous section, it has no programmable definition. We never see f; we only ever see N of its values.
- g is what we produce, and we hope $g \approx f$. But g may differ from f, and perfection is 'impossible' precisely because f is unknown — there is no way to check the answer.

Everything in *Why can machines learn?* is an attempt to say something rigorous about $g \approx f$ despite never seeing f.

Drawing the arrows in the order in which things actually happen gives the learning flow.



§1.4.ii. The Learning Model

The diagram still hides one thing.

> **Question 1.4.2.** What does g look like?

An algorithm cannot search over *all* functions $\mathcal{X} \to \mathcal{Y}$, so we must first decide which formulas are even candidates.

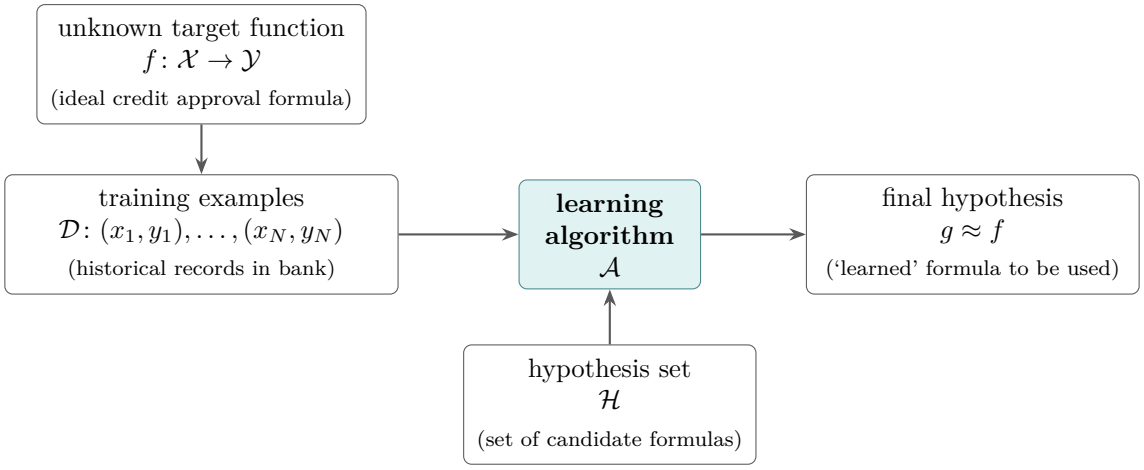
Definition 1.4.3 (Hypothesis set)**.** The **hypothesis set** $\mathcal{H} = \{h_k\}$ is the set of candidate formulas that the algorithm is allowed to return, so that $g \in \mathcal{H}$.

Example 1.4.4. For credit approval, one might approve if

- h_1: annual salary $>$ NTD 800,000;
- h_2: debt $>$ NTD 100,000 (really?);
- h_3: year in job ≤ 2 (really?).

Note that $\mathcal{H}$ happily contains bad hypotheses next to good ones. That is not a defect: it is up to $\mathcal{A}$ to pick the 'best' one out of $\mathcal{H}$ and call it g.

Definition 1.4.5 (Learning model)**.** A **learning model** is the pair consisting of the hypothesis set $\mathcal{H}$ and the learning algorithm $\mathcal{A}$.



This is the picture the rest of the course elaborates. The dashed halves of the story are already visible in it: f and $\mathcal{D}$ are given by the world and we cannot choose them, while $\mathcal{H}$ and $\mathcal{A}$ are ours to design.

Machine learning: use data to compute a hypothesis g that approximates the target f.

Exercise 1.4.6 (Fun Time)**.** How should the four sets below be used to form a learning problem for song recommendation?

$S_1 = [0, 100]$;
$S_2 =$ all possible (userid, songid) pairs;
$S_3 =$ all formulas that 'multiply' user factors and song factors, indexed by all possible combinations of such factors;
$S_4 =$ 1,000,000 pairs of ((userid, songid), rating).

(1) $S_1 = \mathcal{X},\ S_2 = \mathcal{Y},\ S_3 = \mathcal{H},\ S_4 = \mathcal{D}$;
(2) $S_1 = \mathcal{Y},\ S_2 = \mathcal{X},\ S_3 = \mathcal{H},\ S_4 = \mathcal{D}$;
(3) $S_1 = \mathcal{D},\ S_2 = \mathcal{H},\ S_3 = \mathcal{Y},\ S_4 = \mathcal{X}$;
(4) $S_1 = \mathcal{X},\ S_2 = \mathcal{D},\ S_3 = \mathcal{Y},\ S_4 = \mathcal{H}$.

Reference Answer. (2). Reading the whole problem in one line,

$$S_4 \xrightarrow{\mathcal{A} \text{ on } S_3} (g: S_2 \to S_1)\,,$$

i.e. the input space is the set of (user, song) pairs, the output space is the rating scale $[0, 100]$, the hypothesis set is the family of factor-multiplying formulas, and the data is the list of observed ratings. ■

§1.5. Machine Learning and Other Fields

slides p.23–27

Machine learning did not grow up alone, and much of the confusion around the name comes from its neighbours. Comparing the one-line definitions is enough to see both the overlap and the difference in *focus* — which is usually where the real difference lies.

§1.5.i. Machine Learning and Data Mining

Machine Learning	Data Mining
use data to compute a hypothesis g that approximates a target f	use (huge) data to find a property that is *interesting*

- If the 'interesting property' *is* the 'hypothesis that approximates the target', then ML = DM. This is usually what KDDCup does.
- If the 'interesting property' is merely *related* to such a hypothesis, then DM can help ML and vice versa — often, but not always.
- Traditional DM also puts weight on efficient computation in large databases, which ML traditionally does not.

It is difficult to distinguish ML and DM in reality.

§1.5.ii. Machine Learning and Artificial Intelligence

Machine Learning	Artificial Intelligence
use data to compute a hypothesis g that approximates a target f	compute something that shows *intelligent behaviour*

- $g \approx f$ is something that shows intelligent behaviour, so ML can realize AI — among other routes.
- For example, in chess playing: traditional AI builds a game tree, whereas ML for AI learns from board data.

ML is one possible route to realize AI.

§1.5.iii. Machine Learning and Statistics

Machine Learning	Statistics
use data to compute a hypothesis g that approximates a target f	use data to make *inference* about an unknown process

- g is an inference outcome and f is something unknown, so statistics can be used to achieve ML.
- Traditional statistics also focuses on provable results under mathematical assumptions, and cares less about computation.

Statistics: many useful tools for ML.

Exercise 1.5.1 (Fun Time)**.** Which of the following claims is *not* totally true?

(1) machine learning is a route to realize artificial intelligence;

(2) machine learning, data mining and statistics all need data;

(3) data mining is just another name for machine learning;

(4) statistics can be used for data mining.

Reference Answer. (3). While data mining and machine learning do share a huge overlap, they are arguably not equivalent, because of the difference of focus. ■

Summary

Course Introduction foundation oriented and story-like.

What is Machine Learning use data to approximate a target.

Applications of Machine Learning almost everywhere.

Components of Machine Learning $\mathcal{A}$ takes $\mathcal{D}$ and $\mathcal{H}$ to get g.

Machine Learning and Other Fields related to DM, AI and statistics.

We have a frame, $(\mathcal{X}, \mathcal{Y}, f, \mathcal{D}, \mathcal{H}, \mathcal{A}, g)$, but not a single concrete learning model to put in it: we never said what any $h \in \mathcal{H}$ actually computes, nor how $\mathcal{A}$ searches. The next chapter fills in the simplest useful choice of $\mathcal{H}$ and $\mathcal{A}$, for the simplest useful output space $\mathcal{Y} = \{-1, +1\}$ — that is, learning to answer yes/no.

2. Learning to Answer Yes/No

Lecture 2

16 Sep.

> **As previously seen .** Section 1.4 ended with a complete frame for the learning problem: an algorithm $\mathcal{A}$ takes the data $\mathcal{D}$ and a hypothesis set $\mathcal{H}$, and returns a final hypothesis g which we hope satisfies $g \approx f$. Nothing in that frame said what a single $h \in \mathcal{H}$ computes, nor how $\mathcal{A}$ searches.

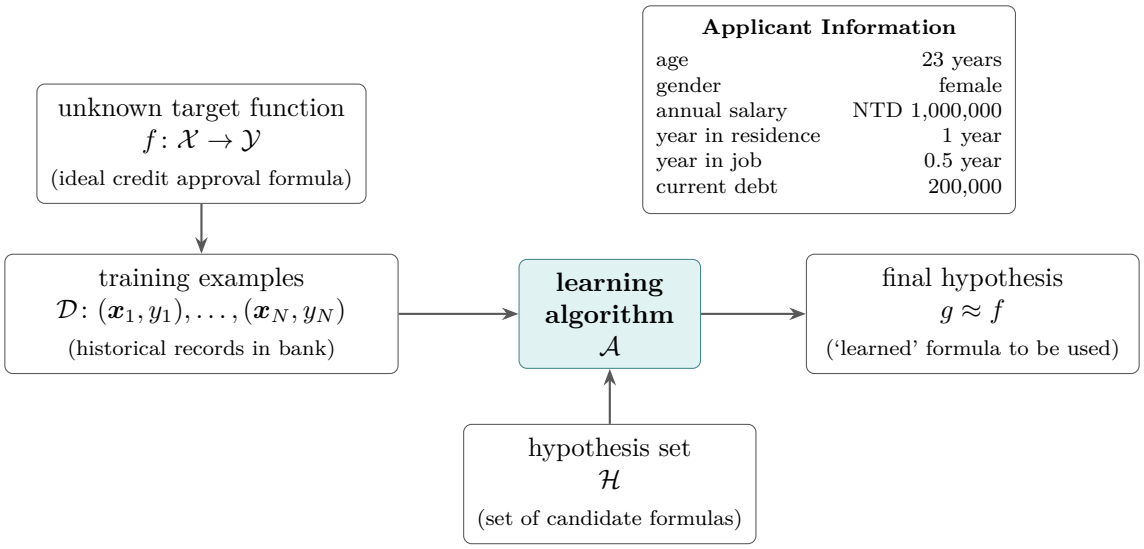
This chapter fills that gap in the simplest interesting case, where the output is a yes/no answer, and gets both an $\mathcal{H}$ and an $\mathcal{A}$ out of it — our first concrete learning model.

§2.1. Perceptron Hypothesis Set

slides p.2–6

§2.1.i. The Credit Approval Problem Revisited

Put the credit approval problem of Section 1.4 back into the flow, this time with the applicant record written out.



> **Question 2.1.1.** What hypothesis set can we use?

§2.1.ii. Features of a Customer

The bank holds a customer record of d numbers,

$$\boldsymbol{x} = (x_1, x_2, \ldots, x_d),$$

the **features** of the customer — age, annual salary, year in job, current debt, and so on — and it must answer yes or no.

Notation . From this chapter on, an input $\boldsymbol{x}$ is a vector of d real features, written in bold. In Section 1.4 the input was an abstract element of $\mathcal{X}$; giving it coordinates is exactly the step that lets us write down formulas for h.

§2.1.iii. The Perceptron

age	23 years
annual salary	NTD 1,000,000
year in job	0.5 year
current debt	200,000

The simplest thing one can do with d numbers is to weigh them against each other. Give each feature x_i an importance w_i, form a weighted *score* $\sum_{i=1}^{d} w_i x_i$, and compare it against a threshold:

$$\text{approve credit if} \quad \sum_{i=1}^{d} w_i x_i > \text{threshold};$$

$$\text{deny credit if} \quad \sum_{i=1}^{d} w_i x_i < \text{threshold}.$$

A positive w_i means the feature speaks in the customer's favour (annual salary), a negative one means it speaks against (current debt), and a large $|w_i|$ means the feature matters a lot.

Since the answer is yes or no, take

$$\mathcal{Y} = \{+1 \text{ (good)},\ -1 \text{ (bad)}\},$$

and ignore the boundary case of a score exactly equal to the threshold, i.e. treat 0 as ignored. Then the rule above is a single formula.

Definition 2.1.2 (Perceptron)**.** For a weight vector $w_1, \ldots, w_d$ and a threshold, the hypothesis

$$h(\boldsymbol{x}) = \operatorname{sign}\left(\left(\sum_{i=1}^{d} w_i x_i\right) - \text{threshold}\right)$$

is called a **perceptron**, for historical reasons. The hypothesis set $\mathcal{H}$ of this chapter is the set of all such h, i.e. one h for every choice of weights and threshold.

§2.1.iv. Vector Form

The threshold is an eyesore: it is a parameter of the same kind as the weights, but it sits outside the sum. The standard repair is to absorb it as the weight of a constant feature:

$$\begin{aligned} h(\boldsymbol{x}) &= \operatorname{sign}\left(\left(\sum_{i=1}^{d} w_i x_i\right) - \text{threshold}\right) \\ &= \operatorname{sign}\left(\left(\sum_{i=1}^{d} w_i x_i\right) + \underbrace{(-\text{threshold})}_{w_0} \cdot \underbrace{(+1)}_{x_0}\right) \\ &= \operatorname{sign}\left(\sum_{i=0}^{d} w_i x_i\right) \\ &= \operatorname{sign}\left(\boldsymbol{w}^{\top} \boldsymbol{x}\right). \end{aligned}$$

Notation . From now on $\boldsymbol{w} = (w_0, w_1, \ldots, w_d)^\top$ and $\boldsymbol{x} = (x_0, x_1, \ldots, x_d)^\top$ are the 'tall' versions, with $x_0 \equiv 1$ always. Each tall $\boldsymbol{w}$ *is* a hypothesis h, so from here on we say 'the hypothesis $\boldsymbol{w}$' and mean $h(\boldsymbol{x}) = \text{sign}(\boldsymbol{w}^\top \boldsymbol{x})$.

The gain is not only cosmetic: everything below manipulates $\boldsymbol{w}$ as a vector, and a stray additive constant would have to be carried through every one of those manipulations.

§2.1.v. Perceptrons in $\mathbb{R}^2$

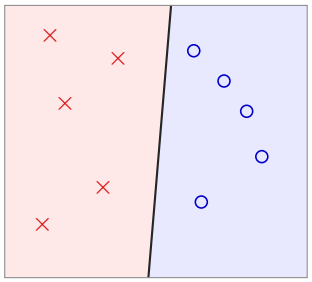
Question 2.1.3. What do perceptrons h 'look like'?

Take $d = 2$, so that

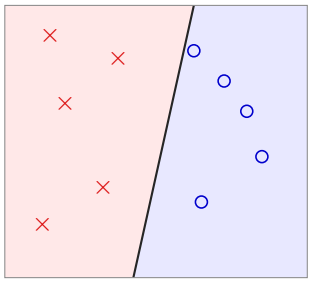
$$h(\boldsymbol{x}) = \text{sign}\left(w_0 + w_1 x_1 + w_2 x_2\right).$$

The set where the score vanishes, $w_0 + w_1 x_1 + w_2 x_2 = 0$, is a line; h says $+1$ on one side of it and -1 on the other. So:

- customer features $\boldsymbol{x}$ are points on the plane (or points in $\mathbb{R}^d$);
- labels y are drawn as $\circ$ for $+1$ and $\times$ for -1;
- a hypothesis h is a line (or a hyperplane in $\mathbb{R}^d$), positive on one side and negative on the other;
- a different line classifies the customers differently.



one hypothesis $\boldsymbol{w}$



another hypothesis $\boldsymbol{w}'$

perceptrons $\Leftrightarrow$ linear (binary) classifiers.

Exercise 2.1.4 (Fun Time)**.** Consider using a perceptron to detect spam messages. Assume that each email is represented by the frequency of keyword occurrence, and that an output of $+1$ indicates spam. Which keywords below should have large *positive* weights in a good perceptron for the task?

(1) coffee, tea, hamburger, steak;

(2) free, drug, fantastic, deal;

(3) machine, learning, statistics, textbook;

(4) national, Taiwan, university, coursera.

Reference Answer. (2). The occurrence of a keyword with a positive weight increases the 'spam score', and hence those keywords should be the ones that often appear in spam. ■

§2.2. Perceptron Learning Algorithm (PLA)

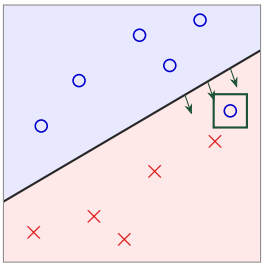
slides p.7–12

Half of the learning model is still missing.

Question 2.2.1. $\mathcal{H}$ = all possible perceptrons, g =?

§2.2.i. Selecting g from $\mathcal{H}$

- What we *want* is $g \approx f$. This is hard to even check, because f is unknown.
- What is *almost necessary* is $g \approx f$ on $\mathcal{D}$, ideally $g(\boldsymbol{x}_n) = f(\boldsymbol{x}_n) = y_n$ for every n. This we can check, because $\mathcal{D}$ is in front of us.
- What makes it *difficult* is that $\mathcal{H}$ is of infinite size, so we cannot look at the candidates one by one.
- The *idea* that resolves this: start from some g_0, and 'correct' its mistakes on $\mathcal{D}$.



Infinitely many candidates are no obstacle if we never enumerate them, but instead walk from one to the next. By the previous subsection we may represent g_0 by its weight vector $\boldsymbol{w}_0$, so the walk happens in $\mathbb{R}^{d+1}$ rather than in the abstract set $\mathcal{H}$.

§2.2.ii. The Algorithm

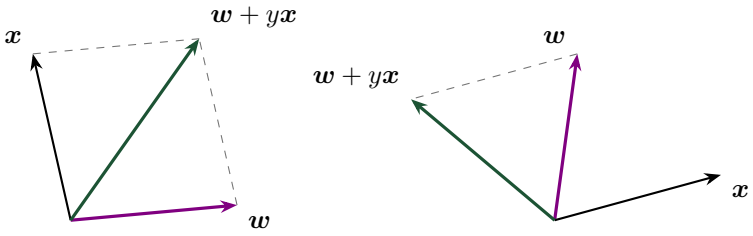
Algorithm 2.1: Perceptron Learning Algorithm (PLA)

Input: training examples $\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}_{n=1}^{N}$
Output: $\boldsymbol{w}_{\text{PLA}}$, returned as g

$\boldsymbol{w}_0 \leftarrow$ some initial vector, say $\mathbf{0}$;
for $t = 0, 1, 2, \ldots$ **do**
  if $\boldsymbol{w}_t$ *makes no mistake on* $\mathcal{D}$ **then return** $\boldsymbol{w}_t$ as $\boldsymbol{w}_{\text{PLA}}$;
  find a mistake of $\boldsymbol{w}_t$, called $(\boldsymbol{x}_{n(t)}, y_{n(t)})$, i.e. one with sign $\left(\boldsymbol{w}_t^\top \boldsymbol{x}_{n(t)}\right) \neq y_{n(t)}$;
  (try to) correct the mistake by $\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + y_{n(t)}\boldsymbol{x}_{n(t)}$;

That is it — a fault confessed is half redressed. :-)

The update rule looks arbitrary until one draws it. A mistake at $(\boldsymbol{x}, y)$ with $y = +1$ means $\boldsymbol{w}^\top \boldsymbol{x} < 0$, i.e. $\boldsymbol{w}$ and $\boldsymbol{x}$ point more than 90° apart; adding $\boldsymbol{x}$ swings $\boldsymbol{w}$ *towards* $\boldsymbol{x}$. A mistake with $y = -1$ means $\boldsymbol{w}^\top \boldsymbol{x} > 0$; subtracting $\boldsymbol{x}$ swings $\boldsymbol{w}$ *away* from $\boldsymbol{x}$. Either way the correction is $\boldsymbol{w} + y\boldsymbol{x}$.



$y = +1$: swing $\boldsymbol{w}$ towards $\boldsymbol{x}$ $y = -1$: swing $\boldsymbol{w}$ away from $\boldsymbol{x}$

§2.2.iii. Practical Implementation of PLA

algorithm 2.1 says "find a mistake" without saying *which* one, and a literal implementation would rescan $\mathcal{D}$ from the beginning after every correction. Cyclic PLA removes both vaguenesses at once: fix an order on the examples, keep a cursor into it, and let "the next mistake" mean *the next mistake the cursor runs into*, continuing from where the last correction happened and wrapping around at the end of the list.

Written out, with $\pi(1), \ldots, \pi(N)$ the chosen order:

Algorithm 2.2: Cyclic PLA

Input: training examples $\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}_{n=1}^{N}$ and a fixed order $\pi(1), \ldots, \pi(N)$ of $1, \ldots, N$

Output: $\boldsymbol{w}_{\text{PLA}}$, returned as g

$\boldsymbol{w} \leftarrow \mathbf{0}$;
$i \leftarrow 1$; `// cursor: where the next scan resumes`
$c \leftarrow 0$; `// examples seen in a row with no mistake`
while $c < N$ **do**
  $n \leftarrow \pi(i)$;
  if $\operatorname{sign}\left(\boldsymbol{w}^{\top}\boldsymbol{x}_n\right) \neq y_n$ **then**
    $\boldsymbol{w} \leftarrow \boldsymbol{w} + y_n\boldsymbol{x}_n$; `// a mistake: correct it`
    $c \leftarrow 0$; `// the clean run is broken`
  else
    $c \leftarrow c + 1$; `// this one is already right`
  $i \leftarrow (i \bmod N) + 1$; `// advance, wrapping around`
return $\boldsymbol{w}$ as $\boldsymbol{w}_{\text{PLA}}$;

Two points deserve spelling out.

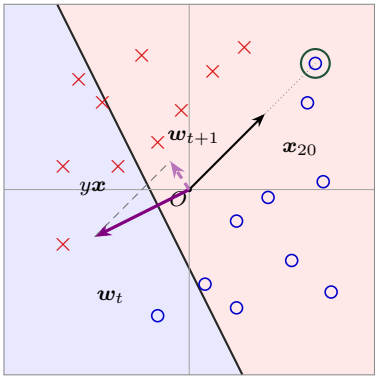
- **When does the loop end?** When c reaches N, i.e. when N examples in a row have been checked and none of them was a mistake. Since the cursor moves cyclically, any N consecutive visits run through every example exactly once — so $c = N$ says precisely that a full cycle passed without encountering a mistake, which is to say that the current $\boldsymbol{w}$ is correct on all of $\mathcal{D}$. There is no need for the scan to be aligned with the start of the list.
- **Why reset c to 0 after every correction?** Because a correction changes $\boldsymbol{w}$, and the new $\boldsymbol{w}$ may well be wrong on examples the cursor has already walked past — exactly what happens to $\boldsymbol{x}_{15}$ in the run below. Everything checked before the correction was checked against a *different* hypothesis and no longer counts, so the tally has to start over.

The order π may be the naïve cycle $1, \ldots, N$, or a precomputed random cycle. Note also that the loop counter here is not the t of algorithm 2.1: this loop turns once per example *examined*, whereas t — and the T bounded in Theorem 2.3.6 — counts only the examples actually *corrected*.

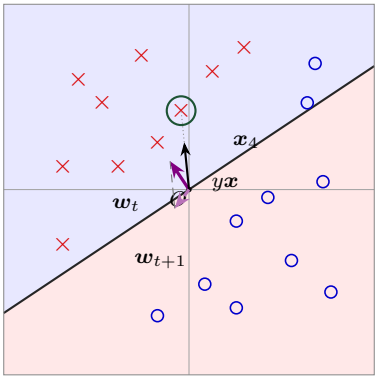
§2.2.iv. Seeing is Believing

Run it on a small separable data set of $N = 20$ examples. Each frame shows the hypothesis $\boldsymbol{w}_t$ that PLA currently holds — its line, and the two shaded sides it calls $+1$ (blue) and -1 (red) — together with the example $\boldsymbol{x}_{n(t)}$ it still gets wrong, circled. A mistake is visible as a $\circ$ sitting in the red half or a $\times$ sitting in the blue half.

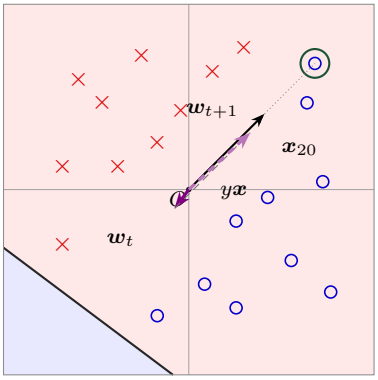
Every frame is drawn with the origin O at its centre, so that all three vectors can be read off from there: the circled $\boldsymbol{x}_{n(t)}$ (black), the current $\boldsymbol{w}_t$ (solid, the one whose line is shaded) and the $\boldsymbol{w}_{t+1}$ the correction is about to produce (dashed) — which is the line drawn in the next frame. All three are at 0.6 of their true length so that they stay beside the data; the dotted continuation of $\boldsymbol{x}_{n(t)}$ runs on to the example itself.



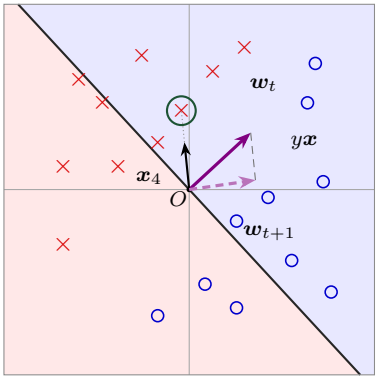
update 1: $\boldsymbol{x}_{20}$ is a mistake



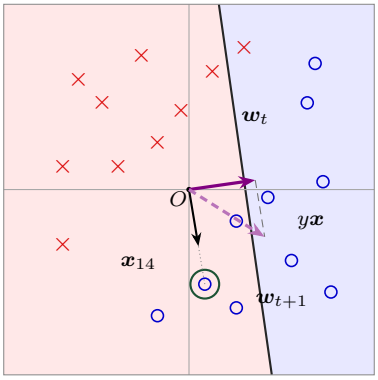
update 2: $\boldsymbol{x}_4$ is a mistake



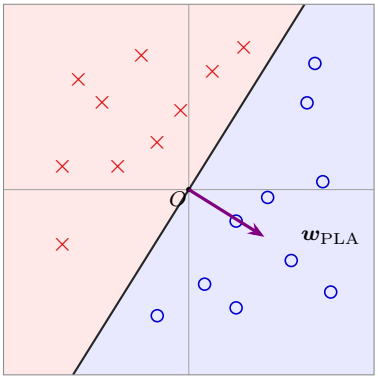
update 3: $\boldsymbol{x}_{20}$ is a mistake



update 4: $\boldsymbol{x}_4$ is a mistake



update 5: $\boldsymbol{x}_{14}$ is a mistake



after 5 updates: none left

Because the three share one scale, the dashed segment joining the two weight tips is the step $y_{n(t)}\boldsymbol{x}_{n(t)}$ — parallel to $\boldsymbol{x}_{n(t)}$ and of the same drawn length — so the update rule is literally the triangle that closes:

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + y_{n(t)}\boldsymbol{x}_{n(t)}.$$

The run is started from an initial $\boldsymbol{w}_0$ that is merely a guess — taking $\boldsymbol{w}_0 = \mathbf{0}$, as algorithm 2.1 suggests, would leave the first frame with no line to look at — so already the first frame shows a genuine classifier, and a genuinely bad one. Each $\boldsymbol{w}_t$ is perpendicular to its own line and points into the $+1$ side, so watch it swing towards a circled $\circ$ and away from a circled $\times$. A correction made for one example happily breaks another: here $\boldsymbol{x}_{20}$ has to be corrected twice, in updates 1 and 3, and so does $\boldsymbol{x}_4$, in updates 2 and 4. After five corrections no mistake is left and the algorithm stops.

It works like a charm in fewer than 20 lines of code. But watching it work once is not a guarantee, and two kinds of issue are left open.

Question 2.2.2 (Some remaining issues of PLA)**. Algorithmic**: does it *halt*, with no mistake?

- naïve cyclic: ??
- random cyclic: ??
- other variant: ??

Learning: is $g \approx f$?

- on $\mathcal{D}$, if it halts: yes, since then there is no mistake;
- outside $\mathcal{D}$: ??
- if not halting: ??

To be shown: if (...), then after 'enough' corrections any PLA variant halts.

The rest of the chapter settles the first question and postpones the "outside $\mathcal{D}$" half of the second one to *Why can machines learn?*

Exercise 2.2.3 (Fun Time)**.** Let us try to think about why PLA may work. Let $n = n(t)$. According to the rule

$$\operatorname{sign}\left(\boldsymbol{w}_t^\top \boldsymbol{x}_n\right) \neq y_n, \qquad \boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + y_n \boldsymbol{x}_n,$$

which formula is true?

(1) $\boldsymbol{w}_{t+1}^\top \boldsymbol{x}_n = y_n$;

(2) $\operatorname{sign}\left(\boldsymbol{w}_{t+1}^\top \boldsymbol{x}_n\right) = y_n$;

(3) $y_n \boldsymbol{w}_{t+1}^\top \boldsymbol{x}_n \geq y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n$;

(4) $y_n \boldsymbol{w}_{t+1}^\top \boldsymbol{x}_n < y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n$.

Reference Answer. (3). Simply multiply the update rule by $y_n \boldsymbol{x}_n$:

$$y_n \boldsymbol{w}_{t+1}^\top \boldsymbol{x}_n = y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n + y_n^2 \|\boldsymbol{x}_n\|^2 \geq y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n,$$

since $y_n^2 = 1$. Note that (1) and (2) are *too strong*: one correction need not fix the very mistake it was made for. All we get is that the quantity $y_n \boldsymbol{w}^\top \boldsymbol{x}_n$, which is positive exactly when $\boldsymbol{x}_n$ is classified correctly, moves in the right direction. The rule somewhat 'tries to correct the mistake'. ■

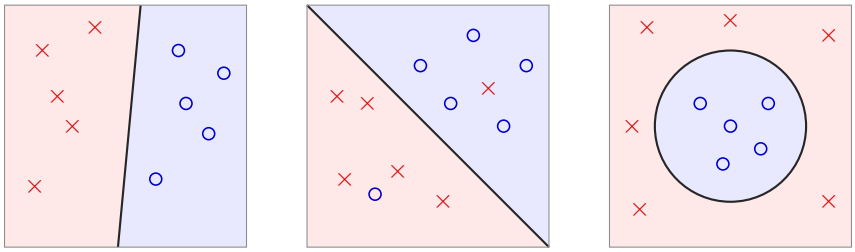
§2.3. Guarantee of PLA

slides p.13–16

§2.3.i. Linear Separability

If PLA halts, then by construction there are no mistakes left, so $\mathcal{D}$ must have admitted a perfect $\boldsymbol{w}$ in the first place. That is a *necessary* condition for halting, and it deserves a name.

Definition 2.3.1 (Linear separability)**.** A data set $\mathcal{D}$ is **linearly separable** if there exists a $\boldsymbol{w}$ that makes no mistake on it, i.e. $y_n = \operatorname{sign}(\boldsymbol{w}^\top \boldsymbol{x}_n)$ for every n.



linearly separable | not linearly separable (noise) | not linearly separable (circular)

The converse is the real question.

Question 2.3.2. Assume a linearly separable $\mathcal{D}$. Does PLA always halt?

It does, and the proof is two facts about what a single correction does.

§2.3.ii. Two Facts

Fact 1: w_t gets more aligned with w_f

Restating Definition 2.3.1 gives the premise the whole argument runs on:

$$\mathcal{D} \text{ linearly separable} \Leftrightarrow \text{exists perfect } \boldsymbol{w}_f \text{ such that } y_n = \operatorname{sign}\left(\boldsymbol{w}_f^\top \boldsymbol{x}_n\right).$$

Fix such a $\boldsymbol{w}_f$. Being **perfect** buys more than "no mistakes": every example is not merely on the correct side of $\boldsymbol{w}_f$'s line but strictly *away* from it, so the worst example over the whole (finite) data set still has room to spare:

$$y_{n(t)} \boldsymbol{w}_f^\top \boldsymbol{x}_{n(t)} \geq \min_n y_n \boldsymbol{w}_f^\top \boldsymbol{x}_n > 0.$$

That strictly positive number is the entire engine of the proof, because it is paid out afresh at every single correction.

Lemma 2.3.3 ($\boldsymbol{w}_f^\top \boldsymbol{w}_t$ goes up)
Updating with any mistake $(\boldsymbol{x}_{n(t)}, y_{n(t)})$ gives

$$\boldsymbol{w}_f^\top \boldsymbol{w}_{t+1} \geq \boldsymbol{w}_f^\top \boldsymbol{w}_t + \min_n y_n \boldsymbol{w}_f^\top \boldsymbol{x}_n > \boldsymbol{w}_f^\top \boldsymbol{w}_t.$$

Proof. Expand the update and apply the displayed bound above:

$$\begin{aligned} \boldsymbol{w}_f^\top \boldsymbol{w}_{t+1} &= \boldsymbol{w}_f^\top \left(\boldsymbol{w}_t + y_{n(t)} \boldsymbol{x}_{n(t)}\right) \\ &\geq \boldsymbol{w}_f^\top \boldsymbol{w}_t + \min_n y_n \boldsymbol{w}_f^\top \boldsymbol{x}_n \\ &> \boldsymbol{w}_f^\top \boldsymbol{w}_t + 0. \end{aligned}$$

■

So $\boldsymbol{w}_t$ *appears* more aligned with $\boldsymbol{w}_f$ after an update — really? Not yet. An inner product can grow for two quite different reasons: because the angle between the two vectors shrank, which is what we want, or merely because $\boldsymbol{w}_t$ got longer, which would mean nothing at all. Lemma 2.3.3 on its own cannot tell these apart. The second fact closes exactly that hole.

Fact 2: $\boldsymbol{w}_t$ does not grow too fast

Here the premise is the other half of how PLA is built — it touches $\boldsymbol{w}_t$ *only* when there is a mistake to fix, and a mistake is precisely a negative agreement:

$$\boldsymbol{w}_t \text{ changed only when mistake} \Leftrightarrow \operatorname{sign}\left(\boldsymbol{w}_t^\top \boldsymbol{x}_{n(t)}\right) \neq y_{n(t)} \Leftrightarrow y_{n(t)} \boldsymbol{w}_t^\top \boldsymbol{x}_{n(t)} \leq 0.$$

Lemma 2.3.4 ($\|\boldsymbol{w}_t\|^2$ is limited)
A mistake 'limits' the growth of $\|\boldsymbol{w}_t\|^2$, even when the update uses the 'longest' $\boldsymbol{x}_n$:

$$\|\boldsymbol{w}_{t+1}\|^2 \leq \|\boldsymbol{w}_t\|^2 + \max_n \|y_n \boldsymbol{x}_n\|^2.$$

Proof. The cross term is the one to watch, and the characterisation of a mistake above says it is never positive:

$$\begin{aligned}
\|\boldsymbol{w}_{t+1}\|^2 &= \left\|\boldsymbol{w}_t + y_{n(t)}\boldsymbol{x}_{n(t)}\right\|^2 \\
&= \|\boldsymbol{w}_t\|^2 + 2y_{n(t)}\boldsymbol{w}_t^\top \boldsymbol{x}_{n(t)} + \left\|y_{n(t)}\boldsymbol{x}_{n(t)}\right\|^2 \\
&\leq \|\boldsymbol{w}_t\|^2 + 0 + \left\|y_{n(t)}\boldsymbol{x}_{n(t)}\right\|^2 \\
&\leq \|\boldsymbol{w}_t\|^2 + \max_n \|y_n \boldsymbol{x}_n\|^2.
\end{aligned}$$

■

Intuition . Updating only on a mistake is what makes the algorithm work. A mistake means $\boldsymbol{w}_t$ and $y_{n(t)}\boldsymbol{x}_{n(t)}$ disagree, so the correction is never a step in the direction $\boldsymbol{w}_t$ already points: it turns $\boldsymbol{w}_t$ rather than stretching it.

Now put the two together. Each fact describes a *single* correction, so apply each of them once per correction and let the bounds telescope. Start PLA from $\boldsymbol{w}_0 = \mathbf{0}$ and suppose it has made T corrections.

Chaining Lemma 2.3.3 through $t = 0, 1, \ldots, T-1$ adds the same positive amount T times over:

$$\boldsymbol{w}_f^\top \boldsymbol{w}_T \geq \underbrace{\boldsymbol{w}_f^\top \boldsymbol{w}_0}_{=0} + T \min_n y_n \boldsymbol{w}_f^\top \boldsymbol{x}_n = T \min_n y_n \boldsymbol{w}_f^\top \boldsymbol{x}_n,$$

so the inner product grows at least *linearly* in T. Chaining Lemma 2.3.4 the same way,

$$\|\boldsymbol{w}_T\|^2 \leq \underbrace{\|\boldsymbol{w}_0\|^2}_{=0} + T \max_n \|y_n \boldsymbol{x}_n\|^2 = T \max_n \|\boldsymbol{x}_n\|^2,$$

where $\|y_n \boldsymbol{x}_n\| = \|\boldsymbol{x}_n\|$ because $|y_n| = 1$. So the *squared* length grows at most linearly, which means $\|\boldsymbol{w}_T\|$ itself grows at most like $\sqrt{T}$.

Dividing the first by the second is what finally answers the '(really?)'. The quantity

$$\frac{\boldsymbol{w}_f^\top}{\|\boldsymbol{w}_f\|} \frac{\boldsymbol{w}_T}{\|\boldsymbol{w}_T\|}$$

is an inner product of two *unit* vectors, i.e. the cosine of the angle between $\boldsymbol{w}_f$ and $\boldsymbol{w}_T$; dividing by $\|\boldsymbol{w}_T\|$ has removed precisely the loophole that made Lemma 2.3.3 inconclusive, so if this grows, the angle really does shrink. Feeding in the two displays — the numerator is bounded below, the denominator above — gives

$$\frac{\boldsymbol{w}_f^\top \boldsymbol{w}_T}{\|\boldsymbol{w}_f\|\|\boldsymbol{w}_T\|} \geq \frac{T \min_n y_n \boldsymbol{w}_f^\top \boldsymbol{x}_n}{\|\boldsymbol{w}_f\| \sqrt{T \max_n \|\boldsymbol{x}_n\|^2}} = \sqrt{T} \cdot \text{constant},$$

the constant being built from $\boldsymbol{w}_f$ and the data alone: every appearance of T has collected into the $T/\sqrt{T} = \sqrt{T}$ out front.

That is already the whole theorem. The left-hand side is a cosine and so never exceeds 1, while the right-hand side grows without bound as T does; the two can hold together only for finitely many T. PLA therefore corrects only finitely many mistakes — it halts. All that is left is to name the constant, which is what the next exercise does.

Exercise 2.3.5 (Fun Time)**.** Let us upper-bound T, the number of mistakes that PLA 'corrects'. Define

$$R^2 = \max_n \|\boldsymbol{x}_n\|^2, \qquad \rho = \min_n y_n \frac{\boldsymbol{w}_f^\top}{\|\boldsymbol{w}_f\|}\boldsymbol{x}_n.$$

We want to show $T \le \square$. Express the upper bound $\square$ by the two terms above.

(1) R/ρ;

(2) R^2/ρ^2;

(3) R/ρ^2;

(4) ρ^2/R^2.

Reference Answer. (2), and the calculation is exactly Theorem 2.3.6 below: naming the two quantities turns the anonymous constant of the previous page into ρ/R, and a cosine's ceiling of 1 then caps T at $1/(\rho/R)^2 = R^2/\rho^2$. ■

§2.3.iii. The Guarantee

Theorem 2.3.6 (PLA halts)

Let $\mathcal{D}$ be linearly separable, let $\boldsymbol{w}_f$ be a perfect weight vector for it, and put

$$R^2 = \max_n \|\boldsymbol{x}_n\|^2, \qquad \rho = \min_n y_n \frac{\boldsymbol{w}_f^\top}{\|\boldsymbol{w}_f\|}\boldsymbol{x}_n > 0.$$

Then PLA started from $\boldsymbol{w}_0 = \boldsymbol{0}$ makes at most

$$T \le \frac{R^2}{\rho^2}$$

corrections, and in particular it halts.

Proof. Everything has been derived already; naming R and ρ is all that is left. Note first that $\rho > 0$: it is $\min_n y_n \boldsymbol{w}_f^\top \boldsymbol{x}_n / \|\boldsymbol{w}_f\|$, and that minimum was shown strictly positive in Fact 1 because $\boldsymbol{w}_f$ is perfect. With this notation the two chained bounds read

$$\boldsymbol{w}_f^\top \boldsymbol{w}_T \ge T \min_n y_n \boldsymbol{w}_f^\top \boldsymbol{x}_n = T\rho\|\boldsymbol{w}_f\|, \qquad \|\boldsymbol{w}_T\|^2 \le T \max_n \|\boldsymbol{x}_n\|^2 = TR^2,$$

so that

$$\frac{\boldsymbol{w}_f^\top \boldsymbol{w}_T}{\|\boldsymbol{w}_f\|\|\boldsymbol{w}_T\|} \ge \frac{T\rho\|\boldsymbol{w}_f\|}{\|\boldsymbol{w}_f\|\sqrt{TR^2}} = \sqrt{T}\cdot\frac{\rho}{R}.$$

The anonymous constant was therefore ρ/R all along. By Remark 2.3.7 the left-hand side is at most 1, so $\sqrt{T}\rho/R \le 1$, i.e. $T \le R^2/\rho^2$. Since the right-hand side is a finite number fixed by $\mathcal{D}$ and $\boldsymbol{w}_f$, PLA cannot correct mistakes forever. ■

Remark 2.3.7 (Is it really a cosine in $\mathbb{R}^{d+1}$?)**.** We have been calling $\boldsymbol{w}_f^\top \boldsymbol{w}_T/(\|\boldsymbol{w}_f\|\|\boldsymbol{w}_T\|)$ a cosine, which is obvious for $d = 2$ but deserves a word once $\boldsymbol{w}$ lives in $\mathbb{R}^{d+1}$ for large d. It is still a cosine, for two reasons, and the proof needs only the weaker of them.

- **What the proof actually uses** is only that the quantity never exceeds 1, and that is the Cauchy–Schwarz inequality $\left|\boldsymbol{u}^\top \boldsymbol{v}\right| \leq \|\boldsymbol{u}\|\|\boldsymbol{v}\|$, valid in $\mathbb{R}^m$ for every m. No geometry is required.
- **Geometrically** it is a cosine in the ordinary sense too. Two vectors always span a subspace of dimension at most 2; inside that plane they make an ordinary angle, and $\boldsymbol{u}^\top \boldsymbol{v}/(\|\boldsymbol{u}\|\|\boldsymbol{v}\|)$ is its cosine. Indeed this is how the angle between two vectors is *defined* in any dimension — Cauchy–Schwarz is exactly the statement that makes the definition legitimate, by putting the ratio in $[-1, 1]$ so that an arccos exists.

So the picture of $\boldsymbol{w}_t$ slowly rotating towards $\boldsymbol{w}_f$ survives intact in any number of dimensions.

Remark 2.3.8. Both constants have a geometric reading, and the bound is more memorable through it. R is the radius of the data: the distance from the origin to the farthest example. And since $\boldsymbol{w}_f/\|\boldsymbol{w}_f\|$ is a unit normal of the perfect separating hyperplane, ρ is the smallest signed distance from an example to that hyperplane — the *margin*. So

$$T \leq \left(\frac{\text{radius of the data}}{\text{margin}}\right)^2 :$$

the more comfortably the data separates, the sooner PLA is done. This quantity returns as the central object of the chapter on support vector machines, which does not merely tolerate a large margin but goes looking for one.

§2.4. Non-Separable Data

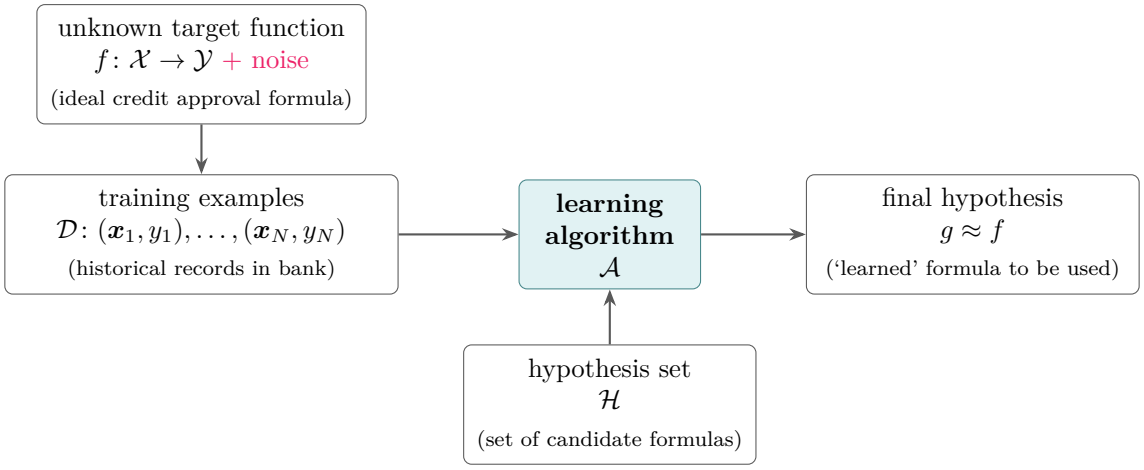
slides p.17–22

§2.4.i. What We Have and What We Lack

- **Guarantee**: as long as $\mathcal{D}$ is linearly separable and we correct by mistake, the inner product of $\boldsymbol{w}_f$ and $\boldsymbol{w}_t$ grows fast while the length of $\boldsymbol{w}_t$ grows slowly; the PLA lines get more and more aligned with $\boldsymbol{w}_f$, so PLA halts.
- **Pros**: simple to implement, fast, and works in any dimension d.
- **Cons**: it *assumes* a linearly separable $\mathcal{D}$ in order to halt — and that property is unknown in advance, with the circularity that if we already knew $\boldsymbol{w}_f$ we would not need PLA at all. Moreover we are not fully sure how long halting takes, since ρ depends on the unknown $\boldsymbol{w}_f$, though in practice it is fast.

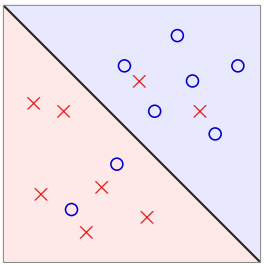
Question 2.4.1. What if $\mathcal{D}$ is not linearly separable?

In practice the usual reason is not that the truth is curved, but that the data is **noisy**: the recorded y_n is sometimes simply wrong, or two customers with identical records behaved differently. The learning flow gains one term.



Question 2.4.2. How can we at least get $g \approx f$ on a noisy $\mathcal{D}$?

§2.4.ii. A Line with Noise Tolerance



If the noise is 'little', then $y_n = f(\boldsymbol{x}_n)$ *usually*, and therefore $g \approx f$ on $\mathcal{D}$ means $y_n = g(\boldsymbol{x}_n)$ *usually*. That suggests giving up on perfection and asking for the fewest mistakes instead:

$$\boldsymbol{w}_g \leftarrow \arg\min_{\boldsymbol{w}} \sum_{n=1}^{N} [\![y_n \neq \operatorname{sign}\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)]\!].$$

Remark 2.4.3. This is exactly the right thing to ask for and, unfortunately, NP-hard to solve. The obstruction is the Iverson bracket: the objective counts, so it is piecewise constant, and there is no slope anywhere to follow downhill. Much of *How can machines learn?* is about replacing this count with a smoother surrogate; here we take the cheaper route instead and modify the algorithm rather than the objective.

§2.4.iii. The Pocket Algorithm

Question 2.4.4. Can we modify PLA to get an 'approximately good' g?

The fix is almost embarrassingly simple. PLA already wanders among candidate lines; it just has no memory, and on non-separable data it wanders forever. So give it a pocket: keep the best line seen so far, and keep wandering.

Algorithm 2.3: Pocket Algorithm

Input: training examples $\mathcal{D}$, an iteration budget
Output: $\hat{\boldsymbol{w}}$, called $\boldsymbol{w}_{\text{POCKET}}$ and returned as g

initialize the pocket weights $\hat{\boldsymbol{w}}$;
for $t = 0, 1, \ldots$ *until enough iterations* **do**
  find a (random) mistake of $\boldsymbol{w}_t$, called $(\boldsymbol{x}_{n(t)}, y_{n(t)})$;
  (try to) correct the mistake by $\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + y_{n(t)}\boldsymbol{x}_{n(t)}$;
  if $\boldsymbol{w}_{t+1}$ *makes fewer mistakes on* $\mathcal{D}$ *than* $\hat{\boldsymbol{w}}$ **then** $\hat{\boldsymbol{w}} \leftarrow \boldsymbol{w}_{t+1}$;
return $\hat{\boldsymbol{w}}$ as g;

Note what changed and what did not. The update rule is untouched — the black lines of PLA are still there — and the only additions are the pocket and the iteration budget, which is now needed because there is no "no more mistakes" to stop on.

A simple modification of PLA to find (somewhat) 'best' weights.

Exercise 2.4.5 (Fun Time)**.** Should we use pocket or PLA? Since we do not know whether $\mathcal{D}$ is linearly separable in advance, we may decide to just go with pocket instead of PLA. If $\mathcal{D}$ is actually linearly separable, what is the difference between the two?

(1) pocket on $\mathcal{D}$ is slower than PLA;

(2) pocket on $\mathcal{D}$ is faster than PLA;

(3) pocket on $\mathcal{D}$ returns a better g in approximating f than PLA;

(4) pocket on $\mathcal{D}$ returns a worse g in approximating f than PLA.

Reference Answer. (1). Because pocket needs to check whether $\boldsymbol{w}_{t+1}$ is better than $\hat{\boldsymbol{w}}$ in each iteration — a full pass over $\mathcal{D}$ — it is slower than PLA. On a linearly separable $\mathcal{D}$, $\boldsymbol{w}_{\text{POCKET}}$ ends up the same as $\boldsymbol{w}_{\text{PLA}}$, both making no mistakes; so the price of not knowing in advance is time, not quality. ■

Summary

Perceptron Hypothesis Set hyperplanes / linear classifiers in $\mathbb{R}^d$.

Perceptron Learning Algorithm (PLA) correct mistakes and improve iteratively.

Guarantee of PLA no mistake eventually, if linearly separable.

Non-Separable Data hold somewhat 'best' weights in a pocket.

We now own one complete learning model, $\mathcal{H}$ and $\mathcal{A}$ both. It answers a *yes/no* question, from a vector of *real* features, given *labelled* examples handed over *all at once*. Every one of those emphasised choices can be varied, and the next chapter takes stock of the resulting zoo of learning problems.

3. Types of Learning

Lecture 3

16 Sep.

> **As previously seen .** Section 2.2 produced one complete learning model: PLA takes a linearly separable $\mathcal{D}$ and the perceptrons $\mathcal{H}$, and returns a hypothesis g. That is one point in a very large space of possibilities.

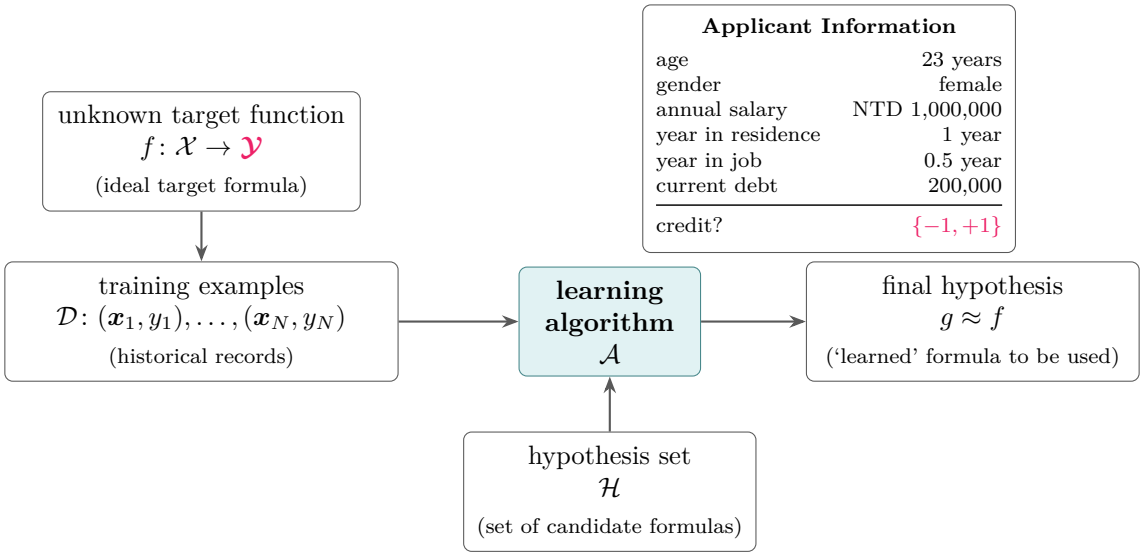
The frame of Section 1.4 has several slots — the output space $\mathcal{Y}$, the labels y_n, the way the data reaches us, the input space $\mathcal{X}$ — and so far every one of them has been filled in the simplest way available. This chapter varies them one at a time, which is the cheapest way to see what the frame can hold. Nothing here is proved; the point is a map of the territory, and the names on it recur for the rest of the course.

§3.1. Learning with Different Output Space $\mathcal{Y}$

slides p.2–8

§3.1.i. The Credit Approval Problem Revisited

Credit approval answers one yes/no question, so its output space has exactly two elements.



$\mathcal{Y} = \{-1, +1\}$**: binary classification.**

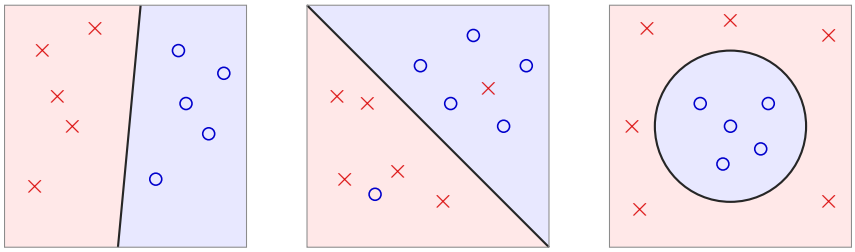
§3.1.ii. Binary Classification

Definition 3.1.1 (Binary classification)**.** A learning problem whose output space has exactly two elements, $\mathcal{Y} = \{-1, +1\}$, is called **binary classification**. The two elements are only names: good/bad, spam/non-spam, sick/not sick.

The same output space covers an enormous range of questions:

- credit approve/disapprove;
- email spam/non-spam;
- patient sick/not sick;
- ad profitable/not profitable;
- answer correct/incorrect (KDDCup 2010).

Geometrically it is the picture we already know — a boundary with one class on each side, easy or hard depending on how the data falls.

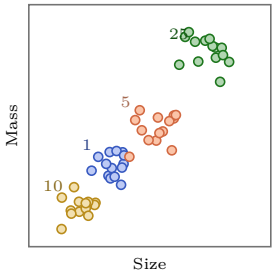


A core and important problem, with many tools that serve as building blocks of other tools.

§3.1.iii. Multiclass Classification

Definition 3.1.2 (Multiclass classification)**.** A learning problem whose output space is finite and explicitly listed, $\mathcal{Y} = \{1, 2, \ldots, K\}$ with $K \geq 2$, is called **multiclass classification**; Definition 3.1.1 is the special case $K = 2$.

Example 3.1.3 (Coin recognition)**.** Classify US coins — 1c, 5c, 10c, 25c — by their (size, mass). The four denominations form four clusters, because a heavier coin is usually a larger one.



Here $\mathcal{Y} = \{1c, 5c, 10c, 25c\}$, so $K = 4$.

Other multiclass problems: written digits ⇒ $0, 1, \ldots, 9$; pictures ⇒ apple, orange, strawberry; emails ⇒ spam, primary, social, promotion, update (Google).

Many applications in practice, especially for 'recognition'.

§3.1.iv. Regression

Definition 3.1.4 (Regression)**.** A learning problem whose output space is the whole real line, $\mathcal{Y} = \mathbb{R}$, is called **regression**; if the output is confined to an interval, $\mathcal{Y} = [\text{lower}, \text{upper}] \subset \mathbb{R}$, it is **bounded regression**. The difference from classification is that $\mathcal{Y}$ now carries an order and a distance, so being wrong by a little is better than being wrong by a lot.

Keep the patient, change the question, and the output space changes with it:

- binary classification: patient features ⇒ sick or not;
- multiclass classification: patient features ⇒ which type of cancer;
- *regression*: patient features ⇒ how many days before recovery.

Regression was studied deeply in statistics long before machine learning existed, which is why so many of its tools arrive ready-made. Other regression problems: company data ⇒ stock price; climate data ⇒ temperature.

Also core and important, with many 'statistical' tools as building blocks of other tools.

§3.1.v. Structured Learning

Definition 3.1.5 (Structured learning)**.** **Structured learning** is a learning problem whose outputs are composite objects — a tag for every word of a sentence, a folding of a protein, a parse tree — whose parts constrain one another. Formally $\mathcal{Y}$ is still a finite set of 'classes', but an enormous one that is defined implicitly by those constraints rather than enumerated.

Example 3.1.6 (Sequence tagging)**.** Multiclass classification sends a word to a word class. Structured learning sends a whole sentence to a structure — the class of every word at once:

$$\underbrace{\text{I}}_{\text{pronoun}}\ \underbrace{\text{love}}_{\text{verb}}\ \underbrace{\text{ML}}_{\text{noun}}$$

so that $\mathcal{Y} = \{\text{PVN}, \text{PVP}, \text{NVN}, \text{PV}, \dots\}$, not including VVVVV.

That exclusion is the whole difficulty. The output space is a huge multiclass problem — a structure is a kind of hyperclass — but without an *explicit* class definition: the valid structures are constrained by grammar, and nobody wrote the list down. Other structured problems: protein data ⇒ protein folding; speech data ⇒ speech parse tree.

A fancy but complicated learning problem.

§3.1.vi. Mini Summary

Note 3.1.7 (Learning with different output space $\mathcal{Y}$)**.**

- binary classification: $\mathcal{Y} = \{-1, +1\}$;
- multiclass classification: $\mathcal{Y} = \{1, 2, \dots, K\}$;
- regression: $\mathcal{Y} = \mathbb{R}$;
- structured learning: $\mathcal{Y} =$ structures;
- ... and a lot more!!

Core tools: binary classification and regression.

Exercise 3.1.8 (Fun Time)**.** The entrance system of the school gym, which does automatic face recognition based on machine learning, is built to charge four different groups of users differently: Staff, Student, Professor, Other. What type of learning problem best fits the need of the system?

(1) binary classification;

(2) multiclass classification;

(3) regression;

(4) structured learning.

Reference Answer. (2). There is an 'explicit' $\mathcal{Y}$ that contains four classes. ■

§3.2. Learning with Different Data Label y_n

slides p.9–15

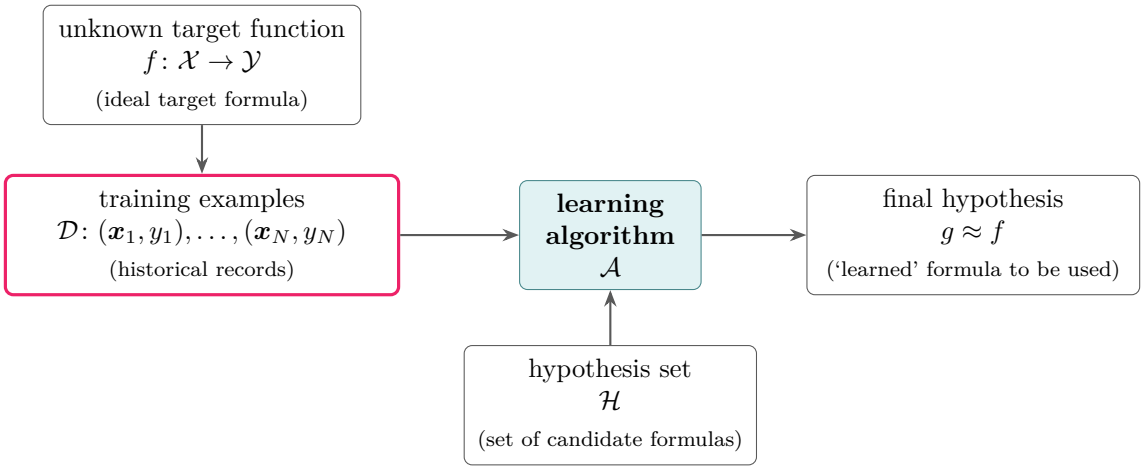
§3.2.i. Supervised Learning

Definition 3.2.1 (Supervised learning)**.** Learning from data in which every input carries its own label,

$$\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}_{n=1}^{N},$$

is called **supervised learning**. This is the only kind we have used so far.

Everything so far has quietly assumed that the data arrives fully labelled.



Supervised learning: every $\boldsymbol{x}_n$ comes with a corresponding y_n.

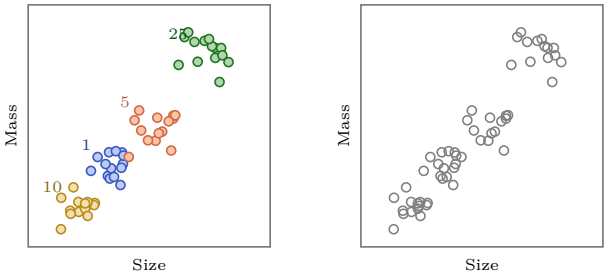
§3.2.ii. Unsupervised Learning

Definition 3.2.2 (Unsupervised learning)**.** Learning from data that carries no labels at all,

$$\mathcal{D} = \{\boldsymbol{x}_n\}_{n=1}^{N},$$

is called **unsupervised learning**. With no y_n there is no target f to approximate in the sense of Section 1.4, so what counts as a good g has to be decided separately for each problem.

Now delete the labels. The coins are still there, and they still fall into clusters — but nothing says which cluster is the quarter.



supervised multiclass classification unsupervised ⇔ 'clustering'

Other clustering problems: articles ⇒ topics; consumer profiles ⇒ consumer groups. Clustering is only one of several unsupervised problems, and they are best remembered as unsupervised shadows of the supervised ones — each keeps the *shape* of a supervised problem but has to invent its own notion of a right answer.

Definition 3.2.3 (Clustering)**.** **Clustering** learns a map

$$\{\boldsymbol{x}_n\} \Rightarrow \text{cluster}(\boldsymbol{x})$$

assigning each input to one of finitely many groups, where the groups themselves are discovered rather than given. It is roughly 'unsupervised multiclass classification', with the essential difference that the classes have no names and no prescribed number — articles ⇒ topics.

Definition 3.2.4 (Density estimation)**.** **Density estimation** learns a map

$$\{\boldsymbol{x}_n\} \Rightarrow \text{density}(\boldsymbol{x})$$

reporting how tightly the inputs crowd around each point of $\mathcal{X}$. Since a density is a non-negative number, this is roughly 'unsupervised bounded regression' — traffic reports with location ⇒ dangerous areas.

Definition 3.2.5 (Outlier detection)**.** **Outlier detection** learns a map

$$\{\boldsymbol{x}_n\} \Rightarrow \text{unusual}(\boldsymbol{x})$$

flagging the inputs that do not look like the rest. It is roughly an extreme 'unsupervised binary classification': extreme because the two classes are wildly unbalanced — almost everything is normal, and it is the rare case that matters — so accuracy is a useless measure of success. Internet logs ⇒ intrusion alert.

. . . and a lot more!!

Unsupervised learning: diverse, with possibly very different performance goals.

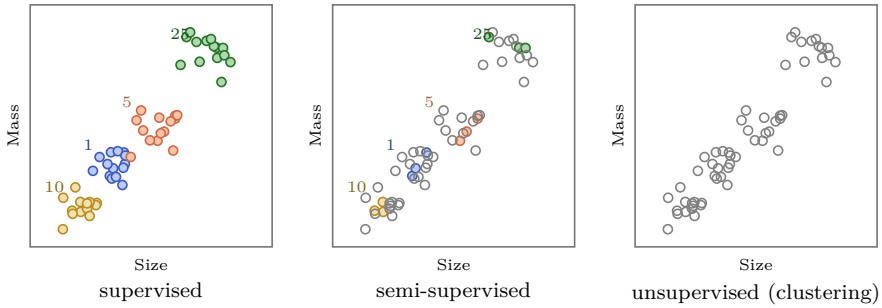
§3.2.iii. Semi-supervised Learning

Definition 3.2.6 (Semi-supervised learning)**.** Learning from data in which only a few inputs carry labels,

$$\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}_{n=1}^{\ell} \cup \{\boldsymbol{x}_n\}_{n=\ell+1}^{N}, \qquad \ell \ll N,$$

is called **semi-supervised learning**. The unlabelled majority is not useless: it shows where the inputs lie, and a boundary is more believable if it passes through a thin region of them.

Between the two extremes sits the case that is commonest in practice: labels exist, but only for a few examples, because labelling costs money.



Other semi-supervised problems: face images with a few labelled ⇒ face identifier (Facebook); medicine data with a few labelled ⇒ medicine effect predictor.

Semi-supervised learning: leverage unlabelled data to avoid 'expensive' labelling.

§3.2.iv. Reinforcement Learning

Definition 3.2.7 (Reinforcement learning)**.** Learning from data of the form

$$\mathcal{D} = \{(\boldsymbol{x}_n, \tilde{y}_n, \text{goodness}(\tilde{y}_n))\}_{n=1}^{N},$$

where $\tilde{y}_n$ is an action that was actually taken and the third entry says only how well it turned out, is called **reinforcement learning**. The correct label y_n is never revealed — one learns that a choice was good or bad, not what the right choice would have been.

A 'very different' but natural way of learning, and the easiest way in is the one the slides take.

Example 3.2.8 (Teach your dog to sit)**.**

You say 'sit down'.



- *The dog pees on the ground.* BAD DOG. THAT'S A VERY WRONG ACTION. You cannot easily show the dog that $y_n = \mathsf{sit}$ when $\boldsymbol{x}_n =$ 'sit down', but you can *punish* it, which says that $\tilde{y}_n = \mathsf{pee}$ is wrong.
- *The dog sits down.* GOOD DOG. LET ME GIVE YOU SOME COOKIES. You still cannot show that $y_n = \mathsf{sit}$, but you can *reward* it, which says that $\tilde{y}_n = \mathsf{sit}$ is good.

Note what the training signal is. It is not the correct label y_n but a triple $(\boldsymbol{x}, \tilde{y}, \text{goodness})$: some action was taken, and we learn only how good *that* action was — never what the best one would have been. Other problems of the same shape: (customer, ad choice, ad click earning) ⇒ ad system; (cards, strategy, winning amount) ⇒ black jack agent.

Reinforcement: learn with 'partial/implicit information' (often sequentially).

§3.2.v. Mini Summary

Note 3.2.9 (Learning with different data label y_n)**.**

- supervised: all y_n;
- unsupervised: no y_n;

- semi-supervised: some y_n;
- reinforcement: implicit y_n by goodness($\tilde{y}_n$);
- ... and more!!

Core tool: supervised learning.

Exercise 3.2.10 (Fun Time)**.** To build a tree recognition system, a company decides to gather one million pictures on the Internet. Then it asks each of its 10 members to view 100 pictures and record whether each picture contains a tree. The pictures and records are then fed to a learning algorithm to build the system. What type of learning problem does the algorithm need to solve?

(1) supervised;

(2) unsupervised;

(3) semi-supervised;

(4) reinforcement.

Reference Answer. (3). The 1,000 records are the labelled $(\boldsymbol{x}_n, y_n)$; the other 999,000 pictures are unlabelled $\boldsymbol{x}_n$. ■

§3.3. Learning with Different Protocol $f \Rightarrow (\boldsymbol{x}_n, y_n)$

slides p.16–21

The third slot is not about what the data *is* but about how it reaches the algorithm — the arrow from f down to $\mathcal{D}$.

§3.3.i. Batch Learning

Definition 3.3.1 (Batch learning)**.** Under the **batch** protocol the entire $\mathcal{D}$ is handed to $\mathcal{A}$ at once, and $\mathcal{A}$ returns a single hypothesis g that is then held fixed.

Example 3.3.2.
1. The coin recogniser of Example 3.1.3 is handed its whole data set at once and learns from all of it: batch supervised multiclass classification.
2. a batch of (email, spam?) ⇒ spam filter
3. a batch of (patient, cancer) ⇒ cancer classifier; a batch of patient data ⇒ groups of patients

Remark 3.3.3. The last one unsupervised, so the protocol is independent of the previous two sections.

Batch learning: a very common protocol.

§3.3.ii. Online Learning

Definition 3.3.4 (Online learning)**.** Under the **online** protocol the examples arrive one at a time: at step t the learner sees $\boldsymbol{x}_t$, predicts with its current g_t, is then told y_t, and updates to g_{t+1}. There is no separate training phase — the hypothesis in use is the one still being learned.

Example 3.3.5 (A spam filter that improves)**.** A *batch* spam filter learns from known (email, spam?) pairs and then predicts with a fixed g. An *online* spam filter instead repeats, for $t = 1, 2, \ldots$:

1. observe an email $\boldsymbol{x}_t$;
2. predict its spam status with the current $g_t(\boldsymbol{x}_t)$;
3. receive the 'desired label' y_t from the user, and then update g_t with $(\boldsymbol{x}_t, y_t)$.

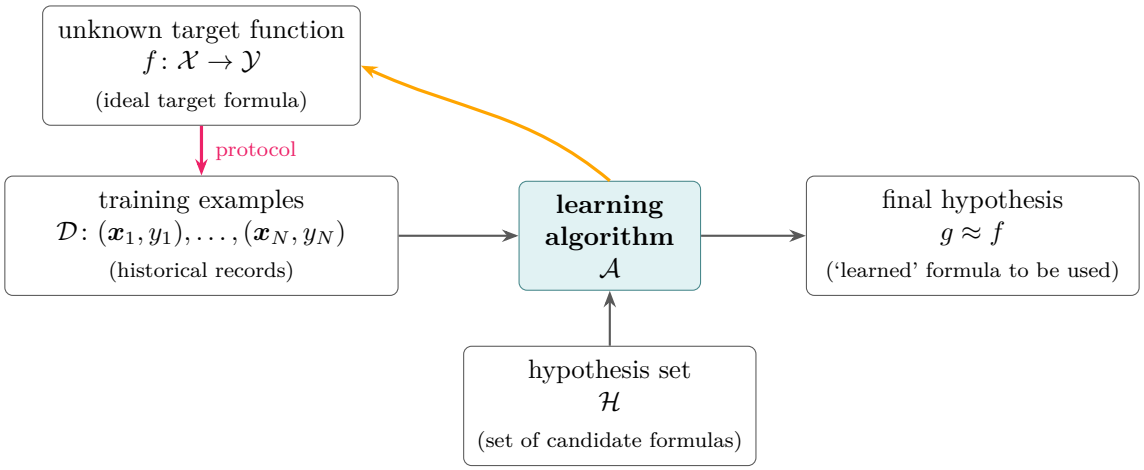
Question 3.3.6. Two connections to what we have already learned: PLA can be easily adapted to the online protocol — how? And reinforcement learning is often done online — why?

Answer. PLA already works one example at a time: it looks at a single $(\boldsymbol{x}_{n(t)}, y_{n(t)})$, and either leaves $\boldsymbol{w}$ alone or corrects it. Feed it a stream instead of a fixed $\mathcal{D}$ and nothing in algorithm 2.1 has to change. Reinforcement learning is online almost by definition: the signal $(\boldsymbol{x}, \tilde{y}, \text{goodness})$ only exists once an action has been taken, so the data cannot be collected in advance — it is generated by the learner's own behaviour. ■

Online: the hypothesis 'improves' through receiving data instances sequentially.

§3.3.iii. Active Learning

Definition 3.3.7 (Active learning)**.** Under the **active** protocol the learner *chooses* which $\boldsymbol{x}_n$ it wants labelled and queries that label, instead of accepting whichever examples happen to arrive. The aim is to reach a good g from fewer labels by asking about the inputs that are most informative.



Note 3.3.8 (Protocol ⇔ learning philosophy)**.**

- batch: 'duck feeding' — everything is pushed in at once;
- online: 'passive sequential' — take what arrives, in order;
- active: 'question asking' (sequentially) — query the y_n of a *chosen* $\boldsymbol{x}_n$.

Active: improve the hypothesis with fewer labels (hopefully) by asking questions strategically.

§3.3.iv. Mini Summary

Note 3.3.9 (Learning with different protocol $f \Rightarrow (\boldsymbol{x}_n, y_n)$)**.**

- batch: all known data;
- online: sequential (passive) data;
- active: strategically-observed data;
- ... and more!!

Core protocol: batch.

Exercise 3.3.10 (Fun Time)**.** A photographer has 100,000 pictures, each containing one baseball player. He wants to categorize the pictures automatically by the player inside. He starts by categorizing 1,000 pictures himself, and then writes an algorithm that tries to categorize the other pictures if it is 'confident' on the category, while pausing for — and learning from — human input if not. What protocol best describes the nature of the algorithm?

(1) batch;

(2) online;

(3) active;

(4) random.

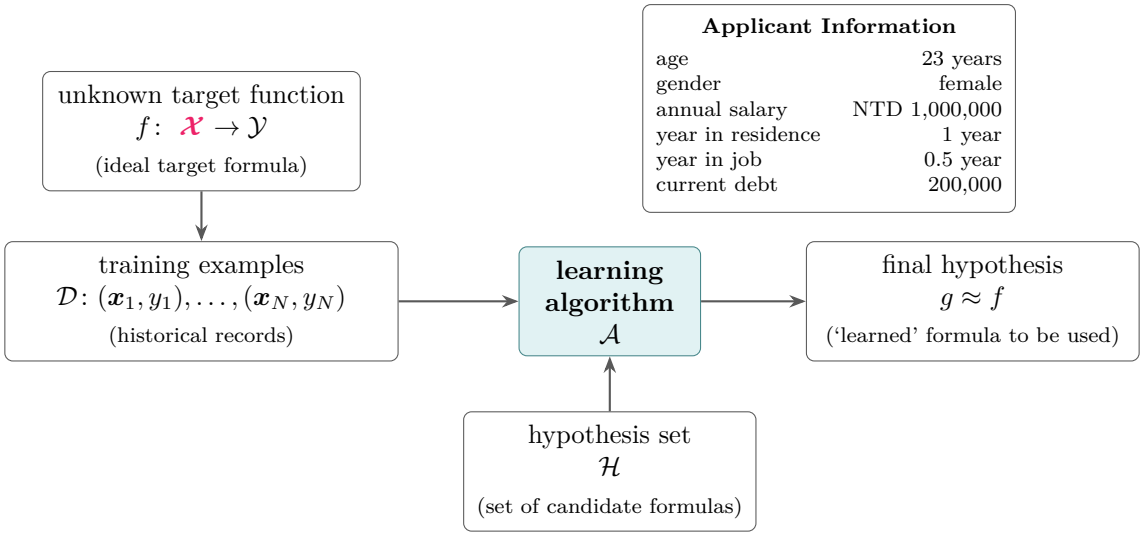
Reference Answer. (3). The algorithm takes an active but naïve strategy: ask when 'confused'. You should probably do the same when taking a class. :-) ■

§3.4. Learning with Different Input Space $\mathcal{X}$

slides p.22–28

§3.4.i. Concrete Features

Definition 3.4.1 (Concrete features)**.** $\mathcal{X} \subseteq \mathbb{R}^d$ consists of **concrete features** when each of its d dimensions carries a sophisticated physical meaning — annual salary, mass, tumour size — normally because a human who understood the task chose to measure it.



Concrete features: each dimension of $\mathcal{X} \subseteq \mathbb{R}^d$ represents 'sophisticated physical meaning'.

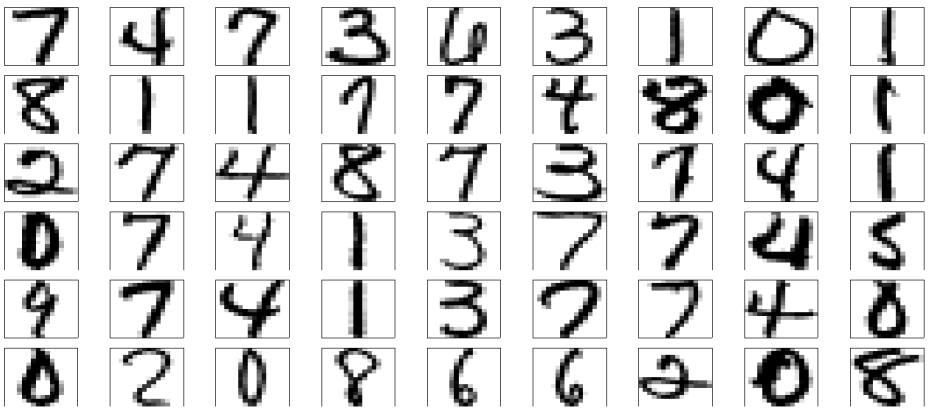
Every input we have met so far is of this kind: (size, mass) for coin classification, customer information for credit approval, patient information for cancer diagnosis. What they share is that somebody decided which quantities were worth measuring — the features carry 'human intelligence' about the learning task, put there before learning began.

Concrete features: the 'easy' ones for ML.

§3.4.ii. Raw Features

Definition 3.4.2 (Raw features)**.** $\mathcal{X}$ consists of **raw features** when each dimension has only a simple physical meaning — the darkness of one pixel, one sample of a waveform. Any dimension on its own says almost nothing; the information lives in combinations of very many of them.

Example 3.4.3 (Digit recognition)**.** Features ⇒ meaning of digit: a typical supervised multi-class classification problem. But what are the features?

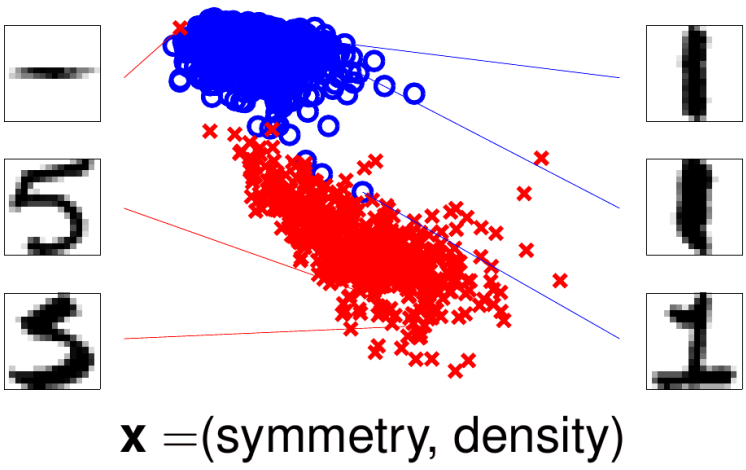


The honest answer is that a 16×16 grey image is just a list of 256 numbers,

$$\boldsymbol{x} \equiv (0, 0, 0.9, 0.6, \ldots) \in \mathbb{R}^{256},$$

and each of those numbers means only 'how dark is this one pixel'. That is a *simple* physical meaning, and it makes the problem harder for machine learning, not easier — no single pixel

tells you anything about the digit. Compare what a human would hand over instead: two numbers, symmetry and density, chosen because they happen to separate the classes.



Other problems with raw features: image pixels, speech signals, and so on.

Raw features: often need humans or machines to convert them to concrete ones.

§3.4.iii. Abstract Features

Definition 3.4.4 (Abstract features)**.** $\mathcal{X}$ consists of **abstract features** when its dimensions have no (or very little) physical meaning at all — identifiers such as a userid or an itemid, where even the ordering of the numbers is an accident of bookkeeping.

Example 3.4.5 (Rating prediction, KDDCup 2011)**.** Given previous (userid, itemid, rating) tuples, predict the rating that some userid would give to some itemid. This is a regression problem with $\mathcal{Y} \subseteq \mathbb{R}$ as the rating and $\mathcal{X} \subseteq \mathbb{N} \times \mathbb{N}$ as (userid, itemid).

User 5,001 is not in any sense one more than user 5,000, so this is harder still than raw features. Other problems with abstract features: student IDs in an online tutoring system (KDDCup 2010), advertisement IDs in an online ad system. That these two competitions were won by extracting factors from bare IDs is exactly the story of Section 1.3.

Abstract: again needs 'feature conversion/extraction/construction'.

§3.4.iv. Mini Summary

Note 3.4.6 (Learning with different input space $\mathcal{X}$)**.**

- concrete: sophisticated (and related) physical meaning;
- raw: simple physical meaning;
- abstract: no (or little) physical meaning;
- ... and more!!

'Easy' input: concrete.

Exercise 3.4.7 (Fun Time)**.** Consider the problem of building an online image advertisement system that shows users the most relevant images. What features can you choose to use?

(1) concrete;

(2) concrete, raw;

(3) concrete, abstract;

(4) concrete, raw, abstract.

Reference Answer. (4). Concrete user features, raw image features, and maybe abstract user/image IDs. ■

Summary

Output space $\mathcal{Y}$ *classification*, *regression*, structured.

Data label y_n *supervised*, un/semi-supervised, reinforcement.

Protocol $f \Rightarrow (\boldsymbol{x}_n, y_n)$ *batch*, online, active.

Input space $\mathcal{X}$ *concrete*, raw, abstract.

The emphasised entry in each line is the one this course treats as its core tool, and the combination of all four — batch supervised binary classification on concrete features — is exactly the setting PLA already solves. Everything else in the list is reached by relaxing one slot at a time.

Which makes the next question awkward. We have a solved case and a map of its neighbours, but not one word yet about whether any of it works on data we have never seen. The next chapter argues, seriously, that learning is impossible.

4. Feasibility of Learning

Lecture 4

23 Sep.

> **As previously seen .** Section 2.2 produced a working learning model, and Section 3.1 onwards mapped the settings around it. Every guarantee obtained so far — PLA halts, PLA makes no mistake on $\mathcal{D}$ — is a statement about the examples we were handed.

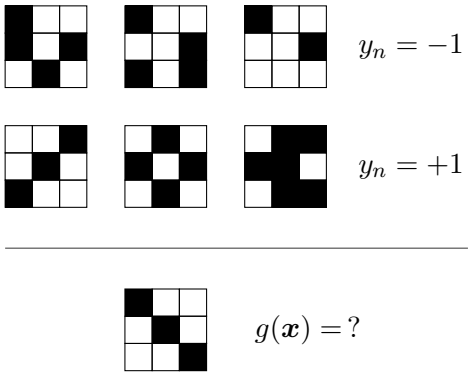
$\mathcal{D}$ is not what we are paid for. The bank wants the next applicant classified, not the ones already in its files, and nothing in the previous three chapters connects the two. This chapter takes that gap seriously enough to argue first that it cannot be bridged at all, and then that it can — once we agree to settle for a guarantee of a weaker kind. Four steps: a proof that learning is impossible, a probabilistic escape, the escape applied to one hypothesis, and finally to a whole hypothesis set.

§4.1. Learning is Impossible?

slides p.2–6

§4.1.i. A Learning Puzzle

Before any mathematics, a test of your own human learning, with six examples.



Each $\boldsymbol{x}_n$ is a 3×3 picture of black and white cells; three of them are labelled -1 and three $+1$. What does your g say about the seventh?

§4.1.ii. Two Controversial Answers

Whatever you say about $g(\boldsymbol{x})$, an answer with reasons behind it is available for the other side as well.

truth $f(\boldsymbol{x}) = +1$ **because ...**	**truth** $f(\boldsymbol{x}) = -1$ **because ...**
• symmetry $\Leftrightarrow +1$; • (black or white count $= 3$) or (black count $= 4$ and middle-top black) $\Leftrightarrow +1$.	• left-top black $\Leftrightarrow -1$; • middle column contains at most 1 black and right-top white $\Leftrightarrow -1$.

Observation . All four rules are correct on all six examples, and all four fire on the test picture. The last one is worth checking by hand: its main diagonal is black, so it is symmetric and its black count is 3, which the left-hand rules read as $+1$; its left-top cell is black and its middle column holds one black with a white right-top cell, which the right-hand rules read as -1.

So the data does not decide. Any answer you give can be called wrong by a teacher who had one of the other rules in mind all along, and that teacher never has to admit to cheating — the rule was consistent with everything you were shown.

All valid reasons: your adversarial teacher can always call you 'didn't learn'. :-(

§4.1.iii. A 'Simple' Binary Classification Problem

The puzzle is easier to argue about once it is small enough to write out completely. Take

$$\mathcal{X} = \{0,1\}^3, \qquad \mathcal{Y} = \{\circ, \times\},$$

so that $\mathcal{X}$ has eight points and there are only $2^8 = 256$ candidate targets f — few enough that $\mathcal{H}$ can be taken to be *all* of them. Suppose five of the eight points come to us labelled:

$\boldsymbol{x}_n$	$y_n = f(\boldsymbol{x}_n)$
000	$\circ$
001	$\times$
010	$\times$
011	$\circ$
100	$\times$

Question 4.1.1. Pick $g \in \mathcal{H}$ with $g(\boldsymbol{x}_n) = y_n$ for every n, the way PLA does. Does $g \approx f$?

§4.1.iv. No Free Lunch

Fixing the five labels kills 2^5 of the 256 degrees of freedom and leaves $2^3 = 8$ targets standing. Here they are, next to any g we might have picked.

	$\boldsymbol{x}$	y	g	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8
$\mathcal{D}$	000	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$
	001	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
	010	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
	011	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$	$\circ$
	100	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
	101		?	$\circ$	$\circ$	$\circ$	$\circ$	$\times$	$\times$	$\times$	$\times$
	110		?	$\circ$	$\circ$	$\times$	$\times$	$\circ$	$\circ$	$\times$	$\times$
	111		?	$\circ$	$\times$	$\circ$	$\times$	$\circ$	$\times$	$\circ$	$\times$

The top block is the same in every column, which is the good news:

- $g \approx f$ inside $\mathcal{D}$: sure!

- $g \approx f$ outside $\mathcal{D}$: **No!** — but that is really what we want.

The bottom block is the bad news, and it is as bad as it can be: for each of the three unseen inputs, four of the surviving targets say $\circ$ and four say $\times$. Whatever g fills into a '?', half of the candidates disagree with it. This is worth saying once in general.

Proposition 4.1.2 (No free lunch)

Let $\mathcal{H}$ contain every $f\colon \mathcal{X} \to \{-1, +1\}$, let $\mathcal{D}$ be any set of labelled examples, and let $\boldsymbol{x} \notin \mathcal{D}$. Among the targets that agree with $\mathcal{D}$, exactly half satisfy $f(\boldsymbol{x}) = +1$ and half satisfy $f(\boldsymbol{x}) = -1$. Consequently every algorithm, no matter how it computes g, is wrong at $\boldsymbol{x}$ for exactly half of them.

Proof. Write S for the set of targets that agree with $\mathcal{D}$ — the eight columns of the table above, in the example. Given $f \in S$, let f' be the target that copies f at every input except $\boldsymbol{x}$, where it answers the other way: $f'(\boldsymbol{x}) = -f(\boldsymbol{x})$.

That f' is again in S is the one place the hypothesis $\boldsymbol{x} \notin \mathcal{D}$ is used: the single value that was changed sits at $\boldsymbol{x}$, which is not one of the examples, so f' still reproduces every label of $\mathcal{D}$. Flipping twice returns the original, $(f')' = f$, and f' is never f itself, since the two differ at $\boldsymbol{x}$ by construction. A rule that pairs each element with another one and is undone by applying it again leaves nobody over, so $f \mapsto f'$ cuts S into pairs, and the two members of a pair give opposite answers at $\boldsymbol{x}$.

Half of S therefore answers $+1$ at $\boldsymbol{x}$ and half answers -1. Now $g(\boldsymbol{x})$ is a single fixed value, whatever $\mathcal{A}$ did to arrive at it, so within each pair it agrees with one member and disagrees with the other. ■

In the table, with $\boldsymbol{x} = 101$, that pairing is $f_1 \leftrightarrow f_5$, $f_2 \leftrightarrow f_6$, $f_3 \leftrightarrow f_7$, $f_4 \leftrightarrow f_8$: the members of a pair agree on every other row and differ only in that one.

The proof uses nothing about the algorithm — it never appears — which is exactly why no cleverer $\mathcal{A}$ can escape. The trouble is the assumption that f is completely unrestricted.

Learning from $\mathcal{D}$ (to infer something outside $\mathcal{D}$) is doomed if any 'unknown' f can happen. :-(

Exercise 4.1.3 (Fun Time)**.** This is a popular 'brain-storming' problem, with a claim that 2% of the world's cleverest population can crack its 'hidden pattern':

$$(5, 3, 2) \to 151022, \qquad (7, 2, 5) \to ?$$

It is a learning problem with $N = 1$, $\boldsymbol{x}_1 = (5, 3, 2)$, $y_1 = 151022$. Learn a hypothesis from the one example to predict on $\boldsymbol{x} = (7, 2, 5)$. What is your answer?

(1) 151026;

(2) 143547;

(3) I need more examples to get the correct answer;

(4) there is no 'correct' answer.

Reference Answer. (4). Following the same nature of the no-free-lunch problems discussed, we cannot hope to be correct under this 'adversarial' setting. The designer's answer is (2): the first two digits are $x_1 \cdot x_2$, the next two $x_1 \cdot x_3$, and the last two $x_1 \cdot x_2 + x_1 \cdot x_3 - x_2$. Knowing

that does not make (2) any more correct than (1) — it only tells you which f this particular teacher had in mind. ■

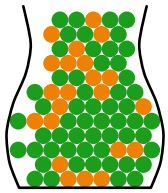
§4.2. Probability to the Rescue

slides p.7–12

§4.2.i. Inferring Something Unknown

Proposition 4.1.2 closes one door: the unknown target f outside $\mathcal{D}$ cannot be inferred. Before giving up, it is worth asking whether *any* unknown quantity is ever inferred from a sample, in any other corner of mathematics — and one is, routinely.

Consider a bin holding very many marbles, each orange or green.



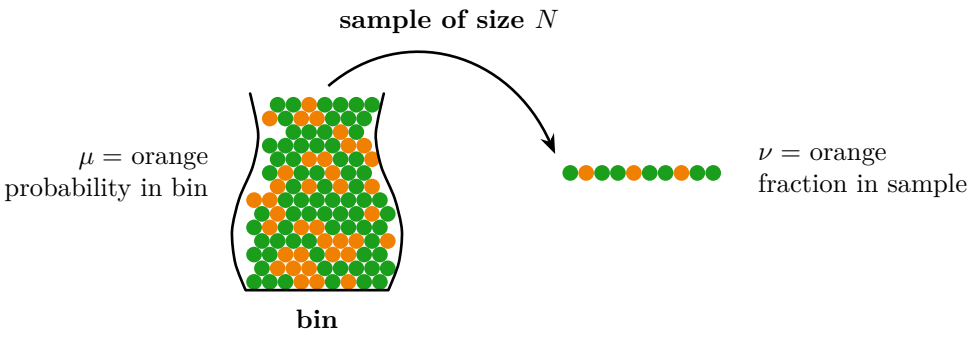
bin

Do we *know* what fraction of them is orange? No: nobody counted, and the bin may hold more marbles than we could ever count.

> **Question 4.2.1.** Can you *infer* the orange probability?

§4.2.ii. Statistics 101: Inferring Orange Probability

The statistician's move is to grab a handful without looking.



> **Notation .** μ is the orange probability in the bin and $1-\mu$ the green one; μ is **unknown**. Draw N marbles independently, each with replacement, and let ν be the orange fraction in that sample, $1-\nu$ the green one; ν is **known**, because it is sitting in front of us. 'In-sample' always refers to ν, 'out-of-sample' to μ.

> **Question 4.2.2.** Does the in-sample ν say anything about the out-of-sample μ?

§4.2.iii. Possible versus Probable

The answer has to be given twice, because the two obvious readings of the question have opposite answers.

No!	**Yes!**
Possibly not: the sample can come out mostly green while the bin is mostly orange.	Probably yes: the in-sample ν is likely to be close to the unknown μ.

Note how little the two claims have to do with each other. The left column is about what a sample *can* be, and a bin of a million marbles of which 999,999 are orange can still deliver ten green ones. The right column is about what a sample *tends* to be, and puts a number on 'tends'. Nothing is being hidden: the bad sample really does exist, we are simply going to bound how often it turns up.

Question 4.2.3. Formally, what does ν say about μ?

§4.2.iv. Hoeffding's Inequality

Theorem 4.2.4 (Hoeffding's inequality)

Draw N marbles independently from a bin whose orange probability is μ, and let ν be the orange fraction among them. For every $\epsilon > 0$,

$$\mathbb{P}\left[|\nu - \mu| > \epsilon\right] \leq 2\exp\left(-2\epsilon^2 N\right).$$

The statement is about marbles, coins, polling and anything else that can be sampled, and the three properties that make it usable are worth separating out.

- It is valid for all N and all ϵ — no 'N large enough' clause, no asymptotics.
- It does not depend on μ. We never need to 'know' μ, which is the whole point, since μ is what we are trying to infer.
- A larger sample size N, or a looser gap ϵ, gives a higher probability for '$\nu \approx \mu$'.

Definition 4.2.5 (PAC)**.** A statement is **probably approximately correct** (PAC) when it may be off by a little (*approximately*, the ϵ) and may fail outright now and then (*probably*, the bound on the right). The statement '$\nu = \mu$' is PAC.

Remark 4.2.6. The factor 2 is a union bound over the two ways of being wrong. Hoeffding proves $\mathbb{P}\left[\nu - \mu > \epsilon\right] \leq \exp\left(-2\epsilon^2 N\right)$ and, by the same argument applied to green, $\mathbb{P}\left[\mu - \nu > \epsilon\right] \leq \exp\left(-2\epsilon^2 N\right)$; adding the two gives the two-sided form. The same cheap trick — bound each bad event, then add — reappears in Section 4.4, where it is the only tool used.

Note 4.2.7. The exponent decays in N, so the bound is far better than its shape suggests. At $\epsilon = 0.1$ and $N = 1{,}000$ it reads $2\exp(-20) \approx 4 \times 10^{-9}$: a thousand marbles pin down the contents of an arbitrarily large bin to within ten percentage points, and the chance of being fooled is one in a quarter of a billion.

If N is large, we can probably infer the unknown μ from the known ν.

Exercise 4.2.8 (Fun Time)**.** Let $\mu = 0.4$. Use Hoeffding's inequality to bound the probability that a sample of 10 marbles will have $\nu \leq 0.1$. What bound do you get?

(1) 0.67;

(2) 0.40;

(3) 0.33;

(4) 0.05.

Reference Answer. (3). The event $\nu \leq 0.1$ is contained in $|\nu - \mu| \geq 0.3$, so set $N = 10$ and $\epsilon = 0.3$:

$$2\exp\left(-2(0.3)^2(10)\right) = 2e^{-1.8} \approx 0.33.$$

The actual probability is about 0.05, which is answer (4); Hoeffding gives only an upper bound, and at $N = 10$ it is a loose one. ■

§4.3. Connection to Learning

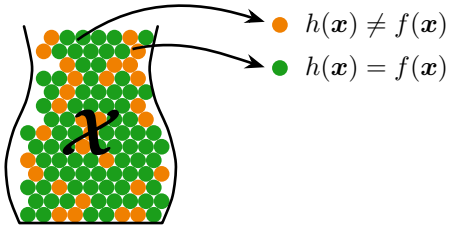
slides p.13–18

§4.3.i. The Bin Analogy

The bin is not an analogy chosen for decoration; every one of its pieces has a counterpart in learning, and the translation is exact.

bin	learning
unknown orange probability μ	a *fixed* hypothesis $h(\boldsymbol{x})$, asked whether it equals the target $f(\boldsymbol{x})$
a marble • in the bin	a point $\boldsymbol{x} \in \mathcal{X}$
orange marble	h is wrong: $h(\boldsymbol{x}) \neq f(\boldsymbol{x})$
green marble	h is right: $h(\boldsymbol{x}) = f(\boldsymbol{x})$
a size-N sample of i.i.d. marbles	checking h on $\mathcal{D} = \left\{ (\boldsymbol{x}_n, \underbrace{y_n}_{f(\boldsymbol{x}_n)}) \right\}$ with i.i.d. $\boldsymbol{x}_n$

So the input space itself is the bin, and it is h that colours it: paint $\boldsymbol{x}$ orange where h disagrees with f and green where it agrees.

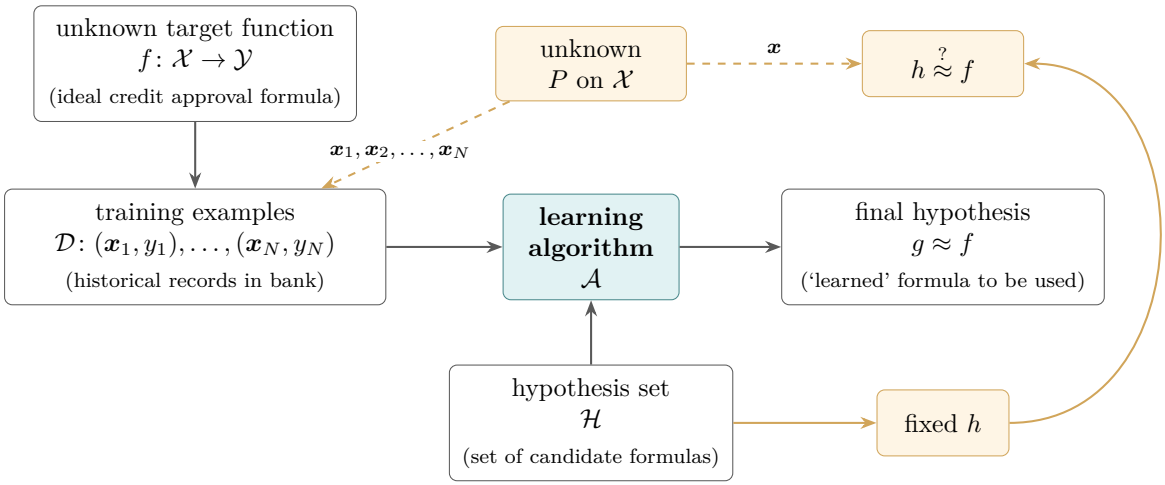


Two conditions make the translation legal, and they are exactly the two that Theorem 4.2.4 needs. The $\boldsymbol{x}_n$ must be drawn independently, and from the same distribution that will later produce the $\boldsymbol{x}$ we are judged on. Nothing is assumed about *which* distribution that is.

If N is large and the $\boldsymbol{x}_n$ are i.i.d., we can probably infer the unknown $[\![h(\boldsymbol{x}) \neq f(\boldsymbol{x})]\!]$ probability from the known $[\![h(\boldsymbol{x}_n) \neq y_n]\!]$ fraction.

§4.3.ii. Added Components

The learning flow of Section 1.4 therefore gains one box it did not have: a probability distribution P on $\mathcal{X}$, which generates both the training inputs and the future ones.



The two quantities the bin analogy named now have proper symbols.

Definition 4.3.1 (In-sample and out-of-sample error)**.** For a hypothesis h, the **out-of-sample error**

$$E_{\text{out}}(h) = \mathop{\mathcal{E}}_{\boldsymbol{x} \sim P} [\![h(\boldsymbol{x}) \neq f(\boldsymbol{x})]\!]$$

is the probability that h is wrong on a fresh input, and the **in-sample error**

$$E_{\text{in}}(h) = \frac{1}{N} \sum_{n=1}^{N} [\![h(\boldsymbol{x}_n) \neq y_n]\!]$$

is the fraction of the examples it gets wrong. In the bin picture $E_{\text{out}}(h)$ is μ and $E_{\text{in}}(h)$ is ν.

For any fixed h, we can probably infer the unknown $E_{\text{out}}(h)$ from the known $E_{\text{in}}(h)$.

§4.3.iii. The Formal Guarantee

Rewriting Theorem 4.2.4 in this notation costs nothing and says everything.

Theorem 4.3.2 (Hoeffding for a fixed hypothesis)
Fix h before seeing $\mathcal{D}$, and let the $\boldsymbol{x}_n$ be drawn i.i.d. from P. Then for every $\epsilon > 0$,

$$\mathbb{P}\left[|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon\right] \leq 2 \exp\left(-2\epsilon^2 N\right).$$

The same three remarks as before apply, and the third one has teeth here.

- Valid for all N and ϵ.

- Does not depend on $E_{\text{out}}(h)$, so there is no need to 'know' $E_{\text{out}}(h)$: both f and P may stay unknown.
- '$E_{\text{in}}(h) = E_{\text{out}}(h)$' is probably approximately correct.

If '$E_{\text{in}}(h) \approx E_{\text{out}}(h)$' and '$E_{\text{in}}(h)$ small', then $E_{\text{out}}(h)$ is small, i.e. $h \approx f$ with respect to P.

That last line deserves to be read slowly, because it says what $g \approx f$ has meant all along. It is not 'g and f agree everywhere' — no amount of data can buy that, by Proposition 4.1.2. It is 'g and f agree on inputs that actually occur', weighted by how often they occur. Regions of $\mathcal{X}$ that P never visits are free to disagree, and we never find out.

§4.3.iv. Verification of One h

So for any fixed h, when the data is large enough, $E_{\text{in}}(h) \approx E_{\text{out}}(h)$. Learning, however, never reports an h of our choosing: it reports $g = \mathcal{A}(\mathcal{D})$, the single hypothesis the algorithm settles on out of $\mathcal{H}$ after seeing $\mathcal{D}$.

Question 4.3.3. Can we then claim 'good learning', that is, $g \approx f$ for the $g = \mathcal{A}(\mathcal{D})$ the algorithm actually returns?

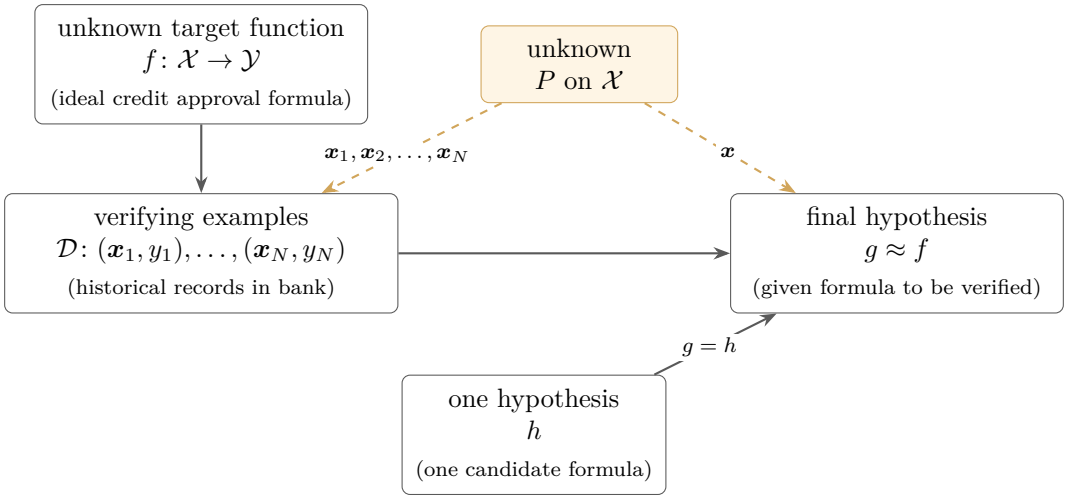
Yes!	**No!**
If $E_{\text{in}}(h)$ is small for the fixed h, and $\mathcal{A}$ picks that h as g, then '$g = f$' is PAC.	If $\mathcal{A}$ is *forced* to pick THE h as g, then $E_{\text{in}}(h)$ is almost always not small, and '$g \neq f$' is PAC!

Both columns describe the same arrangement — one h nominated before the data, returned as g — and differ only in whether $E_{\text{in}}(h)$ turns out small, which is not ours to arrange: an h written down before anyone looks at $\mathcal{D}$ has no reason to fit it. So $E_{\text{in}}(h)$ is almost always large, Theorem 4.3.2 carries *that* over to $E_{\text{out}}(h)$ just as faithfully, and what gets PAC-certified is '$g \neq f$'. The theorem does not fail us; it works perfectly and reports that we are reliably wrong. The obvious repair — read $\mathcal{D}$ first, then return whichever h fits it — is precisely what the premise 'fix h before seeing $\mathcal{D}$' rules out. Reading the data was never the problem, since $E_{\text{in}}(h)$ is computed from it; the problem is that an h chosen that way is a function of $\mathcal{D}$, so it names no one bin whose μ stood fixed while the sample was drawn, and Theorem 4.3.2 has nothing to say about it.

Real learning: $\mathcal{A}$ shall make choices $\in \mathcal{H}$ (like PLA), rather than being forced to pick one h. :-(

§4.3.v. The 'Verification' Flow

What Theorem 4.3.2 does buy is a complete and genuinely useful procedure, one step short of learning. Somebody hands you a formula; you check it against historical records; Hoeffding tells you how much the check is worth.



There is no $\mathcal{A}$ in that picture and no $\mathcal{H}$, only a single given h that becomes g unchanged.

We can now use 'historical records' (data) to verify 'one candidate formula' h.

Exercise 4.3.4 (Fun Time)**.** Your friend tells you her secret rule in investing in a particular stock: 'whenever the stock goes down in the morning, it will go up in the afternoon; vice versa.' To verify the rule, you chose 100 days uniformly at random from the past 10 years of stock data, and found that 80 of them satisfy the rule. What is the best guarantee that you can get from the verification?

(1) You'll definitely be rich by exploiting the rule in the next 100 days.

(2) You'll likely be rich by exploiting the rule in the next 100 days, if the market behaves similarly to the last 10 years.

(3) You'll likely be rich by exploiting the 'best rule' from 20 more friends in the next 100 days.

(4) You'd definitely have been rich if you had exploited the rule in the past 10 years.

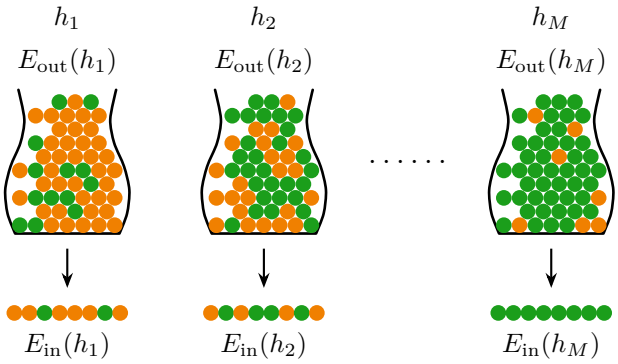
Reference Answer. (2), and every word of the hedge is doing work. (1) is no free lunch. (3) picks the best of 21 rules after looking at the data, which verification says nothing about — that is the subject of Section 4.4. (4) forgets that the verification used only 100 days out of the ten years, so the rule may well be wrong on most of the rest. The clause 'if the market behaves similarly' is the i.i.d. assumption: the next 100 days must be drawn from the same P as the 100 you checked. ■

§4.4. Connection to Real Learning

slides p.19–25

§4.4.i. Multiple h

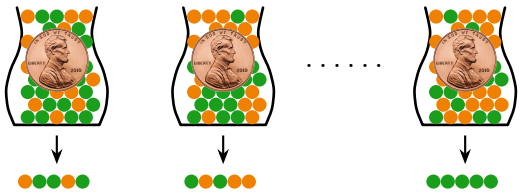
Real learning has M candidates, not one, so there are M bins — one per hypothesis, each with its own $E_{\text{out}}(h_m)$ — and one sample, which is scooped out of all of them at once, because the same $\mathcal{D}$ is used to compute every $E_{\text{in}}(h_m)$.



Question 4.4.1. Real learning, say like PLA: BINGO when getting all green marbles?

§4.4.ii. Coin Game

The temptation is to answer yes, and the following game shows why that is a mistake. It is the same game, with bins replaced by coins: a fair coin is a bin with $\mu = \frac{1}{2}$, and five flips are a sample of size $N = 5$.



Example 4.4.2 (Coin game)**.** Everyone in a size-150 ML class flips a coin 5 times, and one of the students gets 5 heads for her coin 'g'. Is 'g' really magical?

No. Even if all 150 coins are fair, the probability that *one of them* results in 5 heads is

$$1 - \left(\frac{31}{32}\right)^{150} > 99\%.$$

A single fair coin gives five heads with probability $\frac{1}{32}$, which is small; 150 of them give it with probability 99.1%, which is a near certainty. Nothing about any individual coin changed. What changed is that we were allowed to look at all 150 results and then name the coin — and the naming is the part Hoeffding says nothing about.

BAD sample: E_{in} and E_{out} far away — and it can get worse when 'choice' is involved.

§4.4.iii. BAD Sample and BAD Data

Definition 4.4.3 (BAD sample, BAD data)**.** A **BAD sample** is one on which E_{in} and E_{out} are far apart: $E_{\text{out}} = \frac{1}{2}$, say, but all heads came up, so $E_{\text{in}} = 0$. A data set $\mathcal{D}$ is **BAD data for** h when $E_{\text{out}}(h)$ and $E_{\text{in}}(h)$ are far away — for instance $E_{\text{out}}(h)$ big, so h is far from f, while $E_{\text{in}}(h)$ is small, so h is correct on most of the examples.

Intuition . Theorem 4.3.2 holds two things still and lets one vary. Held still are the distribution P on $\mathcal{X}$ and the hypothesis h; what varies is $\mathcal{D}$, drawn i.i.d. from P. In particular $E_{\text{out}}(h)$ does not move at all — it is an expectation over P, fixed by h, f and P alone —

so every bit of the randomness sits in $E_{\text{in}}(h)$. The theorem is therefore a statement about the draw: however $\mathcal{D}$ comes out, as long as N is large enough, $E_{\text{in}}(h)$ lands within ϵ of that one fixed $E_{\text{out}}(h)$ — for all but a $2\exp\left(-2\epsilon^2 N\right)$ fraction of the possible $\mathcal{D}$. Those rare exceptions are exactly the BAD data for h.

That is the whole content of Theorem 4.3.2, restated as a property of the data set rather than of a probability: Hoeffding says the BAD rows of the following table are rare.

	$\mathcal{D}_1$	$\mathcal{D}_2$	…	$\mathcal{D}_{1126}$	…	$\mathcal{D}_{5678}$	Hoeffding
h	**BAD**					**BAD**	$\mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D} \text{ for } h\right] \leq \ldots$

Reading a probability as a sum over data sets makes the next step obvious:

$$\mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D}\right] = \sum_{\text{all possible } \mathcal{D}} \mathbb{P}(\mathcal{D}) \cdot [\![\text{BAD } \mathcal{D}]\!],$$

and Hoeffding says this sum is small.

§4.4.iv. BAD Data for Many h

With M hypotheses there are M rows, each with its own sparse scattering of BAD entries, and the algorithm's freedom is exactly the freedom to avoid them. Say that $\mathcal{D}$ is *BAD for many h* when it is BAD for at least one of them; equivalently, when $\mathcal{A}$ has no 'freedom of choice' on $\mathcal{D}$, because there exists some h whose $E_{\text{out}}(h)$ and $E_{\text{in}}(h)$ are far away.

	$\mathcal{D}_1$	$\mathcal{D}_2$	…	$\mathcal{D}_{1126}$	…	$\mathcal{D}_{5678}$	Hoeffding
h_1	**BAD**					**BAD**	$\mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D} \text{ for } h_1\right] \leq \ldots$
h_2		**BAD**					$\mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D} \text{ for } h_2\right] \leq \ldots$
h_3	**BAD**	**BAD**				**BAD**	$\mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D} \text{ for } h_3\right] \leq \ldots$
…							
h_M	**BAD**					**BAD**	$\mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D} \text{ for } h_M\right] \leq \ldots$
all	**BAD**	**BAD**				**BAD**	?

Question 4.4.4. For M hypotheses, what bounds $\mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D}\right]$?

§4.4.v. Bound of BAD Data

Theorem 4.4.5 (Finite-bin Hoeffding)
Let $\mathcal{H} = \{h_1, \ldots, h_M\}$ be finite and let g be whatever $\mathcal{A}$ returns. For every $\epsilon > 0$,

$$\mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \leq 2M \exp\left(-2\epsilon^2 N\right).$$

Proof. A data set on which g fails is a data set on which *some* h_m fails, whichever h_m the algorithm happened to return, so

$$\begin{aligned}
\mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D}\right] &= \mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D} \text{ for } h_1 \text{ or } \ldots \text{ or BAD } \mathcal{D} \text{ for } h_M\right] \\
&\leq \mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D} \text{ for } h_1\right] + \cdots + \mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D} \text{ for } h_M\right] \qquad \text{(union bound)} \\
&\leq 2\exp\left(-2\epsilon^2 N\right) + \cdots + 2\exp\left(-2\epsilon^2 N\right) \\
&= 2M \exp\left(-2\epsilon^2 N\right),
\end{aligned}$$

the third line by Theorem 4.3.2 applied to each fixed h_m separately. ∎

- The finite-bin version is valid for all M, N and ϵ.
- It does not depend on any $E_{\text{out}}(h_m)$, so there is no need to 'know' them: f and P can stay unknown.
- '$E_{\text{in}}(g) = E_{\text{out}}(g)$' is PAC, *regardless of* $\mathcal{A}$.

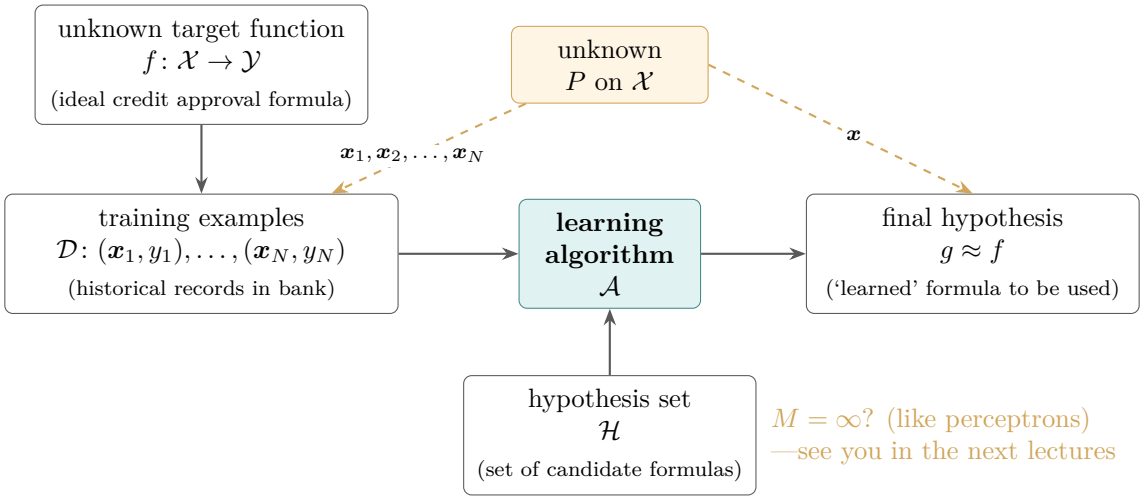
Remark 4.4.6. The union bound is crude — it adds M probabilities as if the bad events never overlapped, when in a typical $\mathcal{H}$ similar hypotheses fail on exactly the same data sets. Paying M for that crudeness is harmless while M is a few thousand and fatal when M is infinite. The next chapters buy the slack back.

The last bullet is what makes the bound useful, because it frees the algorithm completely: whatever rule $\mathcal{A}$ uses to choose, the chosen g is covered. And once every hypothesis is covered, the sensible thing to do is the obvious thing.

The 'most reasonable' $\mathcal{A}$, like PLA or pocket: pick the h_m with the lowest $E_{\text{in}}(h_m)$ as g.

§4.4.vi. The 'Statistical' Learning Flow

Putting the two halves together gives the statement this chapter was after. If $|\mathcal{H}| = M$ is finite and N is large enough, then for whatever g is picked by $\mathcal{A}$ we have $E_{\text{out}}(g) \approx E_{\text{in}}(g)$; and if $\mathcal{A}$ finds one g with $E_{\text{in}}(g) \approx 0$, the PAC guarantee gives $E_{\text{out}}(g) \approx 0$. Learning is possible.



The catch is in the hypothesis of the theorem. $M = |\mathcal{H}|$ must be finite, and the one hypothesis set we own — the perceptrons of Section 2.1 — is a continuum. Theorem 4.4.5 with $M = \infty$ says nothing at all.

Exercise 4.4.7 (Fun Time)**.** Consider 4 hypotheses,

$$h_1(\boldsymbol{x}) = \text{sign}(x_1), \quad h_2(\boldsymbol{x}) = \text{sign}(x_2), \quad h_3(\boldsymbol{x}) = \text{sign}(-x_1), \quad h_4(\boldsymbol{x}) = \text{sign}(-x_2).$$

For any N and ϵ, which of the following statements is *not* true?

(1) the BAD data of h_1 and the BAD data of h_2 are exactly the same;

(2) the BAD data of h_1 and the BAD data of h_3 are exactly the same;

(3) $\mathbb{P}_{\mathcal{D}}[\text{BAD for some } h_k] \leq 8 \exp\left(-2\epsilon^2 N\right)$;

(4) $\mathbb{P}_{\mathcal{D}}\left[\text{BAD for some } h_k\right] \leq 4\exp\left(-2\epsilon^2 N\right)$.

Reference Answer. (1). Settle (2) first, because it decides everything else. Wherever $x_1 \neq 0$ we have $h_3(\boldsymbol{x}) = -h_1(\boldsymbol{x})$, and with $\mathcal{Y} = \{+1, -1\}$ that means h_3 is wrong exactly where h_1 is right:

$$E_{\text{in}}(h_3) = 1 - E_{\text{in}}(h_1), \qquad E_{\text{out}}(h_3) = 1 - E_{\text{out}}(h_1).$$

Subtracting, $E_{\text{in}}(h_3) - E_{\text{out}}(h_3) = -\left(E_{\text{in}}(h_1) - E_{\text{out}}(h_1)\right)$. The two gaps have the same absolute value, so $\mathcal{D}$ is BAD for h_3 exactly when it is BAD for h_1: (2) is true, and the same argument pairs h_4 with h_2.

The four rows of the BAD table are therefore only two distinct rows, and the union bound has two terms to add rather than four:

$$\mathbb{P}_{\mathcal{D}}\left[\text{BAD for some } h_k\right] = \mathbb{P}_{\mathcal{D}}\left[\text{BAD for } h_1 \text{ or BAD for } h_2\right] \leq 2 \cdot 2\exp\left(-2\epsilon^2 N\right).$$

That is (4), and (3) is a weaker bound, so it holds as well. Only (1) is left, and it fails: h_1 and h_2 read different coordinates of $\boldsymbol{x}$, and nothing ties the examples one gets wrong to the examples the other gets wrong.

The moral is worth keeping for the next chapter. Counting hypotheses overcounts BAD data as soon as two hypotheses go wrong on the same data sets — here $M = 4$ while the honest count is 2 — and similar ideas will conquer the $M = \infty$ case. ■

Summary

Learning is impossible? Absolutely no free lunch outside $\mathcal{D}$.

Probability to the rescue Probably approximately correct outside $\mathcal{D}$.

Connection to learning Verification possible if $E_{\text{in}}(h)$ is small for a fixed h.

Connection to real learning Learning possible if $|\mathcal{H}|$ is finite and $E_{\text{in}}(g)$ is small.

The chapter's title keeps its question mark only for the first section. Learning is impossible if the target is arbitrary and the guarantee must be certain; it is possible as soon as we sample i.i.d. from a fixed P and accept a guarantee that is probabilistic, with E_{in} the quantity we minimise and E_{out} the quantity we are actually judged on.

What survives as a debt is the factor M. It is the only place where the size of $\mathcal{H}$ enters, and it is the reason a bigger hypothesis set is not free: more candidates make E_{in} easier to drive down and make the bound worse. The next chapter puts that trade-off at the centre — and starts by replacing M with something that stays finite even when $\mathcal{H}$ does not.

5. Training versus Testing

Lecture 5

23 Sep.

> **As previously seen .** Theorem 4.4.5 closed the last chapter: if $\mathcal{H} = \{h_1, \dots, h_M\}$ is finite and N is large enough, then
>
> $$\mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \leq 2M \exp\left(-2\epsilon^2 N\right)$$
>
> for whatever g the algorithm picks — so if $\mathcal{A}$ also finds a g with $E_{\text{in}}(g) \approx 0$, we get the PAC guarantee $E_{\text{out}}(g) \approx 0$ and learning is possible.

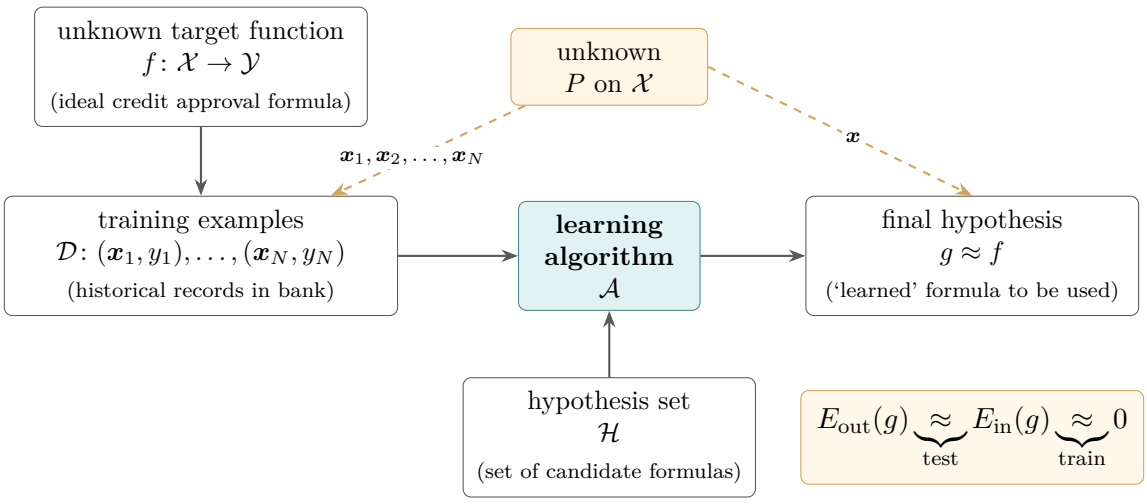
But the perceptrons of Section 2.1 form an infinite set: $M = \infty$ and the bound says nothing. This chapter looks for a finite quantity to put in place of M. The plan is to find where M enters the proof — a union bound over one BAD event per hypothesis — observe that similar hypotheses have almost the same BAD event, and so count hypotheses only up to what N inputs can tell apart. That count is finite, and the last section asks when it grows only polynomially in N.

§5.1. Recap and Preview

slides p.2–6

§5.1.i. The 'Statistical' Learning Flow

We start from where Section 4.4 ended: the learning flow, now with its conclusion attached.



The two $\approx$ signs are achieved in different ways. The right-hand one, $E_{\text{in}}(g) \approx 0$, is about the training data: E_{in} is computed on $\mathcal{D}$, which we have, so the algorithm can measure it and push it down directly — that is what PLA and pocket do. The left-hand one, $E_{\text{out}}(g) \approx E_{\text{in}}(g)$, is about future test data: E_{out} depends on the unknown P, so it cannot be computed, and no work on $\mathcal{D}$ can check it. We can only rely on Theorem 4.4.5, which says it holds with high probability when N is large and M is finite. Learning therefore splits into two separate tasks.

§5.1.ii. Two Central Questions

Each link of the chain has been supplied by an earlier chapter:

$$\underbrace{\text{for batch \& supervised binary classification,}}_{\text{Ch. 3}} \quad \underbrace{g \approx f}_{\text{Ch. 1}} \Leftrightarrow E_{\text{out}}(g) \approx 0,$$

$$\text{achieved through} \quad \underbrace{E_{\text{out}}(g) \approx E_{\text{in}}(g)}_{\text{Ch. 4}} \quad \text{and} \quad \underbrace{E_{\text{in}}(g) \approx 0}_{\text{Ch. 2}}.$$

Chapter 3 fixed the setting and Chapter 1 the goal; Chapter 4 handled $E_{\text{out}} \approx E_{\text{in}}$ and Chapter 2 (PLA) handled $E_{\text{in}} \approx 0$. So learning comes down to two questions:

1. Can we make sure that $E_{\text{out}}(g)$ is close enough to $E_{\text{in}}(g)$?
2. Can we make $E_{\text{in}}(g)$ small enough?

Question 5.1.1. What role does $M = |\mathcal{H}|$ play for the two questions?

§5.1.iii. Trade-off on M

Recall . $\mathcal{D}$ is *BAD* (Definition 4.4.3) when it makes some hypothesis look better than it is — E_{in} and E_{out} far apart, the training error no guide to the real one. Theorem 4.4.5 bounds how often a draw of N examples does that:

$$\mathbb{P}_{\mathcal{D}}\left[\text{BAD } \mathcal{D}\right] = \mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \leq 2 \cdot M \cdot \exp\left(-2\epsilon^2 N\right),$$

so the first of the two questions is answered by making this quantity small.

M pulls the two questions in opposite directions:

small M	**large** M
1. Yes!, $\mathbb{P}\left[\text{BAD}\right] \leq 2 \cdot M \cdot \exp(\ldots)$;	1. No!, $\mathbb{P}\left[\text{BAD}\right] \leq 2 \cdot M \cdot \exp(\ldots)$;
2. No!, too few choices.	2. Yes!, many choices.

A small $\mathcal{H}$ gives a tight bound but may not contain a good fit; a large $\mathcal{H}$ fits easily but its bound is loose.

Using the right M (or $\mathcal{H}$) is important.

Question 5.1.2. Is $M = \infty$ doomed?

§5.1.iv. Preview

What we know is

$$\mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \leq 2 \cdot M \cdot \exp\left(-2\epsilon^2 N\right),$$

and the plan for the next three chapters is to keep this form while replacing the factor that becomes infinite:

- establish a finite quantity $m_{\mathcal{H}}$ that replaces M,

$$\mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \overset{?}{\leq} 2 \cdot m_{\mathcal{H}} \cdot \exp\left(-2\epsilon^2 N\right); \tag{5.1}$$

- justify the feasibility of learning for infinite M;
- study $m_{\mathcal{H}}$ to understand its trade-off for choosing the 'right' $\mathcal{H}$, just as we would for M.

This chapter only constructs $m_{\mathcal{H}}$ and computes it for a few hypothesis sets; the inequality itself, hence the '?', is proved two chapters from now.

Note 5.1.3. The mysterious PLA is to be fully resolved after three more lectures, once the bound covers the infinite perceptron set.

Exercise 5.1.4 (Fun Time). *Data size: how large do we need?* One way to use the inequality

$$\mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \leq \underbrace{2 \cdot M \cdot \exp\left(-2\epsilon^2 N\right)}_{\delta}$$

is to pick a tolerable difference ϵ as well as a tolerable BAD probability δ, and then gather data with size N large enough to achieve those tolerance criteria. Let $\epsilon = 0.1$, $\delta = 0.05$ and $M = 100$. What is the data size needed? (1) 215; (2) 415; (3) 615; (4) 815.

Reference Answer. (2). We can simply express N as a function of those 'known' variables: $2M \exp\left(-2\epsilon^2 N\right) \leq \delta$ rearranges to

$$N \geq \frac{1}{2\epsilon^2} \ln \frac{2M}{\delta} = \frac{1}{2(0.1)^2} \ln \frac{200}{0.05} = 50 \ln 4000 \approx 414.7,$$

so $N = 415$ is the first of the four that works. ■

§5.2. Effective Number of Lines

slides p.7–15

To replace M, we first look at where it came from.

§5.2.i. Where Did M Come From?

Recall the proof of $\mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \leq 2M \exp\left(-2\epsilon^2 N\right)$. For each hypothesis there is a BAD event

$$\mathcal{B}_m: \quad |E_{\text{in}}(h_m) - E_{\text{out}}(h_m)| > \epsilon,$$

and because $\mathcal{A}$ must be free to return any h_m it likes, the quantity to bound is

$$\mathbb{P}\left[\mathcal{B}_1 \text{ or } \mathcal{B}_2 \text{ or } \ldots \text{ or } \mathcal{B}_M\right].$$

We bounded it by assuming the worst case — that all $\mathcal{B}_m$ are non-overlapping:

$$\mathbb{P}\left[\mathcal{B}_1 \text{ or } \mathcal{B}_2 \text{ or } \ldots \text{ or } \mathcal{B}_M\right] \underbrace{\leq}_{\text{union bound}} \mathbb{P}\left[\mathcal{B}_1\right] + \mathbb{P}\left[\mathcal{B}_2\right] + \cdots + \mathbb{P}\left[\mathcal{B}_M\right].$$

Each summand is at most $2\exp\left(-2\epsilon^2 N\right)$ by Hoeffding, so M is simply the number of terms in this sum.

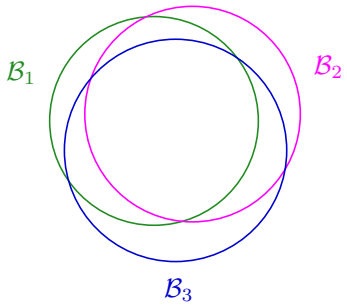
Question 5.2.1. What did the union bound fail to consider when $M = \infty$?

§5.2.ii. Where Did the Union Bound Fail?

The union bound is tight only when the events are disjoint. For similar hypotheses they are not: take $h_1 \approx h_2$, two lines a hair apart. Then

1. $E_{\text{out}}(h_1) \approx E_{\text{out}}(h_2)$, because the two hypotheses disagree on a set of tiny probability; and
2. for most $\mathcal{D}$, $E_{\text{in}}(h_1) = E_{\text{in}}(h_2)$, because the finitely many sample points almost never fall in the sliver where the two disagree.

Hence $\mathcal{B}_1$ and $\mathcal{B}_2$ are almost the same event, and the union bound counts their overlap twice.

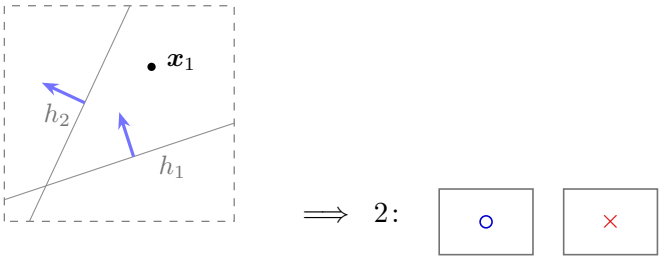


With infinitely many lines this over-counting is infinite, which is exactly why the bound became useless.

Question 5.2.2. To account for the overlap, can we group similar hypotheses by *kind*?

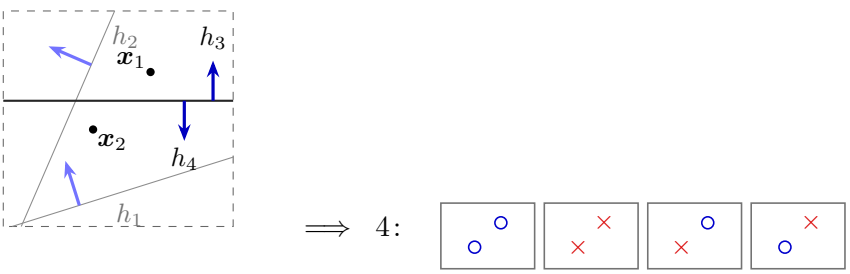
§5.2.iii. How Many Lines Are There?

To group lines by kind, take $\mathcal{H} = \{\text{all lines in } \mathbb{R}^2\}$ (each line calls one side $\circ$ and the other $\times$). There are infinitely many lines, but on a few fixed inputs they can only produce finitely many labellings. We count those, starting from one input.



The arrow on each line points to its positive side. h_1 puts $\boldsymbol{x}_1$ on the positive side and h_2 does not, so on one input there are two kinds of line: h_1-like ($\boldsymbol{x}_1 = \circ$) or h_2-like ($\boldsymbol{x}_1 = \times$).

With two inputs:

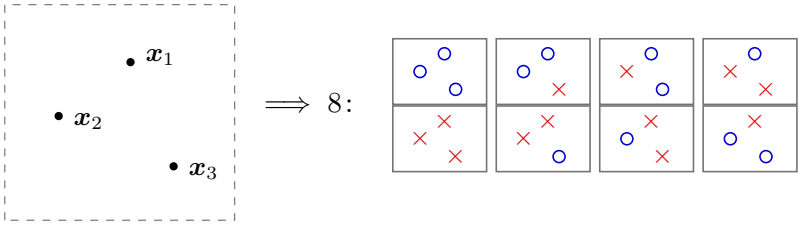


h_1 leaves both points on its positive side and h_2 leaves both on its negative side; the horizontal line between them, read upwards as h_3 and downwards as h_4, splits them both ways. That gives four kinds, all 2^2 labellings.

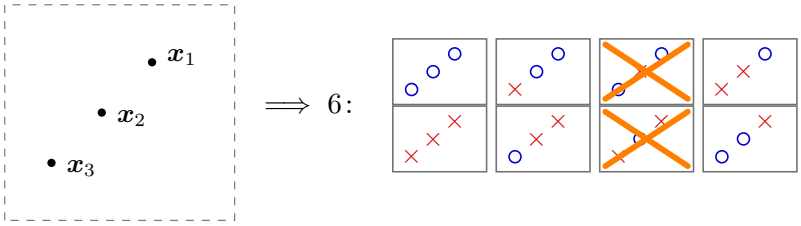
Question 5.2.3. One input: 2; two inputs: 4. Three inputs?

§5.2.iv. Three Inputs, and Why the Answer Is Not One Number

For three inputs forming a triangle, all 2^3 labellings are achievable, since a line can cut off any one vertex (or, read the other way, any two).



If the three inputs are collinear, two of the eight are lost.

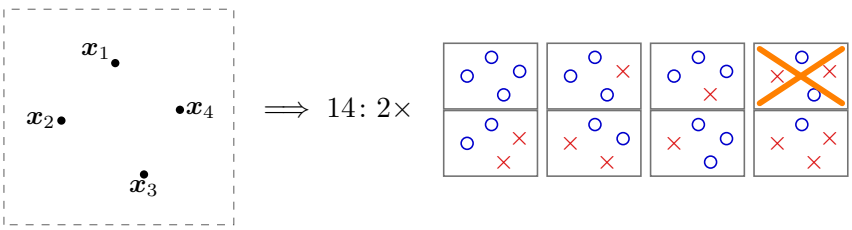


Observation . The two struck-out panels are the two that ask the middle point to differ from both of its neighbours. A line meets the segment $\boldsymbol{x}_1\boldsymbol{x}_3$ at most once, so it can never put $\boldsymbol{x}_2$ alone on one side of it. 'Fewer than 8' is what degeneracy — collinear inputs, or repeated ones — costs.

So the count depends on where the inputs are: 8 or 6. Since the bound must hold for every data set, we will use the largest count, 8.

§5.2.v. Four Inputs

With four inputs the count falls below 2^N for the first time. There are $2^4 = 16$ labellings in theory, but the figure draws only the eight with $\boldsymbol{x}_1 = \circ$: reading a line from its other side swaps every $\circ$ and $\times$, so labellings come in complementary pairs and each panel stands for itself and its mirror image. That is the '2×'.



Only the struck-out panel ($\boldsymbol{x}_1, \boldsymbol{x}_3$ against $\boldsymbol{x}_2, \boldsymbol{x}_4$) and its mirror image are impossible, leaving 14. This is not special to the picture:

Proposition 5.2.4 (At most 14 through four inputs)

For any four inputs $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_4 \in \mathbb{R}^2$, lines realise at most 14 of the $2^4 = 16$ labellings, and 14 is attained.

Proof. Radon's theorem says that any four points of $\mathbb{R}^2$ can be partitioned into two groups whose convex hulls intersect. Fix such a partition and colour one group $\circ$ and the other $\times$. A line realising that labelling would separate the two groups, hence separate their convex hulls — impossible, since the hulls share a point. The same partition with the colours exchanged is a second labelling ruled out for the same reason, and it is a different labelling because a group and its complement are never equal. So at least two of the 16 are missing.

The picture above attains the bound: the four points are in convex position in the cyclic order $\boldsymbol{x}_1, \boldsymbol{x}_4, \boldsymbol{x}_3, \boldsymbol{x}_2$, the intersecting hulls are the two diagonals $\boldsymbol{x}_1\boldsymbol{x}_3$ and $\boldsymbol{x}_2\boldsymbol{x}_4$, and every other labelling is cut off by a line. ■

§5.2.vi. Effective Number of Lines

Definition 5.2.5 (Effective number of lines)**.** The **effective number of lines** for N inputs is the maximum number of kinds of lines with respect to any N inputs $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N$, written effective$(N)$.

This is the grouping asked for in Question 5.2.2: it is finite, and at most 2^N since each kind is a labelling of N inputs. So far:

lines in 2D	
N	effective(N)
1	2
2	4
3	8
4	$14 < 2^N$

If the union bound could be taken over kinds instead of hypotheses, we would get

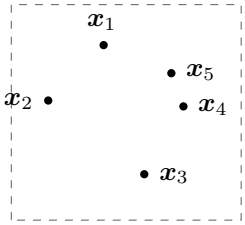
$$\mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \leq 2 \cdot \text{effective}(N) \cdot \exp\left(-2\epsilon^2 N\right).$$

If ① effective(N) can replace M and ② effective$(N) \ll 2^N$, then learning is possible with infinitely many lines. :-)

Intuition . Condition ② is needed because $\exp\left(-2\epsilon^2 N\right)$ decays exponentially: a factor like 2^N could cancel it, while a polynomial factor cannot, and the bound still goes to 0.

Exercise 5.2.6 (Fun Time)**.** What is the effective number of lines for five inputs in $\mathbb{R}^2$? (1) 14; (2) 16; (3) 22; (4) 32.

Reference Answer. (3). If you put the inputs roughly around a circle,



you can then pick any *consecutive* run of inputs to be on one side of the line, and the rest on the other. There are $5 \cdot 4 = 20$ proper runs — five starting points times four lengths — plus the two labellings that use no line at all, all $\circ$ and all $\times$, for a total of 22 kinds, much smaller than $2^5 = 32$. You will find it difficult to generate more kinds by varying the inputs, and a formal proof comes in a later chapter. ■

§5.3. Effective Number of Hypotheses

slides p.16–22

Nothing in Section 5.2 used the geometry of lines, so the same counting applies to any $\mathcal{H}$. This section gives it general names and computes it for a few examples.

§5.3.i. Dichotomies: Mini-hypotheses

Let $\mathcal{H}$ be any set of hypotheses $h\colon \mathcal{X} \to \{\times, \circ\}$.

Definition 5.3.1 (Dichotomy)**.** For fixed inputs $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$, write

$$h(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N) = (h(\boldsymbol{x}_1), h(\boldsymbol{x}_2), \ldots, h(\boldsymbol{x}_N)) \in \{\times, \circ\}^N ,$$

and call it a **dichotomy**: the hypothesis 'limited' to the eyes of the N inputs. Write

$$\mathcal{H}(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N)$$

for the set of all dichotomies implemented by $\mathcal{H}$ on them.

> **Notation .** A dichotomy is written as a string, one symbol per input in index order: $\circ\circ\times\circ$ is the dichotomy that answers $\circ$ on $\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_4$ and $\times$ on $\boldsymbol{x}_3$. The panels of Section 5.2 are exactly such strings, drawn.

A dichotomy is what is left of h when only N inputs are observed, and 'grouping hypotheses by kind' now means grouping them by dichotomy. This is the right grouping for the union bound because of the following.

> **Lemma 5.3.2** (Same dichotomy, same E_{in})
> If h and h' implement the same dichotomy on the inputs of $\mathcal{D}$, then $E_{\text{in}}(h) = E_{\text{in}}(h')$ — not approximately, exactly.

Proof. $E_{\text{in}}(h) = \frac{1}{N}\sum_{n=1}^{N} [\![h(\boldsymbol{x}_n) \neq y_n]\!]$ by Definition 4.3.1, and by hypothesis $h(\boldsymbol{x}_n) = h'(\boldsymbol{x}_n)$ for every n, so the two sums have the same terms. ■

Hypotheses with the same dichotomy thus have the same E_{in} and, being similar, nearly the same E_{out}, so their BAD events nearly coincide — the overlap of Question 5.2.1. Taking one term per dichotomy instead of one per hypothesis reduces the number of terms in the union bound from $|\mathcal{H}|$ to $|\mathcal{H}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)| \leq 2^N$.

	hypotheses $\mathcal{H}$	dichotomies $\mathcal{H}(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N)$
e.g.	all lines in $\mathbb{R}^2$	$\{\circ\circ\circ\circ, \circ\circ\circ\times, \circ\circ\times\times, \ldots\}$
size	possibly infinite	upper bounded by 2^N

$|\mathcal{H}(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N)|$ is our candidate for replacing M.

Note 5.3.3. This is a motivation, not a proof, and two gaps remain. First, $\mathcal{H}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)$ depends on the random inputs; the next subsection removes this dependence. Second, a dichotomy fixes E_{in} exactly but E_{out} only approximately, so hypotheses in a group do not share exactly one BAD event. This second gap is the '?' in Equation 5.1, closed two chapters from now.

§5.3.ii. Growth Function

The first gap is easy to close. $|\mathcal{H}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)|$ depends on the inputs (8 or 6 for three points), so take the worst case:

Definition 5.3.4 (Growth function)**.** The **growth function** of $\mathcal{H}$ is

$$m_{\mathcal{H}}(N) = \max_{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N \in \mathcal{X}} |\mathcal{H}(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N)|.$$

It is at most 2^N, finite however large $\mathcal{H}$ is, and for lines in the plane it is exactly effective(N):

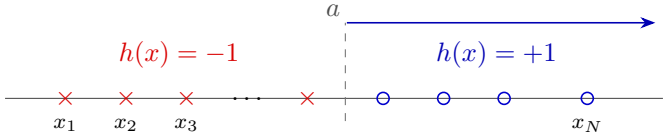
lines in 2D	
N	$m_{\mathcal{H}}(N)$
1	2
2	4
3	$\max(\ldots, 6, 8) = 8$
4	$14 < 2^N$

Question 5.3.5. How does one actually 'calculate' the growth function?

For perceptrons we cannot yet, so we first compute it for three simpler hypothesis sets.

§5.3.iii. Growth Function for Positive Rays

Let $\mathcal{X} = \mathbb{R}$ be one dimensional and let $\mathcal{H}$ contain every $h(x) = \text{sign}(x - a)$ for a threshold $a \in \mathbb{R}$ — the 'positive half' of the 1D perceptrons.



Proposition 5.3.6 (Positive rays)
$m_{\mathcal{H}}(N) = N + 1$.

Proof. Sort the inputs as $x_1 < x_2 < \cdots < x_N$. The dichotomy produced by threshold a depends only on how many inputs lie below a, so all thresholds inside one gap (x_n, x_{n+1}) give the same dichotomy, and different gaps give different ones. There are $N+1$ gaps — the $N-1$ internal ones together with $(-\infty, x_1)$ and (x_N, ∞) — hence $N+1$ dichotomies, and no choice of inputs does better, since repeated inputs only merge gaps. ■

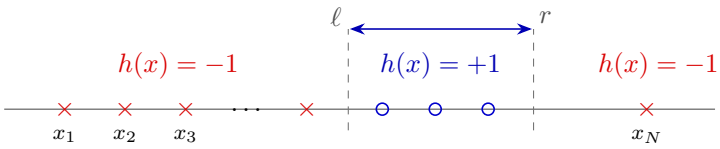
For $N = 4$, the 5 dichotomies:

x_1	x_2	x_3	x_4
○	○	○	○
×	○	○	○
×	×	○	○
×	×	×	○
×	×	×	×

$(N+1) \ll 2^N$ **when** N **is large.**

§5.3.iv. Growth Function for Positive Intervals

Still with $\mathcal{X} = \mathbb{R}$, let $\mathcal{H}$ contain every h with $h(x) = +1$ if and only if $x \in [\ell, r)$, and -1 otherwise.



Proposition 5.3.7 (Positive intervals)
$m_{\mathcal{H}}(N) = \binom{N+1}{2} + 1 = \frac{1}{2}N^2 + \frac{1}{2}N + 1.$

Proof. As before, only the gaps matter: a dichotomy is decided by which of the $N+1$ spots holds ℓ and which holds r. Choosing two *different* spots for the two ends, with ℓ in the earlier one, gives $\binom{N+1}{2}$ distinct dichotomies, each with a non-empty run of ○. Putting both ends in the same spot makes the interval miss every input and produces the all-× dichotomy, one further case however the spot is chosen. Adding the two counts gives $\binom{N+1}{2} + 1$. ■

For $N = 4$, the $10 + 1 = 11$ dichotomies:

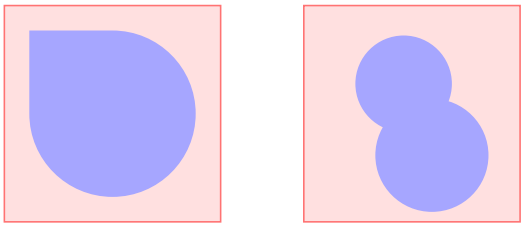
x_1	x_2	x_3	x_4
○	×	×	×
○	○	×	×
○	○	○	×
○	○	○	○
×	○	×	×
×	○	○	×

x_1	x_2	x_3	x_4
×	○	○	○
×	×	○	×
×	×	○	○
×	×	×	○
×	×	×	×

$\left(\frac{1}{2}N^2 + \frac{1}{2}N + 1\right) \ll 2^N$ **when** N **is large.**

§5.3.v. Growth Function for Convex Sets

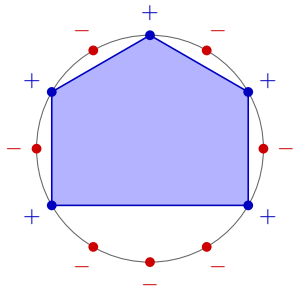
The last example reaches the ceiling 2^N. Let $\mathcal{X} = \mathbb{R}^2$ and let $\mathcal{H}$ contain every h with $h(\boldsymbol{x}) = +1$ if and only if $\boldsymbol{x}$ lies in a convex region, and -1 otherwise. Blue marks the region a hypothesis calls $+1$; the red surround is everything it calls -1.



convex region in blue non-convex region

Question 5.3.8. What is $m_{\mathcal{H}}(N)$ for convex sets?

Since $m_{\mathcal{H}}$ takes the best inputs, put $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$ on a big circle:



Proposition 5.3.9 (Convex sets)
$m_{\mathcal{H}}(N) = 2^N$.

Proof. Place the N inputs on a circle and fix any dichotomy. Take the convex hull of the inputs labelled $+1$ and extend it very slightly. Every positive input is inside by construction. Every negative input is outside, because a point of a circle is a vertex of the convex hull of any subset of that circle containing it — so it cannot lie in the hull of a subset that excludes it. The region is convex, so it is a legitimate h, and it implements the chosen dichotomy. All 2^N dichotomies are therefore implemented by these inputs, and 2^N is also the ceiling. ■

Definition 5.3.10 (Shattering)**.** When $\mathcal{H}$ implements every one of the 2^N dichotomies on $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$, those N inputs are **shattered** by $\mathcal{H}$.

$m_{\mathcal{H}}(N) = 2^N$ if and only if there exist N inputs that can be shattered.

Note the quantifiers: $m_{\mathcal{H}}(N) = 2^N$ needs only *some* N inputs to be shattered, whereas $m_{\mathcal{H}}(N) < 2^N$ says that *no* N inputs are.

Exercise 5.3.11 (Fun Time)**.** Consider positive *and* negative rays as $\mathcal{H}$, which is equivalent to the perceptron hypothesis set in 1D. The hypothesis set is often called a 'decision stump', to describe

the shape of its hypotheses. What is the growth function $m_{\mathcal{H}}(N)$? (1) N; (2) $N+1$; (3) $2N$; (4) 2^N.

Reference Answer. (3). Two dichotomies when the threshold falls in each of the $N-1$ 'internal' spots, one for each reading of the sign, giving $2(N-1)$; then two more for the all-$\circ$ and all-$\times$ cases, which the two outside spots produce whichever way they are read. In total $2(N-1)+2 = 2N$. ■

§5.4. Break Point

slides p.23–26

§5.4.i. The Four Growth Functions

Collecting the growth functions computed so far:

- positive rays: $m_{\mathcal{H}}(N) = N+1$;
- positive intervals: $m_{\mathcal{H}}(N) = \frac{1}{2}N^2 + \frac{1}{2}N + 1$;
- convex sets: $m_{\mathcal{H}}(N) = 2^N$;
- 2D perceptrons: $m_{\mathcal{H}}(N) < 2^N$ in some cases.

Suppose Equation 5.1 were granted and $m_{\mathcal{H}}(N)$ took the place of M:

$$\mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \overset{?}{\leq} 2 \cdot m_{\mathcal{H}}(N) \cdot \exp\left(-2\epsilon^2 N\right).$$

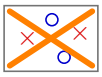
Since $\exp\left(-2\epsilon^2 N\right)$ decays exponentially, a polynomial $m_{\mathcal{H}}(N)$ still lets the bound go to 0 (rays, intervals), while 2^N does not (convex sets).

Polynomial: good; exponential: bad.

Question 5.4.1. For 2D, or general, perceptrons — is $m_{\mathcal{H}}(N)$ polynomial?

§5.4.ii. Break Point of $\mathcal{H}$

We do not know $m_{\mathcal{H}}(N)$ for 2D perceptrons, but we know that some three inputs can be shattered while *no* four inputs can (Proposition 5.2.4).



The second fact is the useful one, and we name it.

Definition 5.4.2 (Break point)**.** If no k inputs can be shattered by $\mathcal{H}$, then k is a **break point** for $\mathcal{H}$.

Immediately:

- $m_{\mathcal{H}}(k) < 2^k$, by Definition 5.3.10.

- $k+1, k+2, k+3, \ldots$ are break points too.
- So the informative quantity is the *minimum* break point, and it is the one we shall study.

Proof of the second bullet. Suppose some $k+1$ inputs were shattered. Delete one of them and restrict every dichotomy to the remaining k: each of the 2^k dichotomies on those k inputs is the restriction of at least one of the 2^{k+1} dichotomies above it, so the k inputs are shattered as well, contradicting the assumption that k is a break point. Induction gives every larger k. ■

2D perceptrons have a break point at 4.

§5.4.iii. The Four Break Points

The four hypothesis sets with their break points:

$\mathcal{H}$	$m_{\mathcal{H}}(N)$	minimum break point
positive rays	$N+1 = O(N)$	2
positive intervals	$\frac{1}{2}N^2 + \frac{1}{2}N + 1 = O(N^2)$	3
convex sets	2^N	none
2D perceptrons	$< 2^N$ in some cases	4

(Rays: $m_{\mathcal{H}}(2) = 3 < 4$; intervals: $m_{\mathcal{H}}(3) = 7 < 8$.) Break point 2 comes with $O(N)$, break point 3 with $O(N^2)$, and no break point with 2^N. This suggests:

Conjecture 5.4.1. No break point: $m_{\mathcal{H}}(N) = 2^N$ — which is certain, being the contrapositive of Definition 5.4.2. Break point k: $m_{\mathcal{H}}(N) = O\big(N^{k-1}\big)$.

If true, a break point alone would guarantee polynomial growth, and hence, via Equation 5.1, feasible learning for 2D perceptrons. Proving it is the next chapter.

Exercise 5.4.3 (Fun Time)**.** Consider positive and negative rays as $\mathcal{H}$, the 1D perceptron set, whose growth function is $m_{\mathcal{H}}(N) = 2N$. What is the minimum break point for $\mathcal{H}$? (1) 1; (2) 2; (3) 3; (4) 4.

Reference Answer. (3). At $k = 3$ we get $m_{\mathcal{H}}(k) = 6$ while $2^k = 8$, so 3 is a break point; and it is minimal because $m_{\mathcal{H}}(2) = 4 = 2^2$ and $m_{\mathcal{H}}(1) = 2 = 2^1$ leave nothing broken below it. Note that Conjecture 5.4.1 predicts $O(N^2)$ here while the truth is $2N$ — the conjecture is an upper bound, not an estimate. ■

Summary

Recap and Preview Two questions: $E_{\text{out}}(g) \approx E_{\text{in}}(g)$, and $E_{\text{in}}(g) \approx 0$.

Effective Number of Lines At most 14 through the eye of 4 inputs.

Effective Number of Hypotheses At most $m_{\mathcal{H}}(N)$ through the eye of N inputs.

Break Point When $m_{\mathcal{H}}(N)$ becomes 'non-exponential'.

Training makes E_{in} small; the bound connecting E_{in} to E_{out} carried a factor M, and we have proposed replacing it by $m_{\mathcal{H}}(N)$, which counts dichotomies instead of hypotheses and is finite even for infinite $\mathcal{H}$.

Two things remain for the next two chapters: proving Conjecture 5.4.1, that a break point k gives $m_{\mathcal{H}}(N) = O(N^{k-1})$; and removing the '?' of Equation 5.1 from

$$\mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \overset{?}{\leq} 2 \cdot m_{\mathcal{H}}(N) \cdot \exp\left(-2\epsilon^2 N\right),$$

Together they give a guarantee for the perceptron.

6. Theory of Generalization

Lecture 6

30 Sep.

> **As previously seen .** Section 5.4 ended with the growth function $m_{\mathcal{H}}(N)$, the number of dichotomies $\mathcal{H}$ can implement on the best N inputs, as the (wishful) effective price of choice in training, and with Conjecture 5.4.1: a break point k should force $m_{\mathcal{H}}(N) = O(N^{k-1})$. Two things were left open — proving that conjecture, and removing the '?' from
>
> $$\mathbb{P}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \overset{?}{\leq} 2 \cdot m_{\mathcal{H}}(N) \cdot \exp\left(-2\epsilon^2 N\right).$$

This chapter does both. The first three sections prove the conjecture without ever looking at a particular $\mathcal{H}$: a break point is a purely combinatorial restriction on which tables of $\circ$ and $\times$ may occur, and counting the largest such table gives a polynomial. The last section sketches why $m_{\mathcal{H}}$, with a few changes of constant, really can stand in for M.

§6.1. Restriction of Break Point

slides p.2–5

§6.1.i. The Four Break Points

The four hypothesis sets of the last chapter, each with the smallest input configuration it fails to shatter:

$\mathcal{H}$	$m_{\mathcal{H}}(N)$	inputs not shattered	break point
positive rays	$N+1$		$m_{\mathcal{H}}(2) = 3 < 2^2$: break point at 2
positive intervals	$\frac{1}{2}N^2 + \frac{1}{2}N + 1$		$m_{\mathcal{H}}(3) = 7 < 2^3$: break point at 3
convex sets	2^N		$m_{\mathcal{H}}(N) = 2^N$ always: no break point
2D perceptrons	$< 2^N$ in some cases		$m_{\mathcal{H}}(4) = 14 < 2^4$: break point at 4

The third column is the dichotomy that fails: a ray cannot say $\circ$ left of $\times$, an interval cannot leave a hole in the middle, a line cannot do XOR. Convex sets never fail, whatever the input.

Section 5.4 already showed one consequence of a break point: break point $k \Rightarrow$ break point $k+1, \ldots$ That concerns only the values $m_{\mathcal{H}}(N) < 2^N$ at $N \geq k$.

> **Question 6.1.1.** What else does a break point imply?

§6.1.ii. What Must Be True When $k = 2$

Suppose the *minimum* break point is $k = 2$. Without knowing $\mathcal{H}$, a few values of $m_{\mathcal{H}}$ are forced:

- $N = 1$: every $m_{\mathcal{H}}(N) = 2$ by definition, since the minimum break point is above 1, so some single input is shattered;

- $N = 2$: every $m_{\mathcal{H}}(N) < 4$ by definition, since 2 is a break point (so the maximum possible is 3).

Question 6.1.2. What is the maximum possible $m_{\mathcal{H}}(N)$ when $N = 3$ and $k = 2$?

The only rule is that no two of the three inputs may be shattered: in the table of dichotomies, no pair of columns may show all four patterns $\circ\circ, \circ\times, \times\circ, \times\times$. So build the table row by row and reject any row that breaks the rule.

- $\circ\,\circ\,\circ$: 1 dichotomy; shatters any two points? No.
- add $\circ\,\circ\,\times$: 2 dichotomies; no.
- add $\circ\,\times\,\circ$: 3 dichotomies; no.
- add $\circ\,\times\,\times$: 4 dichotomies, but now columns $\boldsymbol{x}_2, \boldsymbol{x}_3$ read $\circ\circ, \circ\times, \times\circ, \times\times$ — shattered. Rejected.
- add $\times\,\circ\,\circ$ instead: 4 dichotomies; no pair shattered.
- a fifth row: each of the three remaining candidates shatters some pair (table below).

$\boldsymbol{x}_1$	$\boldsymbol{x}_2$	$\boldsymbol{x}_3$	
$\circ$	$\circ$	$\circ$	kept
$\circ$	$\circ$	$\times$	kept
$\circ$	$\times$	$\circ$	kept
$\times$	$\circ$	$\circ$	kept
$\circ$	$\times$	$\times$	4th row tried: shatters $\boldsymbol{x}_2, \boldsymbol{x}_3$
$\times$	$\circ$	$\times$	5th row tried: shatters $\boldsymbol{x}_1, \boldsymbol{x}_3$
$\times$	$\times$	$\circ$	5th row tried: shatters $\boldsymbol{x}_1, \boldsymbol{x}_2$
$\times$	$\times$	$\times$	5th row tried: shatters $\boldsymbol{x}_1, \boldsymbol{x}_2$ (every pair, in fact)

The four kept rows use up the table: $\circ\times\times$ was already refused, and $\times\circ\times$, $\times\times\circ$, $\times\times\times$ each complete the four patterns on some pair, so no column pair has room left. The maximum possible so far is 4 dichotomies. The search here started from $\circ\,\circ\,\circ$ and so is not a proof that no other table of five exists; that proof comes for free from Theorem 6.3.3 below, which gives $B(3, 2) \leq 4$.

§6.1.iii. Restriction of Break Point

Putting the three cases together, when the minimum break point is $k = 2$:

- $N = 1$: every $m_{\mathcal{H}}(N) = 2$ by definition;
- $N = 2$: every $m_{\mathcal{H}}(N) < 4$ by definition (so maximum possible $= 3$);
- $N = 3$: maximum possible $= 4 \ll 2^3$.

At $N = 3$ the break point already bites twice as hard as the definition asks: the definition only forbids shattering two points, yet it caps three points at half of 2^3.

A break point k restricts the maximum possible $m_{\mathcal{H}}(N)$ a lot for $N > k$.

This is the answer to Question 6.1.1, and it suggests how to prove Conjecture 5.4.1 without knowing anything else about $\mathcal{H}$:

$$m_{\mathcal{H}}(N) \leq \text{maximum possible } m_{\mathcal{H}}(N) \text{ given } k \leq \text{poly}(N). \tag{6.1}$$

The first inequality is trivial; the work is in the second.

Exercise 6.1.3 (Fun Time)**.** When the minimum break point is $k = 1$, what is the maximum possible $m_{\mathcal{H}}(N)$ when $N = 3$? (1) 1; (2) 2; (3) 4; (4) 8.

Reference Answer. (1). Because $k = 1$, the hypothesis set cannot even shatter one point. Thus, every 'column' of the table cannot contain both $\circ$ and $\times$. Then, after including the first dichotomy, it is not possible to include any other different dichotomy:

$\boldsymbol{x}_1$	$\boldsymbol{x}_2$	$\boldsymbol{x}_3$	
$\circ$	$\times$	$\circ$	kept
$\circ$	$\times$	$\times$	rejected: column $\boldsymbol{x}_3$ now shattered

Any second row differs from the first somewhere, and that column then shows both symbols. Thus, the maximum possible $m_{\mathcal{H}}(N)$ is 1. ■

§6.2. Bounding Function: Basic Cases

slides p.6–11

The middle term of Equation 6.1 deserves a name.

§6.2.i. Bounding Function

Definition 6.2.1 (Bounding function)**.** The **bounding function** $B(N, k)$ is the maximum possible $m_{\mathcal{H}}(N)$ when k is a break point.

Two remarks make this concrete.

- It is a *combinatorial quantity*: the maximum number of length-N vectors with entries in $\{\circ, \times\}$ such that no length-k subvector is shattered — that is, no choice of k coordinates shows all 2^k patterns.
- It is *irrelevant of the details of* $\mathcal{H}$. For example, $B(N, 3)$ bounds both positive intervals ($k = 3$) and 1D perceptrons ($k = 3$).

The first remark is why the second holds: the dichotomies of any $\mathcal{H}$ with break point k, on any N inputs, form such a set of vectors, so

$$m_{\mathcal{H}}(N) \leq B(N, k)$$

for every $\mathcal{H}$ with break point k. The price is looseness — B takes the worst table, whether or not any $\mathcal{H}$ realises it.

Question 6.2.2. New goal: is $B(N, k) \leq \text{poly}(N)$?

§6.2.ii. Table of Bounding Function

We fill in a table of $B(N, k)$, rows N, columns k. What is known so far is:

- $B(2, 2) = 3$ (maximum < 4);
- $B(3, 2) = 4$ (the 'pictorial' proof above).

Three whole regions of the table then follow from short arguments.

Proposition 6.2.3 (Boundary cases)

1. $B(N, 1) = 1$ for every N.
2. $B(N, k) = 2^N$ for $N < k$ — including all dichotomies does not violate the 'breaking condition'.
3. $B(N, k) = 2^N - 1$ for $N = k$ — removing a single dichotomy satisfies the 'breaking condition'.

Proof. (1) is the Fun Time above, for any N: if no single coordinate may show both symbols, all rows agree everywhere, so there is one row.

(2) With $N < k$ there are no k coordinates to choose, so the condition is empty and all 2^N vectors are allowed.

(3) With $N = k$ the only k coordinates are all of them. All 2^N vectors would shatter them, so at most $2^N - 1$; and deleting any one vector leaves the full coordinate set missing a pattern, so $2^N - 1$ is attained. ■

Note that (3) contains the first known value, $B(2, 2) = 2^2 - 1 = 3$. The four stages of the table, merged into one — blue: column $k = 1$; black: $N < k$; orange: $N = k$; purple: $B(3, 2)$, the one entry from the pictorial search:

		k						
	$B(N,k)$	1	2	3	4	5	6	…
	1	1	2	2	2	2	2	…
	2	1	**3**	4	4	4	4	…
	3	1	4	**7**	8	8	8	…
N	4	1			**15**	16	16	…
	5	1				**31**	32	…
	6	1					**63**	…
	⋮	⋮						⋱

(The diagonal's first entry $B(1, 1) = 2^1 - 1 = 1$ belongs to both the blue column and the orange diagonal.) Everything on and above the diagonal is now known, and so is the first column; only the region $N > k \geq 2$ below the diagonal remains, which is where N grows with k fixed — the only part that matters for Question 6.2.2.

More than halfway done! :-)

Exercise 6.2.4 (Fun Time)**.** For the 2D perceptrons, which of the following claims is true?

(1) minimum break point $k = 2$;

(2) $m_{\mathcal{H}}(4) = 15$;

(3) $m_{\mathcal{H}}(N) < B(N, k)$ when $N = k =$ minimum break point;

(4) $m_{\mathcal{H}}(N) > B(N, k)$ when $N = k =$ minimum break point.

Reference Answer. (3). As discussed previously, the minimum break point for 2D perceptrons is 4, with $m_{\mathcal{H}}(4) = 14$ (Proposition 5.2.4). Also, note that $B(4, 4) = 15$. So the bounding function $B(N, k)$ can be 'loose' in bounding $m_{\mathcal{H}}(N)$: lines miss two of the sixteen dichotomies of four points (a Radon pair and its mirror), where the break-point rule alone forces only one to be missing. (4) is impossible, since B is by definition an upper bound. ■

§6.3. Bounding Function: Inductive Cases

slides p.12–20

§6.3.i. Estimating $B(4, 3)$

The first unknown entry below the diagonal is $B(4, 3)$, marked '?':

		k						
	$B(N,k)$	1	2	3	4	5	6	…
	1	1	2	2	2	2	2	…
	2	1	3	4	4	4	4	…
	3	1	4	7	8	8	8	…
N	4	1		?	15	16	16	…
	5	1				31	32	…
	6	1					63	…
	⋮	⋮						⋱

The motivation for how to get at it: a table on four inputs is a table on three inputs with one more column, so $B(4, 3)$ shall be related to $B(3, ?)$ — 'adding' one point to a $B(3, ?)$ table. The next step is to reduce $B(4, 3)$ to $B(3, ?)$.

§6.3.ii. 'Achieving' Dichotomies of $B(4, 3)$

First the answer, found the brute-force way: after checking all 2^{2^4} sets of dichotomies on four inputs — every subset of the 16 possible rows — the winner is a set of 11, and $B(4, 3) = 11$:

	$\boldsymbol{x}_1$	$\boldsymbol{x}_2$	$\boldsymbol{x}_3$	$\boldsymbol{x}_4$
01	○	○	○	○
02	×	○	○	○
03	○	×	○	○
04	○	○	×	○
05	○	○	○	×
06	×	×	○	×

	$\boldsymbol{x}_1$	$\boldsymbol{x}_2$	$\boldsymbol{x}_3$	$\boldsymbol{x}_4$
07	×	○	×	○
08	×	○	○	×
09	○	×	×	○
10	○	×	○	×
11	○	○	×	×

		k					
	$B(N,k)$	1	2	3	4	5	6
	1	1	2	2	2	2	2
	2	1	3	4	4	4	4
	3	1	4	7	8	8	8
N	4	1		11	15	16	16
	5	1				31	32
	6	1					63

(I reran the search: no set of 12 avoids shattering three coordinates, and the table above passes.) The shaded cells are the hint for what comes next: the new entry 11 is to be explained by the row $N = 3$ above it.

Question 6.3.1. How to reduce $B(4,3)$ to $B(3,?)$ cases?

§6.3.iii. Reorganized Dichotomies of $B(4,3)$

Sort the 11 rows by their first three entries, and look at what $\boldsymbol{x}_4$ does. Some prefixes $(\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3)$ occur twice, once with $\boldsymbol{x}_4 = \circ$ and once with $\boldsymbol{x}_4 = \times$; the others occur once.

	$\boldsymbol{x}_1$	$\boldsymbol{x}_2$	$\boldsymbol{x}_3$	$\boldsymbol{x}_4$
01	○	○	○	○
05	○	○	○	×
02	×	○	○	○
08	×	○	○	×
03	○	×	○	○
10	○	×	○	×
04	○	○	×	○
11	○	○	×	×
06	×	×	○	×
07	×	○	×	○
09	○	×	×	○

Orange: pair; purple: single. Write α for the number of pairs and β for the number of singles. Here $\alpha = 4$ and $\beta = 3$, and

$$B(4,3) = 11 = 2\alpha + \beta.$$

The point of the split is that the two kinds of prefix face different restrictions, and each restriction is a smaller bounding function.

§6.3.iv. Estimating Part of $B(4,3)$

Delete $\boldsymbol{x}_4$ and merge each pair into one row. What remains is a table on $(\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3)$ with $\alpha+\beta$ distinct rows:

	$\boldsymbol{x}_1$	$\boldsymbol{x}_2$	$\boldsymbol{x}_3$
	○	○	○
α	×	○	○
	○	×	○
	○	○	×
	×	×	○
β	×	○	×
	○	×	×

	$\boldsymbol{x}_1$	$\boldsymbol{x}_2$	$\boldsymbol{x}_3$
	○	○	○
α	×	○	○
	○	×	○
	○	○	×

Left: all $\alpha + \beta$ prefixes. Right: the α paired ones alone.

The whole prefix table (1/2).

- $\alpha + \beta$: dichotomies on $(\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3)$.
- $B(4,3)$ 'no shatter' any 3 inputs $\Rightarrow \alpha + \beta$ 'no shatter' any 3.

The implication holds because deleting a column cannot create a pattern: if the prefix table showed all 8 patterns on $\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3$, so would the original 11 rows. Hence

$$\alpha + \beta \leq B(3,3).$$

The paired prefixes (2/2).

- α: dichotomies on $(\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3)$ with $\boldsymbol{x}_4$ *paired*.
- $B(4,3)$ 'no shatter' any 3 inputs $\Rightarrow \alpha$ 'no shatter' any 2.

The claim is about the α rows alone, not the whole table: rows 01, 02, 03, 06 already shatter $\boldsymbol{x}_1, \boldsymbol{x}_2$, and a break point at 3 says nothing about 2 in general. What makes the α rows special is that each comes with *both* values of $\boldsymbol{x}_4$. So if they shattered two inputs, say $\boldsymbol{x}_1, \boldsymbol{x}_2$, the original table would show each of those four patterns with $\boldsymbol{x}_4 = \circ$ and with $\boldsymbol{x}_4 = \times$ — all eight patterns on $\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_4$, three inputs shattered. The pairing supplies the third input for free. Hence

$$\alpha \leq B(3,2).$$

§6.3.v. Putting It All Together

The two estimates add up, since $2\alpha + \beta = (\alpha + \beta) + \alpha$:

$$\begin{aligned} B(4,3) &= 2\alpha + \beta \\ \alpha + \beta &\leq B(3,3) \\ \alpha &\leq B(3,2) \\ \Rightarrow B(4,3) &\leq B(3,3) + B(3,2). \end{aligned}$$

Nothing in the argument used 4 or 3 specifically. For any N and k, take a largest table for $B(N,k)$ and split it on the last input:

$$\begin{aligned} B(N,k) &= 2\alpha + \beta \\ \alpha + \beta &\leq B(N-1,k) \\ \alpha &\leq B(N-1,k-1) \\ \Rightarrow B(N,k) &\leq B(N-1,k) + B(N-1,k-1). \end{aligned}$$

Lemma 6.3.2 (Inductive formula)
For $N \geq 2$ and $k \geq 2$,

$$B(N,k) \leq B(N-1,k) + B(N-1,k-1).$$

Proof. Let S be a set of $B(N,k)$ vectors in $\{\circ, \times\}^N$ that shatters no k coordinates. Group its vectors by their first $N-1$ entries: α prefixes appear with both values of the last entry and β with one, so $|S| = 2\alpha + \beta$. The $\alpha + \beta$ distinct prefixes shatter no k of the first $N-1$ coordinates, or S would shatter them too; so $\alpha + \beta \leq B(N-1,k)$. The α paired prefixes shatter no $k-1$ of those coordinates, or S would shatter those $k-1$ together with the last one; so $\alpha \leq B(N-1,k-1)$. Add the two. ∎

Running the formula down the table from the known entries fills the region below the diagonal with upper bounds (orange):

		k					
	$B(N,k)$	1	2	3	4	5	6
	1	1	2	2	2	2	2
	2	1	3	4	4	4	4
	3	1	4	7	8	8	8
N	4	1	≤ 5	11	15	16	16
	5	1	≤ 6	≤ 16	≤ 26	31	32
	6	1	≤ 7	≤ 22	≤ 42	≤ 57	63

The shaded cells show the pattern: each entry is at most the one directly above plus the one above-left, as in Pascal's triangle. For instance, $B(4,3) = 11 = 7 + 4$, so the formula is exact there, and $B(5,3) \leq 11 + 5 = 16$.

We now have an upper bound of the bounding function.

§6.3.vi. Bounding Function: The Theorem

The Pascal-like recursion suggests binomial coefficients, and indeed:

Theorem 6.3.3 (Bounding function)

For all $N \geq 1$ and $k \geq 1$,

$$B(N,k) \leq \underbrace{\sum_{i=0}^{k-1} \binom{N}{i}}_{\text{highest term } N^{k-1}}.$$

Proof. Simple induction on N, using the boundary cases and the inductive formula. Write $\Phi(N,k)$ for the right-hand side.

Boundary. If $k = 1$, $B(N,1) = 1 = \binom{N}{0}$ by Proposition 6.2.3. If $N = 1$ and $k \geq 2$, $B(1,k) = 2 = \binom{1}{0} + \binom{1}{1}$ (the sum stops at $\binom{1}{1}$, all later terms being 0).

Induction. For $N \geq 2$, $k \geq 2$, Lemma 6.3.2 and the hypothesis for $N-1$ give

$$B(N,k) \leq \sum_{i=0}^{k-1} \binom{N-1}{i} + \sum_{i=0}^{k-2} \binom{N-1}{i} = \binom{N-1}{0} + \sum_{i=1}^{k-1} \left[\binom{N-1}{i} + \binom{N-1}{i-1} \right] = \sum_{i=0}^{k-1} \binom{N}{i},$$

the last step by Pascal's rule and $\binom{N-1}{0} = \binom{N}{0}$. ∎

- The proof is a simple induction using the boundary and the inductive formula.
- For fixed k, $B(N,k)$ is upper bounded by $\text{poly}(N)$, the leading term being $\binom{N}{k-1} \approx N^{k-1}/(k-1)! \Rightarrow m_{\mathcal{H}}(N)$ is $\text{poly}(N)$ if a break point exists.

That settles Question 6.2.2 and, through $m_{\mathcal{H}}(N) \leq B(N,k)$, proves Conjecture 5.4.1.

Remark 6.3.4. '$\leq$' can be '$=$' actually — go play and prove it if you are a math lover! :-) (One way: exhibit a table of the right size, e.g. all vectors with fewer than k entries $\times$. It has $\sum_{i<k} \binom{N}{i}$ rows and cannot shatter k coordinates, since it never shows $\times$ on all of them at once.) The inequality is also known as the Sauer–Shelah lemma.

§6.3.vii. The Three Break Points

Returning to the hypothesis sets with a break point, Theorem 6.3.3 bounds each growth function from its break point alone (in orange, the bound):

$\mathcal{H}$		$m_{\mathcal{H}}(N)$	break point
positive rays		$N+1 \leq N+1$	$m_{\mathcal{H}}(2) = 3 < 2^2$: at 2
positive intervals		$\frac{1}{2}N^2 + \frac{1}{2}N + 1 \leq \frac{1}{2}N^2 + \frac{1}{2}N + 1$	$m_{\mathcal{H}}(3) = 7 < 2^3$: at 3
2D perceptrons		$m_{\mathcal{H}}(N) =? \leq \frac{1}{6}N^3 + \frac{5}{6}N + 1$	$m_{\mathcal{H}}(4) = 14 < 2^4$: at 4

For rays and intervals the bound is exact: $\binom{N}{0} + \binom{N}{1} = N+1$, and $\binom{N}{0} + \binom{N}{1} + \binom{N}{2} = \frac{1}{2}N^2 + \frac{1}{2}N + 1$. For 2D perceptrons we still do not know $m_{\mathcal{H}}(N)$, but with $k = 4$

$$\sum_{i=0}^{3} \binom{N}{i} = 1 + N + \frac{N(N-1)}{2} + \frac{N(N-1)(N-2)}{6} = \frac{1}{6}N^3 + \frac{5}{6}N + 1,$$

a polynomial, found without computing a single dichotomy beyond $N = 4$.

We can bound $m_{\mathcal{H}}(N)$ by only one break point.

Exercise 6.3.5 (Fun Time)**.** For 1D perceptrons (positive and negative rays), we know that $m_{\mathcal{H}}(N) = 2N$. Let k be the minimum break point. Which of the following is *not* true?

(1) $k = 3$;

(2) for some integers $N > 0$, $m_{\mathcal{H}}(N) = \sum_{i=0}^{k-1} \binom{N}{i}$;

(3) for all integers $N > 0$, $m_{\mathcal{H}}(N) = \sum_{i=0}^{k-1} \binom{N}{i}$;

(4) for all integers $N > 2$, $m_{\mathcal{H}}(N) < \sum_{i=0}^{k-1} \binom{N}{i}$.

Reference Answer. (3). The proof is generally trivial by listing the definitions. (1) is the last Fun Time of Section 5.4. With $k = 3$ the sum is $1 + N + \frac{N(N-1)}{2}$, which equals 2, 4, 7 at $N = 1, 2, 3$: so for (2), $N = 1$ or 2 gives the equality, and $N = 3$ ($6 \neq 7$) breaks (3). One thing to notice is (4): the difference $1 + N + \frac{N(N-1)}{2} - 2N = \frac{(N-1)(N-2)}{2}$ is positive for $N > 2$, so the upper bound can be 'loose'. ■

§6.4. A Pictorial Proof

slides p.21–26

The growth function is now known to be polynomial whenever there is a break point. What remains is the '?' of Equation 5.1: may $m_{\mathcal{H}}$ take the place of M in the bound at all?

§6.4.i. BAD Bound for General $\mathcal{H}$

Because $\mathcal{A}$ may return any hypothesis, the event to control is that *some* h is BAD. What we want is

$$\mathbb{P}\left[\exists h \in \mathcal{H} \text{ s.t. } |E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon\right] \leq 2 \cdot m_{\mathcal{H}}(N) \cdot \exp\left(-2\epsilon^2 N\right).$$

What is actually true, when N is large enough, is

$$\mathbb{P}\left[\exists h \in \mathcal{H} \text{ s.t. } |E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon\right] \leq 2 \cdot 2m_{\mathcal{H}}(2N) \cdot \exp\left(-2 \cdot \frac{1}{16}\epsilon^2 N\right).$$

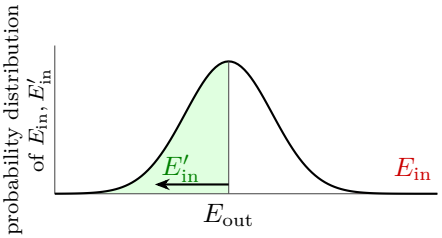
The wish survives with worse constants: $m_{\mathcal{H}}(2N)$ in place of $m_{\mathcal{H}}(N)$ (still polynomial), an extra factor 2, and $\epsilon^2/16$ in the exponent (still exponential decay). Next: a sketch of the proof, in three steps, each of which removes one obstacle and costs one of those constants.

§6.4.ii. Step 1: Replace E_{out} by E'_{in}

$$\frac{1}{2}\mathbb{P}\left[\exists h \in \mathcal{H} \text{ s.t. } |E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon\right] \leq \mathbb{P}\left[\exists h \in \mathcal{H} \text{ s.t. } \left|E_{\text{in}}(h) - E'_{\text{in}}(h)\right| > \frac{\epsilon}{2}\right]$$

- $E_{\text{in}}(h)$ takes finitely many values across $\mathcal{H}$ (one per dichotomy), while $E_{\text{out}}(h)$ takes infinitely many — so replace the evil E_{out} first.
- How? Sample a **verification set** $\mathcal{D}'$ of size N to calculate E'_{in}, the error on $\mathcal{D}'$.
- BAD h of $E_{\text{in}} - E_{\text{out}}$ $\overset{\text{probably}}{\Rightarrow}$ BAD h of $E_{\text{in}} - E'_{\text{in}}$.

$\mathcal{D}'$ is never actually collected; it exists only inside the proof.



The picture is for one fixed h. Both E_{in} and E'_{in} are averages of N independent errors, so they share the same bell-shaped distribution centred on E_{out}. Suppose $\mathcal{D}$ is BAD, with E_{in} out in the red tail, more than ϵ above E_{out}. The ghost sample is drawn independently, so E'_{in} lands in the green half, at or below E_{out}, with probability about $\frac{1}{2}$; and then E_{in} and E'_{in} are more than ϵ apart. So a BAD $\mathcal{D}$ produces a large gap $|E_{\text{in}} - E'_{\text{in}}|$ at least half the time, which is the $\frac{1}{2}$ on the left.

Remark 6.4.1. The rigorous version uses a margin instead of symmetry. On the event that $|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon$ for some h, fix one such h (it depends only on $\mathcal{D}$). By Hoeffding on $\mathcal{D}'$, $|E'_{\text{in}}(h) - E_{\text{out}}(h)| \leq \frac{\epsilon}{2}$ with probability at least $1 - 2\exp\left(-\frac{1}{2}\epsilon^2 N\right)$, which is at least $\frac{1}{2}$ once N is large enough — the 'N large enough' in the statement. When it happens, the triangle inequality gives $|E_{\text{in}}(h) - E'_{\text{in}}(h)| > \epsilon - \frac{\epsilon}{2} = \frac{\epsilon}{2}$. The halved ϵ is the price of this margin.

The evil E_{out} is removed by verification with 'ghost data'.

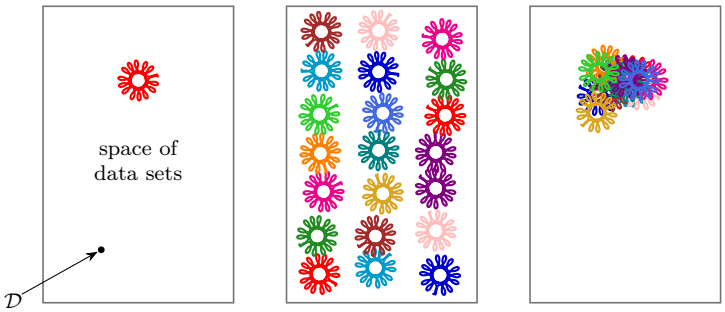
§6.4.iii. Step 2: Decompose $\mathcal{H}$ by Kind

$$\begin{aligned}\text{BAD} &\leq 2\mathbb{P}\left[\exists h \in \mathcal{H} \text{ s.t. } \left|E_{\text{in}}(h) - E'_{\text{in}}(h)\right| > \frac{\epsilon}{2}\right] \\ &\leq 2m_{\mathcal{H}}(2N)\mathbb{P}\left[\text{fixed } h \text{ s.t. } \left|E_{\text{in}}(h) - E'_{\text{in}}(h)\right| > \frac{\epsilon}{2}\right]\end{aligned}$$

- E_{in} is computed with $\mathcal{D}$ and E'_{in} with $\mathcal{D}'$ — and now $m_{\mathcal{H}}$ comes to play.

- How? Both errors depend on h only through its values on the $2N$ points of $\mathcal{D} \cup \mathcal{D}'$, so the infinite $\mathcal{H}$ becomes $|\mathcal{H}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N, \boldsymbol{x}'_1, \ldots, \boldsymbol{x}'_N)|$ kinds, at most $m_{\mathcal{H}}(2N)$.
- Union bound on the $m_{\mathcal{H}}(2N)$ kinds.

The first line is Step 1 with both sides doubled; 'BAD' abbreviates the left-hand probability we started from. The price of this step is $m_{\mathcal{H}}(2N)$ rather than $m_{\mathcal{H}}(N)$: the dichotomies must be counted on the ghost points too.



(a) Hoeffding inequality (b) union bound (c) now

Each box is the space of all possible data sets, and each coloured scribble the BAD region of one hypothesis — the data sets on which it is fooled. (a) Hoeffding says one hypothesis's region is small, so a random $\mathcal{D}$ usually misses it. (b) The union bound adds up all the regions as if they were disjoint, so with many hypotheses they seem to fill the whole space. (c) In truth, hypotheses of the same kind are BAD on nearly the same data sets, and their regions pile on top of one another; counting kinds instead of hypotheses counts each pile once.

Use $m_{\mathcal{H}}(2N)$ to calculate the BAD-overlap properly.

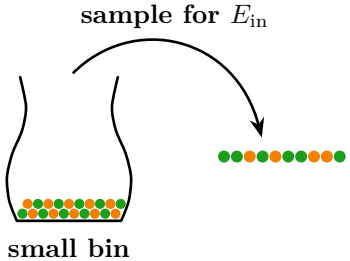
§6.4.iv. Step 3: Use Hoeffding without Replacement

$$\begin{aligned} \text{BAD} &\leq 2m_{\mathcal{H}}(2N)\mathbb{P}\left[\text{fixed } h \text{ s.t. } \left|E_{\text{in}}(h) - E'_{\text{in}}(h)\right| > \frac{\epsilon}{2}\right] \\ &\leq 2m_{\mathcal{H}}(2N) \cdot 2\exp\left(-2\left(\frac{\epsilon}{4}\right)^2 N\right) \end{aligned}$$

- Consider a bin of the $2N$ examples; choose N of them for E_{in}, and leave the others for E'_{in}. Since the average of the two halves is the bin's own error rate,

$$\left|E_{\text{in}} - E'_{\text{in}}\right| > \frac{\epsilon}{2} \Leftrightarrow \left|E_{\text{in}} - \frac{E_{\text{in}} + E'_{\text{in}}}{2}\right| > \frac{\epsilon}{4}.$$

- So? It is just a 'smaller bin', a 'smaller ϵ', and Hoeffding without replacement.



Everything is now conditioned on the $2N$ points in the bin, and h is fixed. Which N of them are $\mathcal{D}$ and which are $\mathcal{D}'$ is a uniformly random split, so E_{in} is the orange fraction of a sample of N drawn *without* replacement from a bin whose orange fraction is $\frac{E_{\text{in}}+E'_{\text{in}}}{2}$. Hoeffding's inequality holds for sampling without replacement as well (Hoeffding proved it in that form), with the tolerance now $\frac{\epsilon}{4}$: that gives $2\exp\big(-2(\epsilon/4)^2N\big)$.

Use Hoeffding after zooming to a fixed h.

§6.4.v. That's All!

Collecting the constants, $2m_{\mathcal{H}}(2N)\cdot 2\exp\left(-2\frac{\epsilon^2}{16}N\right) = 4m_{\mathcal{H}}(2N)\exp\big(-\frac{1}{8}\epsilon^2N\big)$.

Theorem 6.4.2 (Vapnik–Chervonenkis (VC) bound)
For N large enough,

$$\mathbb{P}\left[\exists h\in\mathcal{H}\text{ s.t. }|E_{\text{in}}(h)-E_{\text{out}}(h)|>\epsilon\right]\le 4m_{\mathcal{H}}(2N)\exp\left(-\frac{1}{8}\epsilon^2N\right).$$

Proof sketch.

- replace E_{out} by E'_{in} (Step 1);
- decompose $\mathcal{H}$ by kind (Step 2);
- use Hoeffding without replacement (Step 3). ■

Since the event covers every h, it covers the g returned by any $\mathcal{A}$, exactly as Theorem 4.4.5 did for finite $\mathcal{H}$. Applied to 2D perceptrons:

- break point? 4;
- $m_{\mathcal{H}}(N)$? $O(N^3)$, by Theorem 6.3.3, so $m_{\mathcal{H}}(2N)$ is $O(N^3)$ as well, and the bound goes to 0 as N grows.

Learning with 2D perceptrons is feasible! :-)

Exercise 6.4.3 (Fun Time)**.** For positive rays, $m_{\mathcal{H}}(N)=N+1$. Plug it into the VC bound for $\epsilon=0.1$ and $N=10000$. What is the VC bound of BAD events?

$$\mathbb{P}\left[\exists h\in\mathcal{H}\text{ s.t. }|E_{\text{in}}(h)-E_{\text{out}}(h)|>\epsilon\right]\le 4m_{\mathcal{H}}(2N)\exp\left(-\frac{1}{8}\epsilon^2N\right)$$

(1) 2.77×10^{-87}; (2) 5.54×10^{-83}; (3) 2.98×10^{-1}; (4) 2.29×10^{2}.

Reference Answer. (3). Simple calculation:

$$4\cdot(2\cdot10000+1)\cdot\exp\left(-\frac{1}{8}(0.1)^2\cdot10000\right)=80004\,e^{-12.5}\approx0.298.$$

Note that the BAD probability bound is not very small even with 10000 examples — the constants lost in the three steps are expensive. ■

Summary

Restriction of Break Point A break point 'breaks' consequent points.

Bounding Function: Basic Cases $B(N, k)$ bounds $m_{\mathcal{H}}(N)$ with break point k.

Bounding Function: Inductive Cases $B(N, k)$ is poly(N).

A Pictorial Proof $m_{\mathcal{H}}(N)$ can replace M with a few changes.

The chapter closed both gaps left by the last one. A break point k caps $m_{\mathcal{H}}(N)$ by $\sum_{i<k} \binom{N}{i} = O(N^{k-1})$, and the VC bound puts $m_{\mathcal{H}}(2N)$ where M used to be; so any $\mathcal{H}$ with a break point generalises, however infinite it is. Next: how to 'use' the break point — the minimum break point turns out to be the one number that summarises an $\mathcal{H}$'s capacity.

7. The VC Dimension

Lecture 7

30 Sep.

> **As previously seen .** Theorem 6.3.3 showed that a break point k caps the growth function by a polynomial, $m_{\mathcal{H}}(N) \le B(N,k) = \sum_{i<k} \binom{N}{i}$, and Theorem 6.4.2 put $m_{\mathcal{H}}(2N)$ where M used to be. Together: $E_{\text{out}} \approx E_{\text{in}}$ is possible if $m_{\mathcal{H}}(N)$ breaks somewhere and N is large enough.

That conclusion depends on $\mathcal{H}$ only through one number, its minimum break point. This chapter gives that number a name (shifted by one), computes it for perceptrons in any dimension, explains what it measures physically, and then turns the VC bound around to read off what it says about model complexity and about how much data is needed.

§7.1. Definition of VC Dimension

slides p.2–7

§7.1.i. Recap: More on Growth Function

The last chapter ended with

$$m_{\mathcal{H}}(N) \text{ of break point } k \;\le\; B(N,k) = \underbrace{\sum_{i=0}^{k-1} \binom{N}{i}}_{\text{highest term } N^{k-1}} .$$

The sum is awkward to carry around. Its highest term suggests replacing it by the single power N^{k-1}, and the two tables side by side show when that is safe:

		k				
	$B(N,k)$	1	2	3	4	5
	1	1	2	2	2	2
	2	1	3	4	4	4
	3	1	4	7	8	8
N	4	1	5	11	15	16
	5	1	6	16	26	31
	6	1	7	22	42	57

		k				
	N^{k-1}	1	2	3	4	5
	1	1	1	1	1	1
	2	1	2	4	8	16
	3	1	3	9	27	81
N	4	1	4	16	64	256
	5	1	5	25	125	625
	6	1	6	36	216	1296

Cell by cell, the shaded entries on the right are at least those on the left. Outside the shaded region the comparison fails: column $k = 2$ has $N + 1 > N$, and row $N = 1$ has $2 > 1$. Hence

Provably and loosely, for $N \ge 2$, $k \ge 3$,

$$m_{\mathcal{H}}(N) \le B(N,k) = \sum_{i=0}^{k-1} \binom{N}{i} \le N^{k-1}.$$

Remark 7.1.1. The table is evidence, not a proof. One proof: for $N \geq 2$, $k \geq 3$, induct on k. At $k = 3$, $1 + N + \frac{N(N-1)}{2} \leq N^2$ is $(N-2)(N+1) \geq 0$ after doubling and rearranging, which holds for all $N \geq 2$. Going from k to $k+1$ adds $\binom{N}{k} \leq \frac{N^k}{k!} \leq N^k - N^{k-1}$ whenever $N \geq 2$ and $k \geq 2$, since then $N^{k-1}(N-1) \geq N^k/2 \geq N^k/k!$.

§7.1.ii. Recap: More on Vapnik–Chervonenkis (VC) Bound

Chaining the VC bound with this estimate, for any $g = \mathcal{A}(\mathcal{D}) \in \mathcal{H}$ and a 'statistical' large $\mathcal{D}$, for $N \geq 2$, $k \geq 3$:

$$\begin{aligned}\mathbb{P}_{\mathcal{D}}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] &\leq \mathbb{P}_{\mathcal{D}}\left[\exists h \in \mathcal{H} \text{ s.t. } |E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon\right] \\ &\leq 4m_{\mathcal{H}}(2N)\exp\left(-\frac{1}{8}\epsilon^2 N\right) \\ &\overset{\text{if } k \text{ exists}}{\leq} 4(2N)^{k-1}\exp\left(-\frac{1}{8}\epsilon^2 N\right).\end{aligned}$$

The first step holds because g is one of the h; the last applies the moral above at $2N$ points. The subscript $\mathcal{D}$ is a reminder that the probability is over the draw of the data. Reading the chain as a recipe for learning:

if	(1) $m_{\mathcal{H}}(N)$ breaks at k	(good $\mathcal{H}$)
	(2) N large enough	(good $\mathcal{D}$)
⇒	probably generalized '$E_{\text{out}} \approx E_{\text{in}}$', and	
if	(3) $\mathcal{A}$ picks a g with small E_{in}	(good $\mathcal{A}$)
⇒	probably learned!	(:-) good luck)

The first two conditions are what the theory has been building; the third is an optimisation problem, and is the subject of *How can machines learn?*

§7.1.iii. VC Dimension

The chain carries the exponent $k-1$ everywhere, so it is natural to name that number directly: the formal name of the *maximum non*-break point.

Definition 7.1.2 (VC dimension)**.** The **VC dimension** of $\mathcal{H}$, denoted $d_{\text{VC}}(\mathcal{H})$, is the

$$\text{largest } N \text{ for which } m_{\mathcal{H}}(N) = 2^N.$$

- It is the *most* inputs $\mathcal{H}$ can shatter.
- $d_{\text{VC}} =$ 'minimum k' $- 1$.

If $m_{\mathcal{H}}(N) = 2^N$ for every N there is no break point and $d_{\text{VC}} = \infty$. The definition asks for *some* set of N inputs to be shattered, never all of them. Unwinding it,

$$\begin{aligned}N \leq d_{\text{VC}} &\Rightarrow \mathcal{H} \text{ can shatter some } N \text{ inputs}, \\ k > d_{\text{VC}} &\Rightarrow k \text{ is a break point for } \mathcal{H}.\end{aligned}$$

The second line is the one the bounds use: with $k = d_{\text{VC}} + 1$, the moral of the recap becomes

If $N \geq 2$, $d_{\text{VC}} \geq 2$, then $m_{\mathcal{H}}(N) \leq N^{d_{\text{VC}}}$.

§7.1.iv. The Four VC Dimensions

Reading off the four hypothesis sets of Section 5.3, with a largest shattered configuration drawn next to each:

$\mathcal{H}$	d_{VC}	largest shattered inputs	$m_{\mathcal{H}}(N)$
positive rays	1		$N+1$
positive intervals	2		$\frac{1}{2}N^2+\frac{1}{2}N+1$
convex sets	∞		2^N
2D perceptrons	3		$\leq N^3$ for $N \geq 2$

Each d_{VC} is the minimum break point of the previous chapters minus one: $2-1$, $3-1$, none, $4-1$. The convex-set picture is the circle configuration of Proposition 5.3.9, which is shattered for every N. For 2D perceptrons we still do not know $m_{\mathcal{H}}(N)$ exactly, but $d_{\text{VC}} = 3$ is enough to get $m_{\mathcal{H}}(N) \leq N^3$.

Good: finite d_{VC}.

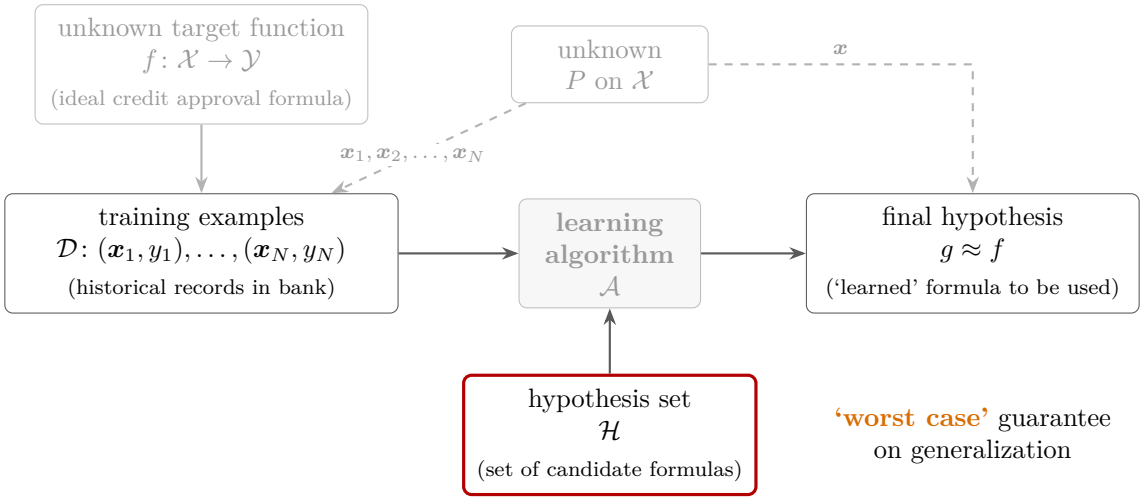
§7.1.v. VC Dimension and Learning

Finite $d_{\text{VC}} \Rightarrow g$ 'will' generalize ($E_{\text{out}}(g) \approx E_{\text{in}}(g)$).

What is remarkable is what the guarantee does *not* need:

- regardless of learning algorithm $\mathcal{A}$;
- regardless of input distribution P;
- regardless of target function f.

In the learning flow, those three boxes can be greyed out; the guarantee depends only on the hypothesis set and the size of the training set.



'Worst case' is the price of that generality: the bound has to hold for the least favourable $\mathcal{A}$, P and f at once, which is one reason it will turn out loose (Section 7.4).

Exercise 7.1.3 (Fun Time)**.** If there is a set of N inputs that cannot be shattered by $\mathcal{H}$. Based only on this information, what can we conclude about $d_{\text{VC}}(\mathcal{H})$? (1) $d_{\text{VC}}(\mathcal{H}) > N$; (2) $d_{\text{VC}}(\mathcal{H}) = N$; (3) $d_{\text{VC}}(\mathcal{H}) < N$; (4) no conclusion can be made.

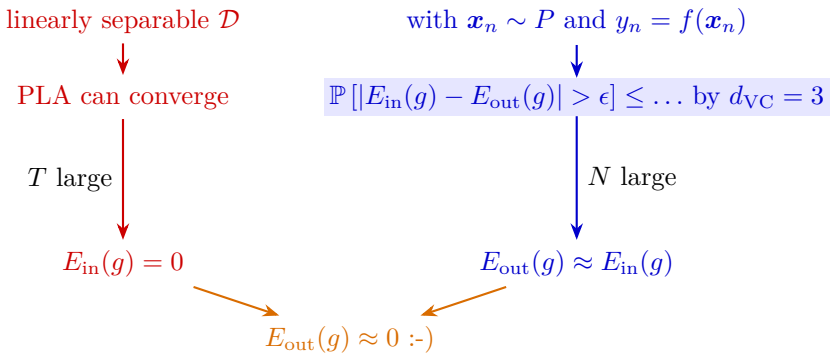
Reference Answer. (4). It is possible that there is another set of N inputs that can be shattered, which means $d_{\text{VC}} \geq N$. It is also possible that no set of N inputs can be shattered, which means $d_{\text{VC}} < N$. Neither case can be ruled out by one non-shattering set. (Three collinear points, which no line shatters, are the example: they say nothing against $d_{\text{VC}} = 3$ for 2D perceptrons.) ■

§7.2. VC Dimension of Perceptrons

slides p.8–16

§7.2.i. 2D PLA Revisited

With d_{VC} in hand, the story of 2D PLA can finally be closed. Two independent chains lead to the same place:



The left chain is optimisation (Section 2.3): after enough iterations T, PLA makes no mistake on $\mathcal{D}$. The right chain is generalisation: with enough examples N, E_{in} is close to E_{out} for any line, including the one PLA returns. Together they give $E_{\text{out}}(g) \approx 0$.

Question 7.2.1. General PLA for $\boldsymbol{x}$ with more than 2 features?

§7.2.ii. VC Dimension of Perceptrons

What is known so far:

- 1D perceptron (pos/neg rays): $d_{\text{VC}} = 2$;
- 2D perceptrons: $d_{\text{VC}} = 3$
  - $d_{\text{VC}} \geq 3$: ∴ can be shattered;
  - $d_{\text{VC}} \leq 3$: × ○ × ○ no four inputs can be shattered;
- d-D perceptrons: $d_{\text{VC}} \stackrel{?}{=} d + 1$.

The first two fit the guess $d+1$: $1+1$ and $2+1$. The extra 1 is the threshold, which Section 2.1 absorbed as w_0 with the constant coordinate $x_0 = 1$, so a d-D perceptron really has $d+1$ weights. Proving $d_{\text{VC}} = d+1$ takes two steps:

- $d_{\text{VC}} \geq d+1$;
- $d_{\text{VC}} \leq d+1$.

The two steps need different kinds of evidence, as the next two quizzes make precise.

Exercise 7.2.2 (Extra Fun Time)**.** What statement below shows that $d_{\text{VC}} \geq d+1$?

(1) There are some $d+1$ inputs we can shatter.

(2) We can shatter any set of $d+1$ inputs.

(3) There are some $d+2$ inputs we cannot shatter.

(4) We cannot shatter any set of $d+2$ inputs.

Reference Answer. (1). d_{VC} is the maximum N such that $m_{\mathcal{H}}(N) = 2^N$, and $m_{\mathcal{H}}(N)$ is the most number of dichotomies of N inputs. So if we can find 2^{d+1} dichotomies on *some* $d+1$ inputs, $m_{\mathcal{H}}(d+1) = 2^{d+1}$ and hence $d_{\text{VC}} \geq d+1$. (2) is also sufficient but asks for far more than needed — and is false for perceptrons, as collinear points show. ■

§7.2.iii. $d_{\text{VC}} \geq d+1$

By the quiz, it is enough that there are *some* $d+1$ inputs we can shatter. Pick some 'trivial' inputs: the origin and the d unit vectors, each with the constant coordinate $x_0 = 1$ in front. As the rows of a matrix,

$$\mathrm{X} = \begin{bmatrix} — \, \boldsymbol{x}_1^\top \, — \\ — \, \boldsymbol{x}_2^\top \, — \\ — \, \boldsymbol{x}_3^\top \, — \\ \vdots \\ — \, \boldsymbol{x}_{d+1}^\top \, — \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 1 & 1 & 0 & \dots & 0 \\ 1 & 0 & 1 & & 0 \\ \vdots & \vdots & & \ddots & 0 \\ 1 & 0 & \dots & 0 & 1 \end{bmatrix}.$$

Visually in 2D these are the three points ⁚ at $(0,0)$, $(1,0)$ and $(0,1)$.

Note 7.2.3. X is $\boldsymbol{(d+1) \times (d+1)}$ and **invertible** — subtracting the first row from every other row leaves the identity matrix.

Remark 7.2.4. The $d+1$ inputs are points of $\mathbb{R}^d$: the origin $\mathbf{0}$ and the d standard unit vectors $\boldsymbol{e}_1, \dots, \boldsymbol{e}_d$, where $\boldsymbol{e}_i$ has 1 in its i-th entry and 0 elsewhere. So $\boldsymbol{x}_1 = (1, \mathbf{0})$ and $\boldsymbol{x}_{i+1} = (1, \boldsymbol{e}_i)$ for $i = 1, \dots, d$. The first column of X is not part of the data: it is the bias coordinate $x_0 = 1$ (Section 2.1). This is why a d-D perceptron works with $(d+1)$-dimensional vectors, and why X is square.

§7.2.iv. Can We Shatter X?

To shatter X means: for any $\boldsymbol{y} = (y_1, \dots, y_{d+1})^\top$ with entries ± 1, find $\boldsymbol{w}$ such that $\operatorname{sign}(\mathrm{X}\boldsymbol{w}) = \boldsymbol{y}$, the n-th entry of $\mathrm{X}\boldsymbol{w}$ being $\boldsymbol{w}^\top \boldsymbol{x}_n$. Demanding the signs is hard; demanding the values is easy, because X is invertible:

$$\operatorname{sign}(\mathrm{X}\boldsymbol{w}) = \boldsymbol{y} \quad \Longleftarrow \quad \mathrm{X}\boldsymbol{w} = \boldsymbol{y} \quad \overset{\text{X invertible!}}{\Leftrightarrow} \quad \boldsymbol{w} = \mathrm{X}^{-1}\boldsymbol{y}.$$

The first implication holds because $\operatorname{sign}(\pm 1) = \pm 1$. So every one of the 2^{d+1} label vectors is realised by some perceptron:

'Special' X can be shattered $\Rightarrow d_{\text{VC}} \geq d+1$.

Exercise 7.2.5 (Extra Fun Time)**.** What statement below shows that $d_{\text{VC}} \leq d+1$?

(1) There are some $d+1$ inputs we can shatter.

(2) We can shatter any set of $d+1$ inputs.

(3) There are some $d+2$ inputs we cannot shatter.

(4) We cannot shatter any set of $d+2$ inputs.

Reference Answer. (4). d_{VC} is the maximum N such that $m_{\mathcal{H}}(N) = 2^N$, and $m_{\mathcal{H}}(N)$ is the most number of dichotomies of N inputs. So if we cannot find 2^{d+2} dichotomies on *any* $d+2$ inputs (i.e. break point), $m_{\mathcal{H}}(d+2) < 2^{d+2}$ and hence $d_{\text{VC}} < d+2$. That is, $d_{\text{VC}} \leq d+1$. Statement (3) is the trap of the Fun Time in Section 7.1: one bad set proves nothing. ■

The upper bound is therefore the harder half: a single well-chosen example no longer suffices, and the argument must work for *every* set of $d+2$ inputs.

§7.2.v. $d_{\text{VC}} \leq d+1$: A 2D Special Case

Start with four inputs in 2D, the corners of the unit square:

$$\begin{matrix}\bullet & \bullet \\ \bullet & \bullet\end{matrix} \qquad \mathrm{X} = \begin{bmatrix} — & \boldsymbol{x}_1^\top & — \\ — & \boldsymbol{x}_2^\top & — \\ — & \boldsymbol{x}_3^\top & — \\ — & \boldsymbol{x}_4^\top & — \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

So $\boldsymbol{x}_1$ is the lower-left corner, $\boldsymbol{x}_2$ lower-right, $\boldsymbol{x}_3$ upper-left, $\boldsymbol{x}_4$ upper-right. Four rows in a three-dimensional space must be linearly dependent, and here the dependence is visible: $\boldsymbol{x}_4 = \boldsymbol{x}_2 + \boldsymbol{x}_3 - \boldsymbol{x}_1$. Now label $\boldsymbol{x}_2, \boldsymbol{x}_3$ as $\circ$ and $\boldsymbol{x}_1$ as $\times$:

$$\begin{matrix}\circ & ? \\ \times & \circ\end{matrix} \qquad ? \text{ cannot be } \times$$

because for every $\boldsymbol{w}$ producing these three labels,

$$\boldsymbol{w}^\top \boldsymbol{x}_4 = \underbrace{\boldsymbol{w}^\top \boldsymbol{x}_2}_{\circ} + \underbrace{\boldsymbol{w}^\top \boldsymbol{x}_3}_{\circ} - \underbrace{\boldsymbol{w}^\top \boldsymbol{x}_1}_{\times} > 0 :$$

the first two terms are positive, and subtracting a negative one only adds. This is the XOR dichotomy again, now explained algebraically.

Linear dependence restricts dichotomy.

§7.2.vi. $d_{\text{VC}} \leq d+1$: The d-D General Case

By the Extra Fun Time above, proving $d_{\text{VC}} \leq d+1$ means proving that $m_{\mathcal{H}}(d+2) < 2^{d+2}$: *no* set of $d+2$ inputs can be shattered. One bad configuration, such as the square, is not enough; the argument has to work for every choice of $d+2$ inputs. So take *any* $d+2$ inputs, as rows of

$$\mathrm{X} = \begin{bmatrix} — & \boldsymbol{x}_1^\top & — \\ — & \boldsymbol{x}_2^\top & — \\ & \vdots & \\ — & \boldsymbol{x}_{d+1}^\top & — \\ — & \boldsymbol{x}_{d+2}^\top & — \end{bmatrix}.$$

There are more rows $(d+2)$ than columns $(d+1)$, so the rows are linearly dependent: some row is a combination of the others. Relabelling so that this row is the last one,

$$\boldsymbol{x}_{d+2} = a_1\boldsymbol{x}_1 + a_2\boldsymbol{x}_2 + \cdots + a_{d+1}\boldsymbol{x}_{d+1}, \qquad \text{some } a_i \text{ non-zero.}$$

- Can you generate $(\operatorname{sign}(a_1), \operatorname{sign}(a_2), \ldots, \operatorname{sign}(a_{d+1}), \times)$? If so, what $\boldsymbol{w}$?

$$\boldsymbol{w}^\top \boldsymbol{x}_{d+2} = a_1 \underbrace{\boldsymbol{w}^\top \boldsymbol{x}_1}_{\circ} + a_2 \underbrace{\boldsymbol{w}^\top \boldsymbol{x}_2}_{\times} + \cdots + a_{d+1} \underbrace{\boldsymbol{w}^\top \boldsymbol{x}_{d+1}}_{\times} > 0 \quad \text{(contradiction!)}$$

The braces show a typical sign pattern, $a_1 > 0$ and the rest negative. Note the direction of the argument. To show that the $d+2$ inputs are *not* shattered, it is not enough to find one $\boldsymbol{w}$ with a bad output; we must exhibit one dichotomy that *no* $\boldsymbol{w}$ produces. The dichotomy is chosen from the coefficients a_i, and then every $\boldsymbol{w}$ that gets the first $d+1$ labels right is shown to be forced to say $\circ$ on $\boldsymbol{x}_{d+2}$.

Lemma 7.2.6 (Linear dependence blocks a dichotomy)

Let $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{d+2} \in \mathbb{R}^{d+1}$ be any $d+2$ inputs (bias coordinate $x_0 = 1$ included). Then some dichotomy $(y_1, \ldots, y_{d+2})$ is not $(\operatorname{sign}(\boldsymbol{w}^\top \boldsymbol{x}_1), \ldots, \operatorname{sign}(\boldsymbol{w}^\top \boldsymbol{x}_{d+2}))$ for any $\boldsymbol{w} \in \mathbb{R}^{d+1}$.

Proof. Step 1: the coefficients. $d+2$ vectors in $\mathbb{R}^{d+1}$ are linearly dependent, so $\sum_{i=1}^{d+2} c_i \boldsymbol{x}_i = \boldsymbol{0}$ for some c_i not all zero. Pick j with $c_j \neq 0$, divide by $-c_j$, and renumber so that $j = d+2$:

$$\boldsymbol{x}_{d+2} = \sum_{i=1}^{d+1} a_i \boldsymbol{x}_i.$$

At least one a_i is non-zero, since otherwise $\boldsymbol{x}_{d+2} = \boldsymbol{0}$, impossible because its first coordinate is $x_0 = 1$.

Step 2: the dichotomy. Define

$$y_i = \begin{cases} \operatorname{sign}(a_i) & \text{if } a_i \neq 0, \\ +1 \text{ (say; any value works)} & \text{if } a_i = 0, \end{cases} \qquad i = 1, \ldots, d+1, \qquad y_{d+2} = -1.$$

Step 3: no $\boldsymbol{w}$ produces it. Suppose $\boldsymbol{w}$ gives $\operatorname{sign}(\boldsymbol{w}^\top \boldsymbol{x}_i) = y_i$ for $i = 1, \ldots, d+1$. Take the inner product of Step 1 with $\boldsymbol{w}$:

$$\boldsymbol{w}^\top \boldsymbol{x}_{d+2} = \sum_{i=1}^{d+1} a_i \, (\boldsymbol{w}^\top \boldsymbol{x}_i).$$

Look at each term. If $a_i = 0$ the term is 0. If $a_i \neq 0$, then $\boldsymbol{w}^\top \boldsymbol{x}_i$ has the sign $y_i = \operatorname{sign}(a_i)$, in particular it is non-zero, so $a_i(\boldsymbol{w}^\top \boldsymbol{x}_i)$ is a product of two non-zero numbers of the same sign, hence > 0. Every term is ≥ 0 and at least one is > 0, so $\boldsymbol{w}^\top \boldsymbol{x}_{d+2} > 0$ and $\operatorname{sign}(\boldsymbol{w}^\top \boldsymbol{x}_{d+2}) = +1 \neq y_{d+2}$.

So every $\boldsymbol{w}$ that is right on the first $d+1$ inputs is wrong on the last, and $(y_1, \ldots, y_{d+2})$ is never produced. ∎

The 2D special case above is the instance $d = 2$, $\boldsymbol{x}_4 = \boldsymbol{x}_2 + \boldsymbol{x}_3 - \boldsymbol{x}_1$: $a = (-1, 1, 1)$, so the blocked dichotomy is $(\times, \circ, \circ, \times)$. Since the lemma holds for *any* $d+2$ inputs, no set of $d+2$ inputs is shattered: $m_{\mathcal{H}}(d+2) < 2^{d+2}$, i.e. $d+2$ is a break point, which is statement (4) of the last Extra Fun Time.

'General' X no-shatter $\Rightarrow$ $d_{\text{VC}} \leq d+1$.

With both steps done:

Theorem 7.2.7 (VC dimension of perceptrons)
For d-D perceptrons, $d_{\text{VC}} = d + 1$.

Exercise 7.2.8 (Fun Time). Based on the proof above, what is d_{VC} of 1126-D perceptrons? (1) 1024; (2) 1126; (3) 1127; (4) 6211.

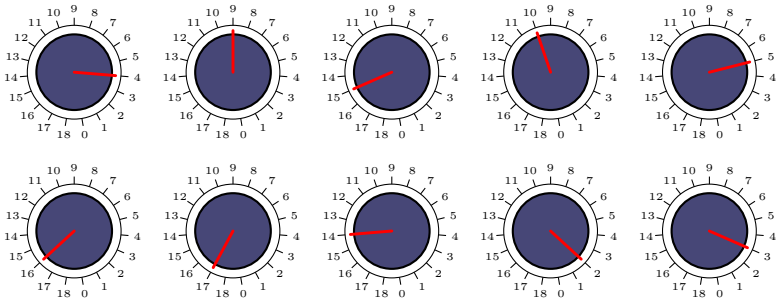
Reference Answer. (3). $1126 + 1 = 1127$. Well, too much fun for this section! :-) ■

§7.3. Physical Intuition of VC Dimension

slides p.17–20

Theorem 7.2.7 says a perceptron has d_{VC} equal to its number of weights. That is suggestive enough to ask what d_{VC} is measuring in general.

§7.3.i. Degrees of Freedom



(modified from the work of Hugues Vermeiren on `http://www.texample.net`)

Think of a hypothesis as a machine with knobs, each weight being one knob. The three quantities met so far are three ways of counting what the knobs can do:

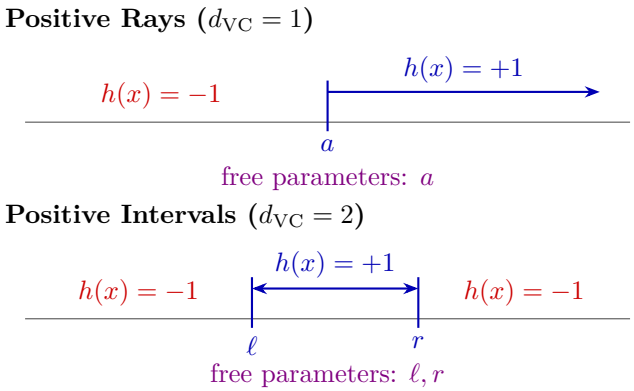
- hypothesis parameters $\boldsymbol{w} = (w_0, w_1, \ldots, w_d)$: **create degrees of freedom**;
- hypothesis quantity $M = |\mathcal{H}|$: 'analog' degrees of freedom;
- hypothesis 'power' $d_{\text{VC}} = d + 1$: **effective 'binary' degrees of freedom**.

M counts every knob setting, and is infinite as soon as a knob turns continuously. d_{VC} counts only how many inputs the knobs can label independently in *every* possible way — how many free binary choices the knobs really control.

$d_{\text{VC}}(\mathcal{H})$**: powerfulness of** $\mathcal{H}$**.**

§7.3.ii. Two Old Friends

The hypothesis sets of Section 5.3 fit the same picture.



One threshold, one binary degree of freedom; two endpoints, two. This suggests

Practical rule of thumb: $d_{\text{VC}} \approx$ #free parameters (but not always).

'Not always' in both directions: parameters can be redundant ($h(x) = \text{sign}(w_1 w_2 x)$ has two parameters but only the sign of the product matters), and a single parameter can be enormously powerful ($h(x) = \text{sign}(\sin(\alpha x))$ has one parameter and $d_{\text{VC}} = \infty$).

§7.3.iii. M and d_{VC}

Copied from Section 5.1 :-), the two central questions are:

(1) can we make sure that $E_{\text{out}}(g)$ is close enough to $E_{\text{in}}(g)$?

(2) can we make $E_{\text{in}}(g)$ small enough?

d_{VC} now plays exactly the role M played there, with M replaced by the polynomial $(2N)^{d_{\text{VC}}}$:

	(1) $E_{\text{out}} \approx E_{\text{in}}$?	(2) E_{in} small?
small M	Yes!, $\mathbb{P}[\textbf{BAD}] \leq 2 \cdot M \cdot \exp(\ldots)$	No!, too few choices
large M	No!, $\mathbb{P}[\textbf{BAD}] \leq 2 \cdot M \cdot \exp(\ldots)$	Yes!, many choices
small d_{VC}	Yes!, $\mathbb{P}[\textbf{BAD}] \leq 4 \cdot (2N)^{d_{\text{VC}}} \cdot \exp(\ldots)$	No!, too limited power
large d_{VC}	No!, $\mathbb{P}[\textbf{BAD}] \leq 4 \cdot (2N)^{d_{\text{VC}}} \cdot \exp(\ldots)$	Yes!, lots of power

Using the right d_{VC} (or $\mathcal{H}$) is important.

Exercise 7.3.1 (Fun Time)**.** Origin-crossing hyperplanes are essentially perceptrons with w_0 fixed at 0. Make a guess about the d_{VC} of origin-crossing hyperplanes in $\mathbb{R}^d$. (1) 1; (2) d; (3) $d + 1$; (4) ∞.

Reference Answer. (2). The proof is almost the same as proving the d_{VC} for usual perceptrons, but it is the intuition ($d_{\text{VC}} \approx$ #free parameters) that you shall use to answer this quiz: fixing w_0 removes one of the $d + 1$ knobs. (In the proof, drop the constant column of X: the d unit vectors are shattered, and any $d + 1$ vectors in $\mathbb{R}^d$ are linearly dependent.) ■

§7.4. Interpreting VC Dimension

slides p.21–25

Everything so far has been about *whether* generalisation happens. Turning the VC bound around shows *how much* it costs, in two currencies: error and data.

§7.4.i. VC Bound Rephrase: Penalty for Model Complexity

For any $g = \mathcal{A}(\mathcal{D}) \in \mathcal{H}$ and a 'statistical' large $\mathcal{D}$, for $N \geq 2$, $d_{\text{VC}} \geq 2$ (the condition '$k \geq 3$' of Section 7.1, now in terms of d_{VC}),

$$\mathbb{P}_{\mathcal{D}}\Big[\underbrace{|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon}_{\textbf{BAD}}\Big] \leq \underbrace{4(2N)^{d_{\text{VC}}} \exp\left(-\frac{1}{8}\epsilon^2 N\right)}_{\delta}.$$

Call the right-hand side δ. The bound then says: …, with probability $\geq 1 - \delta$, **GOOD**: $|E_{\text{in}}(g) - E_{\text{out}}(g)| \leq \epsilon$. Instead of fixing ϵ and asking for δ, fix the confidence δ and solve for ϵ:

$$\begin{aligned} \text{set} \quad \delta &= 4(2N)^{d_{\text{VC}}} \exp\left(-\frac{1}{8}\epsilon^2 N\right) \\ \frac{\delta}{4(2N)^{d_{\text{VC}}}} &= \exp\left(-\frac{1}{8}\epsilon^2 N\right) \\ \ln\left(\frac{4(2N)^{d_{\text{VC}}}}{\delta}\right) &= \frac{1}{8}\epsilon^2 N \\ \sqrt{\frac{8}{N}\ln\left(\frac{4(2N)^{d_{\text{VC}}}}{\delta}\right)} &= \epsilon. \end{aligned}$$

Rephrased: …, with probability $\geq 1 - \delta$, **GOOD!**

$$\text{gen. error } |E_{\text{in}}(g) - E_{\text{out}}(g)| \leq \sqrt{\frac{8}{N}\ln\left(\frac{4(2N)^{d_{\text{VC}}}}{\delta}\right)},$$

that is,

$$E_{\text{in}}(g) - \sqrt{\frac{8}{N}\ln\left(\frac{4(2N)^{d_{\text{VC}}}}{\delta}\right)} \;\leq\; E_{\text{out}}(g) \;\leq\; E_{\text{in}}(g) + \sqrt{\frac{8}{N}\ln\left(\frac{4(2N)^{d_{\text{VC}}}}{\delta}\right)}.$$

The left inequality is greyed out because nobody minds E_{out} being *smaller* than expected; only the upper bound matters.

Definition 7.4.1 (Penalty for model complexity)**.** The square root

$$\Omega(N, \mathcal{H}, \delta) = \sqrt{\frac{8}{N}\ln\left(\frac{4(2N)^{d_{\text{VC}}}}{\delta}\right)}$$

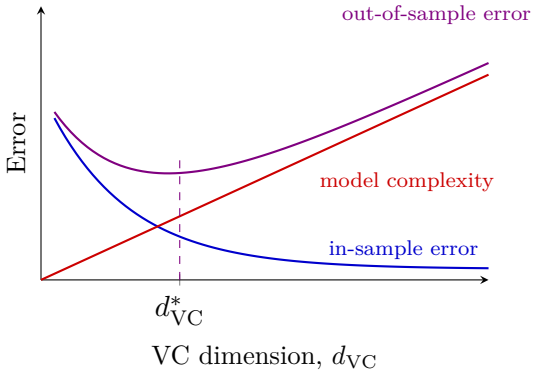
is the **penalty for model complexity**.

It depends on the data through N, on the hypothesis set through d_{VC}, and on how sure we want to be through δ.

§7.4.ii. THE VC Message

With a high probability,

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \underbrace{\sqrt{\frac{8}{N} \ln\left(\frac{4(2N)^{d_{\text{VC}}}}{\delta}\right)}}_{\Omega(N,\mathcal{H},\delta)}.$$



The picture reads the inequality as a function of d_{VC}, with the bound standing in for E_{out}:

- $d_{\text{VC}} \uparrow$: $E_{\text{in}} \downarrow$ but $\Omega \uparrow$;
- $d_{\text{VC}} \downarrow$: $\Omega \downarrow$ but $E_{\text{in}} \uparrow$;
- best d^*_{VC} in the middle.

The first bullet is the intuition of Section 7.3: d_{VC} counts the effective binary degrees of freedom, the powerfulness of $\mathcal{H}$, and a more powerful $\mathcal{H}$ can fit the labels better — the 'Yes!, lots of power' entry of the M-and-d_{VC} table. But the penalty grows with it, and their sum is smallest somewhere in between.

Remark 7.4.2. 'More powerful, hence smaller E_{in}' is an intuition, not a fact about d_{VC} alone. Making it precise takes two assumptions that the picture leaves implicit.

1. The hypothesis sets are *nested*: $\mathcal{H}_1 \subset \mathcal{H}_2 \subset \cdots$ with $d_{\text{VC}}(\mathcal{H}_1) \leq d_{\text{VC}}(\mathcal{H}_2) \leq \cdots$, as when more weights are added to a perceptron (setting the new ones to 0 recovers the old hypotheses).
2. $\mathcal{A}$ returns the g of smallest E_{in} in $\mathcal{H}$.

Under these,

$$\min_{h \in \mathcal{H}_2} E_{\text{in}}(h) \leq \min_{h \in \mathcal{H}_1} E_{\text{in}}(h),$$

since the best hypothesis of $\mathcal{H}_1$ is still available in $\mathcal{H}_2$. A larger d_{VC} means more dichotomies on the data, so a better match to $y_1, \ldots, y_N$ is usually found and the inequality is usually strict. Without nesting there is no such guarantee: on three collinear inputs labelled $\times \circ \times$, positive intervals along that line ($d_{\text{VC}} = 2$) reach $E_{\text{in}} = 0$, while 2D perceptrons ($d_{\text{VC}} = 3$) cannot.

Powerful $\mathcal{H}$ not always good!

§7.4.iii. VC Bound Rephrase: Sample Complexity

The same bound can be solved for N instead. Given specs $\epsilon = 0.1$, $\delta = 0.1$, $d_{\text{VC}} = 3$, we want

$$4(2N)^{d_{\text{VC}}} \exp\left(-\frac{1}{8}\epsilon^2 N\right) \leq \delta.$$

No closed form, but tabulating the left-hand side is enough:

N	bound
100	2.82×10^{7}
1,000	9.17×10^{9}
10,000	1.19×10^{8}
100,000	1.65×10^{-38}
29,300	9.99×10^{-2}

For small N the polynomial factor wins and the 'probability' bound is in the millions; the exponential only takes over between 10^4 and 10^5, and the spec is first met around $N = 29{,}300$. This gives the **sample complexity**: need $N \approx 10{,}000\, d_{\text{VC}}$ in theory.

Practical rule of thumb: $N \approx 10\, d_{\text{VC}}$ often enough!

§7.4.iv. Looseness of VC Bound

$$\mathbb{P}_{\mathcal{D}}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \leq 4(2N)^{d_{\text{VC}}} \exp\left(-\frac{1}{8}\epsilon^2 N\right)$$

Theory: $N \approx 10{,}000\, d_{\text{VC}}$; practice: $N \approx 10\, d_{\text{VC}}$.

Question 7.4.3. Why is the VC bound so loose?

Every step of the derivation paid for generality:

step	it must hold for
Hoeffding for unknown E_{out}	any distribution, any target
$m_{\mathcal{H}}(N)$ instead of $\lvert\mathcal{H}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)\rvert$	'any' data
$N^{d_{\text{VC}}}$ instead of $m_{\mathcal{H}}(N)$	'any' $\mathcal{H}$ of same d_{VC}
union bound on worst cases	any choice made by $\mathcal{A}$

— but hardly better, and 'similarly loose for all models'. Tightening one step usually means assuming something about P, f, the data or $\mathcal{A}$, which gives up the very generality that made the bound useful. And because the looseness is roughly the same for every model, the bound still ranks models correctly even when its numbers are useless.

Philosophical message of VC bound: important for improving ML.

Exercise 7.4.4 (Fun Time)**.** Consider the VC bound below. How can we decrease the probability of getting BAD data?

$$\mathbb{P}_{\mathcal{D}}\left[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\right] \leq 4(2N)^{d_{\text{VC}}} \exp\left(-\frac{1}{8}\epsilon^2 N\right)$$

(1) decrease model complexity d_{VC};

(2) increase data size N a lot;

(3) increase generalization error tolerance ϵ;

(4) all of the above.

Reference Answer. (4). Congratulations on being Master of VC bound! :-) Each of (1)–(3) shrinks the right-hand side; 'a lot' in (2) because the polynomial factor grows with N too, and only wins once N is large (the table above). ■

Summary

Definition of VC Dimension maximum non-break point.

VC Dimension of Perceptrons $d_{\text{VC}}(\mathcal{H}) = d + 1$.

Physical Intuition of VC Dimension $d_{\text{VC}} \approx$ #free parameters.

Interpreting VC Dimension loosely: model complexity & sample complexity.

Learning happens if d_{VC} is finite, N is large and E_{in} is low. All of that was derived for the cleanest possible setting: binary classification with a deterministic target and no noise. Next: more than noiseless binary classification?

8. Noise and Error

Lecture 8

30 Sep.

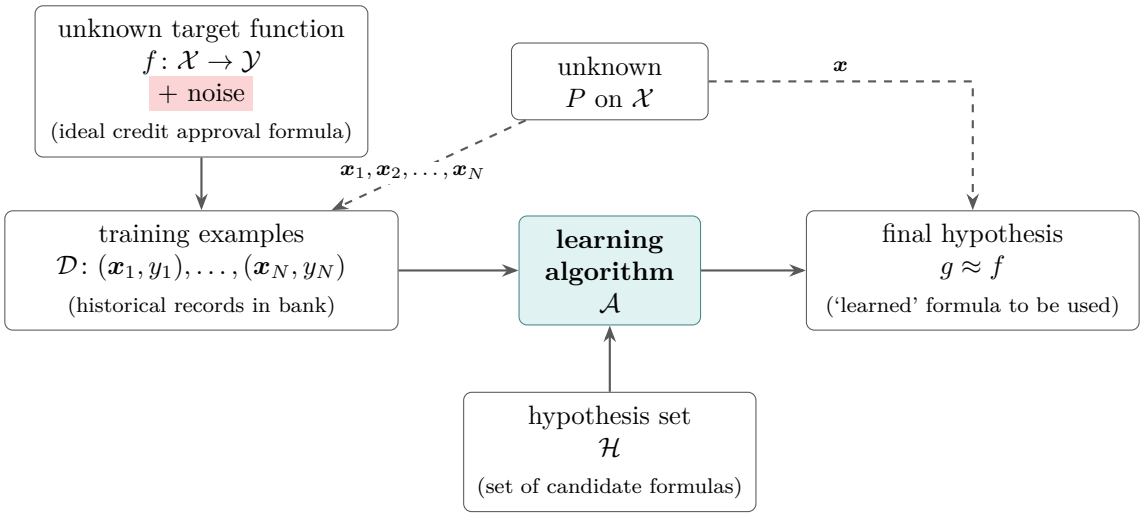
As previously seen . Section 7.4 ended with the VC message: learning happens if d_{VC} is finite, N is large, and E_{in} is low. The whole derivation, however, assumed a deterministic target f, labels $y_n = f(\boldsymbol{x}_n)$ with no noise, and binary classification measured by the fraction of mistakes.

This chapter relaxes the first two assumptions and questions the third. Noise turns the target function into a target *distribution*, and the VC bound survives the change. Once the target is a distribution, 'the best prediction' depends on how errors are measured, so the error measure becomes a component of the learning problem in its own right — chosen by the application, and then often replaced by something the algorithm can actually optimise. The last section works one such case, weighted classification, end to end.

§8.1. Noise and Probabilistic Target

slides p.2–7

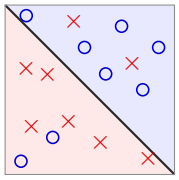
§8.1.i. Recap: The Learning Flow



This is the flow of Section 4.3, with one addition in the target box.

Question 8.1.1. What if there is noise?

§8.1.ii. Noise



Noise was briefly introduced before the **pocket** algorithm (Section 2.4), as the reason a data set might not be linearly separable even when the underlying rule is a line: a few examples sit on the wrong side of the best line.

But there is more! Take the credit applicant again:

age	23 years
gender	female
annual salary	NTD 1,000,000
year in residence	1 year
year in job	0.5 year
current debt	200,000

credit? $\{\text{no}(-1), \text{yes}(+1)\}$

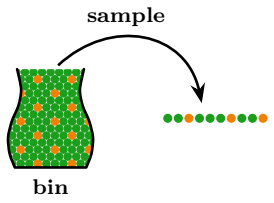
- **noise in y**: good customer, 'mislabeled' as bad?
- **noise in y**: same customers, different labels?
- **noise in $\boldsymbol{x}$**: inaccurate customer information?

The first is a clerical error on the label; the second is two applicants with identical records but different credit histories, which no function of $\boldsymbol{x}$ can fit; the third corrupts the input rather than the label. In every case the pair $(\boldsymbol{x}, y)$ is no longer tied by a function $y = f(\boldsymbol{x})$.

Question 8.1.2. Does the VC bound work under noise?

§8.1.iii. Probabilistic Marbles

One key of the VC bound is the marbles of Section 4.2:



Noise changes how a marble gets its colour, and nothing else:

'deterministic' marbles	'probabilistic' (noisy) marbles
marble $\boldsymbol{x} \sim P(\boldsymbol{x})$	marble $\boldsymbol{x} \sim P(\boldsymbol{x})$
deterministic colour $[\![f(\boldsymbol{x}) \neq h(\boldsymbol{x})]\!]$	probabilistic colour $[\![y \neq h(\boldsymbol{x})]\!]$ with $y \sim P(y \mid \boldsymbol{x})$

A noisy marble is one whose colour is decided afresh, by a coin flip, each time it is drawn. Hoeffding's inequality never cared how the colour was produced: all it needs is that the draws are i.i.d. and each is orange with the same probability.

Same nature: can estimate $\mathbb{P}$[orange] if $\overset{\text{i.i.d.}}{\sim}$.

So the whole chain of the last three chapters goes through if the examples are drawn as

$$\text{VC holds for } \underbrace{\boldsymbol{x} \overset{\text{i.i.d.}}{\sim} P(\boldsymbol{x}),\ y \overset{\text{i.i.d.}}{\sim} P(y \mid \boldsymbol{x})}_{(\boldsymbol{x},y) \overset{\text{i.i.d.}}{\sim} P(\boldsymbol{x},y)}.$$

The out-of-sample error of h now reads $E_{\text{out}}(h) = \mathbb{P}_{(\boldsymbol{x},y)}[h(\boldsymbol{x}) \neq y]$, the orange fraction of the noisy bin.

§8.1.iv. Target Distribution $P(y \mid \boldsymbol{x})$

Definition 8.1.3 (Target distribution)**.** The **target distribution** $P(y \mid \boldsymbol{x})$ characterizes the behaviour of the 'mini-target' on one $\boldsymbol{x}$.

- It can be viewed as 'ideal mini-target' + noise, e.g.
  - $P(\circ \mid \boldsymbol{x}) = 0.7$, $P(\times \mid \boldsymbol{x}) = 0.3$;
  - ideal mini-target $f(\boldsymbol{x}) = \circ$;
  - 'flipping' noise level = 0.3.
- A deterministic target f is a **special case of target distribution**:
  - $P(y \mid \boldsymbol{x}) = 1$ for $y = f(\boldsymbol{x})$;
  - $P(y \mid \boldsymbol{x}) = 0$ for $y \neq f(\boldsymbol{x})$.

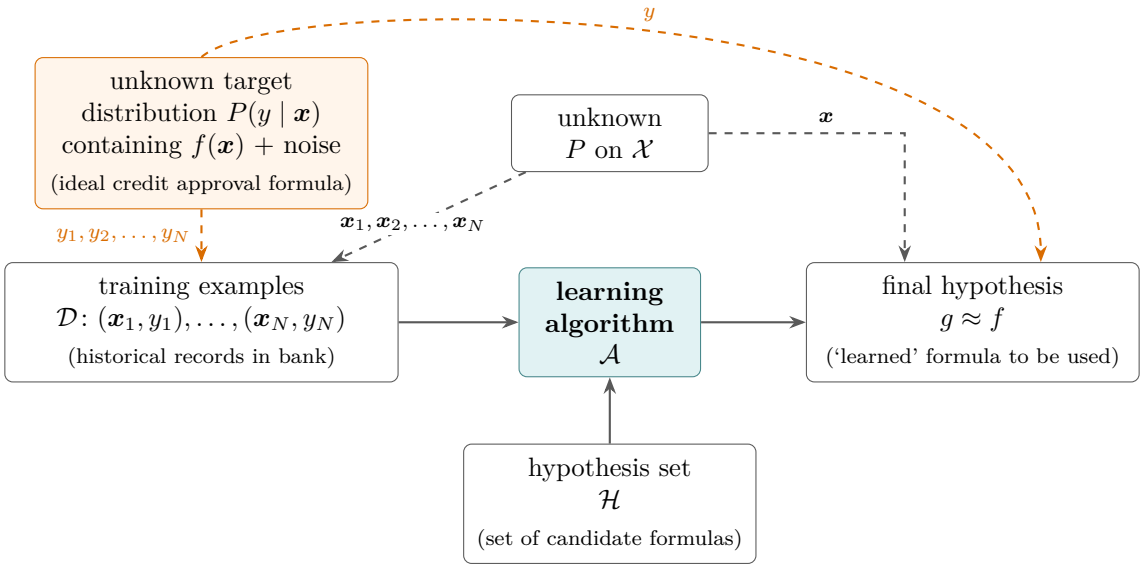
So nothing is lost by switching: every earlier setting is a target distribution concentrated on one label. The two distributions divide the work:

Goal of learning: predict the ideal mini-target (w.r.t. $P(y \mid \boldsymbol{x})$) on often-seen inputs (w.r.t. $P(\boldsymbol{x})$).

$P(y \mid \boldsymbol{x})$ is what we try to learn; $P(\boldsymbol{x})$ only decides where it matters to get it right.

§8.1.v. The New Learning Flow

In the flow, the target function box becomes a target distribution, which now supplies the labels $y_1, \ldots, y_N$ of the training set *and* the label y against which g is judged on a future $\boldsymbol{x}$.



VC still works, pocket algorithm explained :-)

Pocket was introduced for data that no line fits. With noise, that is the normal case even when the ideal mini-target is linear; the VC bound says the line with the smallest E_{in} still has $E_{\text{out}} \approx E_{\text{in}}$, so pocket's choice is justified.

Exercise 8.1.4 (Fun Time)**.** Let's revisit PLA/pocket. Which of the following claim is true?

(1) In practice, we should try to compute if $\mathcal{D}$ is linear separable before deciding to use PLA.

(2) If we know that $\mathcal{D}$ is not linear separable, then the target function f must not be a linear function.

(3) If we know that $\mathcal{D}$ is linear separable, then the target function f must be a linear function.

(4) None of the above.

Reference Answer. (4). (1) After computing if $\mathcal{D}$ is linear separable, we shall know $\boldsymbol{w}^*$ and then there is no need to use PLA. (2) What about noise? A linear target plus flipping noise gives non-separable data. (3) What about 'sampling luck'? :-) A curved target can happen to produce a sample that some line separates. ■

§8.2. Error Measure

slides p.8–13

The target is now a distribution, and g is supposed to be $\approx f$. Before learning anything, we need to say what '$\approx$' means.

§8.2.i. Error Measure

final hypothesis
$g \approx f$

- How well? Previously, we considered the out-of-sample measure

$$E_{\text{out}}(g) = \underset{\boldsymbol{x}\sim P}{\mathcal{E}} [\![g(\boldsymbol{x}) \neq f(\boldsymbol{x})]\!].$$

- More generally, an **error measure** $E(g, f)$.
- Naturally considered:
  - out-of-sample: averaged over unknown $\boldsymbol{x}$;
  - pointwise: evaluated on one $\boldsymbol{x}$;
  - classification: $[\![\text{prediction} \neq \text{target}]\!]$.

Here $\mathcal{E}$ denotes expectation. The three adjectives are three independent choices that E_{out} happened to make at once, and each could be made differently.

Classification error $[\![\ldots]\!]$: often also called '0/1 error'.

§8.2.ii. Pointwise Error Measure

We can often express $E(g, f)$ as an average of $\text{err}(g(\boldsymbol{x}), f(\boldsymbol{x}))$, like

$$E_{\text{out}}(g) = \underset{\boldsymbol{x}\sim P}{\mathcal{E}} \underbrace{[\![g(\boldsymbol{x}) \neq f(\boldsymbol{x})]\!]}_{\text{err}(g(\boldsymbol{x}), f(\boldsymbol{x}))}.$$

Definition 8.2.1 (Pointwise error measure)**.** The function err above is called a **pointwise error measure**. With it,

$$\underbrace{E_{\text{in}}(g) = \frac{1}{N}\sum_{n=1}^{N} \text{err}(g(\boldsymbol{x}_n), f(\boldsymbol{x}_n))}_{\text{in-sample}} \qquad \underbrace{E_{\text{out}}(g) = \mathop{\mathcal{E}}_{\boldsymbol{x}\sim P} \text{err}(g(\boldsymbol{x}), f(\boldsymbol{x}))}_{\text{out-of-sample}}.$$

Not every sensible measure is pointwise — the area under an ROC curve, say, compares predictions on different inputs — but we will mainly consider pointwise err for simplicity.

§8.2.iii. Two Important Pointwise Error Measures

Write the prediction and the target as

$$\text{err}\big(\underbrace{g(\boldsymbol{x})}_{\tilde{y}}, \underbrace{f(\boldsymbol{x})}_{y}\big).$$

0/1 error	**squared error**
$\text{err}(\tilde{y}, y) = [\![\tilde{y} \neq y]\!]$	$\text{err}(\tilde{y}, y) = (\tilde{y} - y)^2$
correct or incorrect?	how far is $\tilde{y}$ from y?
often for classification	often for regression

Question 8.2.2. How does err 'guide' learning?

§8.2.iv. Ideal Mini-Target

The interplay between noise and error:

> $P(y \mid \boldsymbol{x})$ **and** err **define the ideal mini-target** $f(\boldsymbol{x})$**.**

With noise there is no single 'true' label to recover at $\boldsymbol{x}$: what nature gives is the whole distribution $P(y \mid \boldsymbol{x})$. The f we aim at is therefore not a target function handed to us but the best *prediction* against that distribution, and 'best' only means something once err is fixed.

Definition 8.2.3 (Ideal mini-target)**.** Given $P(y \mid \boldsymbol{x})$ and err, the **ideal mini-target** at $\boldsymbol{x}$ is the prediction of smallest expected error,

$$f(\boldsymbol{x}) = \arg\min_{\tilde{y}} \mathop{\mathcal{E}}_{y\sim P(y|\boldsymbol{x})} \text{err}(\tilde{y}, y).$$

So from here on f denotes the error-minimising function, not a function that generated the labels; for a deterministic target ($P(y \mid \boldsymbol{x}) = 1$ at $y = f(\boldsymbol{x})$) and any err with $\text{err}(y, y) = 0 \leq \text{err}(\tilde{y}, y)$, the two coincide.

Take one input whose labels are

$$P(y = 1 \mid \boldsymbol{x}) = 0.2, \quad P(y = 2 \mid \boldsymbol{x}) = 0.7, \quad P(y = 3 \mid \boldsymbol{x}) = 0.1,$$

and ask which prediction $\tilde{y}$ has the smallest average error under each measure:

$\tilde{y}$	$\mathrm{err}(\tilde{y}, y) = [\![\tilde{y} \neq y]\!]$	$\mathrm{err}(\tilde{y}, y) = (\tilde{y} - y)^2$
1	avg. err 0.8	avg. err 1.1
2	avg. err 0.3 $(*)$	avg. err 0.3
3	avg. err 0.9	avg. err 1.5
1.9	avg. err 1.0 (really? :-))	avg. err 0.29 $(*)$
	$f(\boldsymbol{x}) = \arg\max_{y \in \mathcal{Y}} P(y \mid \boldsymbol{x})$	$f(\boldsymbol{x}) = \sum_{y \in \mathcal{Y}} y \cdot P(y \mid \boldsymbol{x})$

The two starred rows are no accident: they are the two minimisers below.

Proposition 8.2.4 (Ideal mini-targets of 0/1 and squared error)
Fix $\boldsymbol{x}$ and write $\mathcal{E}$ for expectation over $y \sim P(y \mid \boldsymbol{x})$.

1. For $\mathrm{err}(\tilde{y}, y) = [\![\tilde{y} \neq y]\!]$, $f(\boldsymbol{x}) = \arg\max_{y \in \mathcal{Y}} P(y \mid \boldsymbol{x})$ (any maximiser, if there are ties).
2. For $\mathrm{err}(\tilde{y}, y) = (\tilde{y} - y)^2$, $f(\boldsymbol{x}) = \sum_{y \in \mathcal{Y}} y \cdot P(y \mid \boldsymbol{x}) = \mathcal{E}[y]$, and this minimiser is unique.

Proof. (1) For any real $\tilde{y}$,

$$\mathcal{E}[\![\tilde{y} \neq y]\!] = \mathbb{P}[y \neq \tilde{y} \mid \boldsymbol{x}] = 1 - P(y = \tilde{y} \mid \boldsymbol{x}),$$

where $P(y = \tilde{y} \mid \boldsymbol{x}) = 0$ if $\tilde{y} \notin \mathcal{Y}$. Minimising the left side is maximising $P(y = \tilde{y} \mid \boldsymbol{x})$. A $\tilde{y}$ outside $\mathcal{Y}$ scores the worst possible value 1, so the minimum is attained exactly at the labels of largest probability.

(2) Let $\mu = \mathcal{E}[y]$. Add and subtract μ:

$$\mathcal{E}(\tilde{y} - y)^2 = \mathcal{E}\big[(\tilde{y} - \mu) + (\mu - y)\big]^2 = (\tilde{y} - \mu)^2 + 2(\tilde{y} - \mu)\,\mathcal{E}[\mu - y] + \mathcal{E}(\mu - y)^2.$$

The middle term vanishes because $\mathcal{E}[\mu - y] = \mu - \mu = 0$, and the last term is $\mathrm{Var}(y)$, which does not depend on $\tilde{y}$. Hence

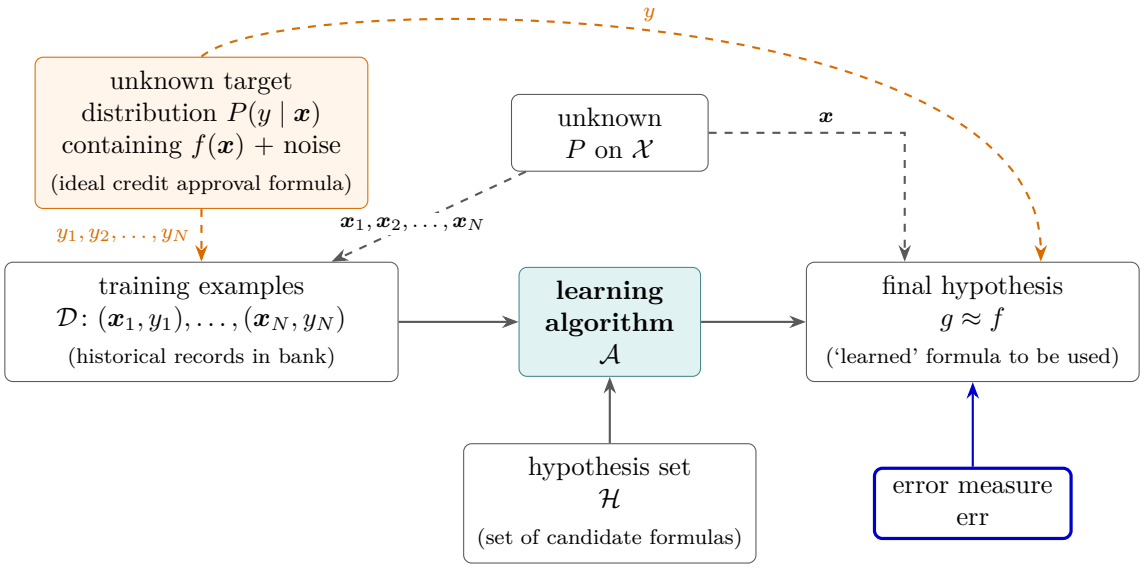
$$\mathcal{E}(\tilde{y} - y)^2 = (\tilde{y} - \mu)^2 + \mathrm{Var}(y) \geq \mathrm{Var}(y),$$

with equality if and only if $\tilde{y} = \mu$. ■

On the example, (1) gives $1 - 0.7 = 0.3$ at $\tilde{y} = 2$, and 1 at $\tilde{y} = 1.9$, which is never a label and so is always wrong. (2) gives $\mu = 0.2 + 1.4 + 0.3 = 1.9$ and $\mathrm{Var}(y) = 3.9 - 1.9^2 = 0.29$, the table's minimum; predicting 2 instead costs $(2 - 1.9)^2 + 0.29 = 0.3$. Same noise, different err, different ideal mini-target.

§8.2.v. Learning Flow with Error Measure

The error measure therefore joins the flow as a component: it is what '$g \approx f$' is judged by.



Extended VC theory/'philosophy' works for most $\mathcal{H}$ and err.

The proof in Section 6.4 was for 0/1 error; analogous bounds exist for other bounded error measures, with d_{VC} replaced by suitable generalisations. We take the philosophy on trust from here.

Exercise 8.2.5 (Fun Time)**.** Consider the following $P(y \mid \boldsymbol{x})$ and $\text{err}(\tilde{y}, y) = |\tilde{y} - y|$. Which of the following is the ideal mini-target $f(\boldsymbol{x})$?

$$P(y = 1 \mid \boldsymbol{x}) = 0.10, \quad P(y = 2 \mid \boldsymbol{x}) = 0.35,$$
$$P(y = 3 \mid \boldsymbol{x}) = 0.15, \quad P(y = 4 \mid \boldsymbol{x}) = 0.40.$$

(1) $2.5 =$ average within $\mathcal{Y} = \{1, 2, 3, 4\}$;

(2) $2.85 =$ weighted mean from $P(y \mid \boldsymbol{x})$;

(3) $3 =$ weighted median from $P(y \mid \boldsymbol{x})$;

(4) $4 = \arg\max P(y \mid \boldsymbol{x})$.

Reference Answer. (3). For the 'absolute error', the weighted median provably results in the minimum average err. The average errors of the four candidates are 1.0, 0.965, 0.95 and 1.15. Moving $\tilde{y}$ to the right by a little changes the average by (mass to the left) − (mass to the right); that is negative until half the mass is on the left, which happens at 3 ($0.10 + 0.35 + 0.15 = 0.6 \geq 0.5$). ■

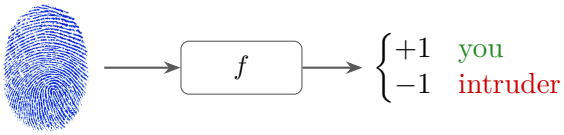
§8.3. Algorithmic Error Measure

slides p.14–19

The last section treated err as given. Who gives it?

§8.3.i. Choice of Error Measure

Fingerprint verification. A fingerprint goes in, and the target says whether it belongs to you:



There are two types of error: false accept and false reject.

		g	
		$+1$	-1
f	$+1$	no error	false reject
	-1	false accept	no error

0/1 error penalizes both types equally.

Whether that is right depends entirely on who uses the verifier.

§8.3.ii. Fingerprint Verification for Supermarket

		g	
		$+1$	-1
f	$+1$	0	10
	-1	1	0

- Supermarket: fingerprint for discount.
- False reject: **very unhappy customer, lose future business**.
- False accept: give away a minor discount, intruder left fingerprint :-)

§8.3.iii. Fingerprint Verification for CIA

		g	
		$+1$	-1
f	$+1$	0	1
	-1	1000	0

- CIA: fingerprint for entrance.
- False accept: **very serious consequences!**
- False reject: unhappy employee, but so what? :-)

Same inputs, same target, same two kinds of mistake — and cost ratios of $10:1$ one way and $1:1000$ the other. The best g for one application is a poor g for the other.

§8.3.iv. Take-home Message for Now

err **is application/user-dependent.**

In practice the user rarely writes the true err down, so the algorithm works with a stand-in.

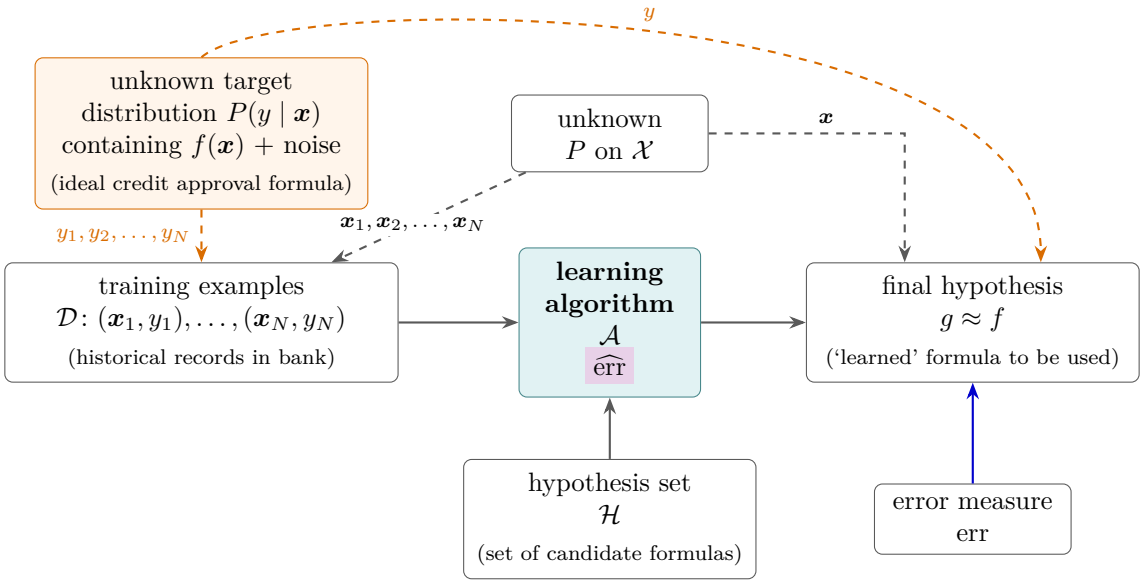
Definition 8.3.1 (Algorithmic error measure)**.** An **algorithmic error measure** $\widehat{\text{err}}$ is the error measure $\mathcal{A}$ actually uses in place of err. It may be

- true: just err;
- plausible:
  - 0/1: minimum 'flipping noise' — NP-hard to optimize, remember? :-)
  - squared: minimum Gaussian noise;
- friendly: easy to optimize for $\mathcal{A}$:
  - closed-form solution;
  - convex objective function.

'Plausible' means justified by a belief about the noise: if labels are flipped at random, the best one can do is make the fewest disagreements, which is 0/1 error — and minimising it over perceptrons is the NP-hard problem of Section 2.4; if the noise is Gaussian, maximum likelihood is least squares. 'Friendly' means justified by what the optimiser can do.

$\widehat{\text{err}}$**: more in next lectures.**

§8.3.v. Learning Flow with Algorithmic Error Measure



err**: application goal;** $\widehat{\text{err}}$**: a key part of many** $\mathcal{A}$**.**

err sits outside the algorithm and judges g; $\widehat{\text{err}}$ sits inside and steers the search. The art is choosing $\widehat{\text{err}}$ close enough to err to aim at the right goal, and friendly enough to be optimised.

Exercise 8.3.2 (Fun Time)**.** Consider err below for CIA. What is $E_{\text{in}}(g)$ when using this err?

		g	
		$+1$	-1
f	$+1$	0	1
	-1	1000	0

(1) $\frac{1}{N}\sum_{n=1}^{N}[\![y_n \neq g(\boldsymbol{x}_n)]\!]$;

(2) $\frac{1}{N}\left(\sum_{y_n=+1}[\![y_n \neq g(\boldsymbol{x}_n)]\!] + 1000\sum_{y_n=-1}[\![y_n \neq g(\boldsymbol{x}_n)]\!]\right)$;

(3) $\frac{1}{N}\left(\sum_{y_n=+1}[\![y_n \neq g(\boldsymbol{x}_n)]\!] - 1000\sum_{y_n=-1}[\![y_n \neq g(\boldsymbol{x}_n)]\!]\right)$;

(4) $\frac{1}{N}\left(1000\sum_{y_n=+1}[\![y_n \neq g(\boldsymbol{x}_n)]\!] + \sum_{y_n=-1}[\![y_n \neq g(\boldsymbol{x}_n)]\!]\right)$.

Reference Answer. (2). When $y_n = -1$, the false positive made on such $(\boldsymbol{x}_n, y_n)$ is penalized 1000 times more! ■

§8.4. Weighted Classification

slides p.20–24

The CIA error of the quiz is a concrete, useful err that is not 0/1. This section shows how to learn with it using nothing but the tools of Section 2.4.

§8.4.i. Weighted Classification

CIA cost (error, loss, ...) matrix.

		$h(\boldsymbol{x})$	
		$+1$	-1
y	$+1$	0	1
	-1	1000	0

Out-of-sample,

$$E_{\text{out}}(h) = \underset{(\boldsymbol{x},y)\sim P}{\mathcal{E}} \begin{Bmatrix} 1 & \text{if } y = +1 \\ 1000 & \text{if } y = -1 \end{Bmatrix} \cdot [\![y \neq h(\boldsymbol{x})]\!];$$

in-sample,

$$E_{\text{in}}(h) = \frac{1}{N}\sum_{n=1}^{N} \begin{Bmatrix} 1 & \text{if } y_n = +1 \\ 1000 & \text{if } y_n = -1 \end{Bmatrix} \cdot [\![y_n \neq h(\boldsymbol{x}_n)]\!].$$

The cost of a mistake depends only on the true label, so the matrix collapses into one weight per example.

Definition 8.4.1 (Weighted classification)**.** **Weighted classification** is classification with a different 'weight' for different $(\boldsymbol{x}, y)$: the pointwise error is $w(\boldsymbol{x}, y) \cdot [\![y \neq h(\boldsymbol{x})]\!]$ for a given weight $w(\boldsymbol{x}, y) \geq 0$.

§8.4.ii. Minimizing E_{in} for Weighted Classification

Write E_{in}^{w} for the weighted in-sample error,

$$E_{\text{in}}^{\text{w}}(h) = \frac{1}{N}\sum_{n=1}^{N}\begin{Bmatrix}1 & \text{if } y_n = +1\\ 1000 & \text{if } y_n = -1\end{Bmatrix} \cdot [\![y_n \neq h(\boldsymbol{x}_n)]\!],$$

and $E_{\text{in}}^{0/1}$ for the plain fraction of mistakes.

Naïve thoughts.

- PLA: doesn't matter if linear separable. :-) Then some $\boldsymbol{w}$ has $E_{\text{in}}^{\text{w}} = 0$, whatever the weights, and PLA finds it.
- Pocket: modify the pocket-replacement rule — if $\boldsymbol{w}_{t+1}$ reaches smaller E_{in}^{w} than $\hat{\boldsymbol{w}}$, replace $\hat{\boldsymbol{w}}$ by $\boldsymbol{w}_{t+1}$.

Question 8.4.2. Pocket has some guarantee on $E_{\text{in}}^{0/1}$; does the modified pocket have a similar guarantee on E_{in}^{w}?

§8.4.iii. Systematic Route: Connect E_{in}^{w} and $E_{\text{in}}^{0/1}$

Rather than re-prove anything about pocket, turn the weighted problem into an unweighted one.

original problem

		$h(\boldsymbol{x})$	
		$+1$	-1
y	$+1$	0	1
	-1	1000	0

$\mathcal{D}$:
$(\boldsymbol{x}_1, +1)$
$(\boldsymbol{x}_2, -1)$
$(\boldsymbol{x}_3, -1)$
$\cdots$
$(\boldsymbol{x}_{N-1}, +1)$
$(\boldsymbol{x}_N, +1)$

equivalent problem

		$h(\boldsymbol{x})$	
		$+1$	-1
y	$+1$	0	1
	-1	1	0

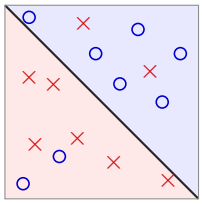
$(\boldsymbol{x}_1, +1)$
$(\boldsymbol{x}_2, -1), (\boldsymbol{x}_2, -1), \ldots, (\boldsymbol{x}_2, -1)$
$(\boldsymbol{x}_3, -1), (\boldsymbol{x}_3, -1), \ldots, (\boldsymbol{x}_3, -1)$
$\cdots$
$(\boldsymbol{x}_{N-1}, +1)$
$(\boldsymbol{x}_N, +1)$

After copying -1 examples 1000 times, E_{in}^{w} for LHS $\equiv$ $E_{\text{in}}^{0/1}$ for RHS!

A mistake on $(\boldsymbol{x}_2, -1)$ costs 1000 on the left and is repeated on all 1000 copies on the right, so the two error counts agree term by term for every h. (The normalisations differ, N against the size of the copied set, but that is a constant factor and does not change which h minimises.) Pocket's guarantee for $E_{\text{in}}^{0/1}$ on the copied data therefore *is* a guarantee for E_{in}^{w} on the original.

§8.4.iv. Weighted Pocket Algorithm

Copying 1000 times is wasteful; the copies only need to exist in how often examples are picked and how errors are counted.



		$h(\boldsymbol{x})$	
		$+1$	-1
y	$+1$	0	1
	-1	1000	0

Using 'virtual copying', the **weighted pocket algorithm** includes:

- weighted PLA: randomly check -1 example mistakes with 1000 times more probability;
- weighted pocket replacement: if $\boldsymbol{w}_{t+1}$ reaches smaller E_{in}^{w} than $\hat{\boldsymbol{w}}$, replace $\hat{\boldsymbol{w}}$ by $\boldsymbol{w}_{t+1}$.

Against algorithm 2.3, only two lines change:

Algorithm 8.1: Weighted Pocket Algorithm

Input: training examples $\mathcal{D}$, weights 1 for $y_n = +1$ and 1000 for $y_n = -1$, an iteration budget

Output: $\hat{\boldsymbol{w}}$, returned as g

initialize the pocket weights $\hat{\boldsymbol{w}}$;
for $t = 0, 1, \ldots$ *until enough iterations* **do**
  find a random mistake of $\boldsymbol{w}_t$, $(\boldsymbol{x}_{n(t)}, y_{n(t)})$, picking each mistake with probability proportional to its weight;
  (try to) correct the mistake by $\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + y_{n(t)}\boldsymbol{x}_{n(t)}$;
  if $E_{\text{in}}^{\text{w}}(\boldsymbol{w}_{t+1}) < E_{\text{in}}^{\text{w}}(\hat{\boldsymbol{w}})$ **then** $\hat{\boldsymbol{w}} \leftarrow \boldsymbol{w}_{t+1}$;
return $\hat{\boldsymbol{w}}$ as g;

Picking a random mistake of the copied data set is exactly picking an original mistake with probability proportional to its weight, and the copied data's $E_{\text{in}}^{0/1}$ is E_{in}^{w} up to a constant; so this *is* pocket on the copied data, run without making the copies.

Systematic route (called 'reduction'): can be applied to many other algorithms!

Exercise 8.4.3 (Fun Time)**.** Consider the CIA cost matrix. If there are 10 examples with $y_n = -1$ (intruder) and 999,990 examples with $y_n = +1$ (you). What would $E_{\text{in}}^{\text{w}}(h)$ be for a constant $h(\boldsymbol{x})$ that always returns $+1$?

		$h(\boldsymbol{x})$	
		$+1$	-1
y	$+1$	0	1
	-1	1000	0

(1) 0.001; (2) 0.01; (3) 0.1; (4) 1.

Reference Answer. (2). The constant h is wrong on exactly the 10 intruders, each costing 1000, out of $N = 10^6$ examples: $E_{\text{in}}^{\text{w}} = 10 \cdot 1000/10^6 = 0.01$. While the quiz is a simple evaluation, it is not uncommon that the data is very unbalanced for such an application. Properly 'setting' the weights can be used to avoid the lazy constant prediction: under plain 0/1 error the same h would score 10^{-5}, and look nearly perfect. ■

Summary

Noise and Probabilistic Target can replace $f(\boldsymbol{x})$ by $P(y \mid \boldsymbol{x})$.

Error Measure affects the 'ideal' target.

Algorithmic Error Measure user-dependent $\Rightarrow$ plausible or friendly.

Weighted Classification easily done by virtual 'example copying'.

The *Why can machines learn?* part closes here: with noise and a general error measure, the VC philosophy still says a finite d_{VC} and enough data make E_{in} a trustworthy proxy for E_{out}. What remains is to make E_{in} small, and so far the only tools are PLA and pocket. Next: more algorithms, please? :-)

9. Linear Regression

Lecture 9

7 Oct.

> **As previously seen .** Section 8.3 closed the *Why can machines learn?* part: learning can happen with a target distribution $P(y \mid \boldsymbol{x})$ and a low E_{in} with respect to err. Among the algorithmic error measures $\widehat{\text{err}}$ of Definition 8.3.1, squared error was called both plausible (it matches Gaussian noise) and friendly (it admits a closed-form solution), but no algorithm has used it yet.

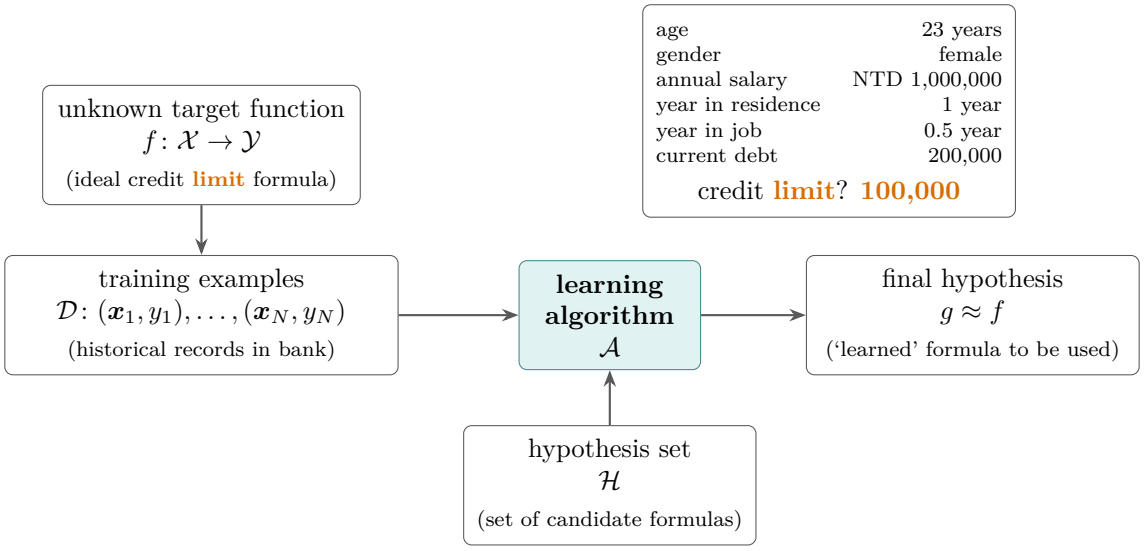
This chapter opens the *How can machines learn?* part with the most direct use of squared error: predict a real number by a weighted sum of the features, and choose the weights that minimise the average squared error. The minimiser comes out in closed form; its in-sample error can be computed exactly in expectation, which gives a generalisation guarantee simpler than VC; and the same weights turn out to be a cheap starting point for binary classification.

§9.1. Linear Regression Problem

slides p.2–6

§9.1.i. Credit Limit Problem

Return to the bank of Section 2.1, but change the question: instead of approving or rejecting the applicant, decide *how much* credit to give.



The record is the same; only the output space has changed, from $\{-1, +1\}$ to amounts of money.

> $\mathcal{Y} = \mathbb{R}$: **regression.**

This is the regression problem of Definition 3.1.4, met there only by name.

§9.1.ii. Linear Regression Hypothesis

age	23 years
annual salary	NTD 1,000,000
year in job	0.5 year
current debt	200,000

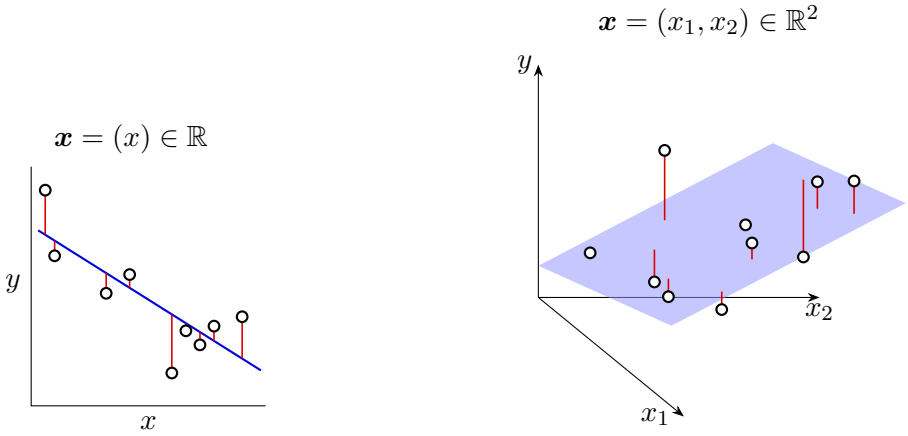
- For $\boldsymbol{x} = (x_0, x_1, x_2, \ldots, x_d)$ 'features of customer', approximate the desired credit limit with a weighted sum:
$$y \approx \sum_{i=0}^{d} w_i x_i.$$
- **Linear regression hypothesis**: $h(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x}$.

As in Section 2.1, $x_0 = 1$ is the constant feature, so w_0 plays the role of an intercept. The perceptron computed the same score and then took its sign to answer yes or no; for an amount, the score itself is the answer.

$h(\boldsymbol{x})$: like perceptron, but without the sign.

§9.1.iii. Illustration of Linear Regression

Each picture shows a data set, the best-fitting linear hypothesis, and the vertical gaps between them. (The data sets are mine, and the drawn line and plane are their actual least-squares fits.)



Definition 9.1.1 (Residual)**.** The **residual** of h on an example $(\boldsymbol{x}_n, y_n)$ is the vertical gap $y_n - h(\boldsymbol{x}_n)$, drawn in red above.

With one feature the hypothesis is a line in the (x, y)-plane; with two it is a plane over the (x_1, x_2)-plane; in general a hyperplane in $\mathbb{R}^{d+1}$.

Linear regression: find lines/hyperplanes with small residuals.

§9.1.iv. The Error Measure

'Small residuals' needs a single number to minimise. The popular/historical error measure is the squared error
$$\mathrm{err}(\hat{y}, y) = (\hat{y} - y)^2,$$

the second pointwise measure of Section 8.2. With it,

$$\underbrace{E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N}\big(\underbrace{h(\boldsymbol{x}_n)}_{\boldsymbol{w}^\top\boldsymbol{x}_n} - y_n\big)^2}_{\text{in-sample}} \qquad \underbrace{E_{\text{out}}(\boldsymbol{w}) = \mathop{\mathcal{E}}_{(\boldsymbol{x},y)\sim P}(\boldsymbol{w}^\top\boldsymbol{x} - y)^2}_{\text{out-of-sample}}.$$

Both errors are written as functions of $\boldsymbol{w}$, since $\boldsymbol{w}$ is the hypothesis. E_{out} averages over the joint $P(\boldsymbol{x}, y)$ of Section 8.1, so the noise in y is part of it.

Question 9.1.2. How do we minimize $E_{\text{in}}(\boldsymbol{w})$?

Exercise 9.1.3 (Fun Time)**.** Consider using linear regression hypothesis $h(\boldsymbol{x}) = \boldsymbol{w}^\top\boldsymbol{x}$ to predict the credit limit of customers $\boldsymbol{x}$. Which feature below shall have a positive weight in a **good hypothesis** for the task?

(1) birth month;

(2) monthly income;

(3) current debt;

(4) number of credit cards owned.

Reference Answer. (2). Customers with higher monthly income should naturally be given a higher credit limit, which is captured by the positive weight on the 'monthly income' feature. ■

§9.2. Linear Regression Algorithm

slides p.7–12

The question of the last section is answered by rewriting E_{in} as a function of one vector and setting its gradient to zero.

§9.2.i. Matrix Form of $E_{\text{in}}(\boldsymbol{w})$

Notation . Stack the inputs as the rows of the **input matrix** X and the outputs into the **output vector** $\boldsymbol{y}$. Roman capitals X, H, I denote matrices, as distinct from the sets $\mathcal{X}$ and $\mathcal{H}$.

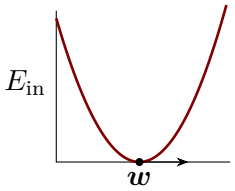
The sum of squares is the squared length of one vector of residuals:

$$\begin{aligned} E_{\text{in}}(\boldsymbol{w}) &= \frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{w}^\top\boldsymbol{x}_n - y_n)^2 = \frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{x}_n^\top\boldsymbol{w} - y_n)^2 \\ &= \frac{1}{N}\left\|\begin{bmatrix}\boldsymbol{x}_1^\top\boldsymbol{w} - y_1\\ \boldsymbol{x}_2^\top\boldsymbol{w} - y_2\\ \dots\\ \boldsymbol{x}_N^\top\boldsymbol{w} - y_N\end{bmatrix}\right\|^2 = \frac{1}{N}\left\|\begin{bmatrix}- & - & \boldsymbol{x}_1^\top & - & -\\ - & - & \boldsymbol{x}_2^\top & - & -\\ & & \dots & & \\ - & - & \boldsymbol{x}_N^\top & - & -\end{bmatrix}\boldsymbol{w} - \begin{bmatrix}y_1\\ y_2\\ \dots\\ y_N\end{bmatrix}\right\|^2 \\ &= \frac{1}{N}\big\| \underbrace{\mathrm{X}}_{N\times d+1}\ \underbrace{\boldsymbol{w}}_{d+1\times 1} - \underbrace{\boldsymbol{y}}_{N\times 1}\big\|^2. \end{aligned}$$

§9.2.ii. Minimizing E_{in}

$$\min_{\boldsymbol{w}} E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}\|\mathrm{X}\boldsymbol{w} - \boldsymbol{y}\|^2$$

- $E_{\text{in}}(\boldsymbol{w})$: continuous, differentiable, **convex**;
- necessary condition of 'best' $\boldsymbol{w}$:



$$\nabla E_{\text{in}}(\boldsymbol{w}) \equiv \begin{bmatrix} \frac{\partial E_{\text{in}}}{\partial w_0}(\boldsymbol{w}) \\ \frac{\partial E_{\text{in}}}{\partial w_1}(\boldsymbol{w}) \\ \cdots \\ \frac{\partial E_{\text{in}}}{\partial w_d}(\boldsymbol{w}) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \cdots \\ 0 \end{bmatrix}$$

— not possible to 'roll down'.

At a point where every partial derivative vanishes, no small move in any direction decreases E_{in} to first order. For a general function that only makes the point a candidate; convexity (shown below the gradient formula) makes every such point a global minimum, so the condition is also sufficient here.

Task: find $\boldsymbol{w}_{\text{LIN}}$ such that $\nabla E_{\text{in}}(\boldsymbol{w}_{\text{LIN}}) = \boldsymbol{0}$.

§9.2.iii. The Gradient $\nabla E_{\text{in}}(\boldsymbol{w})$

Expand the square:

$$E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}\|\mathrm{X}\boldsymbol{w} - \boldsymbol{y}\|^2 = \frac{1}{N}\Big(\boldsymbol{w}^\top \underbrace{\mathrm{X}^\top\mathrm{X}}_{\mathrm{A}}\boldsymbol{w} - 2\boldsymbol{w}^\top \underbrace{\mathrm{X}^\top\boldsymbol{y}}_{\boldsymbol{b}} + \underbrace{\boldsymbol{y}^\top\boldsymbol{y}}_{c}\Big).$$

This is a quadratic in $\boldsymbol{w}$, and its gradient is the vector version of a derivative everyone knows:

one w only	vector $\boldsymbol{w}$
$E_{\text{in}}(w) = \frac{1}{N}\left(aw^2 - 2bw + c\right)$	$E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}\left(\boldsymbol{w}^\top\mathrm{A}\boldsymbol{w} - 2\boldsymbol{w}^\top\boldsymbol{b} + c\right)$
$\nabla E_{\text{in}}(w) = \frac{1}{N}(2aw - 2b)$	$\nabla E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}(2\mathrm{A}\boldsymbol{w} - 2\boldsymbol{b})$
simple! :-)	similar (**derived by definition**)

Lemma 9.2.1 (Gradient of a quadratic form)
For a symmetric A, $\nabla_{\boldsymbol{w}}(\boldsymbol{w}^\top\mathrm{A}\boldsymbol{w}) = 2\mathrm{A}\boldsymbol{w}$ and $\nabla_{\boldsymbol{w}}(\boldsymbol{w}^\top\boldsymbol{b}) = \boldsymbol{b}$.

Proof. By definition, one coordinate at a time. $\boldsymbol{w}^\top\boldsymbol{b} = \sum_j w_j b_j$ has $\partial/\partial w_i = b_i$. For the quadratic form,

$$\frac{\partial}{\partial w_i}\sum_{j,k} w_j A_{jk} w_k = \sum_k A_{ik}w_k + \sum_j w_j A_{ji} = 2(\mathrm{A}\boldsymbol{w})_i,$$

the last step by $A_{ji} = A_{ij}$. ∎

$\mathrm{A} = \mathrm{X}^\top\mathrm{X}$ is symmetric, so Lemma 9.2.1 applies and

$$\nabla E_{\text{in}}(\boldsymbol{w}) = \frac{2}{N}\left(\mathrm{X}^\top\mathrm{X}\boldsymbol{w} - \mathrm{X}^\top\boldsymbol{y}\right).$$

Differentiating once more gives the constant Hessian $\frac{2}{N}\mathrm{X}^\top\mathrm{X}$, which is positive semi-definite because $\boldsymbol{v}^\top\mathrm{X}^\top\mathrm{X}\boldsymbol{v} = \|\mathrm{X}\boldsymbol{v}\|^2 \geq 0$; this is the convexity claimed above.

§9.2.iv. Optimal Linear Regression Weights

Task: find $\boldsymbol{w}_{\mathrm{LIN}}$ such that $\frac{2}{N}\left(\mathrm{X}^\top\mathrm{X}\boldsymbol{w} - \mathrm{X}^\top\boldsymbol{y}\right) = \nabla E_{\mathrm{in}}(\boldsymbol{w}) = \boldsymbol{0}$, i.e. solve the linear system

$$\mathrm{X}^\top\mathrm{X}\boldsymbol{w} = \mathrm{X}^\top\boldsymbol{y} \tag{9.1}$$

(the **normal equations**). Whether it is easy depends on the $(d+1)\times(d+1)$ matrix $\mathrm{X}^\top\mathrm{X}$:

invertible $\mathrm{X}^\top\mathrm{X}$

- **easy!** unique solution

$$\boldsymbol{w}_{\mathrm{LIN}} = \underbrace{\left(\mathrm{X}^\top\mathrm{X}\right)^{-1}\mathrm{X}^\top}_{\text{pseudo-inverse } \mathrm{X}^\dagger}\boldsymbol{y};$$

- often the case because $N \gg d+1$.

singular $\mathrm{X}^\top\mathrm{X}$

- **many** optimal solutions;
- one of the solutions

$$\boldsymbol{w}_{\mathrm{LIN}} = \mathrm{X}^\dagger\boldsymbol{y}$$

by defining $\mathrm{X}^\dagger$ in other ways.

Lemma 9.2.2 (When $\mathrm{X}^\top\mathrm{X}$ is invertible)
$\ker(\mathrm{X}^\top\mathrm{X}) = \ker\mathrm{X}$, so $\mathrm{X}^\top\mathrm{X}$ is invertible iff $\operatorname{rank}\mathrm{X} = d{+}1$; in particular this needs $N \geq d{+}1$. If the $\boldsymbol{x}_n$ are i.i.d. with $\Sigma = \mathcal{E}[\boldsymbol{x}\boldsymbol{x}^\top]$ finite and invertible, then $\mathrm{X}^\top\mathrm{X}$ is invertible for all large enough N, with probability 1.

Proof. $\mathrm{X}^\top\mathrm{X}\boldsymbol{v} = \boldsymbol{0}$ implies $0 = \boldsymbol{v}^\top\mathrm{X}^\top\mathrm{X}\boldsymbol{v} = \|\mathrm{X}\boldsymbol{v}\|^2$, i.e. $\mathrm{X}\boldsymbol{v} = \boldsymbol{0}$; the converse is clear. A square matrix is invertible iff its kernel is $\{\boldsymbol{0}\}$, and by rank–nullity $\ker\mathrm{X} = \{\boldsymbol{0}\}$ iff $\operatorname{rank}\mathrm{X} = d+1$, which is impossible when $\operatorname{rank}\mathrm{X} \leq N < d+1$.

For large N: by the strong law of large numbers, entrywise,

$$\frac{1}{N}\mathrm{X}^\top\mathrm{X} = \frac{1}{N}\sum_{n=1}^{N}\boldsymbol{x}_n\boldsymbol{x}_n^\top \longrightarrow \Sigma \quad \text{a.s.}$$

The determinant is continuous, so $\det\left(\frac{1}{N}\mathrm{X}^\top\mathrm{X}\right) \to \det\Sigma \neq 0$, and it is nonzero for all large N. ∎

Σ fails to be invertible only if $\boldsymbol{v}^\top\Sigma\boldsymbol{v} = \mathcal{E}[(\boldsymbol{x}^\top\boldsymbol{v})^2] = 0$ for some $\boldsymbol{v} \neq \boldsymbol{0}$, i.e. $P(\boldsymbol{x})$ lives on a hyperplane — the features are linearly dependent, such as $x_2 = x_1$ for every input. That is the one case more data cannot fix.

Lemma 9.2.3 (The normal equations are always solvable)
(9.1) has a solution $\boldsymbol{w}^*$ for every $\boldsymbol{y}$, and its solution set is $\boldsymbol{w}^* + \ker\mathrm{X}$. Hence if $\mathrm{X}^\top\mathrm{X}$ is singular there are infinitely many solutions, all with the same predictions $\mathrm{X}\boldsymbol{w}^*$ and the same E_{in}.

Proof. Existence. Clearly $\operatorname{range}(\mathrm{X}^\top\mathrm{X}) \subseteq \operatorname{range}(\mathrm{X}^\top)$. By rank–nullity, Lemma 9.2.2 and row rank = column rank,

$$\operatorname{rank}(\mathrm{X}^\top\mathrm{X}) = (d+1) - \dim\ker\mathrm{X} = \operatorname{rank}\mathrm{X} = \operatorname{rank}\mathrm{X}^\top,$$

so the two subspaces are equal and $\mathrm{X}^\top\boldsymbol{y} \in \operatorname{range}(\mathrm{X}^\top\mathrm{X})$.

Solution set. Two solutions differ by an element of $\ker(\mathrm{X}^\top\mathrm{X}) = \ker \mathrm{X}$ (Lemma 9.2.2), and adding $\boldsymbol{v} \in \ker \mathrm{X}$ changes neither $\mathrm{X}^\top\mathrm{X}\boldsymbol{w}$ nor $\mathrm{X}\boldsymbol{w}$. If $\mathrm{X}^\top\mathrm{X}$ is singular, $\ker \mathrm{X} \neq \{\mathbf{0}\}$ has positive dimension. ■

Geometrically, existence is the projection theorem: $\boldsymbol{y} = \hat{\boldsymbol{y}} + \boldsymbol{r}$ with $\hat{\boldsymbol{y}}$ in the column space of X and $\mathrm{X}^\top\boldsymbol{r} = \mathbf{0}$; writing $\hat{\boldsymbol{y}} = \mathrm{X}\boldsymbol{w}^*$ gives $\mathrm{X}^\top(\boldsymbol{y} - \mathrm{X}\boldsymbol{w}^*) = \mathbf{0}$. Among the solutions, the SVD-based $\mathrm{X}^\dagger\boldsymbol{y}$ is the one in $(\ker \mathrm{X})^\perp$, which by Pythagoras has the smallest norm.

Practical suggestion: use well-implemented † routine instead of $\left(\mathrm{X}^\top\mathrm{X}\right)^{-1}\mathrm{X}^\top$, for numerical stability when almost-singular.

Forming $\mathrm{X}^\top\mathrm{X}$ squares the condition number of X, so an almost-singular problem becomes much worse once that product is formed and inverted.

§9.2.v. Linear Regression Algorithm

Algorithm 9.1: Linear Regression

Input: training examples $\mathcal{D}$

Output: $\boldsymbol{w}_{\mathrm{LIN}}$, returned as g

1 from $\mathcal{D}$, construct input matrix X and output vector $\boldsymbol{y}$ by

$$\mathrm{X} = \underbrace{\begin{bmatrix} - & - & \boldsymbol{x}_1^\top & - & - \\ - & - & \boldsymbol{x}_2^\top & - & - \\ & & \cdots & & \\ - & - & \boldsymbol{x}_N^\top & - & - \end{bmatrix}}_{N \times (d+1)} \qquad \boldsymbol{y} = \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \cdots \\ y_N \end{bmatrix}}_{N \times 1};$$

2 calculate pseudo-inverse $\underbrace{\mathrm{X}^\dagger}_{(d+1)\times N}$;

3 **return** $\underbrace{\boldsymbol{w}_{\mathrm{LIN}}}_{(d+1)\times 1} = \mathrm{X}^\dagger\boldsymbol{y}$;

Simple and efficient with good † routine.

The cost is dominated by the pseudo-inverse, about $O(Nd^2)$ operations, once, with no iteration count to choose.

Exercise 9.2.4 (Fun Time)**.** After getting $\boldsymbol{w}_{\mathrm{LIN}}$, we can calculate the predictions $\hat{y}_n = \boldsymbol{w}_{\mathrm{LIN}}^\top\boldsymbol{x}_n$. If all $\hat{y}_n$ are collected in a vector $\hat{\boldsymbol{y}}$ similar to how we form $\boldsymbol{y}$, what is the matrix formula of $\hat{\boldsymbol{y}}$?

(1) $\boldsymbol{y}$;

(2) $\mathrm{X}\mathrm{X}^\top\boldsymbol{y}$;

(3) $\mathrm{X}\mathrm{X}^\dagger\boldsymbol{y}$;

(4) $\mathrm{X}\mathrm{X}^\dagger\mathrm{X}\mathrm{X}^\top\boldsymbol{y}$.

Reference Answer. (3). Note that $\hat{\boldsymbol{y}} = \mathrm{X}\boldsymbol{w}_{\mathrm{LIN}}$. Then, a simple substitution of $\boldsymbol{w}_{\mathrm{LIN}}$ reveals the answer. ■

§9.3. Generalization Issue

slides p.13–18

Linear regression (algorithm 9.1) returns the E_{in}-minimiser in one shot. Two questions remain: is that learning at all, and how far is E_{out} from E_{in}?

§9.3.i. Is Linear Regression a 'Learning Algorithm'?

$$\boldsymbol{w}_{\text{LIN}} = \mathrm{X}^{\dagger}\boldsymbol{y}$$

No!

- analytic (closed-form) solution, 'instantaneous';
- not improving E_{in} nor E_{out} iteratively.

Yes!

- good E_{in}? **yes, optimal!**
- good E_{out}? **yes, finite d_{VC} like perceptrons**
- improving iteratively? **somewhat, within an iterative pseudo-inverse routine**

The 'No' column judges the algorithm by how it looks; the 'Yes' column by what Section 7.4 says learning requires — low E_{in} and $E_{\text{out}} \approx E_{\text{in}}$. Linear hypotheses in $\mathbb{R}^{d+1}$ have the same $d_{\text{VC}} = d+1$ as perceptrons (Theorem 7.2.7), as far as their sign patterns go.

If $E_{\text{out}}(\boldsymbol{w}_{\text{LIN}})$ is good, learning 'happened'!

§9.3.ii. Benefit of Analytic Solution: 'Simpler-than-VC' Guarantee

Because $\boldsymbol{w}_{\text{LIN}}$ has a formula, its in-sample error can be averaged over data sets exactly, rather than bounded in the worst case. Write $\overline{E_{\text{in}}}$ for that average:

$$\overline{E_{\text{in}}} = \mathop{\mathcal{E}}_{\mathcal{D}\sim P^N}\left\{E_{\text{in}}(\boldsymbol{w}_{\text{LIN}} \text{ w.r.t. } \mathcal{D})\right\} \overset{\text{to be shown}}{=} \text{noise level} \cdot \left(1 - \frac{d+1}{N}\right).$$

Inside the braces, for one fixed $\mathcal{D}$,

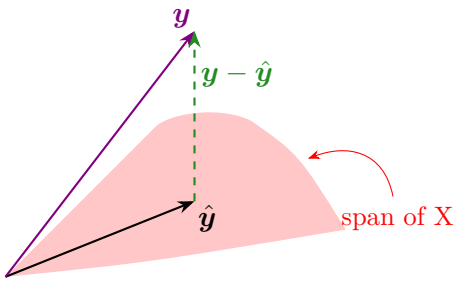
$$E_{\text{in}}(\boldsymbol{w}_{\text{LIN}}) = \frac{1}{N}\left\|\boldsymbol{y} - \underbrace{\hat{\boldsymbol{y}}}_{\text{predictions}}\right\|^2 = \frac{1}{N}\left\|\boldsymbol{y} - \mathrm{X}\underbrace{\mathrm{X}^{\dagger}\boldsymbol{y}}_{\boldsymbol{w}_{\text{LIN}}}\right\|^2$$
$$= \frac{1}{N}\left\|\left(\underbrace{\mathrm{I}}_{\text{identity}} - \mathrm{X}\mathrm{X}^{\dagger}\right)\boldsymbol{y}\right\|^2.$$

Definition 9.3.1 (Hat matrix)**.** Call $\mathrm{H} = \mathrm{X}\mathrm{X}^{\dagger}$ the **hat matrix**, because it puts $\wedge$ on $\boldsymbol{y}$: $\hat{\boldsymbol{y}} = \mathrm{H}\boldsymbol{y}$.

So $E_{\text{in}}(\boldsymbol{w}_{\text{LIN}}) = \frac{1}{N}\|(\mathrm{I}-\mathrm{H})\boldsymbol{y}\|^2$, and the whole question is what $\mathrm{I}-\mathrm{H}$ does to a vector. When $\mathrm{X}^{\top}\mathrm{X}$ is invertible, $\mathrm{H} = \mathrm{X}(\mathrm{X}^{\top}\mathrm{X})^{-1}\mathrm{X}^{\top}$, which is what we use below.

§9.3.iii. Geometric View of Hat Matrix

Think of $\boldsymbol{y}$ and $\hat{\boldsymbol{y}}$ as two points in $\mathbb{R}^N$, one coordinate per example.



In $\mathbb{R}^N$:

- $\hat{\boldsymbol{y}} = \mathrm{X}\boldsymbol{w}_{\mathrm{LIN}}$ within the span of X columns;
- $\boldsymbol{y} - \hat{\boldsymbol{y}}$ smallest: $\boldsymbol{y} - \hat{\boldsymbol{y}} \perp$ span;
- H: project $\boldsymbol{y}$ to $\hat{\boldsymbol{y}} \in$ span;
- $\mathrm{I} - \mathrm{H}$: transform $\boldsymbol{y}$ to $\boldsymbol{y} - \hat{\boldsymbol{y}} \perp$ span.

Every $\mathrm{X}\boldsymbol{w}$ is a combination of the columns of X, so minimising $\|\mathrm{X}\boldsymbol{w} - \boldsymbol{y}\|$ asks for the point of the span nearest to $\boldsymbol{y}$: the foot of the perpendicular. The normal equations say exactly this, since $\mathrm{X}^\top(\boldsymbol{y} - \mathrm{X}\boldsymbol{w}_{\mathrm{LIN}}) = \boldsymbol{0}$ means the residual is orthogonal to every column.

Question 9.3.2. Claim: $\mathrm{trace}(\mathrm{I} - \mathrm{H}) = N - (d+1)$. Why? :-)

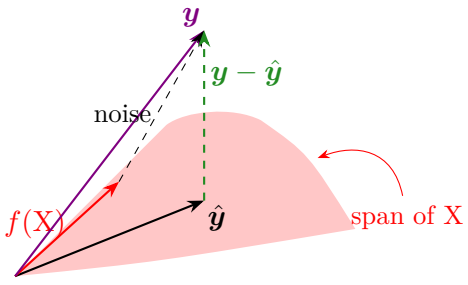
Answer. Assume $\mathrm{X}^\top\mathrm{X}$ invertible. Using $\mathrm{trace}(\mathrm{AB}) = \mathrm{trace}(\mathrm{BA})$,

$$\mathrm{trace}(\mathrm{H}) = \mathrm{trace}\big(\mathrm{X}(\mathrm{X}^\top\mathrm{X})^{-1}\mathrm{X}^\top\big) = \mathrm{trace}\big((\mathrm{X}^\top\mathrm{X})^{-1}\mathrm{X}^\top\mathrm{X}\big) = \mathrm{trace}(\mathrm{I}_{d+1}) = d+1,$$

so $\mathrm{trace}(\mathrm{I}_N - \mathrm{H}) = N - (d+1)$. Geometrically: H projects onto a $(d+1)$-dimensional span, $\mathrm{I} - \mathrm{H}$ onto its $(N-d-1)$-dimensional orthogonal complement, and the trace of a projection is the dimension it projects onto. ■

§9.3.iv. An Illustrative 'Proof'

Now suppose the labels come from an ideal linear target plus noise.



- If $\boldsymbol{y}$ comes from some ideal $f(\mathrm{X}) \in$ span plus **noise**,
- **noise** with per-dimension 'noise level' σ^2 is transformed by $\mathrm{I} - \mathrm{H}$ to be $\boldsymbol{y} - \hat{\boldsymbol{y}}$:

$$E_{\mathrm{in}}(\boldsymbol{w}_{\mathrm{LIN}}) = \frac{1}{N}\|\boldsymbol{y} - \hat{\boldsymbol{y}}\|^2 = \frac{1}{N}\|(\mathrm{I} - \mathrm{H})\mathbf{noise}\|^2 = \frac{1}{N}\big(N - (d+1)\big)\sigma^2.$$

The middle equality holds because $\mathrm{I} - \mathrm{H}$ kills everything in the span, $f(\mathrm{X})$ included; the last one holds on average over the noise, as the following makes precise.

Proposition 9.3.3 (Expected in-sample error of linear regression)
Fix X with $\mathrm{X}^\top\mathrm{X}$ invertible, and let $\boldsymbol{y} = \mathrm{X}\boldsymbol{w}^* + \boldsymbol{\epsilon}$ with $\mathcal{E}[\boldsymbol{\epsilon}] = \mathbf{0}$ and $\mathcal{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^\top] = \sigma^2\mathrm{I}$. Then

$$\mathop{\mathcal{E}}_{\boldsymbol{\epsilon}} E_{\text{in}}(\boldsymbol{w}_{\text{LIN}}) = \sigma^2\left(1 - \frac{d+1}{N}\right).$$

Proof. $(\mathrm{I}-\mathrm{H})\mathrm{X} = \mathrm{X} - \mathrm{X}(\mathrm{X}^\top\mathrm{X})^{-1}\mathrm{X}^\top\mathrm{X} = 0$, so $(\mathrm{I}-\mathrm{H})\boldsymbol{y} = (\mathrm{I}-\mathrm{H})\boldsymbol{\epsilon}$. $\mathrm{M} = \mathrm{I}-\mathrm{H}$ is symmetric and $\mathrm{M}^2 = \mathrm{I} - 2\mathrm{H} + \mathrm{H}^2 = \mathrm{M}$ (as $\mathrm{H}^2 = \mathrm{X}(\mathrm{X}^\top\mathrm{X})^{-1}\mathrm{X}^\top\mathrm{X}(\mathrm{X}^\top\mathrm{X})^{-1}\mathrm{X}^\top = \mathrm{H}$), hence

$$\|\mathrm{M}\boldsymbol{\epsilon}\|^2 = \boldsymbol{\epsilon}^\top\mathrm{M}^\top\mathrm{M}\boldsymbol{\epsilon} = \boldsymbol{\epsilon}^\top\mathrm{M}\boldsymbol{\epsilon} = \operatorname{trace}(\mathrm{M}\boldsymbol{\epsilon}\boldsymbol{\epsilon}^\top).$$

Taking expectations, $\mathcal{E}\|\mathrm{M}\boldsymbol{\epsilon}\|^2 = \operatorname{trace}(\mathrm{M}\cdot\sigma^2\mathrm{I}) = \sigma^2\big(N - (d+1)\big)$ by the claim of the previous subsection. Divide by N. ∎

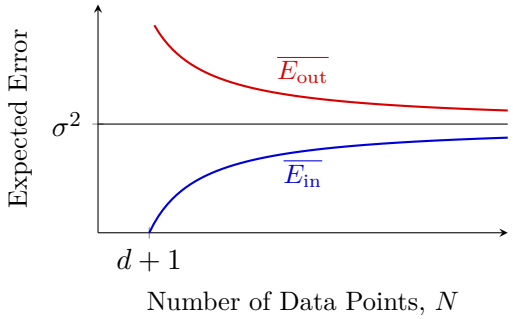
The result does not depend on X, so it survives averaging over $\mathcal{D} \sim P^N$ as well. In words, the fit absorbs $d+1$ of the N dimensions of noise: it is fitted to the noise along the span, which is why E_{in} is optimistically below σ^2. The out-of-sample counterpart is quoted without proof:

$\overline{E_{\text{in}}} = \sigma^2 \cdot \left(1 - \frac{d+1}{N}\right)$, $\overline{E_{\text{out}}} = \sigma^2 \cdot \left(1 + \frac{d+1}{N}\right)$ **(complicated!)**

$\overline{E_{\text{out}}}$ is harder because a fresh test input lies outside the training rows, so the answer involves the distribution of $\boldsymbol{x}$ and not just a trace; for large N the result above holds to first order.

§9.3.v. The Learning Curve

$$\overline{E_{\text{out}}} = \textbf{noise level} \cdot \left(1 + \frac{d+1}{N}\right)$$
$$\overline{E_{\text{in}}} = \textbf{noise level} \cdot \left(1 - \frac{d+1}{N}\right)$$



- both converge to σ^2 (**noise** level) for $N \to \infty$;
- expected generalization error: $\frac{2(d+1)}{N}$ — similar to worst-case guarantee from VC.

The curves are drawn from $N = d+1$, the fewest examples for which $\mathrm{X}^\top\mathrm{X}$ can be invertible; there $\overline{E_{\text{in}}} = 0$, as $d+1$ parameters fit $d+1$ points exactly. The gap $\overline{E_{\text{out}}} - \overline{E_{\text{in}}} = 2\sigma^2(d+1)/N$ (the slide's $\frac{2(d+1)}{N}$ is in units of the noise level) grows with $d+1 = d_{\text{VC}}$ and shrinks with N, the same trade-off Section 7.4 read off the VC bound.

Linear regression (LinReg): learning 'happened'!

Exercise 9.3.4 (Fun Time)**.** Which of the following property about H is not true?

(1) H is symmetric;

(2) $\mathrm{H}^2 = \mathrm{H}$ (double projection = single one);

(3) $(\mathrm{I}-\mathrm{H})^2 = \mathrm{I}-\mathrm{H}$ (double residual transform = single one);

(4) none of the above.

Reference Answer. (4). You can conclude that (2) and (3) are true by their physical meanings! :-) Projecting a point that is already in the span leaves it there, and the residual of a residual is itself. (1) follows from $\mathrm{H} = \mathrm{X}(\mathrm{X}^\top\mathrm{X})^{-1}\mathrm{X}^\top$, whose transpose is itself; all three were used in the proof of Proposition 9.3.3. ■

§9.4. Linear Regression for Binary Classification

slides p.19–22

Linear regression is efficient and comes with a clean guarantee; linear classification by minimum 0/1 error is NP-hard (Section 2.4). Can the first do the second's job?

§9.4.i. Linear Classification vs. Linear Regression

Linear Classification	Linear Regression
$\mathcal{Y} = \{-1, +1\}$	$\mathcal{Y} = \mathbb{R}$
$h(\boldsymbol{x}) = \mathrm{sign}(\boldsymbol{w}^\top\boldsymbol{x})$	$h(\boldsymbol{x}) = \boldsymbol{w}^\top\boldsymbol{x}$
$\mathrm{err}(\hat{y}, y) = [\![\hat{y} \neq y]\!]$	$\mathrm{err}(\hat{y}, y) = (\hat{y} - y)^2$
NP-hard to solve in general	**efficient analytic solution**

$\{-1, +1\} \subset \mathbb{R}$: linear regression for classification?

1. run LinReg on binary classification data $\mathcal{D}$ (efficient);
2. return $g(\boldsymbol{x}) = \mathrm{sign}(\boldsymbol{w}_{\mathrm{LIN}}^\top\boldsymbol{x})$.

The labels ± 1 are real numbers, so step 1 is legal; step 2 turns the real-valued fit back into a yes/no answer.

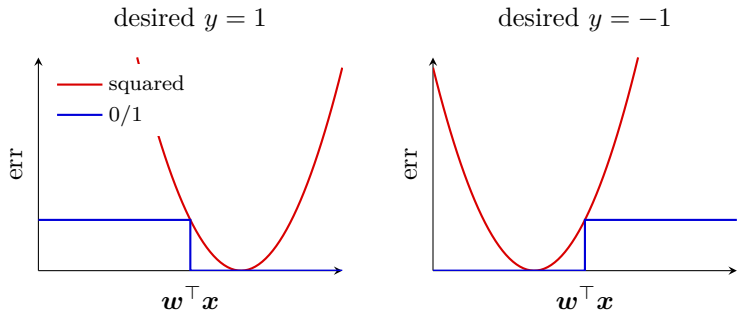
Question 9.4.1. But what is the explanation of this heuristic?

§9.4.ii. Relation of Two Errors

On one example with score $s = \boldsymbol{w}^\top\boldsymbol{x}$, the two errors are

$$\mathrm{err}_{0/1} = [\![\mathrm{sign}(\boldsymbol{w}^\top\boldsymbol{x}) \neq y]\!], \qquad \mathrm{err}_{\mathrm{sqr}} = (\boldsymbol{w}^\top\boldsymbol{x} - y)^2.$$

Plot both against the score:



In both panels the parabola stays on or above the step:

Proposition 9.4.2 (Squared error bounds 0/1 error)
For $y \in \{-1, +1\}$ and every $\boldsymbol{w}$, $\boldsymbol{x}$: $\mathrm{err}_{0/1} \leq \mathrm{err}_{\mathrm{sqr}}$.

Proof. If $\mathrm{err}_{0/1} = 0$ there is nothing to show. Otherwise $\mathrm{sign}(s) \neq y$ for $s = \boldsymbol{w}^\top \boldsymbol{x}$, so $ys \leq 0$, and using $y^2 = 1$,

$$(s - y)^2 = y^2(ys - 1)^2 = (1 - ys)^2 \geq 1 = \mathrm{err}_{0/1}.$$ ■

$\mathrm{err}_{0/1} \leq \mathrm{err}_{\mathrm{sqr}}$.

§9.4.iii. Linear Regression for Binary Classification

Average Proposition 9.4.2 over the examples and combine it with the VC bound of Theorem 6.4.2 for $d_{\mathrm{VC}} = d + 1$:

$$\begin{aligned} \text{classification } E_{\mathrm{out}}(\boldsymbol{w}) &\overset{\mathrm{VC}}{\leq} \text{classification } E_{\mathrm{in}}(\boldsymbol{w}) + \sqrt{\cdots\cdots} \\ &\leq \text{regression } E_{\mathrm{in}}(\boldsymbol{w}) + \sqrt{\cdots\cdots} \end{aligned}$$

- (loose) upper bound $\mathrm{err}_{\mathrm{sqr}}$ as $\widehat{\mathrm{err}}$ to approximate $\mathrm{err}_{0/1}$;
- trade **bound tightness** for **efficiency**.

This is the friendly $\widehat{\mathrm{err}}$ of Definition 8.3.1 in action: minimising the regression E_{in} makes the right-hand side small, so it does push the classification E_{out} down, but only through a bound. The bound is loose because squared error also penalises points that are classified correctly with a large margin ($ys > 1$), pulling the line towards them for no gain in 0/1 error.

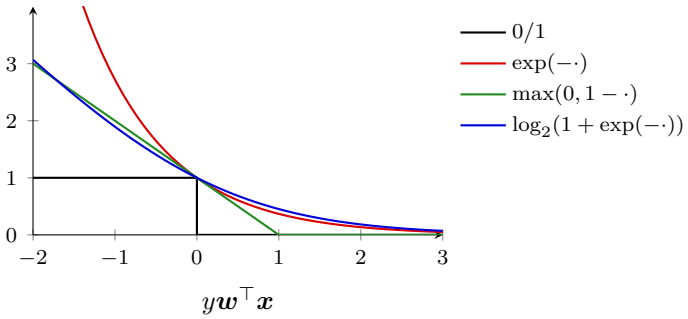
$\boldsymbol{w}_{\mathrm{LIN}}$: useful baseline classifier, or as initial PLA/pocket vector.

As an initial vector it replaces $\boldsymbol{w}_0 = \boldsymbol{0}$ in algorithm 2.1 or algorithm 2.3 by something already roughly right, which typically cuts the number of corrections needed.

Exercise 9.4.3 (Fun Time). Which of the following functions are upper bounds of the pointwise 0/1 error $[\![\mathrm{sign}(\boldsymbol{w}^\top \boldsymbol{x}) \neq y]\!]$ for $y \in \{-1, +1\}$?

(1) $\exp(-y\boldsymbol{w}^\top \boldsymbol{x})$;
(2) $\max(0, 1 - y\boldsymbol{w}^\top \boldsymbol{x})$;
(3) $\log_2(1 + \exp(-y\boldsymbol{w}^\top \boldsymbol{x}))$;
(4) all of the above.

Reference Answer. (4). Plot the curves and you'll see. Thus, all three can be used for binary classification. In fact, all three functions connect to very important algorithms in machine learning and we will discuss one of them soon in the next lecture. Stay tuned. :-)



Each is a function of $y\boldsymbol{w}^\top\boldsymbol{x}$ alone, and where the 0/1 error is 1, i.e. $y\boldsymbol{w}^\top\boldsymbol{x} \leq 0$, each is at least its value at 0, which is 1; elsewhere each is ≥ 0. ■

Summary

Linear Regression Problem use hyperplanes to approximate real values.

Linear Regression Algorithm analytic solution with pseudo-inverse.

Generalization Issue $E_{\text{out}} - E_{\text{in}} \approx \frac{2(d+1)}{N}$ on average.

Linear Regression for Binary Classification 0/1 error $\leq$ squared error.

Linear regression handles real-valued outputs, and through Proposition 9.4.2 it doubles as a crude binary classifier. Between a yes/no answer and an arbitrary real number lies a third kind of output, a probability. Next: binary classification, regression, and then?

10. Logistic Regression

Lecture 10

7 Oct.

> **As previously seen .** Linear regression (algorithm 9.1) found the analytic solution $\boldsymbol{w}_{\text{LIN}} = \mathrm{X}^{\dagger}\boldsymbol{y}$ with linear regression hypotheses and squared error. Linear models so far answer either yes/no (perceptron, sign of the score) or an arbitrary real number (linear regression, the score itself).

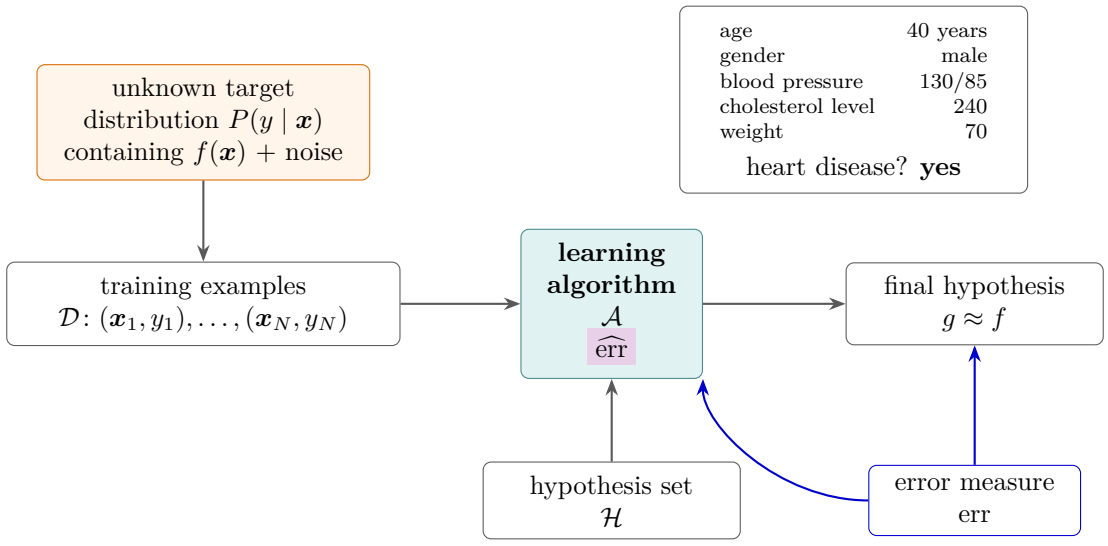
This chapter treats a third kind of question: how *likely* is +1? The answer is a probability, so the score is squashed into $[0, 1]$ by the logistic function. The error measure is then derived rather than chosen, from maximum likelihood, and it has no closed-form minimiser; minimising it calls for an iterative method, gradient descent, which recasts PLA as one instance of a general scheme.

§10.1. Logistic Regression Problem

slides p.2–7

§10.1.i. Heart Attack Prediction Problem (1/2)

A new application, drawn in the noisy flow of Section 8.3: from a patient's record, predict heart disease.



The labels are yes/no, measured by 0/1 error, so by Proposition 8.2.4 the ideal mini-target picks the more probable label:

> **Binary classification: ideal** $f(\boldsymbol{x}) = \operatorname{sign}\left(P(+1 \mid \boldsymbol{x}) - \frac{1}{2}\right) \in \{-1, +1\}$**, because of classification** err**.**

Proposition 10.1.1 (Ideal mini-target of binary classification)

Let $\mathcal{Y} = \{-1, +1\}$, $\mathrm{err}(\tilde{y}, y) = [\![\tilde{y} \neq y]\!]$, fix $\boldsymbol{x}$ and write $p = P(+1 \mid \boldsymbol{x})$. The minimisers in Definition 8.2.3, over all $\tilde{y} \in \mathbb{R}$, are

$$\arg\min_{\tilde{y}} \mathop{\mathcal{E}}_{y \sim P(y|\boldsymbol{x})} [\![\tilde{y} \neq y]\!] = \begin{cases} \{+1\}, & p > \frac{1}{2}, \\ \{-1\}, & p < \frac{1}{2}, \\ \{-1, +1\}, & p = \frac{1}{2}, \end{cases}$$

so $f(\boldsymbol{x}) = \mathrm{sign}\big(p - \frac{1}{2}\big)$ is an ideal mini-target, with either label allowed when $p = \frac{1}{2}$.

Proof. Write $R(\tilde{y})$ for the expected error at the fixed $\boldsymbol{x}$. The expectation of an indicator is the probability of its event, and here y is drawn from $P(y \mid \boldsymbol{x})$, so, as in part (1) of Proposition 8.2.4,

$$R(\tilde{y}) = \mathop{\mathcal{E}}_{y \sim P(y|\boldsymbol{x})} [\![\tilde{y} \neq y]\!] = \mathbb{P}[y \neq \tilde{y} \mid \boldsymbol{x}] = 1 - P(y = \tilde{y} \mid \boldsymbol{x}).$$

Since y takes only the values ± 1, with $P(-1 \mid \boldsymbol{x}) = 1 - p$,

$$R(+1) = 1 - p, \qquad R(-1) = p, \qquad R(\tilde{y}) = 1 \text{ for } \tilde{y} \notin \{-1, +1\}.$$

Values outside $\mathcal{Y}$ never minimise: $\min\{R(+1), R(-1)\} = \min\{p, 1 - p\} \leq \frac{1}{2} < 1$, so the minimum over $\mathbb{R}$ is attained only within $\{-1, +1\}$.

Comparing the two labels:

$$R(-1) - R(+1) = p - (1 - p) = 2\left(p - \tfrac{1}{2}\right).$$

If $p > \frac{1}{2}$ this is positive, so $+1$ is the unique minimiser; if $p < \frac{1}{2}$ it is negative and -1 is; if $p = \frac{1}{2}$ it is zero and both are. In the first two cases the minimiser is $\mathrm{sign}(p - \frac{1}{2})$, and in the third any value of $\mathrm{sign}(0)$ in $\{-1, +1\}$ is a minimiser. ■

So $\mathrm{sign}(p - \frac{1}{2})$ is literally the sign of $R(-1) - R(+1)$: it picks whichever label is cheaper. The threshold is $\frac{1}{2}$ because 0/1 error charges both kinds of mistake the same; the error still left at the optimum, $\min\{p, 1 - p\}$, is the noise no hypothesis can remove at $\boldsymbol{x}$.

§10.1.ii. Heart Attack Prediction Problem (2/2)

Same flow, same patient, a different question:

age	40 years
gender	male
blood pressure	130/85
cholesterol level	240
weight	70

heart attack? **80% risk**

A doctor would rather say '80% risk' than 'yes', since the number tells how urgent treatment is and the yes/no answer is just the number thresholded at $\frac{1}{2}$.

'Soft' binary classification: $f(\boldsymbol{x}) = P(+1 \mid \boldsymbol{x}) \in [0, 1]$.

§10.1.iii. Soft Binary Classification

Definition 10.1.2 (Soft binary classification)**. Soft binary classification** is learning the target function $f(\boldsymbol{x}) = P(+1 \mid \boldsymbol{x}) \in [0, 1]$.

The ideal data for it would carry the probabilities themselves; the data we actually get carry one label drawn from each:

ideal (noiseless) data	actual (noisy) data	actual data, as numbers
$(\boldsymbol{x}_1, y'_1 = 0.9 = P(+1 \mid \boldsymbol{x}_1))$	$(\boldsymbol{x}_1, y_1 = \circ \sim P(y \mid \boldsymbol{x}_1))$	$(\boldsymbol{x}_1, y'_1 = 1 = [\![\circ \overset{?}{\sim} P(y \mid \boldsymbol{x}_1)]\!])$
$(\boldsymbol{x}_2, y'_2 = 0.2 = P(+1 \mid \boldsymbol{x}_2))$	$(\boldsymbol{x}_2, y_2 = \times \sim P(y \mid \boldsymbol{x}_2))$	$(\boldsymbol{x}_2, y'_2 = 0 = [\![\circ \overset{?}{\sim} P(y \mid \boldsymbol{x}_2)]\!])$
$\vdots$	$\vdots$	$\vdots$
$(\boldsymbol{x}_N, y'_N = 0.6 = P(+1 \mid \boldsymbol{x}_N))$	$(\boldsymbol{x}_N, y_N = \times \sim P(y \mid \boldsymbol{x}_N))$	$(\boldsymbol{x}_N, y'_N = 0 = [\![\circ \overset{?}{\sim} P(y \mid \boldsymbol{x}_N)]\!])$

Note 10.1.3. Each actual label is *one draw* from the distribution at its input: $y_n = +1$ with probability $P(+1 \mid \boldsymbol{x}_n)$ and $y_n = -1$ with probability $1 - P(+1 \mid \boldsymbol{x}_n)$, independently across n. The draw usually lands on the likelier side — $y_1 = \circ$ at $P(+1 \mid \boldsymbol{x}_1) = 0.9$, $y_2 = \times$ at $P(+1 \mid \boldsymbol{x}_2) = 0.2$ — but not always: $y_N = \times$ although $P(+1 \mid \boldsymbol{x}_N) = 0.6$. Such a draw on the less likely side is the *noise*; it disagrees with the ideal mini-target $\operatorname{sign}\big(P(+1 \mid \boldsymbol{x}_N) - \frac{1}{2}\big) = +1$ of Proposition 10.1.1, yet it is a perfectly legitimate sample.

The last column rewrites each noisy label as a number, 1 if the draw came out $\circ$ and 0 otherwise: a noisy version of the ideal y'_n, whose average over many draws at $\boldsymbol{x}_n$ would be $P(+1 \mid \boldsymbol{x}_n)$.

Same data as hard binary classification, different target function.

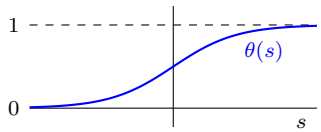
So the task is harder than it looks: we want probabilities but only ever see 0/1 outcomes.

§10.1.iv. Logistic Hypothesis

age	40 years
gender	male
blood pressure	130/85
cholesterol level	240

- For $\boldsymbol{x} = (x_0, x_1, x_2, \ldots, x_d)$ 'features of patient', calculate a weighted 'risk score':

$$s = \sum_{i=0}^{d} w_i x_i.$$

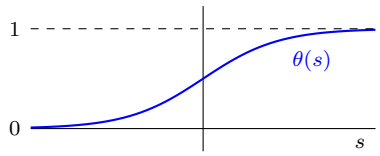


- Convert the score to estimated probability by logistic function $\theta(s)$.

Logistic hypothesis: $h(\boldsymbol{x}) = \theta(\boldsymbol{w}^\top \boldsymbol{x})$.

The score is the perceptron's and linear regression's; only the last step differs, and it is chosen so that the output lands in $[0, 1]$.

§10.1.v. Logistic Function



Definition 10.1.4 (Logistic function)**.** The **logistic function** is

$$\theta(s) = \frac{e^s}{1+e^s} = \frac{1}{1+e^{-s}},$$

with $\theta(-\infty) = 0$, $\theta(0) = \frac{1}{2}$ and $\theta(\infty) = 1$ — a smooth, monotonic, **sigmoid** (S-shaped) function of s.

Two facts are used below. It is symmetric about $(0, \frac{1}{2})$:

$$1 - \theta(s) = \frac{e^{-s}}{1+e^{-s}} = \frac{1}{e^s+1} = \theta(-s).$$

And its derivative is $\theta'(s) = \frac{e^{-s}}{(1+e^{-s})^2} = \theta(s)\,\theta(-s) > 0$.

Logistic regression: use

$$h(\boldsymbol{x}) = \frac{1}{1+\exp(-\boldsymbol{w}^\top \boldsymbol{x})}$$

to approximate target function $f(\boldsymbol{x}) = P(+1 \mid \boldsymbol{x})$.

Exercise 10.1.5 (Fun Time: Logistic Regression and Binary Classification)**.** Consider any logistic hypothesis $h(\boldsymbol{x}) = \frac{1}{1+\exp(-\boldsymbol{w}^\top \boldsymbol{x})}$ that approximates $P(y \mid \boldsymbol{x})$. 'Convert' $h(\boldsymbol{x})$ to a binary classification prediction by taking sign $\left(h(\boldsymbol{x}) - \frac{1}{2}\right)$. What is the equivalent formula for the binary classification prediction?

(1) sign $\left(\boldsymbol{w}^\top \boldsymbol{x} - \frac{1}{2}\right)$;

(2) sign $\left(\boldsymbol{w}^\top \boldsymbol{x}\right)$;

(3) sign $\left(\boldsymbol{w}^\top \boldsymbol{x} + \frac{1}{2}\right)$;

(4) none of the above.

Reference Answer. (2). When $\boldsymbol{w}^\top \boldsymbol{x} = 0$, $h(\boldsymbol{x})$ is exactly $\frac{1}{2}$. So thresholding $h(\boldsymbol{x})$ at $\frac{1}{2}$ is the same as thresholding $\boldsymbol{w}^\top \boldsymbol{x}$ at 0, since θ is increasing. A logistic hypothesis thus carries a perceptron inside it, and soft classification refines the hard one. ■

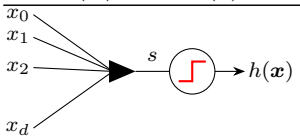
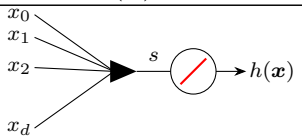
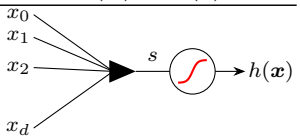
§10.2. Logistic Regression Error

slides p.8–12

The hypothesis set is fixed; what is missing is the error measure that $\mathcal{A}$ minimises.

§10.2.i. Three Linear Models

All three models compute the same linear scoring function $s = \boldsymbol{w}^\top \boldsymbol{x}$ and differ in what they do with it:

linear classification	**linear regression**	**logistic regression**
$h(\boldsymbol{x}) = \text{sign}(s)$	$h(\boldsymbol{x}) = s$	$h(\boldsymbol{x}) = \theta(s)$
		
plausible err = 0/1	friendly err = squared	err = ?
(small flipping noise)	(easy to minimize)	

The first two error measures were justified in Definition 8.3.1. Neither fits the third model: its output is a probability and its labels are ± 1, so 'how far is 0.8 from $+1$' needs a principled answer.

Question 10.2.1. How to define $E_{\text{in}}(\boldsymbol{w})$ for logistic regression?

§10.2.ii. Likelihood

$$\text{target function } f(\boldsymbol{x}) = P(+1 \mid \boldsymbol{x}) \quad \Longleftrightarrow \quad P(y \mid \boldsymbol{x}) = \begin{cases} f(\boldsymbol{x}) & \text{for } y = +1 \\ 1 - f(\boldsymbol{x}) & \text{for } y = -1 \end{cases}$$

Consider $\mathcal{D} = \{(\boldsymbol{x}_1, \circ), (\boldsymbol{x}_2, \times), \ldots, (\boldsymbol{x}_N, \times)\}$. Each example arose by drawing $\boldsymbol{x}_n \sim P(\boldsymbol{x})$ and then $y_n \sim P(y \mid \boldsymbol{x}_n)$, independently, so:

probability that f generates $\mathcal{D}$	**likelihood that h generates $\mathcal{D}$**
$P(\boldsymbol{x}_1)P(\circ \mid \boldsymbol{x}_1) = P(\boldsymbol{x}_1)f(\boldsymbol{x}_1) \times$	$P(\boldsymbol{x}_1)h(\boldsymbol{x}_1) \times$
$P(\boldsymbol{x}_2)P(\times \mid \boldsymbol{x}_2) = P(\boldsymbol{x}_2)\big(1 - f(\boldsymbol{x}_2)\big) \times$	$P(\boldsymbol{x}_2)\big(1 - h(\boldsymbol{x}_2)\big) \times$
$\ldots$	$\ldots$
$P(\boldsymbol{x}_N)P(\times \mid \boldsymbol{x}_N) = P(\boldsymbol{x}_N)\big(1 - f(\boldsymbol{x}_N)\big)$	$P(\boldsymbol{x}_N)\big(1 - h(\boldsymbol{x}_N)\big)$

The right column is the same computation with h pretending to be f; a probability of observed data, viewed as a function of the hypothesis, is called its **likelihood**.

Note 10.2.2. In symbols: read h as a conditional distribution $P_h(y \mid \boldsymbol{x})$, with $P_h(+1 \mid \boldsymbol{x}) = h(\boldsymbol{x})$ and $P_h(-1 \mid \boldsymbol{x}) = 1 - h(\boldsymbol{x})$, exactly as f is read in the display above. Then

$$\text{likelihood}(h) = \prod_{n=1}^{N} P(\boldsymbol{x}_n)\, P_h(y_n \mid \boldsymbol{x}_n),$$

the probability that the process above — $\boldsymbol{x}_n \sim P(\boldsymbol{x})$, then $y_n \sim P_h(y \mid \boldsymbol{x}_n)$ instead of $P(y \mid \boldsymbol{x}_n)$ — produces exactly the observed $\mathcal{D}$; the product comes from the independence of the N examples. Only the label part is swapped: the inputs still come from the true $P(\boldsymbol{x})$, which is why those factors are common to every h. And it is a function *of* h with $\mathcal{D}$ held fixed, not a probability distribution over h.

- if $h \approx f$, then likelihood$(h) \approx$ probability using f;
- probability using f usually **large**.

The second point is because $\mathcal{D}$ really was generated by f: the sample we got is a typical one under f, not one of its rare outcomes. So a hypothesis under which $\mathcal{D}$ looks unlikely is unlikely to be close to f.

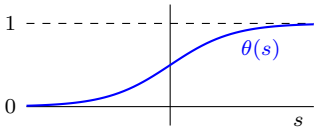
§10.2.iii. Likelihood of Logistic Hypothesis

likelihood$(h) \approx$ **(probability using** f**)** $\approx$ **large** $\implies$ $g = \arg\max_h$ **likelihood**(h)**.**

This is the principle of **maximum likelihood**. For logistic hypotheses it simplifies.

When logistic: $h(\boldsymbol{x}) = \theta(\boldsymbol{w}^\top \boldsymbol{x})$,

$$1 - h(\boldsymbol{x}) = h(-\boldsymbol{x}),$$



by the symmetry $1 - \theta(s) = \theta(-s)$ of Definition 10.1.4, with $s = \boldsymbol{w}^\top \boldsymbol{x}$.

Substituting into the likelihood,

$$\begin{aligned}\text{likelihood}(h) &= P(\boldsymbol{x}_1)h(\boldsymbol{x}_1) \times P(\boldsymbol{x}_2)\big(1 - h(\boldsymbol{x}_2)\big) \times \ldots P(\boldsymbol{x}_N)\big(1 - h(\boldsymbol{x}_N)\big) \\ &= P(\boldsymbol{x}_1)h(+\boldsymbol{x}_1) \times P(\boldsymbol{x}_2)h(-\boldsymbol{x}_2) \times \ldots P(\boldsymbol{x}_N)h(-\boldsymbol{x}_N).\end{aligned}$$

The sign in front of each $\boldsymbol{x}_n$ is its label y_n, and the factors $P(\boldsymbol{x}_n)$ are the same for every h, so they do not affect the arg max:

likelihood(logistic h**)** $\propto \prod_{n=1}^{N} h(y_n \boldsymbol{x}_n)$**.**

§10.2.iv. Cross-Entropy Error

Now turn the maximisation into a minimisation of an average, one step at a time:

$$\begin{aligned}&\max_h \text{ likelihood(logistic } h) \propto \prod_{n=1}^{N} h(y_n \boldsymbol{x}_n) \\ \Leftrightarrow\ &\max_{\boldsymbol{w}} \text{ likelihood}(\boldsymbol{w}) \propto \prod_{n=1}^{N} \theta\left(y_n \boldsymbol{w}^\top \boldsymbol{x}_n\right) \\ \Leftrightarrow\ &\max_{\boldsymbol{w}} \ln \prod_{n=1}^{N} \theta\left(y_n \boldsymbol{w}^\top \boldsymbol{x}_n\right) \\ \Leftrightarrow\ &\min_{\boldsymbol{w}} \frac{1}{N} \sum_{n=1}^{N} -\ln \theta\left(y_n \boldsymbol{w}^\top \boldsymbol{x}_n\right).\end{aligned}$$

The first line becomes the second because a logistic h *is* its $\boldsymbol{w}$; the logarithm is increasing, so it keeps the maximiser while turning the product into a sum; and negating and dividing by N turns the maximum of the sum into the minimum of an average. With $\theta(s) = \frac{1}{1+\exp(-s)}$, $-\ln \theta(s) = \ln(1 + \exp(-s))$, hence

$$\min_{\boldsymbol{w}} \frac{1}{N} \sum_{n=1}^{N} \ln\left(1 + \exp\left(-y_n \boldsymbol{w}^\top \boldsymbol{x}_n\right)\right) \quad \implies \quad \min_{\boldsymbol{w}} \underbrace{\frac{1}{N} \sum_{n=1}^{N} \text{err}(\boldsymbol{w}, \boldsymbol{x}_n, y_n)}_{E_{\text{in}}(\boldsymbol{w})}.$$

Definition 10.2.3 (Cross-entropy error)**.** The pointwise error

$$\text{err}(\boldsymbol{w}, \boldsymbol{x}, y) = \ln\left(1 + \exp\left(-y \boldsymbol{w}^\top \boldsymbol{x}\right)\right) \tag{10.1}$$

is called the **cross-entropy error**.

$\operatorname{err}(\boldsymbol{w}, \boldsymbol{x}, y) = \ln\left(1 + \exp\left(-y\boldsymbol{w}^\top \boldsymbol{x}\right)\right)$**: cross-entropy error.**

The name: write $y' = [\![y = +1]\!]$ and $p = h(\boldsymbol{x})$. Then $-\big(y' \ln p + (1 - y') \ln(1 - p)\big)$, the cross-entropy between the observed label distribution $(y', 1 - y')$ and the predicted one $(p, 1 - p)$, equals $-\ln \theta(y\boldsymbol{w}^\top \boldsymbol{x})$, which is (10.1). Up to the factor $\log_2 e$, (10.1) is also the third upper bound of the last Fun Time of Section 9.4.

Exercise 10.2.4 (Fun Time)**.** The four statements below help us understand more about the cross-entropy error $\operatorname{err}(\boldsymbol{w}, \boldsymbol{x}, y) = \ln\left(1 + \exp\left(-y\boldsymbol{w}^\top \boldsymbol{x}\right)\right)$. Consider $\boldsymbol{w}^\top \boldsymbol{x} \neq 0$. Which statement is not true?

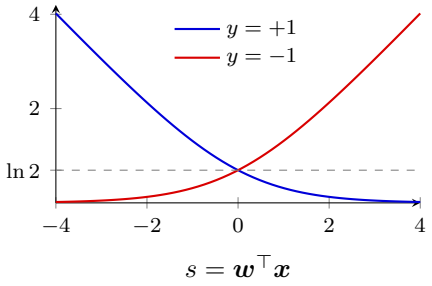
(1) For any $\boldsymbol{w}$, $\boldsymbol{x}$, and y, $\operatorname{err}(\boldsymbol{w}, \boldsymbol{x}, y) > 0$.

(2) For any $\boldsymbol{w}$, $\boldsymbol{x}$, and y, $\operatorname{err}(\boldsymbol{w}, \boldsymbol{x}, y) < 1126$.

(3) When $y = \operatorname{sign}(\boldsymbol{w}^\top \boldsymbol{x})$, $\operatorname{err}(\boldsymbol{w}, \boldsymbol{x}, y) < \ln 2$.

(4) When $y \neq \operatorname{sign}(\boldsymbol{w}^\top \boldsymbol{x})$, $\operatorname{err}(\boldsymbol{w}, \boldsymbol{x}, y) \geq \ln 2$.

Reference Answer. (2). 1126, really? :-) You are highly encouraged to plot the curve of err with respect to some fixed y and some varying score $s = \boldsymbol{w}^\top \boldsymbol{x}$ to know more about the error measure. After plotting, it is easy to see that err is not bounded above, and the other three choices are correct.



Each curve is positive, decreases (for $y = +1$) or increases (for $y = -1$) through $\ln 2$ at $s = 0$, and grows like $|s|$ on the wrong side, which settles (1), (3), (4) and refutes (2). ■

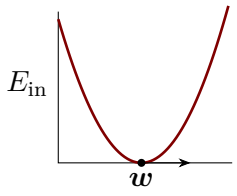
§10.3. Gradient of Logistic Regression Error

slides p.13–17

With E_{in} defined, the remaining task is the one that linear regression solved in Section 9.2: minimise it.

§10.3.i. Minimizing $E_{\text{in}}(\boldsymbol{w})$

$$\min_{\boldsymbol{w}} \; E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N} \sum_{n=1}^{N} \ln\left(1 + \exp\left(-y_n \boldsymbol{w}^\top \boldsymbol{x}_n\right)\right)$$



- $E_{\text{in}}(\boldsymbol{w})$: continuous, differentiable, twice-differentiable, **convex**;
- how to minimize? locate **valley**: want $\nabla E_{\text{in}}(\boldsymbol{w}) = \boldsymbol{0}$.

Convexity holds term by term: $\ln\left(1+e^{-t}\right)$ has second derivative $\theta(t)\theta(-t) > 0$ in t, and composing a convex function of one variable with the linear map $\boldsymbol{w} \mapsto y_n \boldsymbol{w}^\top \boldsymbol{x}_n$ keeps it convex. So, as in Section 9.2, a zero gradient means a global minimum.

First: derive $\nabla E_{\text{in}}(\boldsymbol{w})$.

§10.3.ii. The Gradient $\nabla E_{\text{in}}(\boldsymbol{w})$

Name the two nested pieces of each term, $\bigcirc = -y_n \boldsymbol{w}^\top \boldsymbol{x}_n$ and $\square = 1 + \exp(\bigcirc)$:

$$E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N} \ln\Big(\underbrace{1+\exp\Big(\overbrace{-y_n \boldsymbol{w}^\top \boldsymbol{x}_n}^{\bigcirc}\Big)}_{\square}\Big).$$

The chain rule, one factor per layer:

$$\begin{aligned}
\frac{\partial E_{\text{in}}(\boldsymbol{w})}{\partial w_i} &= \frac{1}{N}\sum_{n=1}^{N} \left(\frac{\partial \ln(\square)}{\partial \square}\right)\left(\frac{\partial(1+\exp(\bigcirc))}{\partial \bigcirc}\right)\left(\frac{\partial(-y_n \boldsymbol{w}^\top \boldsymbol{x}_n)}{\partial w_i}\right) \\
&= \frac{1}{N}\sum_{n=1}^{N} \left(\frac{1}{\square}\right)(\exp(\bigcirc))\,(-y_n x_{n,i}) \\
&= \frac{1}{N}\sum_{n=1}^{N} \left(\frac{\exp(\bigcirc)}{1+\exp(\bigcirc)}\right)(-y_n x_{n,i}) = \frac{1}{N}\sum_{n=1}^{N} \theta(\bigcirc)\,(-y_n x_{n,i}),
\end{aligned}$$

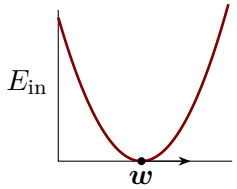
the last step by the first form of θ in Definition 10.1.4. Collecting the $d+1$ coordinates:

$$\nabla E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N} \theta\left(-y_n \boldsymbol{w}^\top \boldsymbol{x}_n\right)(-y_n \boldsymbol{x}_n). \tag{10.2}$$

§10.3.iii. Minimizing $E_{\text{in}}(\boldsymbol{w})$, Again

$$\min_{\boldsymbol{w}} E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N} \ln\left(1+\exp\left(-y_n \boldsymbol{w}^\top \boldsymbol{x}_n\right)\right)$$

$$\text{want } \nabla E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N} \theta\left(-y_n \boldsymbol{w}^\top \boldsymbol{x}_n\right)(-y_n \boldsymbol{x}_n) = \mathbf{0}.$$



(10.2) is a **scaled θ-weighted sum of $-y_n \boldsymbol{x}_n$**:

- all $\theta(\cdot) = 0$: only if $y_n \boldsymbol{w}^\top \boldsymbol{x}_n \gg 0$ — linear separable $\mathcal{D}$;
- weighted sum $= \mathbf{0}$: non-linear equation of $\boldsymbol{w}$.

The weights $\theta(-y_n \boldsymbol{w}^\top \boldsymbol{x}_n)$ are never exactly zero, so the first case is only approached, as $\|\boldsymbol{w}\| \to \infty$ along a direction that separates $\mathcal{D}$; for separable data E_{in} has no finite minimiser at all. In the second case $\boldsymbol{w}$ sits inside θ, so unlike the normal equations of Section 9.2 this is not a linear system.

Closed-form solution? No :-(

§10.3.iv. PLA Revisited: Iterative Optimization

Without a formula, we need a way to walk towards the valley. PLA already walks: it starts from some $\boldsymbol{w}_0$ (say, $\mathbf{0}$) and 'corrects' its mistakes on $\mathcal{D}$. In the form of algorithm 2.1, for $t = 0, 1, \ldots$:

1. find a mistake of $\boldsymbol{w}_t$ called $(\boldsymbol{x}_{n(t)}, y_{n(t)})$,

$$\operatorname{sign}\left(\boldsymbol{w}_t^\top \boldsymbol{x}_{n(t)}\right) \neq y_{n(t)};$$

2. (try to) correct the mistake by

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + y_{n(t)}\boldsymbol{x}_{n(t)};$$

when stop, return last $\boldsymbol{w}$ as g.

The two steps merge into one: (equivalently) pick some n, and update $\boldsymbol{w}_t$ by

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + [\![\operatorname{sign}\left(\boldsymbol{w}_t^\top \boldsymbol{x}_n\right) \neq y_n]\!]\, y_n \boldsymbol{x}_n,$$

which moves only when n is a mistake. Read it as a step size times a direction:

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + \underbrace{1}_{\eta} \cdot \underbrace{\left([\![\operatorname{sign}\left(\boldsymbol{w}_t^\top \boldsymbol{x}_n\right) \neq y_n]\!] \cdot y_n \boldsymbol{x}_n\right)}_{\boldsymbol{v}};$$

when stop, return last $\boldsymbol{w}$ as g.

Choice of $(\eta, \boldsymbol{v})$ and stopping condition defines iterative optimization approach.

Exercise 10.3.1 (Fun Time). Consider the gradient $\nabla E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N} \theta\left(-y_n \boldsymbol{w}^\top \boldsymbol{x}_n\right)\left(-y_n \boldsymbol{x}_n\right)$. That is, each example $(\boldsymbol{x}_n, y_n)$ contributes to the gradient by an amount of $\theta\left(-y_n \boldsymbol{w}^\top \boldsymbol{x}_n\right)$. For any given $\boldsymbol{w}$, which example contributes the most amount to the gradient?

(1) the example with the smallest $y_n \boldsymbol{w}^\top \boldsymbol{x}_n$ value;

(2) the example with the largest $y_n \boldsymbol{w}^\top \boldsymbol{x}_n$ value;

(3) the example with the smallest $\boldsymbol{w}^\top \boldsymbol{x}_n$ value;

(4) the example with the largest $\boldsymbol{w}^\top \boldsymbol{x}_n$ value.

Reference Answer. (1). Using the fact that θ is a monotonic function, we see that the example with the smallest $y_n \boldsymbol{w}^\top \boldsymbol{x}_n$ value contributes to the gradient the most. That is the example most badly misclassified — a soft version of PLA's attention to mistakes. ■

§10.4. Gradient Descent

slides p.18–24

PLA's $\boldsymbol{v}$ comes from a mistake; for a smooth E_{in} we can do better and let the slope pick it.

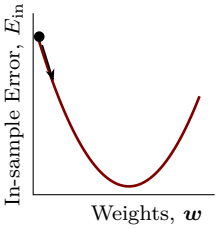
§10.4.i. Iterative Optimization

For $t = 0, 1, \ldots$

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + \eta \boldsymbol{v};$$

when stop, return last $\boldsymbol{w}$ as g.

- PLA: $\boldsymbol{v}$ comes from mistake correction.
- Smooth $E_{\text{in}}(\boldsymbol{w})$ for logistic regression: choose $\boldsymbol{v}$ to get the ball roll '**downhill**'?
  - direction $\boldsymbol{v}$: (assumed) of unit length;
  - step size η: (assumed) positive.



Fixing $\|\boldsymbol{v}\| = 1$ separates the two decisions: $\boldsymbol{v}$ says where to go, η how far.

A greedy approach for some given $\eta > 0$: $\min_{\|\boldsymbol{v}\|=1} E_{\text{in}}(\underbrace{\boldsymbol{w}_t + \eta \boldsymbol{v}}_{\boldsymbol{w}_{t+1}})$.

§10.4.ii. Linear Approximation

A greedy approach for some given $\eta > 0$:

$$\min_{\|\boldsymbol{v}\|=1} E_{\text{in}}(\boldsymbol{w}_t + \eta \boldsymbol{v}).$$

- Still non-linear optimization, now **with constraints** — not any easier than $\min_{\boldsymbol{w}} E_{\text{in}}(\boldsymbol{w})$.
- Local approximation by linear formula makes problem easier:

  $$E_{\text{in}}(\boldsymbol{w}_t + \eta \boldsymbol{v}) \approx E_{\text{in}}(\boldsymbol{w}_t) + \eta \boldsymbol{v}^\top \nabla E_{\text{in}}(\boldsymbol{w}_t)$$

  if η really small (Taylor expansion).

The approximation is the first-order Taylor expansion of E_{in} at $\boldsymbol{w}_t$; its error is $O(\eta^2)$, negligible against the linear term when η is small and $\nabla E_{\text{in}}(\boldsymbol{w}_t) \neq \boldsymbol{0}$.

Theorem 10.4.1 (Taylor expansion of $\boldsymbol{f}\colon \mathbb{R}^n \to \mathbb{R}^m$)

Write $\boldsymbol{f} = (f_1, \ldots, f_m)^\top$ with each $f_j\colon \mathbb{R}^n \to \mathbb{R}$, and let $\boldsymbol{x} = (x_1, \ldots, x_n)^\top$ be the expansion point and $\boldsymbol{h} = (h_1, \ldots, h_n)^\top$ the displacement. For each f_j define

- the gradient $\nabla f_j(\boldsymbol{x}) = \big(\partial f_j/\partial x_1, \ldots, \partial f_j/\partial x_n\big)^\top$, evaluated at $\boldsymbol{x}$;
- the Hessian $\nabla^2 f_j(\boldsymbol{x})$, the $n \times n$ matrix with (a, b)-entry $\partial^2 f_j/\partial x_a \partial x_b$ at $\boldsymbol{x}$;

and let the Jacobian $\mathrm{J}_{\boldsymbol{f}}(\boldsymbol{x})$ be the $m \times n$ matrix whose j-th row is $\nabla f_j(\boldsymbol{x})^\top$.

(i) If $\boldsymbol{f}$ is real analytic at $\boldsymbol{x}$, then for $\|\boldsymbol{h}\|$ small

$$f_j(\boldsymbol{x} + \boldsymbol{h}) = f_j(\boldsymbol{x}) + \sum_{k=1}^{\infty} \frac{1}{k!} \sum_{i_1=1}^{n} \cdots \sum_{i_k=1}^{n} \frac{\partial^k f_j(\boldsymbol{x})}{\partial x_{i_1} \cdots \partial x_{i_k}} h_{i_1} \cdots h_{i_k}, \qquad j = 1, \ldots, m.$$

The $k = 1$ and $k = 2$ terms are

$$\sum_{i=1}^{n} \frac{\partial f_j(\boldsymbol{x})}{\partial x_i} h_i = \nabla f_j(\boldsymbol{x})^\top \boldsymbol{h}, \qquad \frac{1}{2} \sum_{a=1}^{n} \sum_{b=1}^{n} \frac{\partial^2 f_j(\boldsymbol{x})}{\partial x_a \partial x_b} h_a h_b = \frac{1}{2} \boldsymbol{h}^\top \nabla^2 f_j(\boldsymbol{x})\, \boldsymbol{h},$$

so stacking the m components,

$$\boldsymbol{f}(\boldsymbol{x}+\boldsymbol{h}) = \boldsymbol{f}(\boldsymbol{x}) + \mathrm{J}_{\boldsymbol{f}}(\boldsymbol{x})\,\boldsymbol{h} + \frac{1}{2}\begin{pmatrix} \boldsymbol{h}^{\top}\nabla^2 f_1(\boldsymbol{x})\,\boldsymbol{h} \\ \vdots \\ \boldsymbol{h}^{\top}\nabla^2 f_m(\boldsymbol{x})\,\boldsymbol{h} \end{pmatrix} + \cdots .$$

(ii) If $m = 1$ and f is C^2, the series may be cut after $k = 1$ with an exact remainder:

$$f(\boldsymbol{x}+\boldsymbol{h}) = f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^{\top}\boldsymbol{h} + \frac{1}{2}\boldsymbol{h}^{\top}\nabla^2 f(\boldsymbol{x}+\theta\boldsymbol{h})\,\boldsymbol{h} \qquad \text{for some } \theta \in (0,1).$$

Proof. (i) Real analytic means f_j equals a convergent power series in $h_1, \ldots, h_n$ near $\boldsymbol{h} = \boldsymbol{0}$; differentiating it term by term at $\boldsymbol{h} = \boldsymbol{0}$ identifies the coefficients as the partial derivatives above. The $k = 1, 2$ identities are the definitions of ∇f_j and $\nabla^2 f_j$ written out.

(ii) Restrict f to the segment: $\varphi(s) = f(\boldsymbol{u}(s))$ with $\boldsymbol{u}(s) = \boldsymbol{x} + s\boldsymbol{h}$, so $\boldsymbol{u}'(s) = \boldsymbol{h}$. By the chain rule,

$$\varphi'(s) = \sum_{a=1}^{n} \frac{\partial f}{\partial x_a}\big(\boldsymbol{u}(s)\big)\,h_a = \nabla f\big(\boldsymbol{u}(s)\big)^{\top}\boldsymbol{h},$$
$$\varphi''(s) = \sum_{a=1}^{n}\sum_{b=1}^{n} h_a \frac{\partial^2 f}{\partial x_a \partial x_b}\big(\boldsymbol{u}(s)\big)\,h_b = \boldsymbol{h}^{\top}\nabla^2 f\big(\boldsymbol{u}(s)\big)\,\boldsymbol{h},$$

and φ is C^2. Taylor's theorem in one variable with Lagrange remainder gives $\varphi(1) = \varphi(0) + \varphi'(0) + \frac{1}{2}\varphi''(\theta)$ for some $\theta \in (0,1)$, which is the claim. ∎

For $m > 1$, (ii) holds only component by component, each f_j with its own θ_j.

Corollary 10.4.2 (First-order expansion along a direction)
Let E_{in} be twice continuously differentiable, $\|\boldsymbol{v}\| = 1$ and $\eta > 0$. Then

$$E_{\text{in}}(\boldsymbol{w}_t + \eta\boldsymbol{v}) = E_{\text{in}}(\boldsymbol{w}_t) + \eta\boldsymbol{v}^{\top}\nabla E_{\text{in}}(\boldsymbol{w}_t) + \frac{\eta^2}{2}\boldsymbol{v}^{\top}\nabla^2 E_{\text{in}}(\boldsymbol{w}_t + \xi\boldsymbol{v})\,\boldsymbol{v}$$

for some $\xi \in (0, \eta)$. In particular, if $\left\|\nabla^2 E_{\text{in}}\right\| \le M$ (operator norm) on the ball of radius η around $\boldsymbol{w}_t$, the remainder is at most $\frac{M}{2}\eta^2$ for *every* unit $\boldsymbol{v}$.

Proof. Apply Theorem 10.4.1(ii) with $f = E_{\text{in}}$, $\boldsymbol{x} = \boldsymbol{w}_t$, $\boldsymbol{h} = \eta\boldsymbol{v}$, and set $\xi = \theta\eta$. For the bound, write $\mathrm{H} = \nabla^2 E_{\text{in}}(\boldsymbol{w}_t + \xi\boldsymbol{v})$; since $\|\xi\boldsymbol{v}\| = \xi < \eta$, Cauchy–Schwarz gives $\left|\boldsymbol{v}^{\top}\mathrm{H}\boldsymbol{v}\right| \le \|\boldsymbol{v}\|\|\mathrm{H}\boldsymbol{v}\| \le M\|\boldsymbol{v}\|^2 = M$. ∎

So $\boldsymbol{v}^{\top}\nabla E_{\text{in}}(\boldsymbol{w}_t)$ is the slope of E_{in} when walking from $\boldsymbol{w}_t$ along $\boldsymbol{v}$ (the directional derivative), and the linear formula is the tangent plane at $\boldsymbol{w}_t$. Dropping the remainder costs $O(\eta^2)$ uniformly over $\boldsymbol{v}$, while the kept term is of size η, which is what makes 'η really small' a valid excuse. The logistic E_{in} of Section 10.2 is an average of the terms $\ln\big(1 + \exp\big(-y_n\boldsymbol{w}^{\top}\boldsymbol{x}_n\big)\big)$, each smooth in $\boldsymbol{w}$, so the corollary applies.

An approximate greedy approach for some given small η:

$$\min_{\|\boldsymbol{v}\|=1} \underbrace{E_{\text{in}}(\boldsymbol{w}_t)}_{\textbf{known}} + \underbrace{\eta}_{\textbf{given positive}}\ \boldsymbol{v}^{\top}\underbrace{\nabla E_{\text{in}}(\boldsymbol{w}_t)}_{\textbf{known}}.$$

§10.4.iii. Gradient Descent

In the approximate problem the first term is a constant and $\eta > 0$, so only $\boldsymbol{v}^\top \nabla E_{\text{in}}(\boldsymbol{w}_t)$ is to be minimised over unit vectors.

- Optimal $\boldsymbol{v}$: opposite direction of $\nabla E_{\text{in}}(\boldsymbol{w}_t)$,

$$\boldsymbol{v} = -\frac{\nabla E_{\text{in}}(\boldsymbol{w}_t)}{\|\nabla E_{\text{in}}(\boldsymbol{w}_t)\|}.$$

- Gradient descent: for **small** η, $\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t - \eta \frac{\nabla E_{\text{in}}(\boldsymbol{w}_t)}{\|\nabla E_{\text{in}}(\boldsymbol{w}_t)\|}$.

The optimal $\boldsymbol{v}$ follows from Cauchy–Schwarz (as in Remark 2.3.7):

$$\boldsymbol{v}^\top \nabla E_{\text{in}}(\boldsymbol{w}_t) \geq -\|\boldsymbol{v}\|\|\nabla E_{\text{in}}(\boldsymbol{w}_t)\| = -\|\nabla E_{\text{in}}(\boldsymbol{w}_t)\|,$$

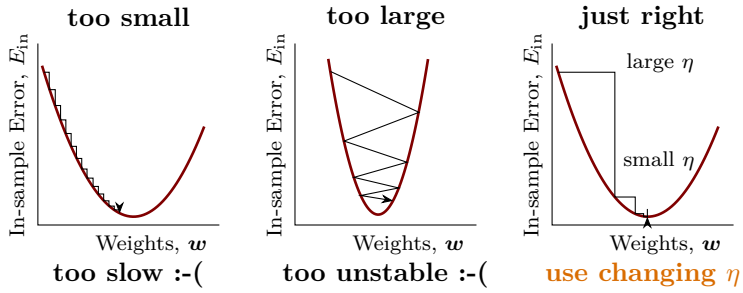
with equality exactly when $\boldsymbol{v}$ points opposite to the gradient.

Definition 10.4.3 (Gradient descent)**.** **Gradient descent** is the iterative optimisation that repeatedly steps against the gradient, $\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t - \eta \nabla E_{\text{in}}(\boldsymbol{w}_t)/\|\nabla E_{\text{in}}(\boldsymbol{w}_t)\|$.

Gradient descent: a simple & popular optimization tool.

§10.4.iv. Choice of η

The derivation assumed η small, but how small? Three runs on a one-dimensional E_{in} of my own (a parabola) show what goes wrong; every iterate drawn is computed, not sketched.



Left, a fixed small step: every move goes the right way, but by the same small amount however steep the slope, so progress is slow. Middle, steps too long for the curvature: each one overshoots the bottom and lands on the other wall, and the iterates bounce across the valley. Right, the step shrinks as the slope flattens: large far away, small near the bottom.

η better be monotonic of $\|\nabla E_{\text{in}}(\boldsymbol{w}_t)\|$.

§10.4.v. Simple Heuristic for Changing η

- If $\eta \propto \|\nabla E_{\text{in}}(\boldsymbol{w}_t)\|$ by ratio η, i.e. $\eta = \eta\|\nabla E_{\text{in}}(\boldsymbol{w}_t)\|$,

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t - \eta \frac{\nabla E_{\text{in}}(\boldsymbol{w}_t)}{\|\nabla E_{\text{in}}(\boldsymbol{w}_t)\|} = \boldsymbol{w}_t - \eta \nabla E_{\text{in}}(\boldsymbol{w}_t).$$

- Call η the **fixed learning rate**.

The normalisation and the proportional step cancel, so the update is simply the raw gradient times a constant. The step length still varies, since it is $\eta\|\nabla E_{\text{in}}(\boldsymbol{w}_t)\|$, and it shrinks automatically near the valley, where the gradient vanishes.

Fixed learning rate gradient descent: $\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t - \eta\nabla E_{\text{in}}(\boldsymbol{w}_t)$.

§10.4.vi. Putting Everything Together

Algorithm 10.1: Logistic Regression

Input: training examples $\mathcal{D}$, learning rate η

Output: the last $\boldsymbol{w}_{t+1}$, returned as g

initialize $\boldsymbol{w}_0$;

for $t = 0, 1, \ldots$ **do**

  compute $\nabla E_{\text{in}}(\boldsymbol{w}_t) = \frac{1}{N}\sum_{n=1}^{N} \theta\left(-y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n\right)(-y_n \boldsymbol{x}_n)$;

  update by $\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t - \eta\nabla E_{\text{in}}(\boldsymbol{w}_t)$;

  if $\nabla E_{\text{in}}(\boldsymbol{w}_{t+1}) = \mathbf{0}$ *or enough iterations* **then** **return** last $\boldsymbol{w}_{t+1}$ as g;

In practice '$= \mathbf{0}$' means 'approximately $\mathbf{0}$': the loop stops when the gradient is tiny or the budget runs out.

Similar time complexity to pocket per iteration.

Each iteration computes N scores and sums N vectors of length $d+1$, $O(Nd)$ work — the same as pocket's check of E_{in} on all of $\mathcal{D}$ after each update in algorithm 2.3.

Exercise 10.4.4 (Fun Time). If $\boldsymbol{w}_0 = \mathbf{0}$, and take $\eta = 0.1$. What is $\boldsymbol{w}_1$ in the logistic regression algorithm?

(1) $+0.1 \cdot \frac{1}{N}\sum_{n=1}^{N} y_n \boldsymbol{x}_n$;

(2) $-0.1 \cdot \frac{1}{N}\sum_{n=1}^{N} y_n \boldsymbol{x}_n$;

(3) $+0.05 \cdot \frac{1}{N}\sum_{n=1}^{N} y_n \boldsymbol{x}_n$;

(4) $-0.05 \cdot \frac{1}{N}\sum_{n=1}^{N} y_n \boldsymbol{x}_n$.

Reference Answer. (3). You can do a simple substitution using the fact that $\theta(0) = \frac{1}{2}$:

$$\nabla E_{\text{in}}(\mathbf{0}) = -\frac{1}{2} \cdot \frac{1}{N}\sum_{n=1}^{N} y_n \boldsymbol{x}_n, \qquad \boldsymbol{w}_1 = \mathbf{0} - 0.1\nabla E_{\text{in}}(\mathbf{0}) = +0.05 \cdot \frac{1}{N}\sum_{n=1}^{N} y_n \boldsymbol{x}_n.$$

This result shows that a scaled average of $y_n \boldsymbol{x}_n$ is somewhat 'one-step' better than the zero vector. ■

Summary

Logistic Regression Problem $P(+1 \mid \boldsymbol{x})$ as target and $\theta(\boldsymbol{w}^\top \boldsymbol{x})$ as hypotheses.

Logistic Regression Error cross-entropy (negative log likelihood).

Gradient of Logistic Regression Error θ-weighted sum of data vectors.

Gradient Descent roll downhill by $-\nabla E_{\text{in}}(\boldsymbol{w})$.

There are now three linear models — linear classification, linear regression and logistic regression — sharing one score $\boldsymbol{w}^\top \boldsymbol{x}$ and differing in their error measures, and two of them have efficient algorithms. Next: linear model'S' for classification, i.e. how the other two can serve the first.

11. Linear Models for Classification

Lecture 11

14 Oct.

As previously seen . Logistic regression (algorithm 10.1) runs gradient descent on the cross-entropy error (10.1) to get a good logistic hypothesis. There are now three linear models — linear classification, linear regression, logistic regression — sharing the score $s = \boldsymbol{w}^\top \boldsymbol{x}$, and only the last two have efficient algorithms.

This chapter makes the linear models work for classification in three steps.

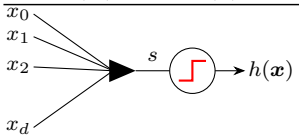
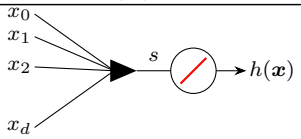
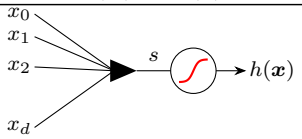
1. The two efficient models are turned into binary classifiers, with a guarantee obtained by bounding the 0/1 error from above.
2. Logistic regression is made as cheap per step as PLA by replacing the gradient with a random one-example estimate.
3. Both kinds of binary classifier are combined into classifiers for more than two classes.

§11.1. Linear Models for Binary Classification

slides p.2–7

§11.1.i. Linear Models Revisited

All three models compute the linear scoring function $s = \boldsymbol{w}^\top \boldsymbol{x}$:

linear classification	**linear regression**	**logistic regression**
$h(\boldsymbol{x}) = \text{sign}(s)$	$h(\boldsymbol{x}) = s$	$h(\boldsymbol{x}) = \theta(s)$
		
plausible err = 0/1	friendly err = squared	plausible err = cross-entropy
discrete $E_{\text{in}}(\boldsymbol{w})$:	quadratic convex $E_{\text{in}}(\boldsymbol{w})$:	smooth convex $E_{\text{in}}(\boldsymbol{w})$:
NP-hard to solve	closed-form solution	gradient descent

Each column records what Section 2.4, Section 9.2 and Section 10.4 found.

- **0/1 error**: the one we actually care about for yes/no answers, but its E_{in} is piecewise constant in $\boldsymbol{w}$, so neither a formula nor a slope can guide the search.
- **The other two**: well-behaved E_{in}, but they answer different questions.

Since every model classifies once its score is passed through sign, the open question is whether their $\boldsymbol{w}$ is a good one for the first column:

Question 11.1.1. Can linear regression or logistic regression help linear classification?

§11.1.ii. Error Functions Revisited

To compare the three, write each error on one example as a function of the score $s = \boldsymbol{w}^\top \boldsymbol{x}$ and the label, for binary classification $y \in \{-1, +1\}$:

linear classification	**linear regression**	**logistic regression**
$h(\boldsymbol{x}) = \operatorname{sign}(s)$	$h(\boldsymbol{x}) = s$	$h(\boldsymbol{x}) = \theta(s)$
$\operatorname{err}(h, \boldsymbol{x}, y) = [\![h(\boldsymbol{x}) \neq y]\!]$	$\operatorname{err}(h, \boldsymbol{x}, y) = \big(h(\boldsymbol{x}) - y\big)^2$	$\operatorname{err}(h, \boldsymbol{x}, y) = -\ln h(y\boldsymbol{x})$
$\operatorname{err}_{0/1}(s, y)$	$\operatorname{err}_{\mathrm{SQR}}(s, y)$	$\operatorname{err}_{\mathrm{CE}}(s, y)$
$= [\![\operatorname{sign}(s) \neq y]\!]$	$= (s - y)^2$	$= \ln\big(1 + \exp(-ys)\big)$
$= [\![\operatorname{sign}(ys) \neq 1]\!]$	$= (ys - 1)^2$	

Each second line uses only $y \in \{-1, +1\}$.

- **0/1**: multiplying by y either keeps a number or flips its sign, so $\operatorname{sign}(s) = y$ exactly when $\operatorname{sign}(ys) = 1$.
- **Squared**: $y^2 = 1$ gives $(s - y)^2 = y^2(s - y)^2 = (ys - 1)^2$.
- **Cross-entropy**: the logistic error is the cross-entropy of Definition 10.2.3: $h(y\boldsymbol{x}) = \theta(y\boldsymbol{w}^\top \boldsymbol{x}) = \theta(ys)$ and $-\ln \theta(t) = \ln(1 + \exp(-t))$.

So all three depend on (s, y) only through the product ys.

(ys): classification correctness score.

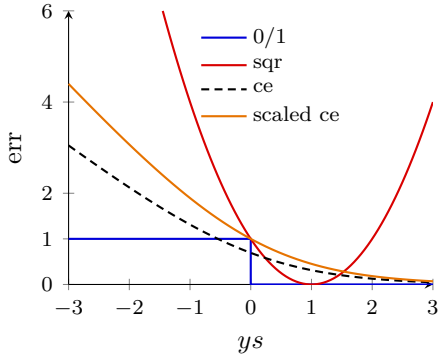
Its sign says whether $\operatorname{sign}(s)$ agrees with y, and its size says by how much the score clears the boundary $s = 0$.

§11.1.iii. Visualizing Error Functions

Four errors, all functions of ys:

$$\begin{aligned} 0/1 \quad & \operatorname{err}_{0/1}(s, y) = [\![\operatorname{sign}(ys) \neq 1]\!] \\ \text{sqr} \quad & \operatorname{err}_{\mathrm{SQR}}(s, y) = (ys - 1)^2 \\ \text{ce} \quad & \operatorname{err}_{\mathrm{CE}}(s, y) = \ln\big(1 + \exp(-ys)\big) \\ \text{scaled ce} \quad & \operatorname{err}_{\mathrm{SCE}}(s, y) = \log_2\big(1 + \exp(-ys)\big) \end{aligned}$$

The last is new: the cross-entropy rescaled so that it passes through $(0, 1)$, the corner of the 0/1 step.



- 0/1: 1 iff $ys \leq 0$.
- sqr: large if $ys \ll 1$, **but** over-charge $ys \gg 1$;
  small $\operatorname{err}_{\mathrm{SQR}} \to$ small $\operatorname{err}_{0/1}$.
- ce: monotonic of ys;
  small $\operatorname{err}_{\mathrm{CE}} \leftrightarrow$ small $\operatorname{err}_{0/1}$.
- scaled ce: a proper upper bound of 0/1;
  small $\operatorname{err}_{\mathrm{SCE}} \leftrightarrow$ small $\operatorname{err}_{0/1}$.

- **0/1 at $ys = 0$**: the convention $\operatorname{sign}(0) \neq 1$ puts $ys = 0$ on the wrong side, which is the safe reading: a score exactly on the boundary has not made a decision.
- **One-way arrow for sqr**: the parabola climbs again to the right of $ys = 1$: an example classified correctly with a large margin still costs much squared error, so a small $\mathrm{err}_{0/1}$ does not force a small $\mathrm{err}_{\mathrm{SQR}}$.
- **Two-way arrows for ce**: both cross-entropy curves instead decrease monotonically in ys, so pushing ys up lowers both errors at once.
- **Why scale**: plain ce is *not* an upper bound: at $ys = 0$ it equals $\ln 2 < 1$. Dividing by $\ln 2$ repairs exactly that.

Proposition 11.1.2 (Scaled cross-entropy bounds 0/1 error)
For $y \in \{-1, +1\}$ and every real s,

$$\mathrm{err}_{0/1}(s, y) \leq \mathrm{err}_{\mathrm{SCE}}(s, y) = \frac{1}{\ln 2}\mathrm{err}_{\mathrm{CE}}(s, y).$$

Proof. The equality is $\log_2 u = \ln u / \ln 2$. Let $\psi(t) = \log_2(1 + e^{-t})$; it is positive and strictly decreasing, since $1 + e^{-t} > 1$ and e^{-t} decreases, and $\psi(0) = \log_2 2 = 1$. If $ys \leq 0$, then $\mathrm{err}_{0/1} = 1 = \psi(0) \leq \psi(ys) = \mathrm{err}_{\mathrm{SCE}}$. If $ys > 0$, then $\mathrm{err}_{0/1} = 0 < \psi(ys)$. ■

Together with Proposition 9.4.2, both efficient models now minimise an upper bound of the 0/1 error — which is precisely what an algorithmic error measure $\widehat{\mathrm{err}}$ (Definition 8.3.1) is meant to do.

Upper bound: useful for designing algorithmic error $\widehat{\mathrm{err}}$.

§11.1.iv. Theoretical Implication of Upper Bound

For any ys where $s = \boldsymbol{w}^\top \boldsymbol{x}$,

$$\begin{aligned} & \mathrm{err}_{0/1}(s, y) \leq \mathrm{err}_{\mathrm{SCE}}(s, y) = \frac{1}{\ln 2}\mathrm{err}_{\mathrm{CE}}(s, y) \\ \Longrightarrow \quad & E_{\mathrm{in}}^{0/1}(\boldsymbol{w}) \leq E_{\mathrm{in}}^{\mathrm{SCE}}(\boldsymbol{w}) = \frac{1}{\ln 2}E_{\mathrm{in}}^{\mathrm{CE}}(\boldsymbol{w}) \\ & E_{\mathrm{out}}^{0/1}(\boldsymbol{w}) \leq E_{\mathrm{out}}^{\mathrm{SCE}}(\boldsymbol{w}) = \frac{1}{\ln 2}E_{\mathrm{out}}^{\mathrm{CE}}(\boldsymbol{w}). \end{aligned}$$

The superscript names the pointwise error being averaged. E_{in} is the mean of the pointwise error over the N examples and E_{out} its expectation over $(\boldsymbol{x}, y) \sim P$; both are linear and preserve $\leq$, so the pointwise inequality of Proposition 11.1.2 passes to them term by term.

Two routes then lead to $E_{\mathrm{out}}^{0/1}$:

VC on 0/1:	**VC-Reg on CE:**
$E_{\mathrm{out}}^{0/1}(\boldsymbol{w}) \leq E_{\mathrm{in}}^{0/1}(\boldsymbol{w}) + \Omega^{0/1}$	$E_{\mathrm{out}}^{0/1}(\boldsymbol{w}) \leq \frac{1}{\ln 2}E_{\mathrm{out}}^{\mathrm{CE}}(\boldsymbol{w})$
$\leq \frac{1}{\ln 2}E_{\mathrm{in}}^{\mathrm{CE}}(\boldsymbol{w}) + \Omega^{0/1}$	$\leq \frac{1}{\ln 2}E_{\mathrm{in}}^{\mathrm{CE}}(\boldsymbol{w}) + \frac{1}{\ln 2}\Omega^{\mathrm{CE}}$

- **Left (VC on 0/1)**: $\Omega^{0/1}$ is the penalty of Definition 7.4.1 for the perceptron set, $d_{\mathrm{VC}} = d + 1$. The VC bound of Theorem 6.4.2 holds with high probability for *every* $\boldsymbol{w}$ at once, so it covers the $\boldsymbol{w}$ that logistic regression returns, even though logistic regression never looked at the 0/1 error; the second line is then Proposition 11.1.2 averaged.

- **Right (VC-Reg on CE)**: the order is reversed: first bound $E_{\text{out}}^{0/1}$ by $E_{\text{out}}^{\text{CE}}$, then generalise the cross-entropy error itself, with a penalty Ω^{CE} of the same kind for that real-valued error, which these notes do not derive.

Either way the right-hand side is small when $E_{\text{in}}^{\text{CE}}(\boldsymbol{w})$ is, which is what logistic regression minimises. Linear regression minimises $E_{\text{in}}^{\text{SQR}}$ instead, so its bound swaps in Proposition 9.4.2, $\text{err}_{0/1} \leq \text{err}_{\text{SQR}}$, where the left route used Proposition 11.1.2:

$$E_{\text{out}}^{0/1}(\boldsymbol{w}) \leq E_{\text{in}}^{0/1}(\boldsymbol{w}) + \Omega^{0/1} \leq E_{\text{in}}^{\text{SQR}}(\boldsymbol{w}) + \Omega^{0/1}.$$

logistic/linear reg. for linear classification:

- **logistic reg.: small $E_{\text{in}}^{\text{CE}}(\boldsymbol{w}) \Longrightarrow$ small $E_{\text{out}}^{0/1}(\boldsymbol{w})$;**
- **linear reg.: small $E_{\text{in}}^{\text{SQR}}(\boldsymbol{w}) \Longrightarrow$ small $E_{\text{out}}^{0/1}(\boldsymbol{w})$.**

§11.1.v. Regression for Classification

1. Run **logistic/linear reg.** on $\mathcal{D}$ with $y_n \in \{-1, +1\}$ to get $\boldsymbol{w}_{\text{REG}}$;
2. return $g(\boldsymbol{x}) = \text{sign}(\boldsymbol{w}_{\text{REG}}^{\top}\boldsymbol{x})$.

This is the recipe of Section 9.4, now with logistic regression as a second choice. Against PLA, the three trade off as follows:

PLA	**linear regression**	**logistic regression**
• pros: **efficient + strong guarantee if lin. separable**	• pros: **'easiest' optimization**	• pros: **'easy' optimization**
• cons: works only if lin. separable, otherwise needing **pocket** heuristic	• cons: loose bound of $\text{err}_{0/1}$ for large $\lvert ys \rvert$	• cons: loose bound of $\text{err}_{0/1}$ for very negative ys

- **'Easiest'**: linear regression is one pseudo-inverse (algorithm 9.1).
- **'Easy'**: logistic regression is a convex descent (algorithm 10.1).

Why is a loose bound a *con*? Suppose a data point sits where the regression error is much larger than $\text{err}_{0/1}$. For the regression error this one point is very expensive, but for $\text{err}_{0/1}$ it is just one mistake (or none), like any other point. So when we minimise the regression error, $\boldsymbol{w}$ may move a lot just to make this one point cheaper, and in doing so push a few other points to the wrong side — giving *more* 0/1 mistakes. This only matters if some data points really lie where the two errors are far apart: $\lvert ys \rvert$ large on either side for squared error, and $ys \ll 0$ for cross-entropy, which goes to 0 on the correct side.

- **linear regression sometimes used to set w_0 for PLA/pocket/logistic regression;**
- **logistic regression often preferred over pocket.**

- **Setting $\boldsymbol{w}_0$**: the use already suggested in Section 9.4: a cheap, roughly right start.

- **Logistic regression over pocket**: both answer the non-separable case, but pocket is a heuristic search on an NP-hard problem, while logistic regression solves its own convex problem and, by the bound above, still controls the 0/1 error.

Exercise 11.1.3 (Fun Time)**.** Following the definition in the lecture, which of the following is not always $\geq \mathrm{err}_{0/1}(y, s)$ when $y \in \{-1, +1\}$?

(1) $\mathrm{err}_{0/1}(y, s)$;

(2) $\mathrm{err}_{\mathrm{SQR}}(y, s)$;

(3) $\mathrm{err}_{\mathrm{CE}}(y, s)$;

(4) $\mathrm{err}_{\mathrm{SCE}}(y, s)$.

Reference Answer. (3). Too simple, uh? :-) Anyway, note that $\mathrm{err}_{0/1}$ is surely an upper bound of itself. Choice (2) is Proposition 9.4.2 and choice (4) is Proposition 11.1.2. Choice (3) fails near the boundary: at $ys = 0$, $\mathrm{err}_{\mathrm{CE}} = \ln 2 < 1 = \mathrm{err}_{0/1}$, and indeed $\mathrm{err}_{\mathrm{CE}} < 1$ for all $ys > -\ln(e - 1) \approx -0.54$. ■

§11.2. Stochastic Gradient Descent

slides p.8–12

Logistic regression is now justified as a classifier; what remains is its cost, which is where PLA still wins.

§11.2.i. Two Iterative Optimization Schemes

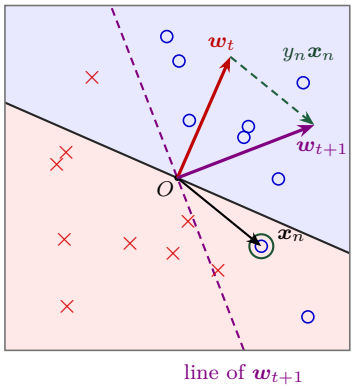
For $t = 0, 1, \ldots$

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + \eta \boldsymbol{v};$$

when stop, return last $\boldsymbol{w}$ as g.

PLA	**logistic regression (pocket)**
pick $(\boldsymbol{x}_n, y_n)$ and decide $\boldsymbol{w}_{t+1}$ by **the one example**	check $\mathcal{D}$ and decide $\boldsymbol{w}_{t+1}$ (or new $\hat{\boldsymbol{w}}$) by **all examples**
$O(1)$ time per iteration **:-)**	$O(N)$ time per iteration **:-(**

- **Logistic regression** needs all N examples because ∇E_{in} in (10.2) is a sum over all of them.
- **Pocket** needs them because it compares E_{in} of the new candidate with that of the one in its pocket (algorithm 2.3).
- **PLA**'s update, by contrast, touches a single example — as in this update of a PLA run on data of my own:



- **Shading**: $\boldsymbol{w}_t$'s: blue where it predicts ○ ($\boldsymbol{w}_t^\top \boldsymbol{x} > 0$), red where it predicts ×.
- **Update**: $\boldsymbol{w}_t$ gets three points wrong. PLA takes one of them, the circled ○ $\boldsymbol{x}_n$ ($y_n = +1$) in the red half, and adds $y_n\boldsymbol{x}_n$ (green, dashed) to $\boldsymbol{w}_t$ (red), giving $\boldsymbol{w}_{t+1}$ (purple), whose line is dashed. Afterwards only one point is wrong.
- **Origin**: as in Section 2.2, the line passes through the origin O, so the vectors can be drawn from O.

The update is one vector addition, whatever N is.

Question 11.2.1. Logistic regression with $O(1)$ time per iteration?

§11.2.ii. Logistic Regression Revisited

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + \eta \underbrace{\frac{1}{N}\sum_{n=1}^{N} \theta\left(-y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n\right)(y_n \boldsymbol{x}_n)}_{-\nabla E_{\text{in}}(\boldsymbol{w}_t)}$$

- Want: update direction $\boldsymbol{v} \approx -\nabla E_{\text{in}}(\boldsymbol{w}_t)$ while computing $\boldsymbol{v}$ by one single $(\boldsymbol{x}_n, y_n)$.
- Technique on removing $\frac{1}{N}\sum_{n=1}^{N}$: view as expectation $\mathcal{E}$ over uniform choice of n!

Write $\text{err}(\boldsymbol{w}, \boldsymbol{x}_n, y_n)$ for the pointwise cross-entropy (10.1), so that

$$E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N} \text{err}(\boldsymbol{w}, \boldsymbol{x}_n, y_n).$$

If n is drawn uniformly from $\{1, \ldots, N\}$, each term has weight $\frac{1}{N}$, so the average over n *is* an expectation, and the gradient passes through the finite sum:

Definition 11.2.2 (Stochastic gradient)**.** With n drawn uniformly at random from $\{1, \ldots, N\}$, $\nabla_{\boldsymbol{w}}\text{err}(\boldsymbol{w}, \boldsymbol{x}_n, y_n)$ is the **stochastic gradient**, and

$$\nabla_{\boldsymbol{w}} E_{\text{in}}(\boldsymbol{w}) = \underset{\text{random } n}{\mathcal{E}} \nabla_{\boldsymbol{w}}\text{err}(\boldsymbol{w}, \boldsymbol{x}_n, y_n)$$

is the **true gradient**.

So instead of averaging the gradients of all N examples, each step picks one n at random and takes the gradient of the error on that single example only. It costs one example to compute, and it is right on average.

§11.2.iii. Stochastic Gradient Descent (SGD)

Stochastic gradient = true gradient + zero-mean 'noise' directions.

That is, $\nabla \mathrm{err}(\boldsymbol{w}, \boldsymbol{x}_n, y_n) = \nabla E_{\mathrm{in}}(\boldsymbol{w}) + \boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} = \nabla \mathrm{err}(\boldsymbol{w}, \boldsymbol{x}_n, y_n) - \nabla E_{\mathrm{in}}(\boldsymbol{w})$, and $\mathcal{E}_n \boldsymbol{\epsilon} = \mathbf{0}$ by Definition 11.2.2.

Definition 11.2.3 (Stochastic gradient descent)**. Stochastic gradient descent** (SGD):

- idea: replace true gradient by stochastic gradient;
- after enough steps, average true gradient $\approx$ average stochastic gradient;
- pros: **simple & cheaper computation :-)** — useful for **big data** or **online learning**;
- cons: less stable in nature.

The second bullet is what makes the replacement legitimate. Each step's noise is zero-mean given everything before it, so the noise terms do not accumulate: their average shrinks like $1/\sqrt{T}$.

Lemma 11.2.4 (Noise averages out)

Run SGD for T steps, drawing n_t uniformly and independently of the past at each step, and let $\boldsymbol{\epsilon}_t = \nabla \mathrm{err}(\boldsymbol{w}_t, \boldsymbol{x}_{n_t}, y_{n_t}) - \nabla E_{\mathrm{in}}(\boldsymbol{w}_t)$. If $\|\boldsymbol{\epsilon}_t\| \le G$ always, then

$$\mathcal{E} \left\| \frac{1}{T} \sum_{t=0}^{T-1} \boldsymbol{\epsilon}_t \right\|^2 \le \frac{G^2}{T}.$$

Proof. $\boldsymbol{w}_t$ is determined by $n_0, \ldots, n_{t-1}$, and n_t is uniform and independent of these, so conditioned on them $\mathcal{E}\boldsymbol{\epsilon}_t = \mathbf{0}$ by Definition 11.2.2. For $s < t$, $\boldsymbol{\epsilon}_s$ is fixed once $n_0, \ldots, n_{t-1}$ are, hence $\mathcal{E}\left[\boldsymbol{\epsilon}_s^\top \boldsymbol{\epsilon}_t\right] = \mathcal{E}\left[\boldsymbol{\epsilon}_s^\top \mathcal{E}[\boldsymbol{\epsilon}_t \mid n_0, \ldots, n_{t-1}]\right] = 0$ by the tower property. Expanding the square, the cross terms vanish:

$$\mathcal{E} \left\| \sum_{t=0}^{T-1} \boldsymbol{\epsilon}_t \right\|^2 = \sum_{t=0}^{T-1} \mathcal{E} \|\boldsymbol{\epsilon}_t\|^2 \le T G^2;$$

divide by T^2. ∎

- G **exists for logistic regression**: $\|\nabla \mathrm{err}(\boldsymbol{w}, \boldsymbol{x}_n, y_n)\| = \theta(\cdot)\|\boldsymbol{x}_n\| \le \max_n \|\boldsymbol{x}_n\|$, the average ∇E_{in} obeys the same bound, and so $G = 2 \max_n \|\boldsymbol{x}_n\|$ works by the triangle inequality.
- **Average, not each step**: the lemma is about the *average* over many steps; each single step can still point well away from $-\nabla E_{\mathrm{in}}(\boldsymbol{w}_t)$, which is the instability of the last bullet.

SGD logistic regression, **looks familiar? :-)**:

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + \eta \underbrace{\theta\left(-y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n\right)(y_n \boldsymbol{x}_n)}_{-\nabla \mathrm{err}(\boldsymbol{w}_t, \boldsymbol{x}_n, y_n)}$$

The underbrace is one term of the sum in (10.2), negated and without the $\frac{1}{N}$. In full:

Algorithm 11.1: SGD Logistic Regression

Input: training examples $\mathcal{D}$, learning rate η
Output: the last $\boldsymbol{w}_{t+1}$, returned as g

initialize $\boldsymbol{w}_0$;
for $t = 0, 1, \ldots$ **do**
  pick $n \in \{1, \ldots, N\}$ uniformly at random;
  update by $\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + \eta\, \theta\left(-y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n\right)(y_n \boldsymbol{x}_n)$;
  if t *large enough* **then return** last $\boldsymbol{w}_{t+1}$ as g;

Each iteration now costs $O(d)$, independent of N.

§11.2.iv. PLA Revisited

$$\text{SGD logistic regression:}\quad \boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + \eta \cdot \theta\left(-y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n\right)(y_n \boldsymbol{x}_n)$$

$$\text{PLA:}\quad \boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t + 1 \cdot [\![y_n \neq \operatorname{sign}(\boldsymbol{w}_t^\top \boldsymbol{x}_n)]\!](y_n \boldsymbol{x}_n)$$

- **SGD logistic regression ≈ 'soft' PLA;**
- **PLA ≈ SGD logistic regression with $\eta = 1$ when $\boldsymbol{w}_t^\top \boldsymbol{x}_n$ large.**

Both move along $y_n \boldsymbol{x}_n$ and differ only in the factor in front.

- **PLA's factor**: 1 on a mistake and 0 otherwise.
- **SGD's factor**: $\theta(-y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n)$, which is near 1 when $y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n \ll 0$ (badly wrong) and near 0 when $y_n \boldsymbol{w}_t^\top \boldsymbol{x}_n \gg 0$ (safely right) — the Iverson bracket smoothed by θ.

The approximation is exact in the limit $\left|\boldsymbol{w}_t^\top \boldsymbol{x}_n\right| \to \infty$, which is why it needs a large score.

Two practical rules of thumb:

- stopping condition? t **large enough**;
- η? 0.1 when $\boldsymbol{x}$ in proper range.
- **Stopping**: forced: checking $\nabla E_{\text{in}}(\boldsymbol{w}_t) \approx \boldsymbol{0}$, the stopping rule of algorithm 10.1, costs $O(N)$ and would undo the saving.
- η: purely empirical: $\eta = 0.1$ is not derived from anything, it is simply a value that tends to work in practice. It presumes the features are scaled to a moderate range, say $|x_i| \lesssim 1$, so that one step $\eta\, \theta(\cdot)\, y_n \boldsymbol{x}_n$ changes each coordinate of $\boldsymbol{w}$ by at most about 0.1.

Exercise 11.2.5 (Fun Time)**.** Consider applying SGD on linear regression for big data. What is the update direction when using the negative stochastic gradient?

(1) $\boldsymbol{x}_n$;
(2) $y_n \boldsymbol{x}_n$;
(3) $2(\boldsymbol{w}_t^\top \boldsymbol{x}_n - y_n)\boldsymbol{x}_n$;
(4) $2(y_n - \boldsymbol{w}_t^\top \boldsymbol{x}_n)\boldsymbol{x}_n$.

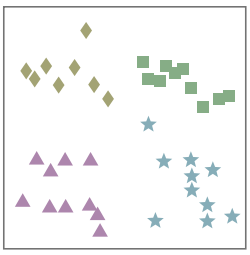
Reference Answer. (4). Go check Section 9.2 if you have forgotten about the gradient of squared error. :-) For one example, $\text{err}(\boldsymbol{w}, \boldsymbol{x}_n, y_n) = (\boldsymbol{w}^\top \boldsymbol{x}_n - y_n)^2$ has gradient $2(\boldsymbol{w}^\top \boldsymbol{x}_n - y_n)\boldsymbol{x}_n$ by the chain rule; its negative is (4). Anyway, the update rule has a nice physical interpretation: improve $\boldsymbol{w}_t$ by 'correcting' proportional to the residual $(y_n - \boldsymbol{w}_t^\top \boldsymbol{x}_n)$. ■

§11.3. Multiclass via Logistic Regression

slides p.13–19

Every tool so far answers a yes/no question. The multiclass problems of Definition 3.1.2 ask for one of K labels.

§11.3.i. Multiclass Classification



- $\mathcal{Y} = \{■, ♦, ▲, ★\}$ (4-class classification);
- **many applications** in practice, especially for 'recognition'.

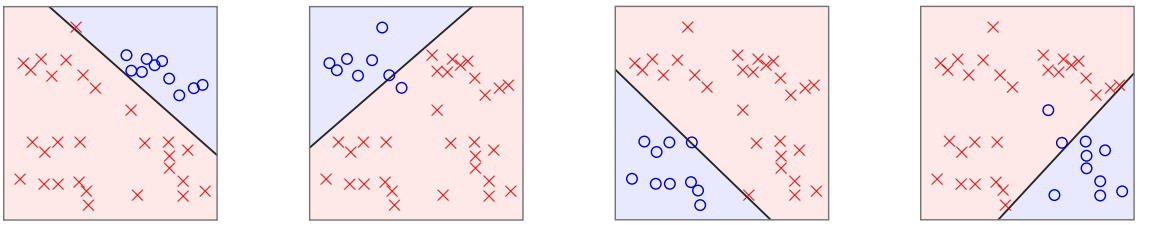
The coin data of Example 3.1.3 looked just like this.

- **Data**: the data set drawn here, and in every figure of this section and the next, is mine: $N = 38$ points (10 each of ■, ▲, ★ and 8 of ♦) placed where the deck's markers sit.
- **Classifiers**: every boundary and shade drawn on it is computed, not sketched: each binary classifier is logistic regression by algorithm 10.1 with $\boldsymbol{w}_0 = \mathbf{0}$, $\eta = 1$ and 150 steps, on the two coordinates rescaled to $[-1, 1]$.

Next: use tools for {×, ○} classification to {■, ♦, ▲, ★} classification.

§11.3.ii. One Class at a Time

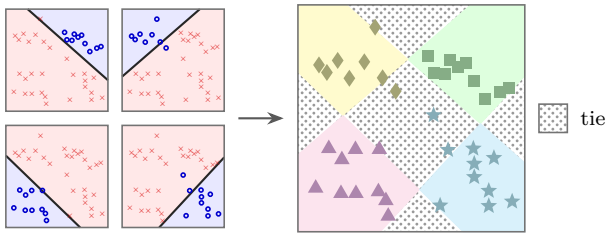
Ask one yes/no question per class: is it ■ or not? Relabel that class ○ and all the others ×, and learn a binary classifier. Four classes, four relabellings of the same 38 points:



Each classifier separates its own class from the rest fairly well, getting one or two points wrong near the middle of the square, where the classes come close. The question is what to do with four answers.

§11.3.iii. Multiclass Prediction: Combine Binary Classifiers

Predict k where exactly one classifier says ○ and it is the k-th one. The four half-planes above, overlaid:

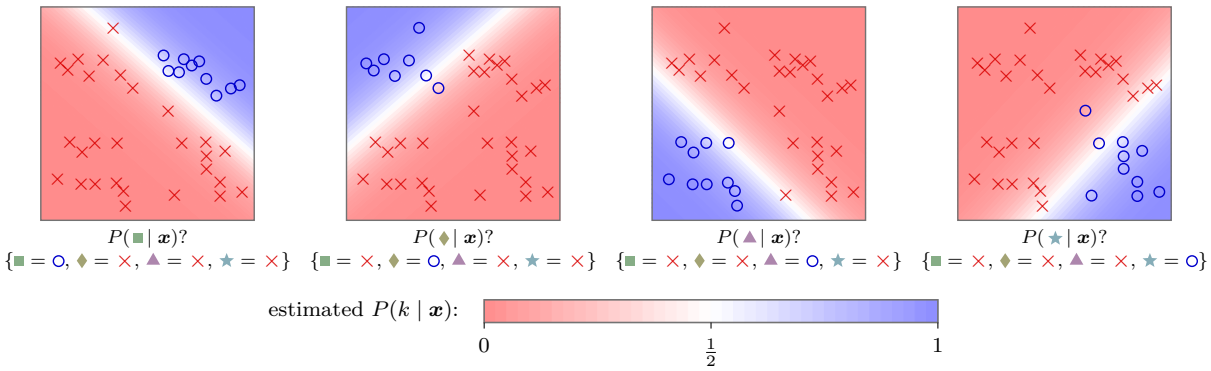


Question 11.3.1. But ties? :-)

The coloured regions are where exactly one classifier claims the input. The hatched ones are not: in the middle all four say ×, and along the diagonals two say ○ at once. Together these take up about a third of the square, and there the hard answers give no reason to prefer any class.

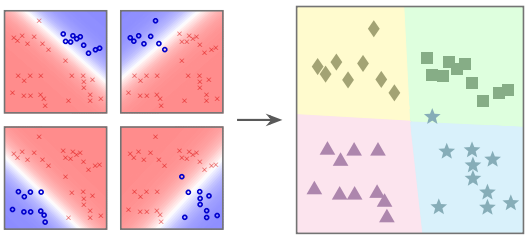
§11.3.iv. One Class at a Time Softly

Instead of '■ or not?', ask 'how likely is ■?'. That is, use each logistic regression as a soft binary classifier (Definition 10.1.2) on the same relabelled data: $\theta(\boldsymbol{w}_{[k]}^{\top}\boldsymbol{x})$ estimates $P(k \mid \boldsymbol{x})$, the probability that $\boldsymbol{x}$ belongs to class k, instead of only answering yes or no.



- **Colours**: each panel shades the estimate $\theta(\boldsymbol{w}_{[k]}^{\top}\boldsymbol{x})$ of $P(k \mid \boldsymbol{x})$ as in the colour bar: the deeper the blue, the closer the probability is to 1; the deeper the red, the closer it is to 0; white is $\frac{1}{2}$.
- **Same classifiers**: these are the same four classifiers as before: thresholding each panel at $\frac{1}{2}$ gives back the corresponding hard panel above, since $\theta(s) \geq \frac{1}{2}$ iff $s \geq 0$.
- **What is new**: the numbers can be compared across panels, even where all four are below $\frac{1}{2}$ or two are above.

§11.3.v. Multiclass Prediction: Combine Soft Classifiers



$$g(\boldsymbol{x}) = \arg\max_{k \in \mathcal{Y}} \ \theta\left(\boldsymbol{w}_{[k]}^{\top}\boldsymbol{x}\right).$$

- **Every input now gets an answer**: the hatched regions of the hard combination are shared out to whichever class has the highest estimated probability, and a tie needs two scores to be exactly equal, which happens only on the boundaries between regions.
- **No θ needed**: since θ is increasing, $g(\boldsymbol{x}) = \arg\max_k \boldsymbol{w}_{[k]}^{\top}\boldsymbol{x}$: θ never needs to be evaluated.
- **Convex regions**: each region is the set where one linear score beats the other three, an intersection of half-planes, hence convex.
- **On this data**: g gets only one of the 38 points wrong: the ★ nearest the centre, which it calls ■.

§11.3.vi. One-Versus-All (OVA) Decomposition

Algorithm 11.2: One-Versus-All (OVA) Decomposition

Input: training examples $\mathcal{D}$ with $y_n \in \mathcal{Y}$
Output: the multiclass classifier g

1 **for** $k \in \mathcal{Y}$ **do**
2 obtain $\boldsymbol{w}_{[k]}$ by running logistic regression on $\mathcal{D}_{[k]} = \left\{\left(\boldsymbol{x}_n, y'_n = 2[\![y_n = k]\!] - 1\right)\right\}_{n=1}^{N}$;
3 **return** $g(\boldsymbol{x}) = \arg\max_{k \in \mathcal{Y}} \left(\boldsymbol{w}_{[k]}^{\top}\boldsymbol{x}\right)$;

$2[\![y_n = k]\!] - 1$ is $+1$ for class k and -1 for every other class, the relabelling of the figures.

- pros: efficient, can be coupled with any logistic regression-like approaches;
  - efficient: K ordinary logistic regressions, which can run in parallel;
  - 'logistic regression-like': step 2 compares $\boldsymbol{w}_{[k]}^{\top}\boldsymbol{x}$ across different classifiers, which only makes sense if every score is on the same scale, an estimate of $P(k \mid \boldsymbol{x})$. A method that outputs only ± 1, such as PLA, gives nothing to compare.
- cons: often **unbalanced** $\mathcal{D}_{[k]}$ when K large;
  - if the K classes are equally frequent, $\mathcal{D}_{[k]}$ has about N/K examples of ○ and about $N(K-1)/K$ of ×: for $K = 100$, 1% against 99%;
  - always answering × then already has $E_{\text{in}} \approx 1/K$, so $\theta(\boldsymbol{w}_{[k]}^{\top}\boldsymbol{x})$ tends to stay small everywhere, even on class k itself;
  - step 2 then compares K small, unreliable numbers, and easily picks the wrong class.
- extension: multinomial ('coupled') logistic regression.
  - the problem it fixes: OVA trains the K estimates separately, so they need not add up to 1. In the centre of our square all four are below $\frac{1}{2}$; where two classifiers overlap, two are above $\frac{1}{2}$;
  - the model: one formula for all classes at once,

$$P(k \mid \boldsymbol{x}) = \frac{\exp\left(\boldsymbol{w}_{[k]}^{\top}\boldsymbol{x}\right)}{\sum_{j \in \mathcal{Y}} \exp\left(\boldsymbol{w}_{[j]}^{\top}\boldsymbol{x}\right)},$$

positive and adding up to 1 by construction. All K weight vectors are learned together by maximum likelihood on the whole $\mathcal{D}$, as in Section 10.2.

Remark 11.3.2 (Error of multinomial logistic regression)**.** Follow Section 10.2 with $\boldsymbol{W} = (\boldsymbol{w}_{[k]})_{k\in\mathcal{Y}}$ and $P_{\boldsymbol{W}}(k \mid \boldsymbol{x})$ the formula above. Dropping the factors $P(\boldsymbol{x}_n)$, common to every $\boldsymbol{W}$,

$$\begin{aligned}
&\max_{\boldsymbol{W}} \text{likelihood}(\boldsymbol{W}) \propto \prod_{n=1}^{N} P_{\boldsymbol{W}}(y_n \mid \boldsymbol{x}_n) = \prod_{n=1}^{N} \frac{\exp\left(\boldsymbol{w}_{[y_n]}^\top \boldsymbol{x}_n\right)}{\sum_{j\in\mathcal{Y}} \exp\left(\boldsymbol{w}_{[j]}^\top \boldsymbol{x}_n\right)} \\
\Leftrightarrow\ &\max_{\boldsymbol{W}} \ln \prod_{n=1}^{N} P_{\boldsymbol{W}}(y_n \mid \boldsymbol{x}_n) \\
\Leftrightarrow\ &\min_{\boldsymbol{W}} \frac{1}{N} \sum_{n=1}^{N} \underbrace{\left(\ln \sum_{j\in\mathcal{Y}} \exp\left(\boldsymbol{w}_{[j]}^\top \boldsymbol{x}_n\right) - \boldsymbol{w}_{[y_n]}^\top \boldsymbol{x}_n\right)}_{\text{err}(\boldsymbol{W},\boldsymbol{x}_n,y_n)}.
\end{aligned}$$

The symmetry $1-\theta(s) = \theta(-s)$ is no longer needed: the formula gives $P_{\boldsymbol{W}}(y_n \mid \boldsymbol{x}_n)$ for every label directly. Pulling $\exp\left(\boldsymbol{w}_{[y]}^\top \boldsymbol{x}\right)$ out of the sum,

$$\text{err}(\boldsymbol{W}, \boldsymbol{x}, y) = \ln\Big(1 + \sum_{j\neq y} \exp\big(-(\boldsymbol{w}_{[y]} - \boldsymbol{w}_{[j]})^\top \boldsymbol{x}\big)\Big).$$

- **Still cross-entropy**: read the observed label as a distribution q over $\mathcal{Y}$ that puts all its mass on y, $q(k) = [\![y = k]\!]$, and the model's output as $p(k) = P_{\boldsymbol{W}}(k \mid \boldsymbol{x})$. Their cross-entropy is

  $$-\sum_{k\in\mathcal{Y}} q(k) \ln p(k) = -\ln P_{\boldsymbol{W}}(y \mid \boldsymbol{x}) = \text{err}(\boldsymbol{W}, \boldsymbol{x}, y),$$

  since $q(k) = 0$ kills every term but $k = y$. This is the two-term formula after (10.1) with $(y', 1-y')$ and $(p, 1-p)$ replaced by K entries each.

- $K = 2$ **gives back** (10.1): take $\mathcal{Y} = \{-1, +1\}$. The sum over $j \neq y$ has the single term $j = -y$, so

  $$\text{err} = \ln\big(1 + \exp\big(-(\boldsymbol{w}_{[y]} - \boldsymbol{w}_{[-y]})^\top \boldsymbol{x}\big)\big).$$

  Put $\boldsymbol{w} = \boldsymbol{w}_{[+1]} - \boldsymbol{w}_{[-1]}$. For $y = +1$, $\boldsymbol{w}_{[y]} - \boldsymbol{w}_{[-y]} = \boldsymbol{w}$; for $y = -1$ it is $-\boldsymbol{w}$. In both cases it equals $y\boldsymbol{w}$, and $\text{err} = \ln\big(1 + \exp\big(-y\boldsymbol{w}^\top \boldsymbol{x}\big)\big)$. Likewise $P_{\boldsymbol{W}}(+1 \mid \boldsymbol{x}) = 1/(1 + \exp\big(-\boldsymbol{w}^\top \boldsymbol{x}\big)) = \theta(\boldsymbol{w}^\top \boldsymbol{x})$: only the difference of the two weight vectors matters, and binary logistic regression learns exactly that difference.

- **Still a 0/1 bound**: we show $\text{err}_{0/1} \leq \text{err}/\ln 2$, the counterpart of Proposition 11.1.2, with $g(\boldsymbol{x}) = \arg\max_k \boldsymbol{w}_{[k]}^\top \boldsymbol{x}$ and a tie counted as a mistake, like sign(0).
  - If $g(\boldsymbol{x}) = y$, then $\text{err}_{0/1} = 0 < \text{err}/\ln 2$, because every term inside the ln is positive, so its argument exceeds 1.
  - If $g(\boldsymbol{x}) \neq y$, then y is not the strict winner, so some $j \neq y$ has $\boldsymbol{w}_{[j]}^\top \boldsymbol{x} \geq \boldsymbol{w}_{[y]}^\top \boldsymbol{x}$. That j's term is $\exp\big(-(\boldsymbol{w}_{[y]} - \boldsymbol{w}_{[j]})^\top \boldsymbol{x}\big) \geq e^0 = 1$, and the other terms are positive, so the argument of the ln is at least $1 + 1 = 2$. Hence $\text{err} \geq \ln 2$, i.e. $\text{err}/\ln 2 \geq 1 = \text{err}_{0/1}$.

  Put differently, a mistake means $P_{\boldsymbol{W}}(y \mid \boldsymbol{x}) \leq \frac{1}{2}$, the multiclass version of $ys \leq 0$.

OVA: a simple multiclass meta-algorithm to keep in your toolbox.

'Meta' because it is built from any binary algorithm of the right kind, called as a subroutine.

Exercise 11.3.3 (Fun Time). Which of the following best describes the training effort of OVA decomposition based on logistic regression on some K-class classification data of size N?

(1) learn K logistic regression hypotheses, each from data of size N/K;

(2) learn K logistic regression hypotheses, each from data of size $N \ln K$;

(3) learn K logistic regression hypotheses, each from data of size N;

(4) learn K logistic regression hypotheses, each from data of size NK.

Reference Answer. (3). Each $\mathcal{D}_{[k]}$ in algorithm 11.2 relabels all N examples. Note that the learning part can be easily done in parallel, while the data is essentially of the same size as the original data. ■

§11.4. Multiclass via Binary Classification

slides p.20–24

OVA works, but its cost and its weakness both come from pitting one class against all the rest.

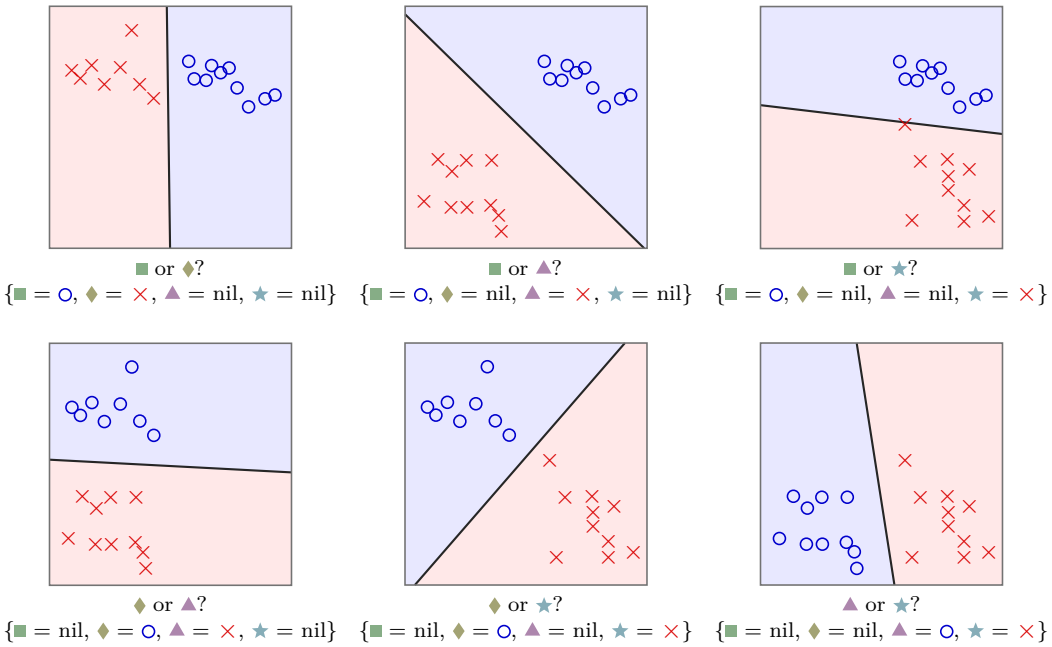
§11.4.i. Source of Unbalance: One versus All

Look again at the four sub-problems behind the soft combination above: in each, one class of 8 or 10 points stands against the other three, 28 or 30 points. With 4 classes that is mild; with 100 classes each sub-problem would be 1% against 99%.

Idea: make binary classification problems more balanced by one versus one.

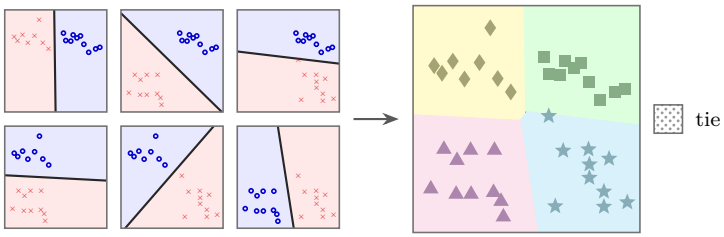
§11.4.ii. One versus One at a Time

Take two classes at a time, relabel one ○ and the other ×, and leave the remaining classes out of that problem entirely ('nil'). With four classes there are six pairs:



- **Each problem**: balanced, uses only the points of its two classes, and here each happens to be linearly separable.
- **Away from its two classes**: a classifier's half-planes say nothing meaningful: the ■-or-♦ classifier still answers on a ★, but its answer is irrelevant.

§11.4.iii. Multiclass Prediction: Combine Pairwise Classifiers



$$g(\boldsymbol{x}) = \textbf{tournament champion}\left\{\boldsymbol{w}_{[k,\ell]}^{\top}\boldsymbol{x}\right\}$$
(voting of classifiers).

- **Tournament champion (voting)**: to label an input $\boldsymbol{x}$, run it through *every* pairwise classifier, $\binom{K}{2}$ of them, and count how many say each class:
  - each classifier $\boldsymbol{w}_{[k,\ell]}$ is a game between k and ℓ: it gives one vote to k if $\boldsymbol{w}_{[k,\ell]}^{\top}\boldsymbol{x} > 0$ and to ℓ otherwise;
  - every class appears in $K-1$ classifiers, one against each of the other classes, so it can collect at most $K-1$ votes;
  - $g(\boldsymbol{x})$ is the class with the most votes, i.e. the class that the most classifiers say $\boldsymbol{x}$ belongs to.

  With our four classes, $\boldsymbol{x}$ goes through all six classifiers. ■, for instance, appears in three of them (■ vs ♦, ■ vs ▲, ■ vs ★), so it can collect 0 to 3 votes.
- **Irrelevant games do no harm**: on a true ★, the three games involving ★ should all go its way, giving 3 votes, while any other class can collect at most 2.
- **Ties**: still possible, where three classes beat each other in a cycle with 2 votes each. Here that happens only in two slivers at the centre, too thin to see, far smaller than the tie regions of the hard OVA combination.

§11.4.iv. One-versus-one (OVO) Decomposition

Algorithm 11.3: One-versus-one (OVO) Decomposition

Input: training examples $\mathcal{D}$ with $y_n \in \mathcal{Y}$
Output: the multiclass classifier g

for $(k,\ell) \in \mathcal{Y} \times \mathcal{Y}$ **do**
  obtain $\boldsymbol{w}_{[k,\ell]}$ by running linear binary classification on $\mathcal{D}_{[k,\ell]} = \left\{\left(\boldsymbol{x}_n, y'_n = 2[\![y_n = k]\!] - 1\right) : y_n = k \text{ or } y_n = \ell\right\}$;
return $g(\boldsymbol{x}) =$ tournament champion $\left\{\boldsymbol{w}_{[k,\ell]}^{\top}\boldsymbol{x}\right\}$;

The loop is over pairs with $k \neq \ell$, each unordered pair once: (ℓ, k) is the same game with the labels swapped. That makes $\binom{K}{2} = K(K-1)/2$ classifiers.

- pros: efficient ('smaller' training problems), stable, can be coupled with any binary classification approaches;
- cons: use $O(K^2)$ $\boldsymbol{w}_{[k,\ell]}$ — **more space, slower prediction, more training**.
- **Any binary method**: unlike OVA, the combination only counts votes, so it never compares scores of different classifiers; any binary method that outputs a sign will do, including PLA or pocket.
- **Smaller problems**: each problem has only the examples of two classes, about $2N/K$ of them; whether the total work is less than OVA's depends on how the binary algorithm's cost grows with data size, which the next Fun Time makes concrete.

OVO: another simple multiclass meta-algorithm to keep in your toolbox.

Exercise 11.4.1 (Fun Time)**.** Assume that some binary classification algorithm takes exactly N^3 CPU-seconds for data of size N. Also, for some 10-class multiclass classification problem, assume that there are $N/10$ examples for each class. Which of the following is total CPU-seconds needed for OVO decomposition based on the binary classification algorithm?

(1) $\frac{9}{200}N^3$;
(2) $\frac{9}{25}N^3$;
(3) $\frac{4}{5}N^3$;
(4) N^3.

Reference Answer. (2). There are 45 binary classifiers, each trained with data of size $(2N)/10$:

$$\binom{10}{2}\left(\frac{2N}{10}\right)^3 = 45 \cdot \frac{8N^3}{1000} = \frac{9}{25}N^3.$$

Note that OVA decomposition with the same algorithm would take $10N^3$ time, much worse than OVO. ■

Summary

Linear Models for Binary Classification three models useful in different ways.

Stochastic Gradient Descent follow negative stochastic gradient.

Multiclass via Logistic Regression predict with maximum estimated $P(k \mid \boldsymbol{x})$.

Multiclass via Binary Classification predict the tournament champion.

The linear models now cover binary classification, soft classification, regression and multiclass classification, with efficient algorithms for all of them. What they share is the limitation of the hypothesis: every decision is made by hyperplanes in the original $\boldsymbol{x}$-space. Next: from linear to nonlinear.

12. Nonlinear Transformation

Lecture 12

21 Oct.

> **As previously seen .** Linear classification, linear regression and logistic regression now cover binary, soft, real-valued and multiclass outputs, all with efficient algorithms. Every one of them decides by the score $s = \boldsymbol{w}^\top \boldsymbol{x}$, so every boundary they can draw is a hyperplane in the original $\boldsymbol{x}$-space.

This chapter removes that restriction without writing a single new algorithm.

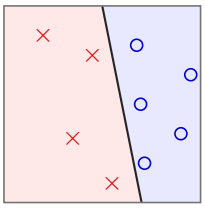
1. A circle in $\mathcal{X}$ turns out to be a line in a transformed space $\mathcal{Z}$.
2. Hence any linear model run on transformed data gives a nonlinear model on the original data.
3. The transform has a price, in computation and in VC dimension.
4. Polynomial transforms nest into a chain of hypothesis sets, which tells us where to start: with the linear model.

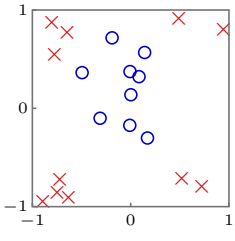
§12.1. Quadratic Hypotheses

slides p.2–7

§12.1.i. Linear Hypotheses

Up to now: linear hypotheses. But limited ...





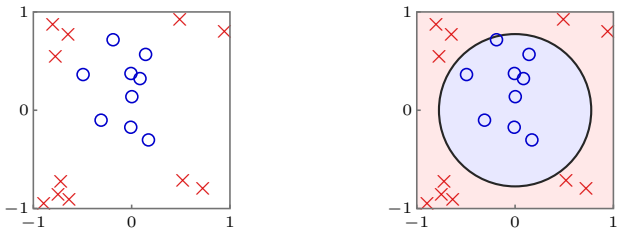
- visually: 'line'-like boundary;
- mathematically: linear scores $s = \boldsymbol{w}^\top \boldsymbol{x}$;
- theoretically: d_{VC} under control :-);
- practically: on some $\mathcal{D}$, large E_{in} for every line :-(.

The first two are the definition of the perceptron (Definition 2.1.2), the third is Theorem 7.2.7: $d_{\text{VC}} = d + 1$, so a modest N already makes $E_{\text{in}} \approx E_{\text{out}}$. The fourth is the right-hand data set, which will be with us for the whole chapter.

Observation . No line separates the right-hand data. The four crosses nearest the corners span a quadrilateral containing the circle at $(0.002, 0.138)$, so that circle is a convex combination $\sum_i \lambda_i \boldsymbol{x}_i$ of crosses. Any $\boldsymbol{w}$ with negative score on every cross then has $\boldsymbol{w}^\top(1, \sum_i \lambda_i \boldsymbol{x}_i) = \sum_i \lambda_i \boldsymbol{w}^\top(1, \boldsymbol{x}_i) < 0$ on the circle too.

Question 12.1.1. How to **break the limit** of linear hypotheses?

§12.1.ii. Circular Separable



- $\mathcal{D}$ not linear separable;
- but **circular separable** by a circle of radius $\sqrt{0.6}$ centered at origin:

$$h_{\text{SEP}}(\boldsymbol{x}) = \text{sign}\left(-x_1^2 - x_2^2 + 0.6\right).$$

h_{SEP} says $+1$ exactly inside the circle $x_1^2 + x_2^2 = 0.6$, and the data agree with it on every example.

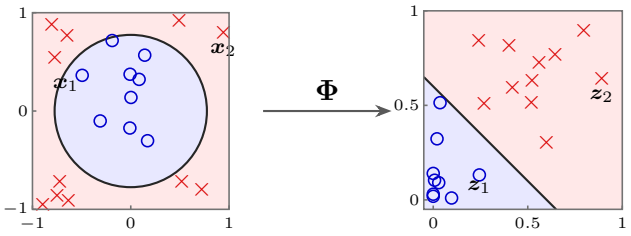
Question 12.1.2. Re-derive **Circular**-PLA, **Circular**-Regression, blahblah ... all over again? :-)

§12.1.iii. Circular Separable and Linear Separable

Read h_{SEP} term by term, as a weighted sum:

$$\begin{aligned} h(\boldsymbol{x}) &= \text{sign}\Big(\underbrace{0.6}_{\tilde{w}_0} \cdot \underbrace{1}_{z_0} + \underbrace{(-1)}_{\tilde{w}_1} \cdot \underbrace{x_1^2}_{z_1} + \underbrace{(-1)}_{\tilde{w}_2} \cdot \underbrace{x_2^2}_{z_2}\Big) \\ &= \text{sign}\left(\tilde{\boldsymbol{w}}^\top \boldsymbol{z}\right). \end{aligned}$$

The formula is still linear — in the new variables $z_1 = x_1^2$, $z_2 = x_2^2$ rather than in x_1, x_2.



- $\{(\boldsymbol{x}_n, y_n)\}$ circular separable $\Longrightarrow$ $\{(\boldsymbol{z}_n, y_n)\}$ linear separable;
- $\boldsymbol{x} \in \mathcal{X} \xmapsto{\boldsymbol{\Phi}} \boldsymbol{z} \in \mathcal{Z}$: **(nonlinear) feature transform $\boldsymbol{\Phi}$**.

Definition 12.1.3 (Feature transform)**.** A **feature transform** is a map $\boldsymbol{\Phi}\colon \mathcal{X} \to \mathcal{Z}$, $\boldsymbol{z} = \boldsymbol{\Phi}(\boldsymbol{x})$, whose first coordinate is the constant $z_0 = 1$. $\mathcal{Z}$ is the **$\mathcal{Z}$-space** and $\boldsymbol{z}_n = \boldsymbol{\Phi}(\boldsymbol{x}_n)$ the transformed examples. Here $\boldsymbol{\Phi}(\boldsymbol{x}) = (1, x_1^2, x_2^2)$.

> **Notation .** A tilde marks objects living in $\mathcal{Z}$: weights $\tilde{\boldsymbol{w}}$, hypotheses $\tilde{h}$, and $\tilde{d}$ for the dimension of $\mathcal{Z}$ not counting z_0. Plain z_i is a coordinate of $\boldsymbol{z}$; bold $\boldsymbol{z}_n$ is the n-th transformed example. In the pictures, $\boldsymbol{x}_1$ (a circle) lands at $\boldsymbol{z}_1$ and $\boldsymbol{x}_2$ (a cross) at $\boldsymbol{z}_2$.

Squaring sends both $\pm x_i$ to the same $z_i \geq 0$, so the whole of $\mathcal{X}$ folds into the corner $z_1, z_2 \geq 0$: circles near the origin of $\mathcal{X}$ land near the origin of $\mathcal{Z}$, crosses near the corners of $\mathcal{X}$ land far out, and the circle $x_1^2 + x_2^2 = 0.6$ becomes the line $z_1 + z_2 = 0.6$.

> **Question 12.1.4.** Circular separable in $\mathcal{X} \implies$ linear separable in $\mathcal{Z}$; vice versa?

Not quite. A line in $\mathcal{Z}$ pulls back to $\tilde{w}_0 + \tilde{w}_1 x_1^2 + \tilde{w}_2 x_2^2 = 0$, which is a circle only for special weights — the next subsection lists what else it can be.

§12.1.iv. Linear Hypotheses in $\mathcal{Z}$-Space

$$(z_0, z_1, z_2) = \boldsymbol{z} = \boldsymbol{\Phi}(\boldsymbol{x}) = (1, x_1^2, x_2^2),$$

$$h(\boldsymbol{x}) = \tilde{h}(\boldsymbol{z}) = \operatorname{sign}\left(\tilde{\boldsymbol{w}}^\top \boldsymbol{\Phi}(\boldsymbol{x})\right) = \operatorname{sign}\left(\tilde{w}_0 + \tilde{w}_1 x_1^2 + \tilde{w}_2 x_2^2\right).$$

$\tilde{\boldsymbol{w}} = (\tilde{w}_0, \tilde{w}_1, \tilde{w}_2)$:

- $(0.6, -1, -1)$: circle ($\circ$ inside);
- $(-0.6, +1, +1)$: circle ($\circ$ outside);
- $(0.6, -1, -2)$: ellipse;
- $(0.6, -1, +2)$: hyperbola;
- $(0.6, +1, +2)$: constant $\circ$:-).

Each case is the sign pattern of $\tilde{\boldsymbol{w}}$.

- **Circles**: $\tilde{w}_1 = \tilde{w}_2$ with the opposite sign to $\tilde{w}_0$; negating all three keeps the curve and swaps the sides, which is why $\circ$ moves from inside to outside.
- **Ellipse**: unequal weights of the same sign, $x_1^2 + 2x_2^2 = 0.6$.
- **Hyperbola**: weights of opposite signs, $x_1^2 - 2x_2^2 = 0.6$.
- **Constant**: all positive, so $0.6 + x_1^2 + 2x_2^2 > 0$ for every $\boldsymbol{x}$: there is no boundary at all.

What is missing is just as telling: every curve here is centred at the origin, with axes along x_1 and x_2, because $\boldsymbol{\Phi}$ sees only x_1^2 and x_2^2 and cannot tell x_i from $-x_i$. A circle centred anywhere else, or a tilted ellipse, is out of reach.

Lines in $\mathcal{Z}$-space $\Leftrightarrow$ special quadratic curves in $\mathcal{X}$-space.

§12.1.v. General Quadratic Hypothesis Set

The missing curves need the missing terms. A 'bigger' $\mathcal{Z}$-space with

$$\boldsymbol{\Phi}_2(\boldsymbol{x}) = (1, x_1, x_2, x_1^2, x_1x_2, x_2^2)$$

keeps all monomials of degree at most 2: perceptrons in $\mathcal{Z}$-space $\Leftrightarrow$ quadratic hypotheses in $\mathcal{X}$-space.

Definition 12.1.5 (Quadratic hypothesis set)**.**

$$\mathcal{H}_{\boldsymbol{\Phi}_2} = \left\{ h(\boldsymbol{x}) : h(\boldsymbol{x}) = \tilde{h}\left(\boldsymbol{\Phi}_2(\boldsymbol{x})\right) \text{ for some linear } \tilde{h} \text{ on } \mathcal{Z} \right\}.$$

- can implement **all possible quadratic curve boundaries**: circle, ellipse, **rotated** ellipse, hyperbola, parabola, ...

$$\text{ellipse } 2(x_1 + x_2 - 3)^2 + (x_1 - x_2 - 4)^2 = 1 \quad \Longleftarrow \quad \tilde{\boldsymbol{w}}^\top = [33, -20, -4, 3, 2, 3];$$

- include **lines and constants as degenerate cases**.

Example 12.1.6 (The rotated ellipse)**.** Expanding,

$$2(x_1 + x_2 - 3)^2 = 2x_1^2 + 4x_1x_2 + 2x_2^2 - 12x_1 - 12x_2 + 18,$$
$$(x_1 - x_2 - 4)^2 = x_1^2 - 2x_1x_2 + x_2^2 - 8x_1 + 8x_2 + 16,$$

so the ellipse is $33 - 20x_1 - 4x_2 + 3x_1^2 + 2x_1x_2 + 3x_2^2 = 0$, whose coefficients in the order of $\boldsymbol{\Phi}_2$ are exactly $\tilde{\boldsymbol{w}}$. The cross term $2x_1x_2$ tilts the axes, and the linear terms move the centre to $(3.5, -0.5)$, where $x_1 + x_2 = 3$ and $x_1 - x_2 = 4$. Far from the centre the quadratic part dominates and is positive, so this $\tilde{\boldsymbol{w}}$ says $\circ$ outside the ellipse.

Setting $\tilde{w}_3 = \tilde{w}_4 = \tilde{w}_5 = 0$ leaves $\text{sign}(\tilde{w}_0 + \tilde{w}_1x_1 + \tilde{w}_2x_2)$, an ordinary perceptron; zeroing all but $\tilde{w}_0$ leaves a constant. So the perceptrons sit inside $\mathcal{H}_{\boldsymbol{\Phi}_2}$ — the quadratic set only adds options.

Next: learn a good quadratic hypothesis g.

Exercise 12.1.7 (Fun Time)**.** Using the transform $\boldsymbol{\Phi}_2(\boldsymbol{x}) = (1, x_1, x_2, x_1^2, x_1x_2, x_2^2)$, which of the following weights $\tilde{\boldsymbol{w}}^\top$ in the $\mathcal{Z}$-space implements the parabola $2x_1^2 + x_2 = 1$?

(1) $[-1, 2, 1, 0, 0, 0]$;

(2) $[0, 2, 1, 0, -1, 0]$;

(3) $[-1, 0, 1, 2, 0, 0]$;

(4) $[-1, 2, 0, 0, 0, 1]$.

Reference Answer. (3). Too simple, uh? :-) Flexibility to implement arbitrary quadratic curves opens new possibilities for minimizing E_{in}! Indeed $2x_1^2 + x_2 = 1$ is $-1 + 0 \cdot x_1 + 1 \cdot x_2 + 2x_1^2 + 0 \cdot x_1x_2 + 0 \cdot x_2^2 = 0$. The others are the line $2x_1 + x_2 = 1$, the hyperbola $2x_1 + x_2 = x_1x_2$, and the sideways parabola $2x_1 + x_2^2 = 1$. ■

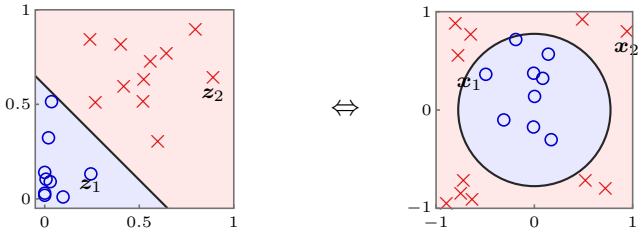
§12.2. Nonlinear Transform

slides p.8–12

$\mathcal{H}_{\boldsymbol{\Phi}_2}$ is defined through perceptrons on $\mathcal{Z}$; what is still missing is a way to *learn* in it.

§12.2.i. Good Quadratic Hypothesis

$\mathcal{Z}$-space		$\mathcal{X}$-space
perceptrons	$\Leftrightarrow$	quadratic hypotheses
good perceptron	$\Leftrightarrow$	**good quadratic hypothesis**
separating perceptron	$\Leftrightarrow$	**separating quadratic hypothesis**



- want: get **good perceptron** in $\mathcal{Z}$-space;
- known: get **good perceptron** in $\mathcal{X}$-space with data $\{(\boldsymbol{x}_n, y_n)\}$.

The two rows of '$\Leftrightarrow$' are one identity, applied example by example:

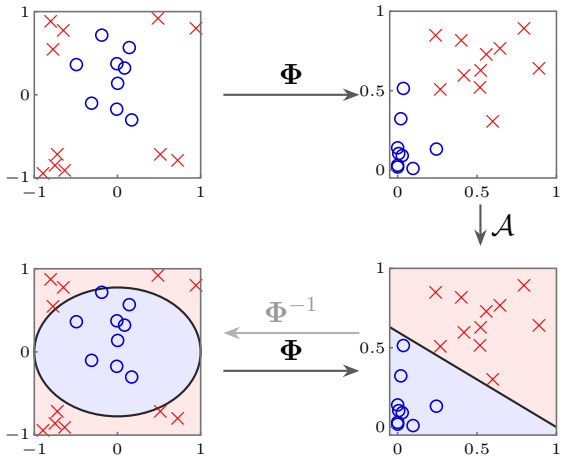
> **Lemma 12.2.1** (Same in-sample error after the transform)
> Let $h(\boldsymbol{x}) = \tilde{h}(\boldsymbol{\Phi}(\boldsymbol{x}))$. Then h on $\{(\boldsymbol{x}_n, y_n)\}$ and $\tilde{h}$ on $\{(\boldsymbol{z}_n, y_n)\}$ make mistakes on exactly the same n, so their E_{in} are equal.

Proof. $h(\boldsymbol{x}_n) = \tilde{h}(\boldsymbol{\Phi}(\boldsymbol{x}_n)) = \tilde{h}(\boldsymbol{z}_n)$, and the label y_n is the same in both data sets. ■

So a good (or separating) perceptron for the transformed data *is* a good (or separating) quadratic hypothesis for the original data, and the algorithm needs no change — only its input does.

Todo: get good perceptron in $\mathcal{Z}$-space with data $\{(\boldsymbol{z}_n = \boldsymbol{\Phi}_2(\boldsymbol{x}_n), y_n)\}$.

§12.2.ii. The Nonlinear Transform Steps



Algorithm 12.1: Learning with a Feature Transform

Input: training examples $\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}$, a feature transform $\boldsymbol{\Phi}$, a linear classification algorithm $\mathcal{A}$

Output: the hypothesis g

1 transform original data $\{(\boldsymbol{x}_n, y_n)\}$ to $\{(\boldsymbol{z}_n = \boldsymbol{\Phi}(\boldsymbol{x}_n), y_n)\}$ by $\boldsymbol{\Phi}$;

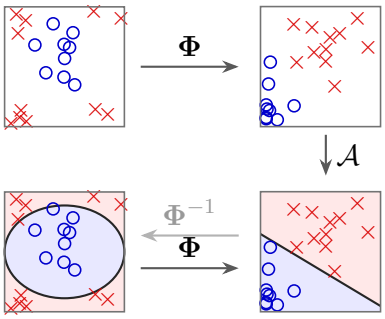
2 get a good perceptron $\tilde{\boldsymbol{w}}$ using $\{(\boldsymbol{z}_n, y_n)\}$ and your favorite linear classification algorithm $\mathcal{A}$;

3 **return** $g(\boldsymbol{x}) = \text{sign}\left(\tilde{\boldsymbol{w}}^\top \boldsymbol{\Phi}(\boldsymbol{x})\right)$;

Two things in the picture deserve a second look.

- **An ellipse, not the circle**: the line $\mathcal{A}$ returned (bottom right) is $0.6z_1 + z_2 = 0.6$, i.e. $\tilde{\boldsymbol{w}} = (0.6, -0.6, -1)$, not the $(0.6, -1, -1)$ we designed by hand. It separates the transformed data just as well, and back in $\mathcal{X}$ it is the ellipse $x_1^2 + x_2^2/0.6 = 1$. By Lemma 12.2.1 both have $E_{\text{in}} = 0$; $\mathcal{A}$ has no reason to prefer the circle.
- **The grey $\boldsymbol{\Phi}^{-1}$**: $\boldsymbol{\Phi}$ is not invertible — $\boldsymbol{x}$ and $-\boldsymbol{x}$ have the same $\boldsymbol{z}$ — and the algorithm never inverts it. Step 3 carries the boundary back by *composing* with $\boldsymbol{\Phi}$: g says $\circ$ on the set of $\boldsymbol{x}$ whose image lies on the $\circ$ side of the line. The arrow is only a picture of that preimage.

§12.2.iii. Nonlinear Model via Nonlinear Φ + Linear Models



Two choices:

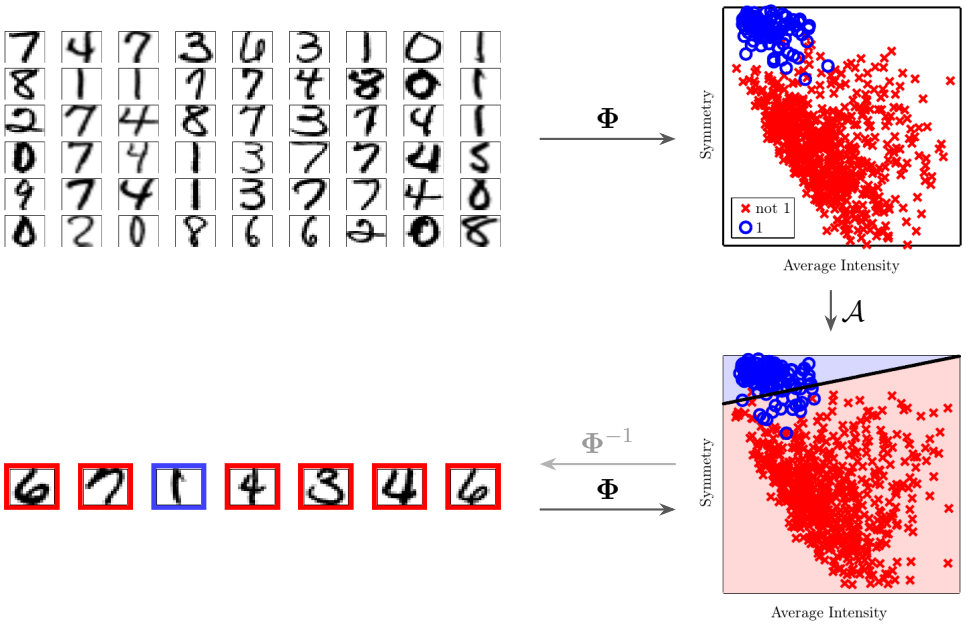
- feature transform $\boldsymbol{\Phi}$;
- linear model $\mathcal{A}$, **not just binary classification**.

algorithm 12.1 never used that $\mathcal{A}$ is a classifier: step 2 hands $\{(\boldsymbol{z}_n, y_n)\}$ to *any* linear model of the previous chapters, and step 3 returns that model's own output applied to $\boldsymbol{\Phi}(\boldsymbol{x})$ — $\tilde{\boldsymbol{w}}^\top \boldsymbol{\Phi}(\boldsymbol{x})$ for linear regression, $\theta(\tilde{\boldsymbol{w}}^\top \boldsymbol{\Phi}(\boldsymbol{x}))$ for logistic regression. The two knobs are independent.

Pandora's box :-): can now freely do quadratic PLA, quadratic regression, cubic regression, . . . , polynomial regression.

§12.2.iv. Feature Transform Φ

The same diagram, on the digit data of Definition 3.4.2: each image is reduced to two numbers, its average intensity and its symmetry, and the task is '1' ($\circ$) against 'not 1' ($\times$).



Top left are raw 16×16 images, each a point of $\mathbb{R}^{256}$; $\mathbf{\Phi}$ computes the two features, after which the '1's (thin, so dark on few pixels, and symmetric) crowd into the upper left. A linear classifier in that plane draws one line, and a new image is classified by computing its features and checking the side: the bottom row shows the result, a blue frame for '1' and red for 'not 1'. As before, $\mathbf{\Phi}^{-1}$ is grey — two numbers cannot be turned back into an image, and need not be.

Not new, not just polynomial:
raw (pixels) $\xrightarrow{\text{domain knowledge}}$ concrete (intensity, symmetry).

This is the conversion of Definition 3.4.2 into Definition 3.4.1 features, seen in a new light: hand-made features were feature transforms all along, and a polynomial transform is just one that needs no domain knowledge.

Question 12.2.2. The force, too good to be true? :-)

Exercise 12.2.3 (Fun Time)**.** Consider the quadratic transform $\mathbf{\Phi}_2(\boldsymbol{x})$ for $\boldsymbol{x} \in \mathbb{R}^d$ instead of in $\mathbb{R}^2$. The transform should include all different quadratic, linear, and constant terms formed by $(x_1, x_2, \ldots, x_d)$. What is the number of dimensions of $\boldsymbol{z} = \mathbf{\Phi}_2(\boldsymbol{x})$?

(1) d;

(2) $\frac{d^2}{2} + \frac{3d}{2} + 1$;

(3) $d^2 + d + 1$;

(4) 2^d.

Reference Answer. (2). Number of different quadratic terms is $\binom{d}{2} + d$; number of different linear terms is d; number of different constant term is 1. The quadratic terms are the cross terms $x_i x_j$, $i < j$, and the squares x_i^2; in total

$$\frac{d(d-1)}{2} + d + d + 1 = \frac{d^2}{2} + \frac{3d}{2} + 1.$$

For $d = 2$ this is 6, the length of $\mathbf{\Phi}_2$ above. ■

§12.3. Price of Nonlinear Transform

slides p.13–17

The Pandora's box of Section 12.2 opens onto ever richer transforms; this section counts what each one costs.

§12.3.i. Computation/Storage Price

The Q-th order polynomial transform keeps every monomial of degree at most Q:

$$\boldsymbol{\Phi}_Q(\boldsymbol{x}) = \big(1,\ x_1, x_2, \dots, x_d,\ x_1^2, x_1x_2, \dots, x_d^2,\ \dots,\ x_1^Q, x_1^{Q-1}x_2, \dots, x_d^Q\big).$$

Its length is

$$\underbrace{1}_{\tilde{w}_0} + \underbrace{\tilde{d}}_{\text{others}} = \#\text{ ways of } \le Q\text{-combination from } d \text{ kinds with repetitions}$$
$$= \binom{Q+d}{Q} = \binom{Q+d}{d} = O(Q^d).$$

Lemma 12.3.1 (Number of monomials)
The number of monomials $x_1^{a_1}\cdots x_d^{a_d}$ with $a_1+\cdots+a_d \le Q$ is $\binom{Q+d}{d}$, and for fixed d this is $O(Q^d)$.

Proof. Add a slack exponent $a_0 = Q-(a_1+\cdots+a_d) \ge 0$. The monomials are then in bijection with the solutions of $a_0+a_1+\cdots+a_d = Q$ in non-negative integers, i.e. with the arrangements of Q stars and d bars in a row: $\binom{Q+d}{d}$ of them. Finally $\binom{Q+d}{d} = \prod_{i=1}^{d}\frac{Q+i}{i} \le (Q+1)^d$. ■

That count is the effort needed for computing/storing $\boldsymbol{z} = \boldsymbol{\Phi}_Q(\boldsymbol{x})$ and $\tilde{\boldsymbol{w}}$: every example is stored as $1+\tilde{d}$ numbers, and every step of $\mathcal{A}$ works with vectors of that length.

Q large $\Longrightarrow$ difficult to compute/store.

§12.3.ii. Model Complexity Price

The same $\boldsymbol{\Phi}_Q(\boldsymbol{x})$, the same count $1+\tilde{d} = O(Q^d)$, now read as degrees of freedom:

- number of free parameters $\tilde{w}_i = \tilde{d}+1 \approx d_{\text{VC}}(\mathcal{H}_{\boldsymbol{\Phi}_Q})$;
- $d_{\text{VC}}(\mathcal{H}_{\boldsymbol{\Phi}_Q}) \le \tilde{d}+1$, why?

$$\begin{gathered}\text{any } \tilde{d}+2 \text{ inputs not shattered in } \mathcal{Z}\\ \Longrightarrow \text{any } \tilde{d}+2 \text{ inputs not shattered in } \mathcal{X}\end{gathered}$$

Proposition 12.3.2 (VC dimension after a transform)
For any feature transform $\boldsymbol{\Phi}$ into $\mathcal{Z}$ of dimension $\tilde{d}$, $d_{\text{VC}}(\mathcal{H}_{\boldsymbol{\Phi}}) \le \tilde{d}+1$.

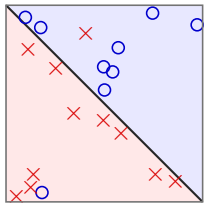
Proof. Suppose $\mathcal{H}_{\boldsymbol{\Phi}}$ shatters $\boldsymbol{x}_1, \dots, \boldsymbol{x}_{\tilde{d}+2}$ (Definition 5.3.10). Each dichotomy on them is produced by some $h = \tilde{h}\circ\boldsymbol{\Phi}$, and $\tilde{h}$ produces the same dichotomy on $\boldsymbol{z}_n = \boldsymbol{\Phi}(\boldsymbol{x}_n)$. So $\tilde{d}$-D perceptrons would shatter $\tilde{d}+2$ points of $\mathcal{Z}$ (necessarily distinct, as equal $\boldsymbol{z}_n$ could not get different labels), contradicting Theorem 7.2.7. ■

The slide's '≈' is because equality may fail: $\mathcal{H}_{\boldsymbol{\Phi}}$ can only use points of the form $\boldsymbol{\Phi}(\boldsymbol{x})$, a curved surface inside $\mathcal{Z}$, and that surface need not offer $\tilde{d}+1$ points that perceptrons shatter. Still, the bound grows like Q^d.

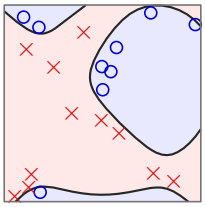
Q **large** $\Longrightarrow$ **large** d_{VC}**.**

§12.3.iii. Generalization Issue

The same twenty inputs as before, now with labels that no circle explains:



$\boldsymbol{\Phi}_1$ (original $\boldsymbol{x}$)



$\boldsymbol{\Phi}_4$

Which one do you prefer? :-)

- $\boldsymbol{\Phi}_1$ 'visually' preferred;
- $\boldsymbol{\Phi}_4$: $E_{\text{in}}(g)=0$ but overkill.

The line misses two examples, the circle at the bottom left and the cross just left of centre at the top, which look like noise. The quartic boundary ($1+\tilde{d}=\binom{6}{2}=15$ weights) bends three separate ∘ regions into the square to catch every one of them. The two questions of Section 5.1 decide between them:

(1) can we make sure that $E_{\text{out}}(g)$ is close enough to $E_{\text{in}}(g)$?

(2) can we make $E_{\text{in}}(g)$ small enough?

The table answers both questions for a higher or a lower $\tilde{d}$, which Q controls; its columns are the questions (1) and (2):

$\tilde{d}$ (Q)	1: $E_{\text{out}}(g)\approx E_{\text{in}}(g)$?	2: small $E_{\text{in}}(g)$?
higher	:-(	**:-D**
lower	**:-D**	:-(

- **Higher** $\tilde{d}$: $\mathcal{H}_{\boldsymbol{\Phi}}$ holds more curved boundaries, so one fitting $\mathcal{D}$ can be found, as $\boldsymbol{\Phi}_4$ does with $E_{\text{in}}=0$: question (2) is answered. But d_{VC} may now be as large as $\tilde{d}+1$ (Proposition 12.3.2), the penalty Ω grows, and nothing guarantees $E_{\text{out}}\approx E_{\text{in}}$: question (1) fails.
- **Lower** $\tilde{d}$: the reverse. $d_{\text{VC}}\leq\tilde{d}+1$ is small, so Ω is small and E_{in} is a trustworthy estimate of E_{out}; but the few boundaries available may not separate $\mathcal{D}$, as $\boldsymbol{\Phi}_1$ misses two examples.

No single Q gets **:-D** in both columns. This is the trade-off of Definition 7.4.1 once more: in $E_{\text{out}}\leq E_{\text{in}}+\Omega$, raising Q shrinks the first term and enlarges the second.

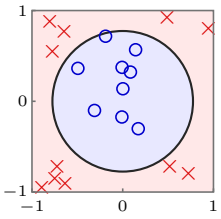
Question 12.3.3. How to pick Q? **Visually**, maybe?

§12.3.iv. Danger of Visual Choices

First of all, can you really 'visualize' when $\mathcal{X} = \mathbb{R}^{10}$? (Well, I can't :-))

Visualize $\mathcal{X} = \mathbb{R}^2$:

- full $\mathbf{\Phi}_2$: $\boldsymbol{z} = (1, x_1, x_2, x_1^2, x_1x_2, x_2^2)$, $d_{\text{VC}} = 6$;
- or $\boldsymbol{z} = (1, x_1^2, x_2^2)$, $d_{\text{VC}} = 3$, **after visualizing**?
- or better $\boldsymbol{z} = (1, x_1^2 + x_2^2)$, $d_{\text{VC}} = 2$?
- or even better $\boldsymbol{z} = \big(\text{sign}(0.6 - x_1^2 - x_2^2)\big)$?



— careful about **your brain's 'model complexity'**.

Every line of the list is chosen by looking at the picture. Each one still separates the data, so $E_{\text{in}} = 0$ every time, while $d_{\text{VC}} = \tilde{d}+1$ keeps going down. Read off the VC bound $E_{\text{out}} \leq E_{\text{in}} + \Omega$, each step looks like an improvement. It is not.

What was really learned. Look at the whole process as one algorithm:

$$\mathcal{D} \longrightarrow \text{you look at } \mathcal{D} \text{ and pick } \mathbf{\Phi} \longrightarrow \mathcal{A} \text{ picks } g \in \mathcal{H}_{\mathbf{\Phi}}.$$

If the data had looked different — an ellipse, or a circle away from the origin — you would have picked a different $\mathbf{\Phi}$. So the hypotheses this process can output are not just those of $\mathcal{H}_{(1,x_1^2+x_2^2)}$, but those of every $\mathcal{H}_{\mathbf{\Phi}}$ you might have picked: at least all quadratic curves, $d_{\text{VC}} \geq 6$. Writing $d_{\text{VC}} = 2$ counts only the last step and leaves out the search done in your head.

Why that matters. The VC bound needs $\mathcal{H}$ to be fixed *before* the data are seen. For a fixed $\mathcal{H}$, BAD data (Definition 4.4.3) are rare, and that is all Theorem 6.4.2 promises. A set chosen *after* looking at $\mathcal{D}$ is chosen to suit this very $\mathcal{D}$, and the bound says nothing about such a choice.

The last line. $z = \text{sign}(0.6 - x_1^2 - x_2^2)$ is already the finished classifier; $\mathcal{A}$ has nothing left to learn. All the learning was done by you, and its complexity — your brain's 'model complexity' — is never charged.

For VC-safety, $\mathbf{\Phi}$ shall be decided without 'peeking' data.

Exercise 12.3.4 (Fun Time)**.** Consider the Q-th order polynomial transform $\mathbf{\Phi}_Q(\boldsymbol{x})$ for $\boldsymbol{x} \in \mathbb{R}^2$. Recall that $\tilde{d} = \binom{Q+2}{2} - 1$. When $Q = 50$, what is the value of $\tilde{d}$? (1) 1126; (2) 1325; (3) 2651; (4) 6211.

Reference Answer. (2). It's just a simple calculation, but shows you how $\tilde{d}$ becomes hundreds of times of $d = 2$ after the transform:

$$\binom{52}{2} - 1 = \frac{52 \cdot 51}{2} - 1 = 1326 - 1 = 1325.$$

■

§12.4. Structured Hypothesis Sets

slides p.18–21

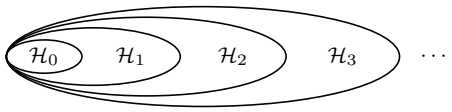
Section 12.3 left Q to be chosen, and forbade choosing it by looking at the data. The polynomial transforms themselves suggest an order in which to try.

§12.4.i. Polynomial Transform Revisited

Each $\boldsymbol{\Phi}_Q$ extends the previous one by the monomials of degree exactly Q:

$$\boldsymbol{\Phi}_0(\boldsymbol{x}) = (1), \qquad \begin{aligned} \boldsymbol{\Phi}_1(\boldsymbol{x}) &= \big(\boldsymbol{\Phi}_0(\boldsymbol{x}),\ x_1, x_2, \dots, x_d\big), \\ \boldsymbol{\Phi}_2(\boldsymbol{x}) &= \big(\boldsymbol{\Phi}_1(\boldsymbol{x}),\ x_1^2, x_1x_2, \dots, x_d^2\big), \\ \boldsymbol{\Phi}_3(\boldsymbol{x}) &= \big(\boldsymbol{\Phi}_2(\boldsymbol{x}),\ x_1^3, x_1^2x_2, \dots, x_d^3\big), \\ &\vdots \\ \boldsymbol{\Phi}_Q(\boldsymbol{x}) &= \big(\boldsymbol{\Phi}_{Q-1}(\boldsymbol{x}),\ x_1^Q, x_1^{Q-1}x_2, \dots, x_d^Q\big). \end{aligned}$$

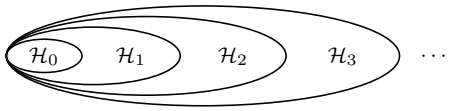
$$\begin{array}{ccccccccccc} \mathcal{H}_{\boldsymbol{\Phi}_0} & \subset & \mathcal{H}_{\boldsymbol{\Phi}_1} & \subset & \mathcal{H}_{\boldsymbol{\Phi}_2} & \subset & \mathcal{H}_{\boldsymbol{\Phi}_3} & \subset & \cdots & \subset & \mathcal{H}_{\boldsymbol{\Phi}_Q} \\ \| & & \| & & \| & & \| & & & & \| \\ \mathcal{H}_0 & & \mathcal{H}_1 & & \mathcal{H}_2 & & \mathcal{H}_3 & & \cdots & & \mathcal{H}_Q \end{array}$$



The inclusions hold because $\boldsymbol{\Phi}_{Q-1}(\boldsymbol{x})$ is the first block of $\boldsymbol{\Phi}_Q(\boldsymbol{x})$: putting weight 0 on every new monomial turns any $\tilde{\boldsymbol{w}}$ of the smaller set into one of the larger set with the same hypothesis, exactly as lines sat inside $\mathcal{H}_{\boldsymbol{\Phi}_2}$. $\mathcal{H}_0$ holds the two constants and $\mathcal{H}_1$ the ordinary perceptrons.

Structure: nested $\mathcal{H}_i$.

§12.4.ii. Structured Hypothesis Sets



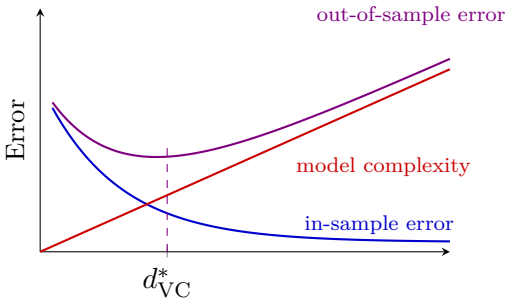
Let $g_i = \arg\min_{h\in\mathcal{H}_i} E_{\text{in}}(h)$:

$$\begin{array}{ccccccccc} \mathcal{H}_0 & \subset & \mathcal{H}_1 & \subset & \mathcal{H}_2 & \subset & \mathcal{H}_3 & \subset & \cdots \\ d_{\text{VC}}(\mathcal{H}_0) & \le & d_{\text{VC}}(\mathcal{H}_1) & \le & d_{\text{VC}}(\mathcal{H}_2) & \le & d_{\text{VC}}(\mathcal{H}_3) & \le & \cdots \\ E_{\text{in}}(g_0) & \ge & E_{\text{in}}(g_1) & \ge & E_{\text{in}}(g_2) & \ge & E_{\text{in}}(g_3) & \ge & \cdots \end{array}$$

Proposition 12.4.1 (Monotonicity along a nested chain)
If $\mathcal{H} \subset \mathcal{H}'$, then $d_{\text{VC}}(\mathcal{H}) \le d_{\text{VC}}(\mathcal{H}')$ and $\min_{h\in\mathcal{H}'} E_{\text{in}}(h) \le \min_{h\in\mathcal{H}} E_{\text{in}}(h)$.

Proof. Any dichotomy produced by $\mathcal{H}$ is produced by $\mathcal{H}'$, so whatever $\mathcal{H}$ shatters, $\mathcal{H}'$ shatters too (Definition 5.3.10), and the largest shattered size (Definition 7.1.2) can only grow. The minimum over a larger set is no larger. ■

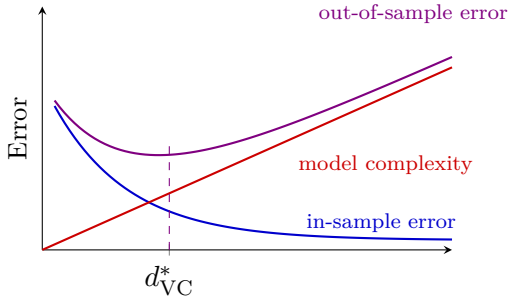
Put on the picture of Section 7.4, moving along the chain means moving right on the d_{VC} axis: E_{in} keeps dropping while the model complexity keeps rising.



Use $\mathcal{H}_{1126}$ won't be good! :-(

$\mathcal{H}_{1126}$ (after the course number) sits far to the right of d^*_{VC}, where E_{in} is tiny but the bound on E_{out} is dominated by the model-complexity term.

§12.4.iii. Linear Model First



- tempting sin: use $\mathcal{H}_{1126}$, low $E_{\text{in}}(g_{1126})$ to fool your boss — **really? :-(a dangerous path of no return**;
- safe route: $\mathcal{H}_1$ first
  - if $E_{\text{in}}(g_1)$ good enough, **live happily thereafter :-)**;
  - otherwise, move right of the curve **with nothing lost except 'wasted' computation**.

The asymmetry is what makes the safe route safe. A small $E_{\text{in}}(g_{1126})$ says nothing about E_{out}, and there is no way back from it: having seen how $\mathcal{H}_{1126}$ did, falling back to a simpler model is itself a choice made by peeking. Starting at $\mathcal{H}_1$ instead, a good $E_{\text{in}}(g_1)$ comes with the strong guarantee of a small d_{VC}; a bad one only means moving one step right, and by Proposition 12.4.1 the larger sets still contain everything $\mathcal{H}_1$ could do.

Linear model first: simple, efficient, safe, and workable!

Exercise 12.4.2 (Fun Time)**.** Consider two hypothesis sets, $\mathcal{H}_1$ and $\mathcal{H}_{1126}$, where $\mathcal{H}_1 \subset \mathcal{H}_{1126}$. Which of the following relationship between $d_{\text{VC}}(\mathcal{H}_1)$ and $d_{\text{VC}}(\mathcal{H}_{1126})$ is not possible?

(1) $d_{\text{VC}}(\mathcal{H}_1) = d_{\text{VC}}(\mathcal{H}_{1126})$;

(2) $d_{\text{VC}}(\mathcal{H}_1) \neq d_{\text{VC}}(\mathcal{H}_{1126})$;

(3) $d_{\text{VC}}(\mathcal{H}_1) < d_{\text{VC}}(\mathcal{H}_{1126})$;

(4) $d_{\text{VC}}(\mathcal{H}_1) > d_{\text{VC}}(\mathcal{H}_{1126})$.

Reference Answer. (4). Every input combination that $\mathcal{H}_1$ shatters can be shattered by $\mathcal{H}_{1126}$, so d_{VC} cannot decrease — the first half of Proposition 12.4.1. ■

Summary

Quadratic Hypotheses linear hypotheses on quadratic-transformed data.

Nonlinear Transform happy linear modeling after $\mathcal{Z} = \mathbf{\Phi}(\mathcal{X})$.

Price of Nonlinear Transform computation/storage/[model complexity].

Structured Hypothesis Sets linear/simpler model first.

A feature transform turns every linear model into a nonlinear one, at a price in $\tilde{d}$ that the VC bound collects through d_{VC}. The $\mathbf{\Phi}_4$ picture already showed what happens when the price is ignored: zero E_{in}, and a boundary nobody believes. Next: the dark side of the force :-).

13. Hazard of Overfitting

Lecture 13

21 Oct.

> **As previously seen .** A nonlinear feature transform $\mathbf{\Phi}$ turns every linear model into a nonlinear one, at a price in d_{VC}. The $\mathbf{\Phi}_4$ boundary of Section 12.3 paid that price in full: zero E_{in}, and a boundary nobody believes. The advice was to start from the linear model, but the danger itself was only pointed at.

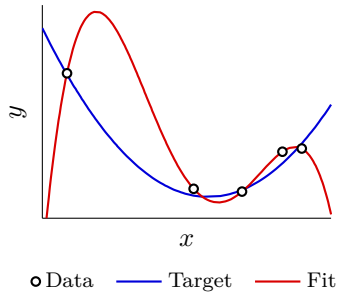
This chapter opens topic 4, *How Can Machines Learn Better?*, by naming that danger and taking it apart. Overfitting turns out to be a *move* to a larger model that lowers E_{in} but raises E_{out}. It is driven by small data, noise and excessive model power, and also by a complicated target even when there is no noise at all. That last effect is a second kind of noise, *deterministic* noise: the part of f that $\mathcal{H}$ cannot capture. The chapter ends with a first set of remedies that work on the data themselves.

§13.1. What is Overfitting?

slides p.2–5

§13.1.i. Bad Generalization

The smallest example of things going wrong:

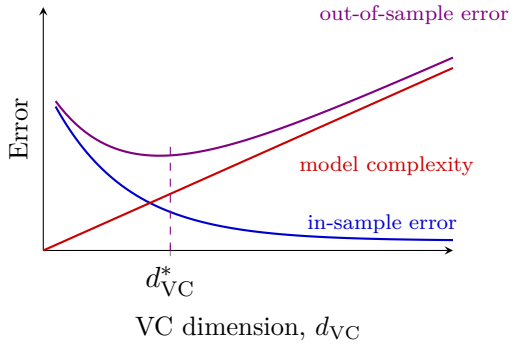


- The task is regression for $x \in \mathbb{R}$, with only $N = 5$ examples.
- The target $f(x)$ is a 2nd order polynomial (the blue curve).
- Each label is $y_n = f(x_n)+$ very small noise, so the five points sit almost exactly on the parabola.
- We learn by linear regression in $\mathcal{Z}$-space, with $\mathbf{\Phi}$ = 4th order polynomial, $\mathbf{\Phi}_4(x) = (1, x, x^2, x^3, x^4)$.
- That gives 5 weights for 5 examples. Z is a square Vandermonde matrix, invertible because the x_n are distinct, so there is a unique solution passing all examples (the red curve), and $E_{\text{in}}(g) = 0$.
- To pass exactly through points that the noise has nudged off the parabola, the quartic must swing far from f in the wide gap between the leftmost point and the other four, where there is no data. $E_{\text{out}}(g)$ averages the error over all x, so it is **huge**.

Bad generalization: low E_{in}, high E_{out}.

§13.1.ii. Bad Generalization and Overfitting

The VC picture of Section 7.4 says the same thing in general. Moving right along the d_{VC} axis, E_{in} falls while the model-complexity term rises; E_{out}, bounded by their sum, is smallest at some d^*_{VC}.



Three places on this curve get three names:

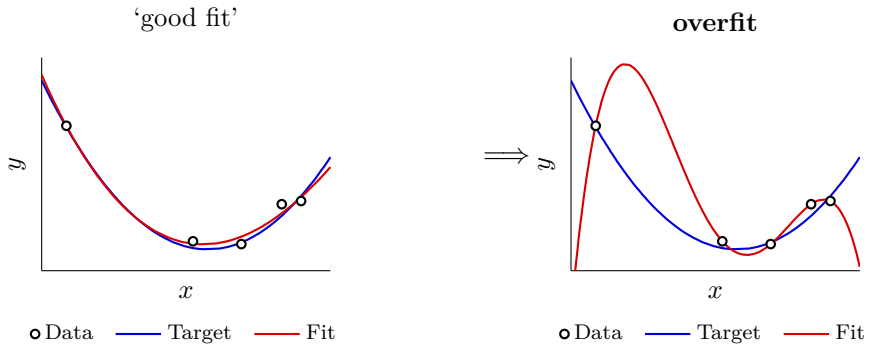
- Taking $d_{\text{VC}} = 1126$ for learning lands far to the right, where $(E_{\text{out}} - E_{\text{in}})$ is large. This is **bad generalization**, a property of a single hypothesis set.
- Switching from $d_{\text{VC}} = d^*_{\text{VC}}$ to $d_{\text{VC}} = 1126$ compares two sets: $E_{\text{in}} \downarrow$ but $E_{\text{out}} \uparrow$, because the gap grows faster than E_{in} shrinks. This move is **overfitting**.
- Switching from $d_{\text{VC}} = d^*_{\text{VC}}$ to $d_{\text{VC}} = 1$ goes the other way and makes both worse, $E_{\text{in}} \uparrow$ and $E_{\text{out}} \uparrow$. This is **underfitting**.

The quartic of the previous subsection is both: it generalizes badly, and it is an overfit compared with the quadratic.

Bad generalization: low E_{in}, high E_{out};
overfitting: lower E_{in}, higher E_{out}.

§13.1.iii. Cause of Overfitting: A Driving Analogy

Fit the same five examples twice. On the left is a 'good fit', a curve that lies almost on top of f; how to obtain it from the same 4th-order model is the subject of the next chapter. On the right is the 4th-order interpolant from before, the overfit.



What turns the left picture into the right one is easiest to see with an analogy to driving.

learning	driving
overfit	commit a car accident
use excessive d_{VC}	'drive too fast'
noise	bumpy road
limited data size N	limited observations about road condition

Speed alone does not cause an accident: the accident comes from speed on a bumpy road that the driver cannot see well. In the pictures, the 4th-order model is the speed. The noise and the mere five examples are the road. The good fit drives the same road slowly and stays on f. Excessive d_{VC} was already the subject of the VC curve. The other two causes are new.

Question 13.1.1. How does **noise** & **data size** affect overfitting?

Exercise 13.1.2 (Fun Time)**.** Based on our discussion, for data of fixed size, which of the following situation is relatively of the lowest risk of overfitting?

(1) small noise, fitting from small d_{VC} to median d_{VC};

(2) small noise, fitting from small d_{VC} to large d_{VC};

(3) large noise, fitting from small d_{VC} to median d_{VC};

(4) large noise, fitting from small d_{VC} to large d_{VC}.

Reference Answer. (1). Two causes of overfitting are noise and excessive d_{VC}. So if both are relatively 'under control', the risk of overfitting is smaller. ■

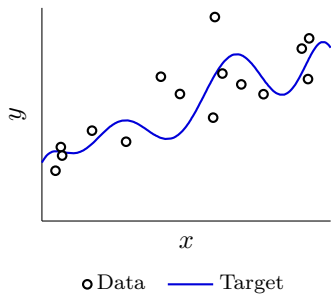
§13.2. The Role of Noise and Data Size

slides p.6–11

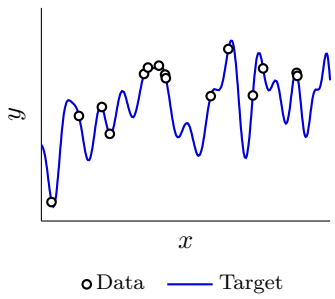
§13.2.i. Case Study

Take two data sets of $N = 15$ examples each. On the left the target is a 10th-order polynomial and the labels carry noise. On the right the target is a 50th-order polynomial, but the labels are noiseless: every point lies exactly on f.

10-th order target function + **noise**



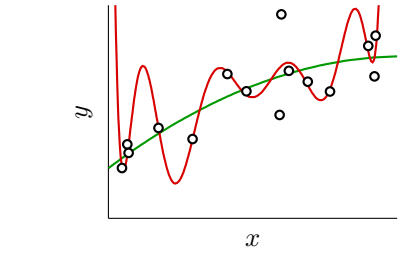
50-th order target function **noiselessly**



On each, compare two hypothesis sets. $\mathcal{H}_Q$ is the polynomials of order Q, i.e. linear regression after $\mathbf{\Phi}_Q$, and g_Q is its least-squares best on the 15 examples.

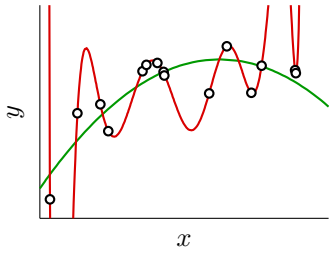
Question 13.2.1. Overfitting from best $g_2 \in \mathcal{H}_2$ to best $g_{10} \in \mathcal{H}_{10}$?

10-th order target function + **noise**



Data 2nd Order Fit 10th Order Fit

50-th order target function **noiselessly**



Data 2nd Order Fit 10th Order Fit

In both plots the red 10th-order fit passes closer to the points than the green parabola, so its E_{in} is lower. Between and beyond the points, most of all near ± 1, it swings wildly: 11 weights fitted to 15 points leave little to keep it in place where there is no data. The table puts numbers on this.

10-th order + noise

	$g_2 \in \mathcal{H}_2$	$g_{10} \in \mathcal{H}_{10}$
E_{in}	0.050	0.034
E_{out}	0.127	**9.00**

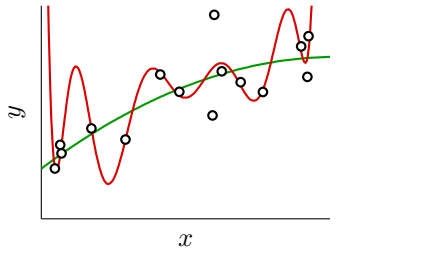
50-th order, noiseless

	$g_2 \in \mathcal{H}_2$	$g_{10} \in \mathcal{H}_{10}$
E_{in}	0.029	0.00001
E_{out}	0.120	**7680**

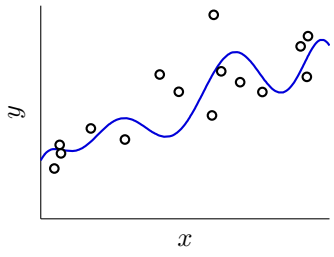
E_{in} improves a little in both cases, while E_{out} grows by a factor of about 70 on the left and 6×10^4 on the right. Both moves are overfitting in the sense of Section 13.1.

Overfitting from g_2 to g_{10}? both yes!

§13.2.ii. Irony of Two Learners



Data 2nd Order Fit 10th Order Fit



Data Target

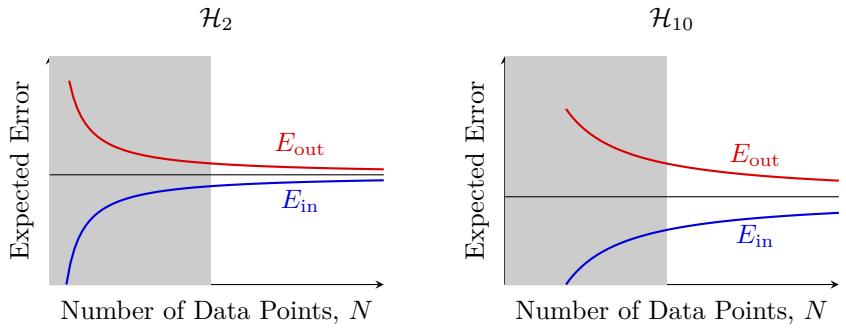
Name the two choices after two learners.

- Learner *Overfit* (O) picks $g_{10} \in \mathcal{H}_{10}$.
- Learner *Restrict* (R) picks $g_2 \in \mathcal{H}_2$.
- Suppose both know that the target is 10th order. Then O seems to have the right model, since $f \in \mathcal{H}_{10}$, while R 'gives up' the ability to fit f at all.

Yet the left plot shows the green parabola following the overall trend of the target and the red curve oscillating far from it. $\mathcal{H}_{10}$ does contain a hypothesis with $E_{\text{out}} = \sigma^2$, the best possible, but O has to find it from 15 noisy examples, and 11 weights estimated from 15 points mostly follow the noise. R estimates only 3 weights from the same points, which is reliable.

But R wins in E_{out} a lot!
Philosophy: concession for advantage? :-)

§13.2.iii. Learning Curves Revisited

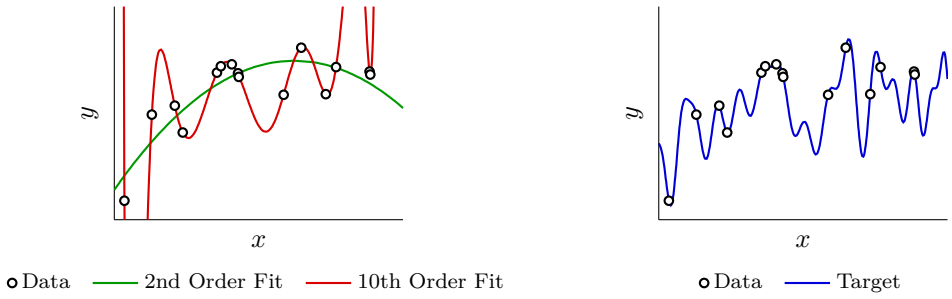


These are the learning curves of Section 9.3.v, drawn for $\mathcal{H}_2$ and $\mathcal{H}_{10}$: expected E_{in} (blue) and E_{out} (red) against N. Both approach the black line, the error that remains with unlimited data. That line is lower for $\mathcal{H}_{10}$, which contains the target, than for $\mathcal{H}_2$, which does not.

- $\mathcal{H}_{10}$ has the lower E_{out} when $N \to \infty$, but a much larger generalization error (the gap between red and blue) for small N, because it has more weights to estimate.
- In the gray area, O overfits: compared with $\mathcal{H}_2$, its E_{in} is lower but its E_{out} is higher. The case study, with $N = 15$, sits in this region.

R always wins in E_{out} if N small!

§13.2.iv. The 'No Noise' Case



Now take the right-hand data set of the case study.

- Learner *Overfit* (O) again picks $g_{10} \in \mathcal{H}_{10}$.
- Learner *Restrict* (R) again picks $g_2 \in \mathcal{H}_2$.
- This time both know that there is no noise, so O can argue that every point is exact and there is nothing to overfit. Still, R wins: the red fit chases the points and swings away from the target between them.

Question 13.2.2. Is there really **no noise**?

'Target complexity' acts like noise.

Neither $\mathcal{H}_2$ nor $\mathcal{H}_{10}$ contains the 50th-order target, so no hypothesis can follow the fast wiggles of f. Seen from inside $\mathcal{H}_{10}$, the labels y_n deviate from the best hypothesis in a way nothing can explain. That is exactly what noise did on the left of the case study. $\mathcal{H}_{10}$, with more weights, bends further to fit those deviations, and pays for it the same way.

Exercise 13.2.3 (Fun Time)**.** When having limited data, in which of the following case would learner R perform better than learner O?

(1) limited data from a 10-th order target function with some noise;

(2) limited data from a 1126-th order target function with no noise;

(3) limited data from a 1126-th order target function with some noise;

(4) all of the above.

Reference Answer. (4). We discussed about (1) and (2), but you shall be able to 'generalize' :-) that R also wins in the more difficult case of (3). ■

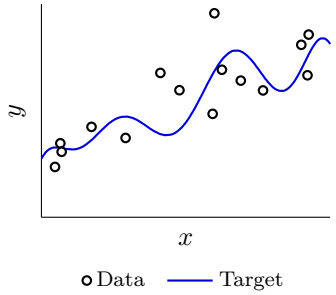
§13.3. Deterministic Noise

slides p.12–17

§13.3.i. A Detailed Experiment

The case study had two data sets. To see *how much* noise and target complexity it takes, the same comparison is repeated over a whole grid of settings.

$$y = f(x) + \epsilon \sim \text{Gaussian}\left(\underbrace{\sum_{q=0}^{Q_f} \alpha_q x^q}_{f(x)}, \sigma^2\right)$$



Each label is $f(x)$ plus Gaussian noise, and the experiment has three knobs:

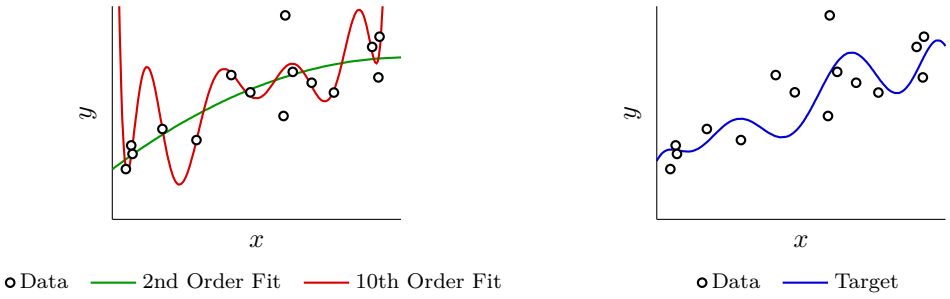
- Gaussian iid noise ϵ with level σ^2;
- a target f drawn from some 'uniform' distribution with complexity level Q_f, the order of the polynomial;
- the data size N.

The plot shows one draw, with $Q_f = 10$ and $N = 15$.

'Uniform' means that the coefficients are drawn at random and rescaled so that every f has the same overall size. A given σ^2 then means the same amount of noise relative to the signal, whatever Q_f is.

Question 13.3.1. Goal: 'overfit level' for different (N, σ^2) and (N, Q_f)?

§13.3.ii. The Overfit Measure



For every setting, draw a data set and fit it twice, exactly as in the case study.

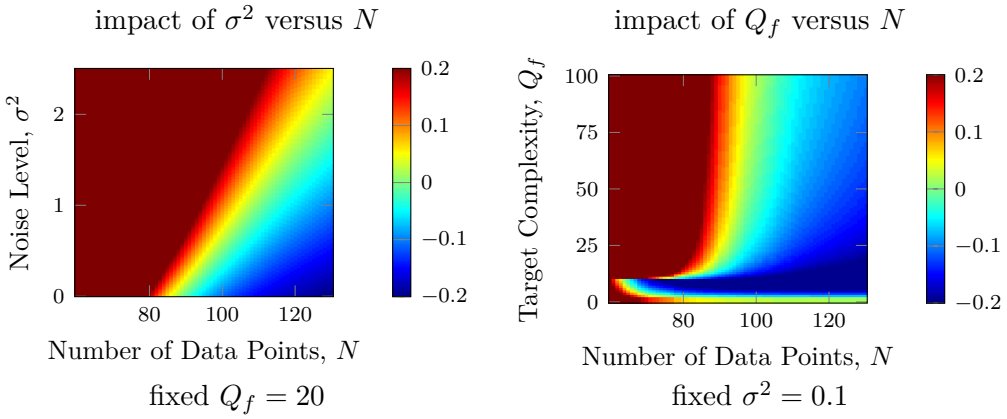
- $g_2 \in \mathcal{H}_2$ is the least-squares best quadratic.
- $g_{10} \in \mathcal{H}_{10}$ is the least-squares best 10th-order polynomial.
- $E_{\text{in}}(g_{10}) \leq E_{\text{in}}(g_2)$ holds for sure, by Proposition 12.4.1: $\mathcal{H}_2 \subset \mathcal{H}_{10}$, and the minimum over a larger set is no larger.

So half of overfitting, $E_{\text{in}} \downarrow$, is automatic, and whether the move overfits depends only on E_{out}.

Overfit measure $E_{\text{out}}(g_{10}) - E_{\text{out}}(g_2)$**.**

Positive means the move to $\mathcal{H}_{10}$ overfitted; negative means it was worth it.

§13.3.iii. The Results

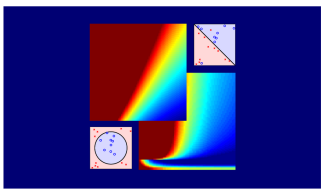


Each pixel is one setting, coloured by the overfit measure clipped to $[-0.2, 0.2]$. Dark red is serious overfitting; dark blue means $\mathcal{H}_{10}$ clearly won.

The left map varies the noise σ^2 against N, with $Q_f = 20$ fixed. Red fills the upper left, where examples are few and noise is high. Its edge is a rising diagonal: the more noise, the more examples are needed before $\mathcal{H}_{10}$ is safe.

The right map varies the target complexity Q_f against N, with $\sigma^2 = 0.1$ fixed and small. Above $Q_f \approx 10$, red still fills the left side and widens as Q_f grows, although the noise never changes. The thin band at the bottom depends on which sets contain f. For $Q_f \leq 2$ both do, so the extra weights of $\mathcal{H}_{10}$ only fit noise and the measure is positive. For $3 \leq Q_f \leq 10$ only $\mathcal{H}_{10}$ does, so it wins (dark blue). Above 10 neither does, and the red returns.

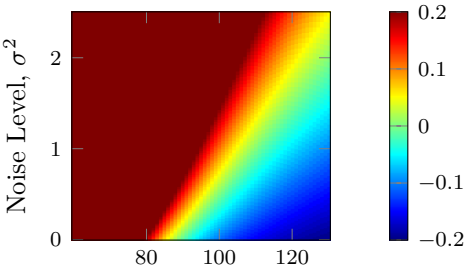
ring a bell? :-)



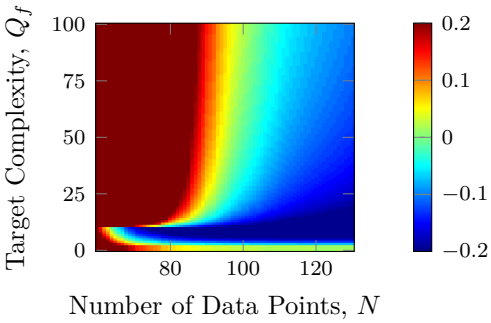
This is the picture on the title slide of every handout, the course's logo. It is made of these two maps together with the linear-separable and circular-separable data sets of Section 12.1.

§13.3.iv. Impact of Noise and Data Size

impact of σ^2 versus N:
stochastic noise



impact of Q_f versus N:
deterministic noise



These are the same two maps. The new titles name the quantity on each vertical axis.

- On the left, the vertical axis σ^2 is the level of the **stochastic noise**: the noise ϵ of Section 8.1, drawn at random for each example. Moving up the map means more stochastic noise.
- On the right, the vertical axis Q_f measures the **deterministic noise**. σ^2 is fixed here, so the only thing that changes is the target complexity. A more complex f leaves more of itself outside $\mathcal{H}_2$ and $\mathcal{H}_{10}$, and that part acts as noise. Moving up the map means more deterministic noise, which the next subsection defines.

Read this way, the two maps say the same thing: going up (more noise of either kind) or left (fewer examples) makes overfitting worse. Collecting everything so far:

four reasons of serious overfitting:

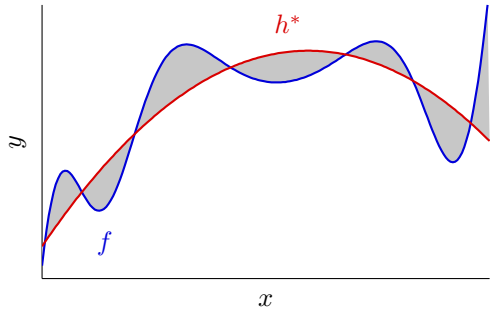
data size N ↓	overfit ↑
stochastic noise ↑	overfit ↑
deterministic noise ↑	overfit ↑
excessive power ↑	overfit ↑

The first three are read off the two maps. The fourth, *excessive power*, means a hypothesis set more flexible (larger d_{VC}) than the data can support. It is the 'use excessive d_{VC}', or 'drive too fast', of the driving analogy. In this experiment it is the switch from $\mathcal{H}_2$ to $\mathcal{H}_{10}$, which is what every pixel measures. The 11 weights of $\mathcal{H}_{10}$ give it the power to follow noise that the 3 weights of $\mathcal{H}_2$ cannot. The more power beyond what is needed, the more overfitting. Real problems usually have little data and some noise, and the target is rarely simple, so all four are common.

Overfitting 'easily' happens.

§13.3.v. Deterministic Noise

Why does target complexity act like noise? In the picture, f is a wiggly target, h^* is the best hypothesis in a simpler $\mathcal{H}$ (closest to f in squared error), and the gray area between them is what $\mathcal{H}$ can never reach.



- If $f \notin \mathcal{H}$, something of f cannot be captured by $\mathcal{H}$.
- The **deterministic noise** is the difference between the best $h^* \in \mathcal{H}$ and f: the gray area.
- To a learner using $\mathcal{H}$, it acts like 'stochastic noise'. That is not new to CS: a pseudo-random generator outputs a fixed function of its seed, yet the output looks random to anyone who cannot model that function.
- It differs from stochastic noise in two ways.
  - It depends on $\mathcal{H}$: a larger set gets closer to f and leaves less of it.
  - It is fixed for a given $\boldsymbol{x}$: the same x_n always carries the same $f(x_n) - h^*(x_n)$, whereas ϵ is redrawn for each example.

For every hypothesis in $\mathcal{H}$, this gap costs about as much as stochastic noise of the same size would. That is why $\mathcal{H}_2$'s black line in the learning curves sits higher. The philosophy below is the same idea applied to people: a target too complicated for what a kid can grasp only looks like noise to them.

Remark 13.3.2. 'Deterministic noise ↑' in Section 13.3 (the right map and the table) means the *target* getting more complex, with $\mathcal{H}$ fixed. It does not mean the hypothesis set getting more complex. Making $\mathcal{H}$ more complex goes the other way: h^* gets closer to f, so the deterministic noise drops (the first difference above). It also adds power, though, and with little data the extra power costs more than the smaller deterministic noise saves, so a larger $\mathcal{H}$ can still overfit.

Philosophy: when teaching a kid, perhaps better not to use examples from a complicated target function? :-)

The exercise below asks for the deterministic noise point by point. For each x, the **level of deterministic noise** at x is the size of the gap there,

$$|f(x) - h^*(x)|.$$

Exercise 13.3.3 (Fun Time). Consider the target function being $\sin(1126x)$ for $x \in [0, 2\pi]$. When x is uniformly sampled from the range, and we use all possible linear hypotheses $h(x) = \boldsymbol{w}^\top \boldsymbol{x}$ to approximate the target function with respect to the squared error, what is the level of deterministic noise for each x?

(1) $|\sin(1126x)|$;

(2) $|\sin(1126x) - x|$;

(3) $|\sin(1126x) + x|$;

(4) $|\sin(1126x) - 1126x|$.

Reference Answer. (1). You can try a few different $\boldsymbol{w}$ and convince yourself that the best hypothesis h^* is $h^*(x) = 0$. The deterministic noise is the difference between f and h^*. ■

With $h^* \approx 0$, the level at x is $|\sin(1126x) - 0| = |\sin(1126x)|$. A line cannot follow the $1126/\pi \approx 358$ full periods of $\sin(1126x)$ on $[0, 2\pi]$, so the best it can do is stay near their average, 0.

§13.4. Dealing with Overfitting

slides p.18–21

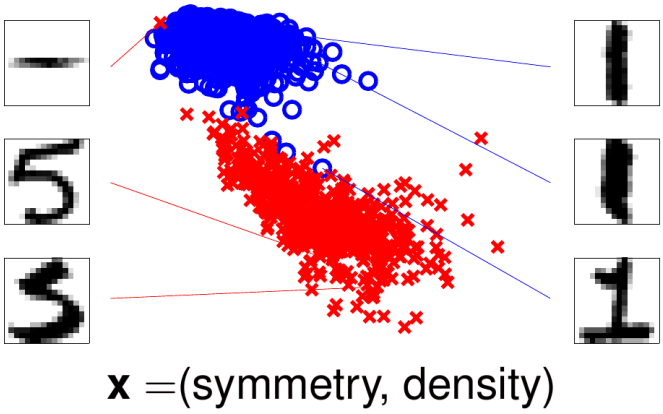
§13.4.i. Driving Analogy Revisited

learning	driving
overfit	commit a car accident
use excessive d_{VC}	'drive too fast'
noise	bumpy road
limited data size N	limited observations about road condition
start from simple model	drive slowly
data cleaning/pruning	use more accurate road information
data hinting	exploit more road information
regularization	put the brakes
validation	monitor the dashboard

The lower half of the table extends the analogy to remedies, each aimed at a cause in the upper half. Starting from a simple model is driving slowly: it keeps d_{VC} small, the 'linear model first' of Section 12.4. Data cleaning/pruning gives more accurate road information, i.e. less noise. Data hinting exploits more road information, i.e. more data. Both are treated in the rest of this section. Regularization puts on the brakes, and validation watches the dashboard; they take the next chapters.

All very practical techniques to combat overfitting.

§13.4.ii. Data Cleaning/Pruning



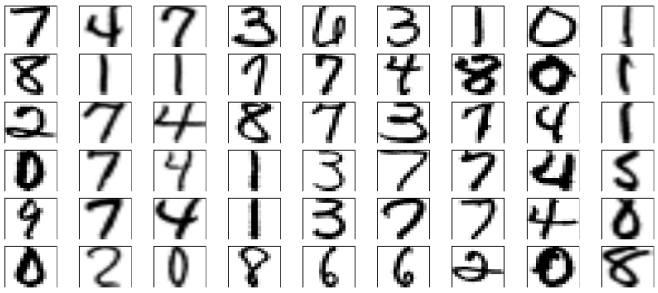
This is the '1' (∘) versus '5' (×) data of Definition 3.4.2, each digit reduced to its symmetry and density. One × sits at the very top, inside the cloud of ∘'s. It is labelled '5', but its image, the top-left thumbnail, is a short horizontal stroke: a symmetric, light image, with exactly the features of a '1'. To the learner it is noise.

- We might 'detect' the outlier 5 at the top
  - because it is too close to other ∘, or too far from other ×;
  - because it is wrong by the current classifier;
  - or by other rules (...).
- Possible action 1 is to correct its label (**data cleaning**).
- Possible action 2 is to remove the example (**data pruning**).

Either action lowers the noise the learner sees. Both depend on the detection, though, and the same rules also flag examples that are correct but unusual. Removing those leaves a less representative data set.

Possibly helps, but effect varies.

§13.4.iii. Data Hinting



The grid shows handwritten digits from the training data.

- Slightly shifted or rotated digits carry the same meaning: a '7' moved by one pixel or tilted a few degrees is still a '7'.
- The possible action is to add **virtual examples** by shifting or rotating the given digits, keeping their labels (**data hinting**).

This turns prior knowledge about f into data, making N larger without collecting anything new.

Possibly helps, but watch out — virtual example not $\overset{\text{iid}}{\sim} P(\boldsymbol{x}, y)$!

The catch is that the Hoeffding and VC guarantees of the earlier chapters assume examples drawn iid from the same P that E_{out} is measured on. Virtual examples are copies of existing ones, so they are not independent of them and not drawn from P. A rotation large enough to turn a '6' into a '9' would even teach the wrong label.

Exercise 13.4.1 (Fun Time)**.** Assume we know that $f(x)$ is symmetric for some 1D regression application. That is, $f(x) = f(-x)$. One possibility of using the knowledge is to consider symmetric hypotheses only. On the other hand, you can also generate virtual examples from the original data $\{(x_n, y_n)\}$ as hints. What virtual examples suit your needs best?

(1) $\{(x_n, -y_n)\}$;

(2) $\{(-x_n, -y_n)\}$;

(3) $\{(-x_n, y_n)\}$;

(4) $\{(2x_n, 2y_n)\}$.

Reference Answer. (3). We want the virtual examples to encode the invariance when $x \to -x$. Since $f(-x_n) = f(x_n)$, the label y_n is just as valid at $-x_n$. ■

Summary

What is Overfitting? lower E_{in} but higher E_{out}.

The Role of Noise and Data Size overfitting 'easily' happens!

Deterministic Noise what $\mathcal{H}$ cannot capture acts like noise.

Dealing with Overfitting data cleaning/pruning/hinting, and more.

Overfitting is driven by small N, stochastic noise, deterministic noise and excessive power, and only the last of these is fully in the learner's hands. The data-side remedies help when they apply, but they need extra knowledge about outliers or invariances. Next: putting the brakes with regularization.

14. Regularization

Lecture 14

28 Oct.

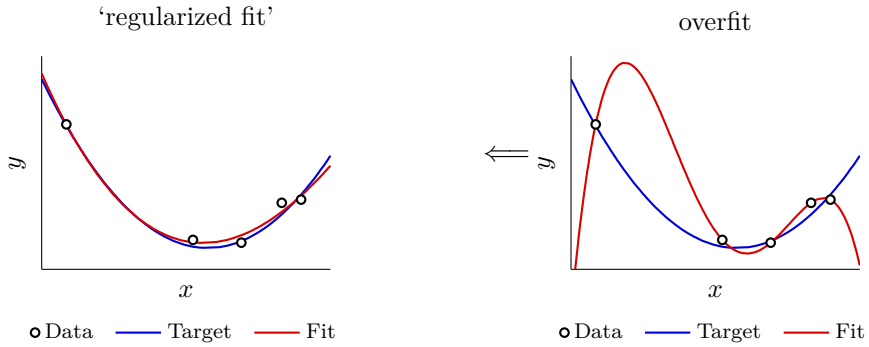
> **As previously seen .** Overfitting happens with excessive power, stochastic and deterministic noise, and limited data. Of the remedies in the driving analogy, 'start from a simple model' and the data-side ones were covered; 'put the brakes', regularization, was left for later.

This chapter builds the brakes. Stepping back from a large hypothesis set to a smaller one is the same as adding a constraint on the weights. Softening that constraint gives a family of sets $\mathcal{H}(C)$, and solving the constrained problem turns into minimizing an *augmented error*, which for linear regression has a closed form. VC theory explains why this works, and the chapter ends with other choices of regularizer and the question of how much to regularize.

§14.1. Regularized Hypothesis Set

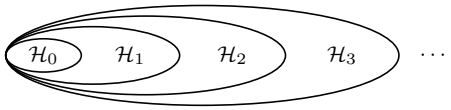
slides p.2–7

§14.1.i. Regularization: The Magic



The right plot is the overfit of Section 13.1: five examples from a 2nd-order target, fitted exactly by a 4th-order polynomial. The left plot is what regularization makes of the same data, a curve that stays close to the target. The arrow points from right to left: regularization starts from the overfit and pulls it back.

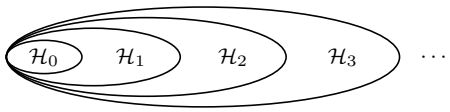
- The idea is to 'step back' from $\mathcal{H}_{10}$ to $\mathcal{H}_2$ in the nested chain of polynomial hypothesis sets, i.e. to move left in the picture below.



- The name has a history: it comes from function approximation for **ill-posed problems**, where many functions fit the given information equally well and an extra condition is needed to pick one.

Question 14.1.1. How to step back?

§14.1.ii. Stepping Back as Constraint



Use the Q-th order polynomial transform for $x \in \mathbb{R}$,

$$\boldsymbol{\Phi}_Q(x) = \left(1, x, x^2, \ldots, x^Q\right),$$

followed by linear regression, and from now on write the weights $\tilde{\boldsymbol{w}}$ of $\mathcal{Z}$-space simply as $\boldsymbol{w}$. A hypothesis in each set is then

$$\begin{aligned} \text{hypothesis } \boldsymbol{w} \text{ in } \mathcal{H}_{10}: &\quad w_0 + w_1 x + w_2 x^2 + w_3 x^3 + \ldots + w_{10} x^{10}, \\ \text{hypothesis } \boldsymbol{w} \text{ in } \mathcal{H}_2: &\quad w_0 + w_1 x + w_2 x^2. \end{aligned}$$

The second line is the first one with the last eight terms gone. That is, $\mathcal{H}_2 = \mathcal{H}_{10}$ AND 'constraint that $w_3 = w_4 = \ldots = w_{10} = 0$'.

Step back = constraint.

§14.1.iii. Regression with Constraint

Written as sets of weight vectors, both live in the same space $\mathbb{R}^{10+1}$:

$$\mathcal{H}_{10} \equiv \left\{\boldsymbol{w} \in \mathbb{R}^{10+1}\right\}, \qquad \mathcal{H}_2 \equiv \left\{\boldsymbol{w} \in \mathbb{R}^{10+1} \text{ while } w_3 = w_4 = \ldots = w_{10} = 0\right\}.$$

Regression on each is the same minimization, with or without the constraint:

$$\begin{aligned} \text{regression with } \mathcal{H}_{10}: &\quad \min_{\boldsymbol{w} \in \mathbb{R}^{10+1}} E_{\text{in}}(\boldsymbol{w}), \\ \text{regression with } \mathcal{H}_2: &\quad \min_{\boldsymbol{w} \in \mathbb{R}^{10+1}} E_{\text{in}}(\boldsymbol{w}) \quad \text{s.t. } w_3 = w_4 = \ldots = w_{10} = 0. \end{aligned}$$

Step back = constrained optimization of E_{in}.

Question 14.1.2. Why don't you just use $\boldsymbol{w} \in \mathbb{R}^{2+1}$? :-)

For $\mathcal{H}_2$ itself that would indeed be simpler. The point of writing it as a constraint is that the constraint can now be changed, giving sets between $\mathcal{H}_2$ and $\mathcal{H}_{10}$ that have no such shortcut.

§14.1.iv. Regression with Looser Constraint

The first change: instead of insisting that the *last* eight weights are zero, ask only that *some* eight are.

$$\mathcal{H}_2 \equiv \left\{\boldsymbol{w} \in \mathbb{R}^{10+1} \text{ while } w_3 = \ldots = w_{10} = 0\right\}, \qquad \mathcal{H}_2' \equiv \left\{\boldsymbol{w} \in \mathbb{R}^{10+1} \text{ while } \geq 8 \text{ of } w_q = 0\right\}.$$

Counting the nonzero weights with the Iverson bracket, regression becomes

$$\begin{aligned} \text{regression with } \mathcal{H}_2: &\quad \min_{\boldsymbol{w} \in \mathbb{R}^{10+1}} E_{\text{in}}(\boldsymbol{w}) \quad \text{s.t. } w_3 = \ldots = w_{10} = 0, \\ \text{regression with } \mathcal{H}_2': &\quad \min_{\boldsymbol{w} \in \mathbb{R}^{10+1}} E_{\text{in}}(\boldsymbol{w}) \quad \text{s.t. } \sum_{q=0}^{10} [\![w_q \neq 0]\!] \leq 3. \end{aligned}$$

- $\mathcal{H}_2'$ is more flexible than $\mathcal{H}_2$: $\mathcal{H}_2 \subset \mathcal{H}_2'$, since any three weights may be the free ones, not just w_0, w_1, w_2.
- $\mathcal{H}_2'$ is less risky than $\mathcal{H}_{10}$: $\mathcal{H}_2' \subset \mathcal{H}_{10}$, since it still uses at most three weights.

Bad news for sparse hypothesis set $\mathcal{H}_2'$: NP-hard to solve :-(

The difficulty is the choice of which three weights to keep. There are $\binom{11}{3}$ choices here, and in general the count grows combinatorially, just like the 0/1 error of the perceptron.

§14.1.v. Regression with Softer Constraint

The second change replaces 'how many weights are nonzero' by 'how large the weights are':

$$\mathcal{H}_2' \equiv \left\{\boldsymbol{w} \in \mathbb{R}^{10+1} \text{ while } \geq 8 \text{ of } w_q = 0\right\}, \qquad \mathcal{H}(C) \equiv \left\{\boldsymbol{w} \in \mathbb{R}^{10+1} \text{ while } \|\boldsymbol{w}\|^2 \leq C\right\},$$

$$\text{regression with } \mathcal{H}_2': \quad \min_{\boldsymbol{w}\in\mathbb{R}^{10+1}} E_{\text{in}}(\boldsymbol{w}) \quad \text{s.t.} \quad \sum_{q=0}^{10} [\![w_q \neq 0]\!] \leq 3,$$

$$\text{regression with } \mathcal{H}(C): \quad \min_{\boldsymbol{w}\in\mathbb{R}^{10+1}} E_{\text{in}}(\boldsymbol{w}) \quad \text{s.t.} \quad \sum_{q=0}^{10} w_q^2 \leq C.$$

- $\mathcal{H}(C)$ overlaps with $\mathcal{H}_2'$ but is not exactly the same: both keep the weights 'small', one by count and one by size.
- Over $C \geq 0$, the sets form a soft and smooth structure:

$$\mathcal{H}(0) \subset \mathcal{H}(1.126) \subset \ldots \subset \mathcal{H}(1126) \subset \ldots \subset \mathcal{H}(\infty) = \mathcal{H}_{10}.$$

  Unlike $\mathcal{H}_0 \subset \mathcal{H}_1 \subset \cdots$, the size can now be tuned continuously by C.

Definition 14.1.3 (Regularized hypothesis)**.** The **regularized hypothesis** $\boldsymbol{w}_{\text{REG}}$ is the optimal solution from the **regularized hypothesis set** $\mathcal{H}(C)$.

Exercise 14.1.4 (Fun Time)**.** For $Q \geq 1$, which of the following hypothesis (weight vector $\boldsymbol{w} \in \mathbb{R}^{Q+1}$) is not in the regularized hypothesis set $\mathcal{H}(1)$?

(1) $\boldsymbol{w}^\top = [0, 0, \ldots, 0]$;
(2) $\boldsymbol{w}^\top = [1, 0, \ldots, 0]$;
(3) $\boldsymbol{w}^\top = [1, 1, \ldots, 1]$;
(4) $\boldsymbol{w}^\top = \left[\sqrt{\frac{1}{Q+1}}, \sqrt{\frac{1}{Q+1}}, \ldots, \sqrt{\frac{1}{Q+1}}\right]$.

Reference Answer. (3). The squared length of $\boldsymbol{w}$ in (3) is $Q+1$, which is not ≤ 1. ■

§14.2. Weight Decay Regularization

slides p.8–13

§14.2.i. Matrix Form of Regularized Regression Problem

The regularized regression problem, with $\boldsymbol{z}_n = \boldsymbol{\Phi}_Q(x_n)$:

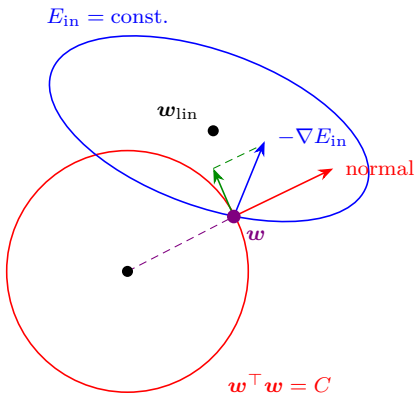
$$\min_{\boldsymbol{w}\in\mathbb{R}^{Q+1}} E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}\underbrace{\sum_{n=1}^{N}\left(\boldsymbol{w}^\top \boldsymbol{z}_n - y_n\right)^2}_{(\mathrm{Z}\boldsymbol{w}-\boldsymbol{y})^\top(\mathrm{Z}\boldsymbol{w}-\boldsymbol{y})} \quad \text{s.t.} \quad \underbrace{\sum_{q=0}^{Q} w_q^2}_{\boldsymbol{w}^\top\boldsymbol{w}} \le C.$$

- The sum $\sum_n \ldots$ equals $(\mathrm{Z}\boldsymbol{w} - \boldsymbol{y})^\top(\mathrm{Z}\boldsymbol{w} - \boldsymbol{y})$, the matrix form of linear regression in Section 9.2, remember? :-)
- $\boldsymbol{w}^\top\boldsymbol{w} \le C$ says the feasible $\boldsymbol{w}$ lie within a radius-$\sqrt{C}$ hypersphere centred at the origin.

Question 14.2.1. How to solve the constrained optimization problem?

§14.2.ii. The Lagrange Multiplier

$$\min_{\boldsymbol{w}\in\mathbb{R}^{Q+1}} E_{\text{in}}(\boldsymbol{w}) = \frac{1}{N}(\mathrm{Z}\boldsymbol{w} - \boldsymbol{y})^\top(\mathrm{Z}\boldsymbol{w} - \boldsymbol{y}) \quad \text{s.t.} \quad \boldsymbol{w}^\top\boldsymbol{w} \le C$$



The picture is the problem in weight space. $\boldsymbol{w}_{\text{lin}}$, the unconstrained minimizer, lies outside the red sphere, so the best feasible $\boldsymbol{w}$ sits on the sphere's boundary. The blue ellipse is the contour of E_{in} through the current $\boldsymbol{w}$.

- The decreasing direction of E_{in} is $-\nabla E_{\text{in}}(\boldsymbol{w})$, as in gradient descent (Section 10.4), remember? :-)
- The normal vector of the boundary $\boldsymbol{w}^\top\boldsymbol{w} = C$ at $\boldsymbol{w}$ is $\boldsymbol{w}$ itself (red arrow), pointing straight out of the sphere.
- If $-\nabla E_{\text{in}}(\boldsymbol{w})$ and $\boldsymbol{w}$ are not parallel, then $-\nabla E_{\text{in}}(\boldsymbol{w})$ has a component along the boundary (green arrow). Moving that way decreases $E_{\text{in}}(\boldsymbol{w})$ without violating the constraint, so $\boldsymbol{w}$ is not yet optimal.
- Hence at the optimal solution $\boldsymbol{w}_{\text{REG}}$ no such component is left: $-\nabla E_{\text{in}}(\boldsymbol{w}_{\text{REG}}) \propto \boldsymbol{w}_{\text{REG}}$.

Want: find Lagrange multiplier $\lambda > 0$ and $\boldsymbol{w}_{\text{REG}}$ such that $\nabla E_{\text{in}}(\boldsymbol{w}_{\text{REG}}) + \frac{2\lambda}{N}\boldsymbol{w}_{\text{REG}} = \boldsymbol{0}$.

The proportionality constant is written as $\frac{2\lambda}{N}$ only to make the next formulas clean; any positive constant would do.

Remark 14.2.2. Why must λ be positive? The picture assumes $\boldsymbol{w}_{\text{lin}}$ lies outside the sphere, so the constraint is active and $\boldsymbol{w}_{\text{REG}}$ is on the boundary. There, E_{in} decreases *outward*, towards $\boldsymbol{w}_{\text{lin}}$, which means $-\nabla E_{\text{in}}(\boldsymbol{w}_{\text{REG}})$ points the same way as $\boldsymbol{w}_{\text{REG}}$ and the constant is positive. If it pointed inward ($\lambda < 0$), moving inward would lower E_{in} and stay feasible, so $\boldsymbol{w}_{\text{REG}}$ would not be optimal. The remaining case is $\lambda = 0$: then $\nabla E_{\text{in}}(\boldsymbol{w}_{\text{REG}}) = \boldsymbol{0}$, i.e. $\boldsymbol{w}_{\text{REG}} = \boldsymbol{w}_{\text{lin}}$, which happens when $\boldsymbol{w}_{\text{lin}}$ is already inside the sphere and the constraint does nothing. That is why the next subsection allows '$\lambda > 0$ or $\lambda = 0$'.

§14.2.iii. Augmented Error

- If an oracle tells you $\lambda > 0$, then solving

$$\nabla E_{\text{in}}(\boldsymbol{w}_{\text{REG}}) + \frac{2\lambda}{N}\boldsymbol{w}_{\text{REG}} = \boldsymbol{0}$$

  means, with the gradient of linear regression from Section 9.2,

$$\frac{2}{N}\left(\mathrm{Z}^{\top}\mathrm{Z}\boldsymbol{w}_{\text{REG}} - \mathrm{Z}^{\top}\boldsymbol{y}\right) + \frac{2\lambda}{N}\boldsymbol{w}_{\text{REG}} = \boldsymbol{0}.$$

- The optimal solution is

$$\boldsymbol{w}_{\text{REG}} \leftarrow (\mathrm{Z}^{\top}\mathrm{Z} + \lambda\mathrm{I})^{-1}\mathrm{Z}^{\top}\boldsymbol{y},$$

  called **ridge regression** in Statistics. For $\lambda > 0$ the matrix $\mathrm{Z}^{\top}\mathrm{Z} + \lambda\mathrm{I}$ is always invertible, unlike $\mathrm{Z}^{\top}\mathrm{Z}$ alone.

The same equation can also be read the other way round: it is exactly the condition 'gradient $= \boldsymbol{0}$' of an unconstrained function.

- If an oracle tells you $\lambda > 0$, then solving $\nabla E_{\text{in}}(\boldsymbol{w}_{\text{REG}}) + \frac{2\lambda}{N}\boldsymbol{w}_{\text{REG}} = \boldsymbol{0}$ is equivalent to minimizing

$$\underbrace{E_{\text{in}}(\boldsymbol{w}) + \frac{\lambda}{N}\overbrace{\boldsymbol{w}^{\top}\boldsymbol{w}}^{\text{regularizer}}}_{\text{augmented error } E_{\text{aug}}(\boldsymbol{w})},$$

  since the gradient of $\frac{\lambda}{N}\boldsymbol{w}^{\top}\boldsymbol{w}$ is $\frac{2\lambda}{N}\boldsymbol{w}$.

- So regularization can be done with the **augmented error** instead of the constrained E_{in}:

$$\boldsymbol{w}_{\text{REG}} \leftarrow \arg\min_{\boldsymbol{w}} E_{\text{aug}}(\boldsymbol{w}) \quad \text{for given } \lambda > 0 \text{ or } \lambda = 0.$$

Minimizing unconstrained E_{aug} effectively minimizes some C-constrained E_{in}.

The trade is that we no longer specify C but λ.

Remark 14.2.3. 'λ corresponds to C' means that both problems have the same answer: the $\boldsymbol{w}$ that minimizes E_{aug} with this λ is also the $\boldsymbol{w}$ that minimizes E_{in} under $\boldsymbol{w}^{\top}\boldsymbol{w} \leq C$. Here is how to find that C for a given $\lambda > 0$.

(1) **Solve the λ problem.** Let $\boldsymbol{w}_{\text{REG}}$ be the minimizer of $E_{\text{aug}}(\boldsymbol{w}) = E_{\text{in}}(\boldsymbol{w}) + \frac{\lambda}{N}\boldsymbol{w}^{\top}\boldsymbol{w}$.

(2) **Choose C from its answer.** Set $C = \boldsymbol{w}_{\text{REG}}^{\top}\boldsymbol{w}_{\text{REG}}$. Then $\boldsymbol{w}_{\text{REG}}$ itself satisfies the constraint $\boldsymbol{w}^{\top}\boldsymbol{w} \leq C$, with equality.

(3) **Compare with every other feasible $\boldsymbol{w}$.** Take any $\boldsymbol{w}$ with $\boldsymbol{w}^\top\boldsymbol{w} \le C$. Since $\boldsymbol{w}_{\text{REG}}$ minimizes E_{aug},

$$E_{\text{in}}(\boldsymbol{w}) + \frac{\lambda}{N}\boldsymbol{w}^\top\boldsymbol{w} \;\ge\; E_{\text{in}}(\boldsymbol{w}_{\text{REG}}) + \frac{\lambda}{N}\underbrace{\boldsymbol{w}_{\text{REG}}^\top\boldsymbol{w}_{\text{REG}}}_{=C}.$$

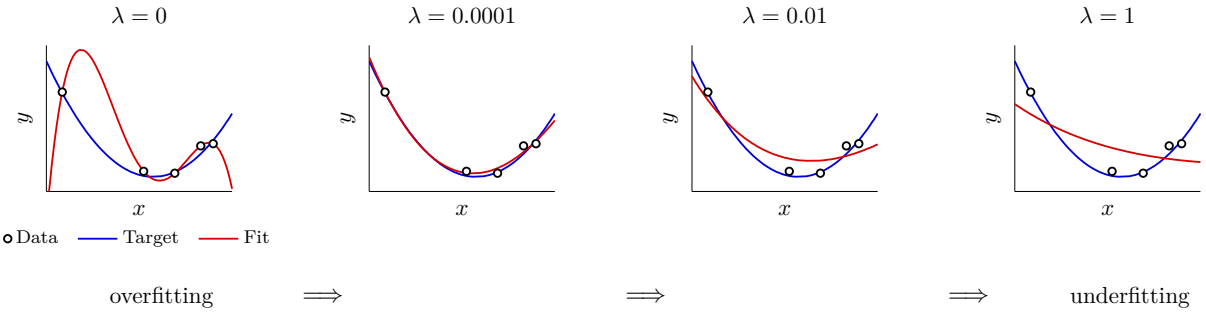
Move the penalty term to the right:

$$E_{\text{in}}(\boldsymbol{w}) \;\ge\; E_{\text{in}}(\boldsymbol{w}_{\text{REG}}) + \frac{\lambda}{N}\underbrace{\left(C - \boldsymbol{w}^\top\boldsymbol{w}\right)}_{\ge 0 \text{ since } \boldsymbol{w} \text{ is feasible}} \;\ge\; E_{\text{in}}(\boldsymbol{w}_{\text{REG}}).$$

(4) **Conclude.** No feasible $\boldsymbol{w}$ has a smaller E_{in} than $\boldsymbol{w}_{\text{REG}}$, so $\boldsymbol{w}_{\text{REG}}$ is the solution of the C-constrained problem.

So each λ comes with the C equal to the squared length of its own solution. A larger λ charges more for length, so the solution comes out shorter, and its C is smaller.

§14.2.iv. The Results



The same five examples, fitted with a 4th-order transform and increasing λ. At $\lambda = 0$ there is no regularization and the fit is the overfit from before. Already $\lambda = 0.0001$ removes the large swing and gives the 'regularized fit' of the first section. At $\lambda = 1$ the weights are pressed so short that the fit can no longer follow the data, which is underfitting.

Philosophy: a little regularization goes a long way!

Definition 14.2.4 (Weight-decay regularization)**.** Adding $+\frac{\lambda}{N}\boldsymbol{w}^\top\boldsymbol{w}$ to E_{in} is called **weight-decay regularization**:

$$\text{larger } \lambda \Leftrightarrow \text{prefer shorter } \boldsymbol{w} \Leftrightarrow \text{effectively smaller } C.$$

It goes with 'any' transform + linear model.

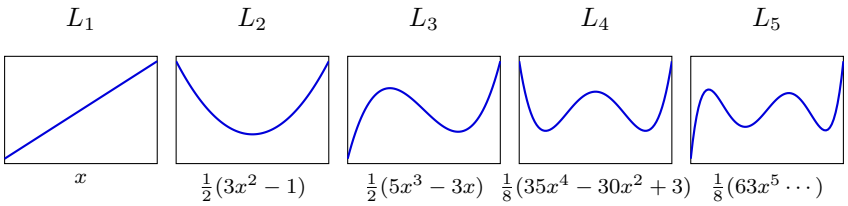
The name comes from gradient descent: each step on E_{aug} first shrinks the current $\boldsymbol{w}$ by a constant factor and then moves along $-\nabla E_{\text{in}}$, so without support from the data the weights decay towards zero.

§14.2.v. Some Detail: Legendre Polynomials

$$\min_{\boldsymbol{w}\in\mathbb{R}^{Q+1}} \frac{1}{N}\sum_{n=1}^{N}\left(\boldsymbol{w}^\top\boldsymbol{\Phi}(x_n) - y_n\right)^2 + \frac{\lambda}{N}\sum_{q=0}^{Q} w_q^2$$

Weight decay penalizes every w_q by the same amount, so the transform matters.

- The naive polynomial transform is $\mathbf{\Phi}(x) = \left(1, x, x^2, \dots, x^Q\right)$. When $x_n \in [-1, +1]$, x_n^q is really small for large q, so a high-order term needs a large w_q to have any effect, and weight decay punishes it unfairly.
- The normalized polynomial transform is $(1, L_1(x), L_2(x), \dots, L_Q(x))$, using the 'orthonormal basis functions' called **Legendre polynomials**. Each L_q spans the whole range $[-1, 1]$, so equal weights mean comparable effects.



The plots show why: every L_q swings between -1 and 1 on the whole interval, with q zero crossings, whereas x^q stays near 0 except close to ± 1.

Exercise 14.2.5 (Fun Time)**.** When would $\boldsymbol{w}_{\text{REG}}$ equal $\boldsymbol{w}_{\text{LIN}}$?

(1) $\lambda = 0$;

(2) $C = \infty$;

(3) $C \geq \|\boldsymbol{w}_{\text{LIN}}\|^2$;

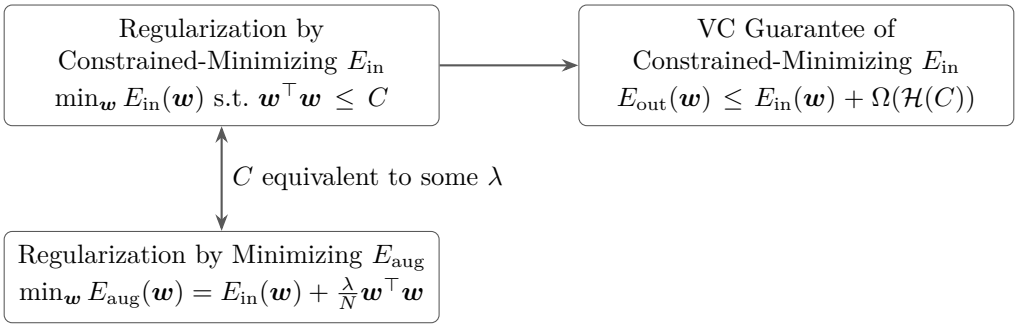
(4) all of the above.

Reference Answer. (4). (1) and (2) shall be easy; (3) means that there are effectively no constraint on $\boldsymbol{w}$, hence the equivalence: $\boldsymbol{w}_{\text{LIN}}$ already lies inside the sphere, so the constraint never binds. ■

§14.3. Regularization and VC Theory

slides p.14–17

§14.3.i. Regularization and VC Theory



Read the diagram as a chain. Minimizing E_{in} under $\boldsymbol{w}^\top\boldsymbol{w} \leq C$ is learning with the hypothesis set $\mathcal{H}(C)$, so the VC bound applies with the complexity of $\mathcal{H}(C)$ (top arrow). Minimizing E_{aug} gives the same solution as the constrained problem for some C (vertical arrow). So minimizing E_{aug} inherits that guarantee, even though it searches all of $\mathbb{R}^{Q+1}$.

Minimizing E_{aug}: indirectly getting VC guarantee without confining to $\mathcal{H}(C)$.

§14.3.ii. Another View of Augmented Error

Put the two formulas side by side:

$$\text{augmented error: } E_{\text{aug}}(\boldsymbol{w}) = E_{\text{in}}(\boldsymbol{w}) + \frac{\lambda}{N}\boldsymbol{w}^\top\boldsymbol{w}, \qquad \text{VC bound: } E_{\text{out}}(\boldsymbol{w}) \leq E_{\text{in}}(\boldsymbol{w}) + \Omega(\mathcal{H}).$$

Both add a complexity term to E_{in}, but of different things.

- The regularizer $\boldsymbol{w}^\top\boldsymbol{w}$, written $\Omega(\boldsymbol{w})$, measures the complexity of a *single hypothesis*.
- The generalization price $\Omega(\mathcal{H})$ measures the complexity of a *hypothesis set*.
- If $\frac{\lambda}{N}\Omega(\boldsymbol{w})$ 'represents' $\Omega(\mathcal{H})$ well, then E_{aug} is a better proxy of E_{out} than E_{in} is.

Minimizing E_{aug}:
(heuristically) operating with the better proxy;
(technically) enjoying flexibility of whole $\mathcal{H}$.

§14.3.iii. Effective VC Dimension

$$\min_{\boldsymbol{w}\in\mathbb{R}^{\tilde{d}+1}} E_{\text{aug}}(\boldsymbol{w}) = E_{\text{in}}(\boldsymbol{w}) + \frac{\lambda}{N}\Omega(\boldsymbol{w})$$

What is the model complexity of this learning?

- Counted by the hypothesis set, $d_{\text{VC}}(\mathcal{H}) = \tilde{d}+1$, because all $\boldsymbol{w}$ are 'considered' during the minimization.
- The $\boldsymbol{w}$ that are 'actually needed', however, form only $\mathcal{H}(C)$, with some C equivalent to λ.
- $d_{\text{VC}}(\mathcal{H}(C))$ is therefore the more honest count. It depends on the algorithm as well as on $\mathcal{H}$, and is called the **effective VC dimension** $d_{\text{EFF}}(\mathcal{H}, \mathcal{A})$, where $\mathcal{A}$ is 'min E_{aug}'.

Explanation of regularization: $d_{\text{VC}}(\mathcal{H})$ large, while $d_{\text{EFF}}(\mathcal{H}, \mathcal{A})$ small if $\mathcal{A}$ regularized.

Exercise 14.3.1 (Fun Time)**.** Consider the weight-decay regularization with regression. When increasing λ in $\mathcal{A}$, what would happen with $d_{\text{EFF}}(\mathcal{H}, \mathcal{A})$?

(1) $d_{\text{EFF}} \uparrow$;
(2) $d_{\text{EFF}} \downarrow$;
(3) $d_{\text{EFF}} = d_{\text{VC}}(\mathcal{H})$ and does not depend on λ;
(4) $d_{\text{EFF}} = 1126$ and does not depend on λ.

Reference Answer. (2). Larger $\lambda \Leftrightarrow$ smaller $C \Leftrightarrow$ smaller $\mathcal{H}(C) \Leftrightarrow$ smaller d_{EFF}. ■

§14.4. General Regularizers

slides p.18–21

§14.4.i. General Regularizers $\Omega(\boldsymbol{w})$

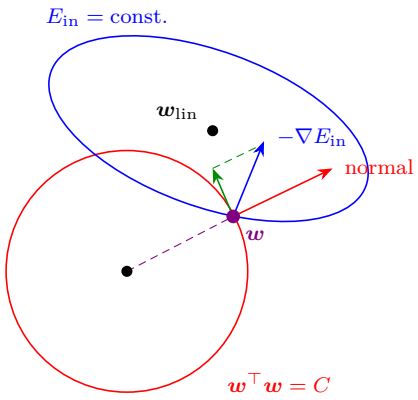
Weight decay is one choice of $\Omega(\boldsymbol{w})$. In general we want the constraint to point in the 'direction' of the target function, so that what it removes is mostly what we would have got wrong anyway.

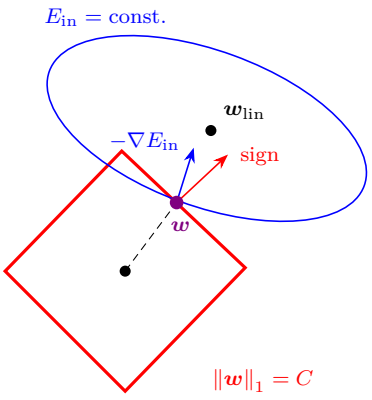
- *Target-dependent*: use some properties of the target, if known.
  - For a symmetric target, the **symmetry** regularizer $\sum [\![q \text{ is odd}]\!] w_q^2$ penalizes only the odd terms.
- *Plausible*: push in a direction towards **smoother** or **simpler** hypotheses, because stochastic and deterministic noise are both **non-smooth**, and fitting them is what overfitting does.
  - The **sparsity** (L1) regularizer $\sum |w_q|$ is one such choice (next subsection).
- *Friendly*: make the problem easy to optimize.
  - The **weight-decay** (L2) regularizer $\sum w_q^2$ keeps a closed-form solution.
- *Bad?* :-) No worries: a poor regularizer is guarded by λ, since $\lambda = 0$ switches it off.

Augmented error = error $\widehat{\text{err}}$ + regularizer Ω.
Regularizer: target-dependent, plausible, or friendly.
Ringing a bell? :-)
Error measure: user-dependent, plausible, or friendly.

The bell is Definition 8.3.1: the algorithmic error measure was chosen by the same three criteria. Both parts of E_{aug} are design choices of the algorithm, made for the same kinds of reasons.

§14.4.ii. L2 and L1 Regularizer





L2 Regularizer

$$\Omega(\boldsymbol{w}) = \sum_{q=0}^{Q} w_q^2 = \|\boldsymbol{w}\|_2^2$$

- convex, differentiable everywhere;
- easy to optimize.

L1 Regularizer

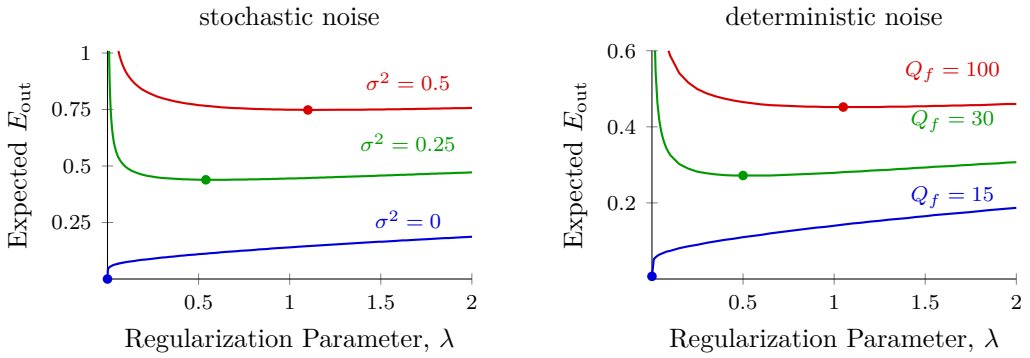
$$\Omega(\boldsymbol{w}) = \sum_{q=0}^{Q} |w_q| = \|\boldsymbol{w}\|_1$$

- convex, **not** differentiable everywhere;
- **sparsity** in solution.

The left picture is the one from the Lagrange multiplier. On the right the sphere becomes the diamond $\|\boldsymbol{w}\|_1 = C$. Its normal direction on each flat face is the **sign** vector of $\boldsymbol{w}$, the same at every point of that face. So along a face, $-\nabla E_{\text{in}}$ keeps a component that slides $\boldsymbol{w}$ further, until it reaches a corner. The corners lie on the axes, where some $w_q = 0$: the solution is often sparse.

L1 useful if needing sparse solution.

§14.4.iii. The Optimal λ



Each curve is the expected E_{out} of weight decay as λ grows, and the dot marks its minimum, the best λ for that setting. On the left the stochastic noise level σ^2 varies; on the right the target complexity Q_f, i.e. the deterministic noise.

- More noise $\Leftrightarrow$ more regularization needed: the dot moves right as σ^2 or Q_f grows, and without noise ($\sigma^2 = 0$) the best λ is 0. In driving terms, a more bumpy road $\Leftrightarrow$ putting on the brakes more.
- The noise is unknown in practice, so it is important to make proper choices of λ. Too little leaves the overfitting; too much climbs the right side of each curve.

Question 14.4.1. How to choose? Stay tuned for the next lecture! :-)

Exercise 14.4.2 (Fun Time)**.** Consider using a regularizer $\Omega(\boldsymbol{w}) = \sum_{q=0}^{Q} 2^q w_q^2$ to work with Legendre polynomial regression. Which kind of hypothesis does the regularizer prefer?

(1) symmetric polynomials satisfying $h(x) = h(-x)$;

(2) low-dimensional polynomials;

(3) high-dimensional polynomials;

(4) no specific preference.

Reference Answer. (2). There is a higher 'penalty' for higher-order terms, and hence the regularizer prefers low-dimensional polynomials. ■

Summary

Regularized Hypothesis Set original $\mathcal{H}$ + constraint.

Weight Decay Regularization add $\frac{\lambda}{N}\boldsymbol{w}^\top\boldsymbol{w}$ in E_{aug}.

Regularization and VC Theory regularization decreases d_{EFF}.

General Regularizers target-dependent, [plausible], or [friendly].

Regularization trades a little E_{in} for a large drop in effective complexity, and its strength is set by a single number λ. The optimal-λ curves show that the right value depends on noise we cannot see. Next: choosing from the so-many models and parameters.